

i have to share my react js with express and mongodb project so first i share my all project files you analyze and give me suggestion on last after sending all files ok

backend files

config/database.js

```
"import mongoose from 'mongoose'
```

```
const connectDB = async () => {
```

```
  try {
```

```
    if (!process.env.MONGODB_URI) {
```

```
      console.error('✗ MONGODB_URI is required in environment variables')
```

```
      process.exit(1)
```

```
    }
```

```
    const conn = await mongoose.connect(process.env.MONGODB_URI)
```

```
    console.log(`✔ MongoDB Connected: ${conn.connection.host}`)
```

```
    return conn
```

```
  } catch (error) {
```

```
    console.error('✗ MongoDB connection error:', error.message)
```

```
    process.exit(1)
```

```
  }
```

```
}
```

```
export default connectDB
```

```
",
```

constants/categories.js

```
"export const CATEGORIES = {
```

```
  web: {
```

```
name: 'Web Exploitation',
icon: '🌐',
color: '#3B82F6',
description: 'Find and exploit web vulnerabilities'
},
crypto: {
name: 'Cryptography',
icon: '🔒',
color: '#10B981',
description: 'Decrypt and break cryptographic systems'
},
forensics: {
name: 'Forensics',
icon: '🔍',
color: '#F59E0B',
description: 'Analyze files and recover hidden data'
},
pwn: {
name: 'Binary Exploitation',
icon: '💣',
color: '#EF4444',
description: 'Exploit binary vulnerabilities'
},
reverse: {
name: 'Reverse Engineering',
icon: '🔧',
color: '#8B5CF6',
description: 'Reverse engineer applications'
```

```

},
misc: {
  name: 'Miscellaneous',
  icon: '🔒',
  color: '#6B7280',
  description: 'Various security challenges'
}
};

```

```

export const DIFFICULTIES = {
  easy: { name: 'Easy', color: '#10B981', points: 100 },
  medium: { name: 'Medium', color: '#F59E0B', points: 300 },
  hard: { name: 'Hard', color: '#EF4444', points: 500 }
};",

```

middleware/advancedRateLimit.js

```
"import Redis from 'ioredis'
```

```
let redis = null
```

```
// ✔ Only connect to Redis if explicitly enabled
```

```
if (process.env.REDIS_ENABLED === 'true') {
```

```
  redis = new Redis(process.env.REDIS_URL)
```

```
  redis.on('connect', () => console.log('✔ Redis connected'))
```

```
  redis.on('error', (err) =>
```

```
    console.warn('⚠ Redis connection error (ignored in dev):', err.message)
```

```
)
```

```

} else {

  console.log('⚠ Redis disabled — using in-memory fallback for rate limiting')
}

// Helper for safe Redis commands or mock
const safeIncr = async (key) => {
  if (!redis) {
    // Fake increment counter
    if (!global.__fakeCounters) global.__fakeCounters = {}
    global.__fakeCounters[key] = (global.__fakeCounters[key] || 0) + 1
    return global.__fakeCounters[key]
  }
  return await redis.incr(key)
}

const safeExpire = async (key, seconds) => {
  if (!redis) return
  await redis.expire(key, seconds)
}

// === Rate limit middlewares ===
export const challengeSpecificRateLimit = (maxAttempts = 10, windowMinutes = 60) => {
  return async (req, res, next) => {
    const key = `rate_limit:${req.userId}:${req.params.challengeId}`

    try {
      const current = await safeIncr(key)

      if (current === 1) await safeExpire(key, windowMinutes * 60)
    }
  }
}

```

```

    if (current > maxAttempts) {
      return res.status(429).json({
        error: `Too many attempts. Try again in ${windowMinutes} minutes.`,
        retryAfter: windowMinutes * 60
      })
    }

    req.attemptCount = current
    next()
  } catch (error) {
    console.error('Rate limit error:', error)
    next()
  }
}

export const authRateLimit = (maxAttempts = 5, windowMinutes = 15) => {
  return async (req, res, next) => {
    const identifier = req.ip || req.connection.remoteAddress
    const key = `auth_rate_limit:${identifier}`

    try {
      const current = await safeIncr(key)
      if (current === 1) await safeExpire(key, windowMinutes * 60)

      if (current > maxAttempts) {
        return res.status(429).json({
          error: `Too many authentication attempts. Try again in ${windowMinutes} minutes.`
        })
      }
    }
  }
}

```

```

    }

    next()
  } catch (error) {
    console.error('Auth rate limit error:', error)
    next()
  }
}
}

export const globalRateLimit = (maxRequests = 1000, windowMinutes = 15) => {
  return async (req, res, next) => {
    const identifier = req.ip || req.connection.remoteAddress
    const key = `global_rate_limit:${identifier}`

    try {
      const current = await safeIncr(key)
      if (current === 1) await safeExpire(key, windowMinutes * 60)

      if (current > maxRequests) {
        return res.status(429).json({
          error: 'Too many requests. Please slow down.'
        })
      }

      next()
    } catch (error) {
      console.error('Global rate limit error:', error)
      next()
    }
  }
}

```

```
}  
}  
}  
",
```

auth.js

```
"import jwt from 'jsonwebtoken'
```

```
import User from '../models/User.js'
```

```
const authMiddleware = async (req, res, next) => {
```

```
  try {
```

```
    const token = req.header('Authorization')?.replace('Bearer ', '')
```

```
    if (!token) {
```

```
      return res.status(401).json({ error: 'No token, authorization denied' })
```

```
    }
```

```
    // Validate JWT_SECRET exists
```

```
    if (!process.env.JWT_SECRET) {
```

```
      console.error('JWT_SECRET not configured')
```

```
      return res.status(500).json({ error: 'Server configuration error' })
```

```
    }
```

```
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
```

```
    const user = await User.findById(decoded.userId).select('-password')
```

```
    if (!user) {
```

```
      return res.status(401).json({ error: 'Token is not valid' })
```

```
    }
```

```
    if (!user.isActive) {
      return res.status(401).json({ error: 'Account is deactivated' })
    }

    req.userId = user._id
    req.user = user
    next()
  } catch (error) {
    console.error('Auth middleware error:', error)

    if (error.name === 'JsonWebTokenError') {
      return res.status(401).json({ error: 'Invalid token' })
    } else if (error.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'Token expired' })
    }

    res.status(401).json({ error: 'Authentication failed' })
  }
}

const adminMiddleware = async (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Access denied. Admin required.' })
  }
  next()
}

export { authMiddleware, adminMiddleware },
```


scripts/createAdmin.js

```
"import User from '../models/User.js'
```

```
import connectDB from '../config/database.js'
```

```
import { randomBytes } from 'crypto'
```

```
import Badge from '../models/Badge.js'
```

```
const createAdminUser = async () => {
```

```
  try {
```

```
    await connectDB()
```

```
    console.log('🌀 Setting up admin user and system...')
```

```
    // Check if admin already exists
```

```
    const existingAdmin = await User.findOne({ role: 'admin' })
```

```
    if (existingAdmin) {
```

```
      console.log('✅ Admin user already exists')
```

```
      console.log('📁 Admin email:', existingAdmin.email)
```

```
      return
```

```
    }
```

```
    // Generate secure random password
```

```
    const tempPassword = randomBytes(12).toString('hex')
```

```
    // Create admin user
```

```
    const adminUser = new User({
```

```
      username: 'admin',
```

```
      email: 'admin@hackathon.com',
```

```
      password: tempPassword,
```

```
    fullName: 'System Administrator',  
    role: 'admin',  
    score: 0  
  })
```

```
await adminUser.save()
```

```
// Initialize default badges
```

```
const badgeService = await import('../services/badgeService.js')
```

```
await badgeService.default.initializeDefaultBadges()
```

```
console.log('🎉 System setup completed successfully!')
```

```
console.log('=====')
```

```
console.log('📁 Admin Login Credentials:')
```

```
console.log(` ✉ Email: admin@hackathon.com`)
```

```
console.log(` 🔑 Password: ${tempPassword}`)
```

```
console.log(` 👤 Username: admin`)
```

```
console.log('=====')
```

```
console.log('🚀 Features Enabled:')
```

```
console.log(' ✔ Dynamic Scoring System')
```

```
console.log(' ✔ Team Management')
```

```
console.log(' ✔ Achievement System')
```

```
console.log(' ✔ Badge Rewards')
```

```
console.log(' ✔ Real-time Leaderboard')
```

```
console.log(' ✔ Docker Challenge Support')
```

```
console.log(' ✔ File Upload/Download')
```

```
console.log(' ✔ Writeup System')
```

```
console.log('=====')
```

```

console.log('⚠ IMPORTANT:')
console.log(' 1. Change the admin password immediately!')
console.log(' 2. Configure environment variables for production')
console.log(' 3. Set up Docker if using container challenges')
console.log(' 4. Configure Redis for rate limiting and caching')

} catch (error) {
  console.error('❌ Error during system setup:', error.message)
  process.exit(1)
} finally {
  process.exit()
}
}

```

```
createAdminUser()",
```

```
initializeBadges.js
```

```
"import '../config/database.js'
```

```
import Badge from '../models/Badge.js'
```

```
const initializeBadges = async () => {
```

```
  try {
```

```
    console.log('🔄 Initializing default badges...')
```

```
const defaultBadges = [
```

```
{
```

```
  name: 'First Blood',
```

```
  description: 'Get the first solve on a challenge',
```

```
  icon: '🏆',
```

```
color: '#DC2626',
criteria: { type: 'first_blood' },
rarity: 'rare'
},
{
name: 'Web Warrior',
description: 'Solve 5 web challenges',
icon: '🌐',
color: '#3B82F6',
criteria: { type: 'category_mastery', category: 'web', threshold: 5 },
rarity: 'common'
},
{
name: 'Crypto Expert',
description: 'Solve 5 cryptography challenges',
icon: '🔒',
color: '#10B981',
criteria: { type: 'category_mastery', category: 'crypto', threshold: 5 },
rarity: 'common'
},
{
name: 'Forensics Master',
description: 'Solve 5 forensics challenges',
icon: '🔍',
color: '#F59E0B',
criteria: { type: 'category_mastery', category: 'forensics', threshold: 5 },
rarity: 'common'
},
{
```

```
name: 'Pwn Guru',
description: 'Solve 5 binary exploitation challenges',
icon: '🌟',
color: '#EF4444',
criteria: { type: 'category_mastery', category: 'pwn', threshold: 5 },
rarity: 'common'
},
{
name: 'Reverse Engineer',
description: 'Solve 5 reverse engineering challenges',
icon: '🔪',
color: '#8B5CF6',
criteria: { type: 'category_mastery', category: 'reverse', threshold: 5 },
rarity: 'common'
},
{
name: 'Score Hunter',
description: 'Reach 1000 points',
icon: '🎯',
color: '#F59E0B',
criteria: { type: 'score_threshold', threshold: 1000 },
rarity: 'common'
},
{
name: 'CTF Legend',
description: 'Reach 5000 points',
icon: '🏆',
color: '#8B5CF6',
```

```

    criteria: { type: 'score_threshold', threshold: 5000 },
    rarity: 'legendary'
  },
  {
    name: 'Team Player',
    description: 'Be part of a winning team',
    icon: '👥',
    color: '#6B7280',
    criteria: { type: 'team_work' },
    rarity: 'epic'
  },
  {
    name: 'Speed Demon',
    description: 'Solve a challenge within 5 minutes of starting',
    icon: '⚡',
    color: '#10B981',
    criteria: { type: 'speed_demon', threshold: 300 }, // 5 minutes in seconds
    rarity: 'rare'
  }
]

```

```

let createdCount = 0
for (const badgeData of defaultBadges) {
  const existing = await Badge.findOne({ name: badgeData.name })
  if (!existing) {
    await Badge.create(badgeData)
    createdCount++
    console.log(`✔ Created badge: ${badgeData.name}`)
  }
}

```

```
}
```

```
console.log(`🏆 Badge initialization complete! Created ${createdCount} new badges.`)
```

```
  } catch (error) {
```

```
    console.error('❌ Error initializing badges:', error)
```

```
  } finally {
```

```
    process.exit()
```

```
  }
```

```
}
```

```
initializeBadges()",
```

```
utils/backupManager.js
```

```
"import fs from 'fs/promises'
```

```
import path from 'path'
```

```
import { fileURLToPath } from 'url'
```

```
import archiver from 'archiver'
```

```
import { createWriteStream } from 'fs'
```

```
import Challenge from '../models/Challenge.js'
```

```
import User from '../models/User.js'
```

```
import Submission from '../models/Submission.js'
```

```
const __filename = fileURLToPath(import.meta.url)
```

```
const __dirname = path.dirname(__filename)
```

```
export class BackupManager {
```

```
  constructor() {
```

```
    this.backupDir = path.join(__dirname, '../backups')
```

```
}
```

```
async ensureBackupDir() {
```

```
  try {
```

```
    await fs.access(this.backupDir)
```

```
  } catch {
```

```
    await fs.mkdir(this.backupDir, { recursive: true })
```

```
  }
```

```
}
```

```
async createBackup() {
```

```
  await this.ensureBackupDir()
```

```
  const timestamp = new Date().toISOString().replace(/[:.]/g, '-')
```

```
  const backupPath = path.join(this.backupDir, `backup-${timestamp}.json`)
```

```
  const zipPath = path.join(this.backupDir, `backup-${timestamp}.zip`)
```

```
  try {
```

```
    // Collect all data
```

```
    const backupData = {
```

```
      timestamp: new Date(),
```

```
      metadata: {
```

```
        version: '1.0',
```

```
        records: {}
```

```
      },
```

```
      data: {
```

```
        challenges: await Challenge.find().lean(),
```

```
        users: await User.find().select('-password').lean(),
```

```
        submissions: await Submission.find().lean(),
```



```

    teams: await (await import('../models/Team.js')).default.find().lean(),
    achievements: await (await import('../models/Achievement.js')).default.find().lean()
  }
}

// Save as JSON
await fs.writeFile(backupPath, JSON.stringify(backupData, null, 2))

// Create zip file
await this.createZipFile(backupPath, zipPath)

// Remove JSON file
await fs.unlink(backupPath)

console.log(`Backup created: ${zipPath}`)
return zipPath

} catch (error) {
  console.error('Backup creation failed:', error)
  throw error
}
}

async createZipFile(sourcePath, zipPath) {
  return new Promise((resolve, reject) => {
    const output = createWriteStream(zipPath)
    const archive = archiver('zip', { zlib: { level: 9 } })

    output.on('close', () => resolve(zipPath))
  })
}

```

```

    archive.on('error', reject)

    archive.pipe(output)
    archive.file(sourcePath, { name: 'backup.json' })
    archive.finalize()
  })
}

async restoreBackup(backupPath) {
  try {
    // This would involve reading the backup and restoring data
    // For security, this should be done carefully in production
    console.log('Restore functionality would be implemented here')

    return { success: true, message: 'Restore process completed' }
  } catch (error) {
    console.error('Restore failed:', error)
    throw error
  }
}

async listBackups() {
  await this.ensureBackupDir()

  try {
    const files = await fs.readdir(this.backupDir)
    const backups = files
      .filter(file => file.endsWith('.zip'))
      .map(file => {

```

```

    const filePath = path.join(this.backupDir, file)

    return {
      filename: file,
      path: filePath,
      created: file.split('-').slice(1).join('-').replace('.zip', '')
    }
  })
  .sort((a, b) => b.created.localeCompare(a.created))

  return backups
} catch (error) {
  console.error('Failed to list backups:', error)
  return []
}
}

async cleanupOldBackups(maxAgeDays = 30) {
  const backups = await this.listBackups()
  const cutoff = new Date()
  cutoff.setDate(cutoff.getDate() - maxAgeDays)

  for (const backup of backups) {
    const backupDate = new Date(backup.created)
    if (backupDate < cutoff) {
      await fs.unlink(backup.path)
      console.log(`Deleted old backup: ${backup.filename}`)
    }
  }
}

```

```
}
```

```
export default new BackupManager()",
```

```
utils/commandExecutor.js
```

```
"import User from '../models/User.js'
```

```
import Submission from '../models/Submission.js'
```

```
import Hint from '../models/Hint.js'
```

```
import UserHint from '../models/UserHint.js'
```

```
import achievementService from '../services/achievementService.js'
```

```
import socketService from '../services/socketService.js'
```

```
export const executeCommand = async (command, challenge, user, sessionData = {}) => {
```

```
  const args = command.split(' ');
```

```
  const cmd = args[0].toLowerCase();
```

```
  // Update session data for time tracking
```

```
  if (!sessionData.startTime) {
```

```
    sessionData.startTime = new Date();
```

```
  }
```

```
  try {
```

```
    switch (cmd) {
```

```
      case 'help':
```

```
        return await handleHelpCommand();
```

```
      case 'ls':
```

```
        return await handleLsCommand(challenge);
```

```
case 'cat':  
    return await handleCatCommand(args, challenge);  
  
case 'check':  
    return await handleCheckCommand(args, challenge, user, sessionData);  
  
case 'hint':  
    return await handleHintCommand(args, challenge, user);  
  
case 'pwd':  
    return { output: `/challenges/${challenge._id}` };  
  
case 'clear':  
    return { output: '\n'.repeat(50) + 'Terminal cleared' };  
  
case 'find':  
    return await handleFindCommand(args, challenge);  
  
case 'download':  
    return await handleDownloadCommand(args, challenge);  
  
case 'status':  
    return await handleStatusCommand(challenge, user);  
  
case 'whoami':  
    return { output: `You are: ${user.username} (${user.role})` };  
  
default:  
    return { output: `Command not found: ${cmd}. Type 'help' for available commands.` };
```

```

    }
  } catch (error) {
    console.error('Command execution error:', error);
    return { output: 'Error executing command' };
  }
};

```

```

const handleHelpCommand = async () => {
  return {
    output: `Enhanced CTF Terminal - Available Commands:

```

Command	Description
ls	List directory contents
cat <file>	Display file content
find <pattern>	Search for files and content
check <flag>	Submit flag for verification
hint [level]	Get a challenge hint (levels 1-3)
download <file>	Download challenge files
status	Show challenge progress and stats
pwd	Show current directory
whoami	Show current user information
clear	Clear terminal screen

💡 Tip: Use 'hint 1' for the cheapest hint, 'hint 3' for the most helpful!

```

  };
};

const handleLsCommand = async (challenge) => {

```

```

try {
  // Enhanced file listing with challenge-specific files

  let mockFiles = ['flag.txt', 'hint.md', 'readme.txt'];

  // Add category-specific files
  const categoryFiles = {
    web: ['index.html', 'server.js', 'config.php', 'database.sql'],
    crypto: ['encrypted.txt', 'key.pem', 'cipher.py', 'instructions.md'],
    forensics: ['image.jpg', 'memory.dmp', 'logfile.log', 'packet.pcap'],
    pwn: ['vulnerable', 'source.c', 'exploit.py', 'flag.bin'],
    reverse: ['binary.exe', 'disassembly.txt', 'strings.txt', 'license.key'],
    misc: ['challenge.zip', 'data.csv', 'script.sh', 'output.log']
  };

  mockFiles = [...mockFiles, ...(categoryFiles[challenge.category] || [])];

  // Add attachments if any
  if (challenge.attachments && challenge.attachments.length > 0) {
    challenge.attachments.forEach(att => mockFiles.push(att.name));
  }

  return {
    output: 'Directory contents:',
    files: mockFiles,
    metadata: {
      totalFiles: mockFiles.length,
      category: challenge.category
    }
  };
}

```

```
    } catch (error) {  
      return { output: 'Error reading directory' };  
    }  
  };  
};
```

```
const handleCatCommand = async (args, challenge) => {  
  if (args.length < 2) {  
    return { output: 'Usage: cat <filename>' };  
  }  
}
```

```
const filename = args[1];
```

```
// Security: Prevent path traversal
```

```
if (filename.includes('.') || filename.includes('/') || filename.includes('\\')) {  
  return { output: 'Invalid filename' };  
}
```

```
// Enhanced file contents with challenge-specific data
```

```
const fileContents = {  
  'flag.txt': 'The flag is hidden somewhere... Use your skills to find it!\n\n💡 Hint: The flag format is CTF{...}',
```

```
  'hint.md': challenge.hint || 'Look for vulnerabilities in the system.',
```

```
  'readme.txt': `Challenge: ${challenge.title}
```

```
Category: ${challenge.category}
```

```
Difficulty: ${challenge.difficulty}
```

```
Points: ${challenge.points}
```

```
Solves: ${challenge.solveCount}
```

```
Description:
```


`${challenge.description}`

Good luck! 🍀,

`'instructions.md': `# ${challenge.title}`

`## Challenge Instructions`

`${challenge.description}`

`## Flag Format`

The flag follows this pattern: CTF{...}

`## Tips`

- Read all files carefully
 - Look for hidden information
 - Check file permissions and metadata
 - Use appropriate tools for the challenge category`
- `};`

`// Add category-specific file contents`

`const categoryContents = {`

`web: {`

`'index.html': `<html>`

`<head><title>${challenge.title}</title></head>`

`<body>`

`<h1>Welcome to ${challenge.title}</h1>`

`<p>Look around for vulnerabilities...</p>`

`<!-- TODO: Add actual challenge content -->`

`</body>`

```
</html>`,

    'config.php': `<?php

// Database configuration

$host = 'localhost';

$dbname = 'ctf_challenge';

$username = 'admin';

$password = 'super_secret_password_123';


// TODO: Add actual challenge logic

?>`

    },

    crypto: {

        'encrypted.txt':

'VmXSQ2ExWXISWGxTYmxKV1lrZFNWRmx0ZUV0WFJteFpZMGh3VjFadGVlcFdNbIF3WVd4S2RGVnRNVk5
XYIZGNIZqSjRVMkpHYkZWV2JYaFRZbTFvTkZklpHOVVSbHB5V2taT2FWSnJOWEZWYkZKWFUxWmFkR1Z
GT1ZwaVZscEIWakxVDJOc1duUIVibFpYVWpOb2IxbHNWbmRsYkZwWVpVZDBWMVpzY0RCV01XUXdWa
kZrYzFkdFJsZGhhMXB5V1ZSR2QxWldXbk5YYIVacVRWWndTRII5ZUdGU01rMTVWVzVhYVZKR1NraFdNV2
h2VmpGa2MxZHVUbGRXUIVwaFZtMHhNRmxYVfHoVGJrNXFVbTFvVjFsWGRHRmpNV1lwVTJ0a2FsSlhVb
GhXYIRGM1V6RlplRk5yYUZaaVdHaHlWV3BHZDJGV1NuUINiR1JxVFZkU2VsWlhlR0ZaVmXsNVpFaGtWbUp
IVW5aV2FrWmhZekZ3UIZWdGFGZGISMUo2V1RCV1IWZHRWa2RYYms1cFlrWndjRIZ0ZUhkTk1WcDBUVIZ
rYUJd1ZqVldWM0JIVmpGT2RGVnNaRmROYWxaUVZqQmtTMUp0U2tkVGJHeFdZbGhvTTFac1kzaE9SMU
pHV2taT2FWSnJOVEJXVjNoclZqRmtWMVp1UmxOaVJscFIWVzAxZDJWc1duRlJiR1JwVjBkb2VsWXhaREJUT
VZsNFIVWk9XbFpzY0ROV01qVIBZVlpPEZac1pGaGISMUp4VldwR1NtVkdUblJoUm1ScFVqRktkbGxWV25k
VFJscHpZMGh3VjJkVVJUQldWRVpoVmpGd1JWSnRSbHBpVmxwSVZrZDRhMkZHU25OalJtaGFWak5OZUZ
adE1UUlpWMFY1Vkd4a1YxWkZxbFZWTW5SaFlRNdSMVp0Um1wTIYxSkIWakxVDFkR1pGaE5WV1JZVW
pCd1NWcFZXbXRqYIVWNVZXeGtWMDf1YUZSV2JURTBWakpHUjFOdVRsWmlSa3BoV1ZSS1UxSXhjRWRh
Ums1WfVsUnNWMVJYY0VkbGJGcFIUVWhvV0dKc1NsaFdiWEJIVIRKS1IyTkhiRk5OYTFwSVZteGtKMIZzV1h
oYVJtUlhZbGhDU0ZkV1dtdFhSa3B6WTBac1dtSkhhRIJXUjNoaFpERlplV1JIT1doTIZXdzJWVzAxUTJJeFduTlhi
a3BxVWxkU1dGUldXbmRYYkZwR1YyeHdXROV4Y0U5V1Z6RTBWMnhhZEUXWVFsWk5WbHA2VmpKMGE
WnRSWHBSYkdoVFIYcFdWRlpyWkRSWIZteFhVMnhXYVZKR2NGbFdNakZHWIVaa2MyRkhhRk5pUjFKMld
WUktORmxYVfhsU2EzQm9VakpvVkZsc2FGTlNNV1JIVjJ0b1ZHSiViRmRaVkvWTFpWWmtjbHBIYkZOV1JYQ
k1WVEo0YjJGR1NuVlJiR2hvVFRCS1dWbFZHR3RrTVZaSFUyeGtUbFp1UWxoV1J6RjNZekZaZUZkdVpGZG
ISMUUp4VkJaagplZ3BSYXpsWFYyeGtUMVpzVmpWWk1GcExWMjFpUjFwSGJGTmISWEJZV1ZkNFQxWXNdNV
mRqUlHSWFRWWndNRnBGV2s5VklrcFpZVpPVjJkR2NGUldha3B2VmpGU2RGtNJaRmRpVkvVaVIZGWmF
ZVmRHU2xWU2JYUlhUV3R3U0ZadGRHRmpWbVJIVjJ4V1YySlIVbGxXYIRCM1pVWmFkR1ZIZUdGU2JlQjVX
VEJhVjJSSfZrZFRiR3hwVW01Q1dGbFVTazlqTVZwMFpFVTWVbUpHY0ZoWk1HaExWMnhhVIZGdFJscGhN
bW6VlcweE5GWXhXa2RYYmtaVllrWndjRIZ0TIVOak1XUjFWbXhhVjJkSFVubFhhMXB2VmpBeFNHUKdaR2
xTYmtKUFZtMHhORII4VW5OWGJsWlIVakZ3V0Zsc2FFTlhiRnBWVW14a2FFMVIRalpYVkvKafDWwVlplV1ZIT
```

1ZkTIZYQkpWbTE0YzJGR1NuTlhiR1JxVfZad01GVnRkSGRYUm14V1YyNUdXR0pIVW5wV01uUlhVMFpzY2xwR1pGTmlSbkJaVjFSS05GVXITa2RUymxKV1IXdGFURmw2U2tabFJrNTFZVWRvVGxadVFtRldiVEUwWWpGV2MxcEdaRk5OYIdoWIZtMHhORlI3TVZoVmEyeFhWbnBHU0ZaWE1UUmINv1JIVjJ0b2JGSIIrbGxXYIhoM1lVWktjbUZHY0ZOV1JUVkdWbTE0YTFkR1duRlNiR1JxVfZac05WVnRIR0ZoTURWSFdrWk9UbFp1UWxoV1J6RjNZekZaZUZkdVpGZGISWEJZV1ZkNFQxWXDnVmRqUlHsWFRWWndNRnBGV2s5Vk1rcFpZVVPpVjJKR2NGUldha3B2VmpGU2RGTnJaRmRpVkVaVIZGWmFZVmRHU2xWU2JYUlhZa2hDV0Zac1kzZGxSazV6V2taa2FWsnNjR2hWYm5CSFI6RndSMXBIYUZOTIYxSIIWVEowYTFZeFdYcFJhMIJZWwtK2U2IxVnRNVk5YUmxwMFRWaG9VMkpXU2toV1Z6RTBWVzFHY2xOc2FGZGIXR2h5VmpCV1MxWXhXblJTV0doVVlrWktWMVpYTVRCaE1VbDRXa2hPYUzJeWFESldNRnBoWkVkv1IxUnVUbGROVmtwWIZXMTRkMIzZV25STINHUIhUVlp3TUZWdGRHRmhNa1pIVj1S1dHskhhRIJXYlhCTFZqRk9kV0ZHY0ZkT1tZ3lWbXRhUzFkR1ZuRIJibFpYWWxob00xcFdXbUZqTVdSMVUyeGtWbUpHU2tsV2JYUIRvZzVjFkdVRsaGISMUp4VkZaa2V3cFJiemxYVj4a1QxWnNWaZaTUZWTFYyMvdSMXBIYkZOaVJYQlIXVmQ0VDFZd01WZGpSWFJYVfZad01GcEZxazlWTWtwWlIVWk9WMkpHY0ZSV2FrcHZWakZTZEZOclpGZGIWRVpVWkZaYVIWZEdTbFZTYlhSWFlraENXRlpzWTNkbFJRNXpXa1prYVZKc2NHaFZibkJIWXpGd1lxcEhhRk5OVjFKWVZUSjBhMVI4V1hwUmEyUIIza2RTYjFWdE1WTlhSbHawVFZob1UySldTa2hXVnpFMFZXMUdjRk5zVWxaTIYzaFIXVlJLuzFVeFduUIRXR2hVWWxaS1dWWnRNVFJaVmxwMFRWUkNWMDfYV25sYVJFWmhZekZ3U0dWR1RsVk5Wbkj5VmpKek5WWXITa2RqU0d4VVIItMVNjRIZ0ZUd0T1JteFhWMjVTVGxKc2JETldNblJoVmpKT2RGSnJaRIJpVjNoVVdXeG9iMVpHYkZobFJyaFhZVEpTV0ZscmFFTlpWbHB4VW14a2FWSnJOVmRWYm5BMVZESkdWbFJxVGxwaE1YQnlXVEJXVDJGc1NuVlJiR1JvVFRCS1dsWXhhrTVUTVZsNvUydG9WbUpHY0ZSV2JYQkhVekZrUjJKSVRtaFNWR3hZV1ZkNFYxZEdXbkphUms1WfVVsNnWMMVI5ZERSV01rWnpXa1prYVZkRlNsaFdWRW93VmpBeGntTkZhRIJTV0VKVZGVmFZV013TIVkWfDHeHFvbXmXyJfWdE1VZFhiRnBZWIVkR1YwMXJXbnBaVIZwUFZqRmtjbHBIZEZkTIJGWktWbGR3UjFVeFpITmlSbVJwVTBaS1dGWnRNSGRsUm1SWFdrVmthMDFzU2toV2JYUIRWa1pzVjFwRVRSZGISMUo1V1ZSR1UxSXhjRWRhUm1OTfZGYzFVMIF4V2xkWGJtUlHza2hDV1ZadE1UQlpWbGw0Vj1U2JGSIIvbGxXYIhoM1lVWktjbUZHY0ZOV1JUVkdWbTE0YTFkR1duRlNiR1JxVfZac05WVnRIR0ZoTURWSFdrWk9UbFp1UWxoV1J6RjNZekZaZUZkdVpGZGISWEJZV1ZkNFQxWXDnVmRqUlHsWFRWWndNRnBGV2s5Vk1rcFpZVVPpVjJKR2NGUldha3B2VmpGU2RGTnJaRmRpVk'

}

};

// Merge base contents with category-specific contents

const allContents = { ...fileContents, ...categoryContents[challenge.category] };

if (allContents[filename]) {

return { output: allContents[filename] };

} else {

return { output: `File not found: \${filename}` };

}

};

```
const handleCheckCommand = async (args, challenge, user, sessionData) => {  
  if (args.length < 2) {  
    return { output: 'Usage: check <flag>' };  
  }  
  
  const submittedFlag = args.slice(1).join(' '); // Handle flags with spaces  
  const isCorrect = submittedFlag === challenge.flag;  
  
  try {  
    // Check if already solved  
    const existingSubmission = await Submission.findOne({  
      user: user._id,  
      challenge: challenge._id,  
      isCorrect: true  
    });  
  
    if (existingSubmission) {  
      return { output: '✔ Challenge already completed!' };  
    }  
  
    // Calculate time spent  
    const timeSpent = Math.floor((new Date() - sessionData.startTime) / 1000);  
  
    if (isCorrect) {  
      // Update user progress  
      if (!user.solvedChallenges.includes(challenge._id)) {  
        user.solvedChallenges.push(challenge._id);  
        user.score += challenge.points;  
      }  
    }  
  }  
}
```

```

    user.updateStreak();

    await user.save();
}

// Record successful submission
const submission = new Submission({
    user: user._id,
    challenge: challenge._id,
    flagSubmitted: submittedFlag,
    isCorrect: true,
    pointsAwarded: challenge.points,
    timeSpent: timeSpent
});

await submission.save();

// Update challenge solve count
await challenge.updateOne({ $inc: { solveCount: 1 } });

// Check for achievements
await achievementService.checkFirstBlood(challenge._id, user._id);
await achievementService.checkCategoryMaster(user._id, challenge.category);

// Send notification
socketService.broadcastChallengeSolve(user, challenge, submission.solveOrder === 1);

return {
    output: 🚩 Flag verified! Challenge completed!

```

🏆 Points awarded: \${challenge.points}

🕒 Time spent: $\text{\${Math.floor(timeSpent / 60)}m \text{\${timeSpent \% 60}s}$

📄 Solve position: $\text{\#\${submission.solveOrder}}$

Type 'status' to see your updated progress!`,

```
      solved: true,
      points: challenge.points,
      solveOrder: submission.solveOrder,
      timeSpent: timeSpent
    };
  } else {
    // Record failed attempt
    const submission = new Submission({
      user: user._id,
      challenge: challenge._id,
      flagSubmitted: submittedFlag,
      isCorrect: false
    });
    await submission.save();

    // Update user statistics
    await user.updateOne({ $inc: { 'statistics.totalAttempts': 1 } });

    return {
      output: '❌ Incorrect flag. Keep searching!\n💡 Use the hint command if you need help.'
    };
  }
} catch (error) {
  console.error('Check command error:', error);
  return { output: 'Error processing flag submission' };
}
```

```

    }
};

const handleHintCommand = async (args, challenge, user) => {
    const hintLevel = args.length > 1 ? parseInt(args[1]) : 1;

    if (isNaN(hintLevel) || hintLevel < 1 || hintLevel > 3) {
        return { output: 'Usage: hint [level] - where level is 1, 2, or 3' };
    }

    try {
        // Check if hint exists
        const hint = await Hint.findOne({
            challenge: challenge._id,
            level: hintLevel,
            isActive: true
        });

        if (!hint) {
            return { output: `No hint available for level ${hintLevel}` };
        }

        // Check if user already purchased this hint
        const existingUserHint = await UserHint.findOne({
            user: user._id,
            hint: hint._id
        });

        if (existingUserHint) {

```

```
    return {  
      output: `Hint #${hintLevel} (Already purchased):\n${hint.content}`  
    };  
  }  
  
  // Check if user can afford the hint  
  if (user.score < hint.cost) {  
    return {  
      output: `Insufficient points! Hint #${hintLevel} costs ${hint.cost} points, but you only have  
${user.score}.`  
    };  
  }  
  
  // Purchase hint  
  const userHint = new UserHint({  
    user: user._id,  
    hint: hint._id,  
    challenge: challenge._id,  
    pointsDeducted: hint.cost  
  });  
  
  await userHint.save();  
  
  // Deduct points from user  
  user.score -= hint.cost;  
  await user.save();  
  
  return {
```



```
    output: `💡 Hint #${hintLevel} (Cost: ${hint.cost} points):\n${hint.content}\n\nRemaining points:
${user.score}`,
```

```
    pointsDeducted: hint.cost,
```

```
    newScore: user.score
```

```
  };
```

```
  } catch (error) {
```

```
    console.error('Hint command error:', error);
```

```
    return { output: 'Error retrieving hint' };
```

```
  }
```

```
};
```

```
const handleFindCommand = async (args, challenge) => {
```

```
  if (args.length < 2) {
```

```
    return { output: 'Usage: find <pattern>' };
```

```
  }
```

```
  const pattern = args.slice(1).join(' ');
```

```
  // Mock search results
```

```
  const searchResults = [
```

```
    `Searching for "${pattern}"...`,
```

```
    `Found 3 results:`,
```

```
    `- /challenges/${challenge._id}/readme.txt (line 5)`,
```

```
    `- /challenges/${challenge._id}/instructions.md (line 12)`,
```

```
    `- /challenges/${challenge._id}/hidden/secret.txt (line 1)`
```

```
  ];
```

```
  return { output: searchResults.join('\n') };
```

```
};
```

```
const handleDownloadCommand = async (args, challenge) => {
```

```
  if (args.length < 2) {
```

```
    return { output: 'Usage: download <filename>' };
```

```
  }
```

```
  const filename = args[1];
```

```
  // Check if file exists in attachments
```

```
  if (challenge.attachments && challenge.attachments.some(att => att.name === filename)) {
```

```
    return {
```

```
      output: `📄 Downloading ${filename}...`,
```

```
      download: {
```

```
        filename: filename,
```

```
        url: `/api/files/download/${challenge._id}/${filename}`,
```

```
        message: 'File ready for download'
```

```
      }
```

```
    };
```

```
  } else {
```

```
    return { output: `File not found or not available for download: ${filename}` };
```

```
  }
```

```
};
```

```
const handleStatusCommand = async (challenge, user) => {
```

```
  try {
```

```
    const submissions = await Submission.find({
```

```
      user: user._id,
```

```
      challenge: challenge._id
```

```

});

const correctSubmission = submissions.find(sub => sub.isCorrect);
const attemptCount = submissions.filter(sub => !sub.isCorrect).length;

const status = correctSubmission ? '✔ SOLVED' : '🔍 IN PROGRESS';
const pointsInfo = correctSubmission ?
  `Points earned: ${correctSubmission.pointsAwarded}` :
  `Potential points: ${challenge.points}`;

const output = [
  `Challenge: ${challenge.title}`,
  `Status: ${status}`,
  `Category: ${challenge.category}`,
  `Difficulty: ${challenge.difficulty}`,
  pointsInfo,
  `Your attempts: ${attemptCount}`,
  `Total solves: ${challenge.solveCount}`,
  correctSubmission ? `Solved at: ${correctSubmission.createdAt.toLocaleString()}` : 'Keep working
on it! 🚀'
];

return { output: output.join('\n') };
} catch (error) {
  console.error('Status command error:', error);
  return { output: 'Error retrieving status' };
}
};",

```

services/

AchievementService.js

```
"import Achievement from '../models/Achievement.js'
```

```
import Submission from '../models/Submission.js'
```

```
import User from '../models/User.js'
```

```
export class AchievementService {
```

```
  async checkFirstBlood(challengeId, userId) {
```

```
    const solveCount = await Submission.countDocuments({
```

```
      challenge: challengeId,
```

```
      isCorrect: true
```

```
    })
```

```
    if (solveCount === 1) {
```

```
      await Achievement.create({
```

```
        user: userId,
```

```
        type: 'first_blood',
```

```
        challenge: challengeId,
```

```
        metadata: { solveNumber: 1 }
```

```
      })
```

```
    // Notify via socket
```

```
    const socketService = await import('../socketService.js')
```

```
    socketService.default.sendNotification(userId, {
```

```
      type: 'achievement',
```

```
      title: 'First Blood!',
```

```
      message: 'You got the first solve on this challenge!',
```

```
      achievement: 'first_blood'
```

```
    })
```

```
}  
}
```

```
async checkCategoryMaster(userId, category) {  
  const categorySolves = await Submission.countDocuments({  
    user: userId,  
    isCorrect: true  
  }).populate('challenge')
```

```
// This is simplified - you'd need to check actual category solves
```

```
const categoryCount = await Submission.aggregate([  
  { $match: { user: userId, isCorrect: true } },  
  {  
    $lookup: {  
      from: 'challenges',  
      localField: 'challenge',  
      foreignField: '_id',  
      as: 'challenge'  
    }  
  },  
  { $unwind: '$challenge' },  
  { $match: { 'challenge.category': category } },  
  { $count: 'count' }  
])
```

```
const count = categoryCount[0]?.count || 0
```

```
if (count >= 5) {  
  const existingAchievement = await Achievement.findOne({
```

```
    user: userId,  
    type: 'category_master',  
    category: category  
  })
```

```
  if (!existingAchievement) {  
    await Achievement.create({  
      user: userId,  
      type: 'category_master',  
      category: category,  
      metadata: { count: count }  
    })  
  }  
}  
}
```

```
async checkPointMilestones(userId) {  
  const user = await User.findById(userId)  
  const milestones = [100, 500, 1000, 2500, 5000]
```

```
  for (const milestone of milestones) {  
    if (user.score >= milestone) {  
      const existing = await Achievement.findOne({  
        user: userId,  
        type: 'point_milestone',  
        points: milestone  
      })
```

```
      if (!existing) {
```

```
    await Achievement.create({
      user: userId,
      type: 'point_milestone',
      points: milestone,
      metadata: { milestone: milestone }
    })
  }
}
}
```

```
async awardAchievement(userId, achievementType, metadata = {}) {
  const achievement = new Achievement({
    user: userId,
    type: achievementType,
    metadata: metadata
  })

  await achievement.save()

  return achievement
}
```

```
async getUserAchievements(userId) {
  return Achievement.find({ user: userId })
    .populate('challenge', 'title category points')
    .sort({ awardedAt: -1 })
}
}
```

```
export default new AchievementService()",
```

BadgeService.js

```
"import Badge from '../models/Badge.js'
```

```
import User from '../models/User.js'
```

```
import Submission from '../models/Submission.js'
```

```
class BadgeService {
```

```
  constructor() {
```

```
    // Initialize badges once (non-blocking)
```

```
    this.initializeDefaultBadges()
```

```
    .then(() => console.log('🎉 Default badges ready'))
```

```
    .catch(err => console.error('❌ Badge init failed:', err.message))
```

```
  }
```

```
  // Create default badges if not present
```

```
  async initializeDefaultBadges() {
```

```
    const defaultBadges = [
```

```
      {
```

```
        name: 'First Blood',
```

```
        description: 'Get the first solve on a challenge',
```

```
        icon: '🩸',
```

```
        color: '#DC2626',
```

```
        criteria: { type: 'first_blood' },
```

```
        rarity: 'rare'
```

```
      },
```

```
      {
```

```
        name: 'Web Warrior',
```

```
        description: 'Solve 5 web challenges',
```



```
icon: '🌐',
color: '#3B82F6',
criteria: { type: 'category_mastery', category: 'web', threshold: 5 },
rarity: 'common'
},
{
  name: 'Crypto Expert',
  description: 'Solve 5 cryptography challenges',
  icon: '🔒',
  color: '#10B981',
  criteria: { type: 'category_mastery', category: 'crypto', threshold: 5 },
  rarity: 'common'
},
{
  name: 'Score Hunter',
  description: 'Reach 1000 points',
  icon: '🏆',
  color: '#F59E0B',
  criteria: { type: 'score_threshold', threshold: 1000 },
  rarity: 'common'
},
{
  name: 'CTF Legend',
  description: 'Reach 5000 points',
  icon: '🏆',
  color: '#8B5CF6',
  criteria: { type: 'score_threshold', threshold: 5000 },
  rarity: 'legendary'
}
```

```

    },
    {
      name: 'Team Player',
      description: 'Be part of a winning team',
      icon: '👥',
      color: '#6B7280',
      criteria: { type: 'team_work' },
      rarity: 'epic'
    }
  ]

```

```

for (const badgeData of defaultBadges) {
  await Badge.updateOne(
    { name: badgeData.name },
    { $setOnInsert: badgeData },
    { upsert: true }
  )
}

```

// Check and award eligible badges to a user

```

async checkAndAwardBadges(userId) {
  const user = await User.findById(userId).populate('solvedChallenges badges')
  const badges = await Badge.find({ isActive: true })

  if (!user) return []

  const awardedBadges = []

```

```

for (const badge of badges) {
  const meetsCriteria = await this.meetsBadgeCriteria(user, badge)
  const alreadyHas = user.badges.some(b => b._id.equals(badge._id))

  if (meetsCriteria && !alreadyHas) {
    user.badges.push(badge._id)
    awardedBadges.push(badge)
  }
}

if (awardedBadges.length > 0) {
  await user.save()

  // Notify user via socket
  const { default: socketService } = await import('./socketService.js')
  for (const badge of awardedBadges) {
    socketService.sendNotification(userId, {
      type: 'badge',
      title: 'New Badge Earned!',
      message: `You earned the ${badge.name} badge!`,
      badge
    })
  }
}

return awardedBadges
}

// Badge condition logic

```

```

async meetsBadgeCriteria(user, badge) {
  const { criteria } = badge

  switch (criteria.type) {
    case 'score_threshold':
      return user.score >= criteria.threshold

    case 'challenges_solved':
      return user.solvedChallenges.length >= criteria.threshold

    case 'category_mastery': {
      const count = user.solvedChallenges.filter(
        ch => ch.category?.toLowerCase() === criteria.category.toLowerCase()
      ).length
      return count >= criteria.threshold
    }

    case 'first_blood': {
      const { default: Achievement } = await import('../models/Achievement.js')
      return !! (await Achievement.findOne({
        user: user._id,
        type: 'first_blood'
      }))
    }

    case 'team_work': {
      const { default: Team } = await import('../models/Team.js')
      return !! (await Team.findOne({
        members: user._id,

```

```

        'eventWins.0': { $exists: true }
    )))
}

default:
    return false
}
}

// List badges owned by a user
async getUserBadges(userId) {
    const user = await User.findById(userId).populate('badges')
    return user?.badges || []
}

// Leaderboard of users with most badges
async getBadgeLeaderboard() {
    return User.aggregate([
        { $match: { role: 'participant' } },
        {
            $project: {
                username: 1,
                badgeCount: { $size: '$badges' },
                badges: 1,
                score: 1
            }
        },
        { $sort: { badgeCount: -1, score: -1 } },
        { $limit: 20 }
    ])
}

```

```
  ])  
}  
}
```

```
export default new BadgeService()
```

```
,
```

```
dockerservice.js
```

```
"import Docker from 'dockerode'
```

```
import { randomBytes } from 'crypto'
```

```
const docker = new Docker()
```

```
export class DockerChallengeManager {
```

```
  constructor() {
```

```
    this.activeContainers = new Map()
```

```
  }
```

```
  async spawnChallengeInstance(userId, challengeId, challengeConfig) {
```

```
    const containerName = `user-${userId}-challenge-${challengeId}-${randomBytes(8).toString('hex')}`
```

```
    try {
```

```
      const container = await docker.createContainer({
```

```
        Image: challengeConfig.image,
```

```
        name: containerName,
```

```
        HostConfig: {
```

```
          Memory: challengeConfig.memoryLimit || 256 * 1024 * 1024,
```

```
          MemorySwap: challengeConfig.memoryLimit || 256 * 1024 * 1024,
```

```
          CpuShares: 1024,
```

```
          PidsLimit: 100,
```

```

    NetworkMode: 'none', // Isolated network
    AutoRemove: true,
    ReadonlyRootfs: true // Read-only filesystem
  },
  Env: challengeConfig.environment || [],
  Cmd: challengeConfig.command || [],
  WorkingDir: challengeConfig.workingDir || '/app'
})

await container.start()

// Store container info
this.activeContainers.set(`${userId}-${challengeId}`, {
  containerId: container.id,
  name: containerName,
  startedAt: new Date()
})

// Get container info
const info = await container.inspect()

return {
  containerId: container.id,
  status: 'running',
  ports: challengeConfig.ports || [],
  expiresAt: new Date(Date.now() + (challengeConfig.timeout || 3600) * 1000)
}

} catch (error) {

```

```
    console.error('Failed to spawn container:', error)
    throw new Error('Failed to start challenge instance')
  }
}
```

```
async stopChallengeInstance(userId, challengeId) {
  const key = `${userId}-${challengeId}`
  const containerInfo = this.activeContainers.get(key)

  if (!containerInfo) return

  try {
    const container = docker.getContainer(containerInfo.containerId)
    await container.stop({ t: 10 }) // 10 second timeout
    await container.remove()

    this.activeContainers.delete(key)
  } catch (error) {
    console.error('Failed to stop container:', error)
  }
}
```

```
async cleanupExpiredContainers() {
  const now = new Date()

  for (const [key, containerInfo] of this.activeContainers.entries()) {
    if (containerInfo.expiresAt && containerInfo.expiresAt < now) {
      await this.stopChallengeInstance(
        key.split('-')[0],
```



```
        key.split('-')[1]
    )
}
}
```

```
async getContainerLogs(containerId, tail = 100) {
    try {
        const container = docker.getContainer(containerId)
        const logs = await container.logs({
            stdout: true,
            stderr: true,
            tail: tail,
            timestamps: true
        })

        return logs.toString()
    } catch (error) {
        console.error('Failed to get container logs:', error)
        return ''
    }
}
```

```
async getContainerStats(containerId) {
    try {
        const container = docker.getContainer(containerId)
        const stats = await container.stats({ stream: false })

        return {
```

```
    cpu: stats.cpu_stats,  
    memory: stats.memory_stats,  
    network: stats.networks  
  }  
} catch (error) {  
  console.error('Failed to get container stats:', error)  
  return null  
}  
}  
}
```

```
export default new DockerChallengeManager(),
```

```
services/
```

```
faceDetection.js
```

```
"import * as faceapi from 'face-api.js'
```

```
export class FaceDetectionService {
```

```
  constructor() {
```

```
    this.modelsLoaded = false
```

```
    this.detectionOptions = new faceapi.TinyFaceDetectorOptions({
```

```
      inputSize: 512,
```

```
      scoreThreshold: 0.5
```

```
    })
```

```
  }
```

```
  async loadModels() {
```

```
    if (this.modelsLoaded) return
```

```
try {  
  await faceapi.nets.tinyFaceDetector.loadFromUri('/models')  
  await faceapi.nets.faceLandmark68TinyNet.loadFromUri('/models')  
  await faceapi.nets.faceExpressionNet.loadFromUri('/models')  
  await faceapi.nets.faceRecognitionNet.loadFromUri('/models')  
  
  this.modelsLoaded = true  
  console.log('Face detection models loaded successfully')  
} catch (error) {  
  console.error('Error loading face detection models:', error)  
  throw error  
}  
}
```

```
async detectFaces(videoElement) {  
  if (!this.modelsLoaded) {  
    await this.loadModels()  
  }  

```

```
try {  
  const detections = await faceapi  
    .detectAllFaces(videoElement, this.detectionOptions)  
    .withFaceLandmarks(true)  
    .withFaceExpressions()  
  
  return this.analyzeDetections(detections)  
} catch (error) {  
  console.error('Error detecting faces:', error)  
  return { faces: [], alerts: [] }  
}
```

```
}  
}
```

```
analyzeDetections(detections) {
```

```
  const result = {  
    faces: detections.length,  
    alerts: [],  
    confidence: 0,  
    expressions: {}  
  }
```

```
  if (detections.length === 0) {
```

```
    result.alerts.push({  
      type: 'no_face',  
      confidence: 1.0,  
      message: 'No face detected in frame'  
    })
```

```
  } else if (detections.length > 1) {
```

```
    result.alerts.push({  
      type: 'multiple_faces',  
      confidence: 1.0,  
      message: `Multiple faces detected: ${detections.length}`  
    })  
  }
```

```
  // Analyze each face
```

```
  detections.forEach((detection, index) => {
```

```
    const { detection: face, expressions } = detection
```

```

result.confidence = Math.max(result.confidence, face.score)

// Check if face is properly centered and visible
if (face.box.width < 100 || face.box.height < 100) {
  result.alerts.push({
    type: 'face_too_small',
    confidence: 0.8,
    message: 'Face appears too small in frame'
  })
}

// Analyze facial expressions for suspicious activity
const suspiciousExpressions = this.analyzeExpressions(expressions)
if (suspiciousExpressions.length > 0) {
  result.alerts.push(...suspiciousExpressions)
}

// Store expression data
result.expressions[`face_${index}`] = expressions
})

return result
}

analyzeExpressions(expressions) {
  const alerts = []
  const threshold = 0.7

  if (expressions.surprised > threshold) {

```

```
alerts.push({
  type: 'suspicious_expression',
  confidence: expressions.surprised,
  message: 'Suspicious expression detected: surprise'
})
}
```

```
if (expressions.fearful > threshold) {
  alerts.push({
    type: 'suspicious_expression',
    confidence: expressions.fearful,
    message: 'Suspicious expression detected: fear'
  })
}
```

```
if (expressions.angry > threshold) {
  alerts.push({
    type: 'suspicious_expression',
    confidence: expressions.angry,
    message: 'Suspicious expression detected: anger'
  })
}
```

```
return alerts
}
```

```
async captureFrame(videoElement, quality = 0.7) {
  return new Promise((resolve) => {
    const canvas = document.createElement('canvas')
```

```
canvas.width = videoElement.videoWidth  
canvas.height = videoElement.videoHeight
```

```
const ctx = canvas.getContext('2d')  
ctx.drawImage(videoElement, 0, 0, canvas.width, canvas.height)
```

```
const imageData = canvas.toDataURL('image/jpeg', quality)  
resolve(imageData)  
})  
}
```

```
validateWebcamStream(stream) {  
  const track = stream.getVideoTracks()[0]  
  if (!track) {  
    throw new Error('No video track found in webcam stream')  
  }  
}
```

```
const settings = track.getSettings()
```

```
// Validate minimum requirements
```

```
if (settings.width < 640 || settings.height < 480) {  
  throw new Error('Webcam resolution too low. Minimum 640x480 required.')  
}
```

```
if (settings.frameRate < 15) {  
  throw new Error('Webcam frame rate too low. Minimum 15 FPS required.')  
}
```

```
return true
```

```
}  
}
```

```
export default new FaceDetectionService()",
```

scoringservice.js

```
"import Challenge from '../models/Challenge.js'
```

```
export class ScoringService {
```

```
  calculateDynamicPoints(challenge) {
```

```
    if (!challenge.dynamicScoring) {
```

```
      return challenge.points
```

```
    }
```

```
    const { solveCount, basePoints, minPoints, decay } = challenge
```

```
    const calculatedPoints = Math.max(
```

```
      minPoints,
```

```
      Math.floor(basePoints * Math.pow(decay, solveCount))
```

```
    )
```

```
    return calculatedPoints
```

```
  }
```

```
  async updateChallengePoints(challengeId) {
```

```
    const challenge = await Challenge.findById(challengeId)
```

```
    if (!challenge.dynamicScoring) return
```

```
    const newPoints = this.calculateDynamicPoints(challenge)
```



```

if (newPoints !== challenge.points) {
  await Challenge.findByIdAndUpdate(challengeId, {
    points: newPoints
  })

  // Notify about point change
  const socketService = await import('./socketService.js')
  socketService.default.io.emit('points_updated', {
    challengeId,
    newPoints,
    oldPoints: challenge.points
  })
}
}

async initializeDynamicScoring() {
  // Initialize dynamic scoring for existing challenges
  const challenges = await Challenge.find({ dynamicScoring: true })

  for (const challenge of challenges) {
    const solveCount = await this.getSolveCount(challenge._id)
    const newPoints = this.calculateDynamicPoints({
      ...challenge.toObject(),
      solveCount
    })

    if (newPoints !== challenge.points) {
      await Challenge.findByIdAndUpdate(challenge._id, { points: newPoints })
    }
  }
}

```

```
}  
}
```

```
async getSolveCount(challengeId) {  
  const Submission = await import('../models/Submission.js')  
  return Submission.default.countDocuments({  
    challenge: challengeId,  
    isCorrect: true  
  })  
}  
}
```

```
export default new ScoringService()",
```

```
securitymonitoring.js
```

```
"import TestSession from '../models/TestSession.js'  
import User from '../models/User.js'  
import Challenge from '../models/Challenge.js'  
import socketService from './socketService.js'
```

```
export class SecurityMonitor {  
  constructor() {  
    this.activeSessions = new Map()  
    this.violationHandlers = new Map()  
  }
```

```
// Initialize monitoring for a session
```

```
async initializeSession(sessionId, userId, challengeId, securityConfig) {  
  try {
```

```

const session = await TestSession.findOne({ sessionId })

if (!session) {
  throw new Error('Session not found')
}

// Store session in active sessions
this.activeSessions.set(sessionId, {
  session,
  startTime: new Date(),
  violations: 0,
  lastScreenshot: null,
  webcamStatus: 'initializing'
})

// Setup violation handlers
this.setupViolationHandlers(sessionId, securityConfig)

// Start monitoring intervals
this.startMonitoring(sessionId, securityConfig)

return session
} catch (error) {
  console.error('Error initializing security session:', error)
  throw error
}
}

setupViolationHandlers(sessionId, config) {
  this.violationHandlers.set(sessionId, {

```

```
handleWebcamViolation: async (violation) => {  
  const session = this.activeSessions.get(sessionId)  
  if (!session) return  
  
  await session.session.addWebcamAlert(  
    violation.type,  
    violation.confidence,  
    violation.screenshot  
  )  
  
  if (violation.type === 'multiple_faces' || violation.type === 'no_face') {  
    await this.handleCriticalViolation(sessionId, 'webcam', violation)  
  }  
},
```

```
handleTabSwitch: async (data) => {  
  const session = this.activeSessions.get(sessionId)  
  if (!session) return  
  
  session.session.monitoring.tabSwitches.push({  
    timestamp: new Date(),  
    count: data.count,  
    urls: data.urls || []  
  })  
  
  if (data.count > config.browserRestrictions.maxTabSwitches) {  
    await this.handleCriticalViolation(sessionId, 'tab_switch', data)  
  }  
}
```

```

    await session.session.save()
  },

  handleFocusLoss: async (duration) => {
    const session = this.activeSessions.get(sessionId)
    if (!session) return

    session.session.monitoring.focusEvents.push({
      timestamp: new Date(),
      type: 'blur',
      duration
    })

    if (duration > config.violationThresholds.maxFocusLoss) {
      await this.handleCriticalViolation(sessionId, 'focus_loss', { duration })
    }

    await session.session.save()
  }
})

}

async handleCriticalViolation(sessionId, violationType, data) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return

  const config = await this.getSecurityConfig()
  let action = 'warning'

```

```
switch (violationType) {  
    case 'webcam':  
        action = config.violationActions.webcamViolation  
        break  
    case 'tab_switch':  
        action = config.violationActions.tabSwitchViolation  
        break  
    case 'focus_loss':  
        action = config.violationActions.focusViolation  
        break  
}
```

```
await session.session.addViolation(  
    violationType,  
    'high',  
    `Automatic detection: ${violationType}`,  
    action  
)
```

```
// Notify via socket
```

```
socketService.sendNotification(session.session.user.toString(), {  
    type: 'security_violation',  
    title: 'Security Violation Detected',  
    message: `Violation: ${violationType}. Action: ${action}`,  
    severity: 'high'  
})
```

```
// Terminate session if required
```

```
if (action === 'terminate' || action === 'disqualify') {
```

```
    await this.terminateSession(sessionId, `Automatic termination due to ${violationType}`)
  }
}
```

```
startMonitoring(sessionId, config) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return

  // Webcam monitoring interval
  session.webcamInterval = setInterval(async () => {
    await this.checkWebcamStatus(sessionId)
  }, config.monitoring.webcamCheckInterval * 1000)
```

```
  // Screenshot interval
  session.screenshotInterval = setInterval(async () => {
    await this.captureScreenshot(sessionId)
  }, config.monitoring.screenshotInterval * 1000)
```

```
  // Auto-terminate timer
  if (config.sessionSettings.maxDuration) {
    session.autoTerminateTimer = setTimeout(async () => {
      await this.terminateSession(sessionId, 'Maximum duration reached')
    }, config.sessionSettings.maxDuration * 1000)
  }
}
```

```
async checkWebcamStatus(sessionId) {
  // This would integrate with the frontend webcam monitoring
  // For now, it's a placeholder for the actual implementation
```

```
const session = this.activeSessions.get(sessionId)

if (!session) return

// In a real implementation, this would receive status from the frontend
console.log(`Checking webcam status for session ${sessionId}`)
}

async captureScreenshot(sessionId) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return

  // This would be triggered by the frontend
  // The frontend would capture and send screenshots periodically
  console.log(`Capturing screenshot for session ${sessionId}`)
}

async terminateSession(sessionId, reason = 'Session terminated') {
  const session = this.activeSessions.get(sessionId)
  if (!session) return

  // Clear intervals
  if (session.webcamInterval) clearInterval(session.webcamInterval)
  if (session.screenshotInterval) clearInterval(session.screenshotInterval)
  if (session.autoTerminateTimer) clearTimeout(session.autoTerminateTimer)

  // Update session
  await session.session.terminateSession(reason)

  // Remove from active sessions
```



```
this.activeSessions.delete(sessionId)
this.violationHandlers.delete(sessionId)
```

```
// Notify user
```

```
socketService.sendNotification(session.session.user.toString(), {
  type: 'session_terminated',
  title: 'Session Terminated',
  message: reason,
  severity: 'critical'
})
```

```
return session.session
}
```

```
async getSessionStatus(sessionId) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return null

  return {
    sessionId,
    status: session.session.status,
    startTime: session.startTime,
    violations: session.violations,
    webcamStatus: session.webcamStatus,
    duration: Math.floor((new Date() - session.startTime) / 1000)
  }
}
```

```
async getSecurityConfig() {
```

```

const SecuritySettings = await import('../models/SecuritySettings.js')

let config = await SecuritySettings.default.findOne()

if (!config) {
  config = new SecuritySettings.default()
  await config.save()
}

return config
}

export default new SecurityMonitor()",

```

socketservice.js

```

"import { Server } from 'socket.io'
import jwt from 'jsonwebtoken'
import User from '../models/User.js'

```

```

class SocketService {
  constructor() {
    this.io = null
    this.connectedUsers = new Map()
  }

```

```

async initialize(server) {
  this.io = new Server(server, {
    cors: {
      origin: process.env.FRONTEND_URL || 'http://localhost:3000',

```

```

    methods: ['GET', 'POST']
  }
})

// Optional Redis adapter (only if available)
try {
  if (process.env.REDIS_ENABLED === 'true' && process.env.REDIS_URL) {
    try {
      const { createAdapter } = await import('@socket.io/redis-adapter')
      const Redis = (await import('ioredis')).default
      const pubClient = new Redis(process.env.REDIS_URL)
      const subClient = pubClient.duplicate()
      this.io.adapter(createAdapter(pubClient, subClient))
      console.log('✔ Socket.io Redis adapter connected')
    } catch (err) {
      console.warn('⚠ Redis connection failed:', err.message)
    }
  } else {
    console.log('⚠ Redis adapter disabled — using in-memory sockets only')
  }
} catch (err) {
  console.warn('⚠ Redis adapter not available:', err.message)
}

this.setupMiddleware()
this.setupEventHandlers()

```

```
    return this.io
  }
```

```
setupMiddleware() {
  this.io.use(async (socket, next) => {
    try {
      const token = socket.handshake.auth?.token
      if (!token) {
        return next(new Error('Authentication error: missing token'))
      }

      const decoded = jwt.verify(token, process.env.JWT_SECRET)
      const user = await User.findById(decoded.userId).select('-password')

      if (!user) {
        return next(new Error('Authentication error: user not found'))
      }

      socket.userId = user._id
      socket.user = user
      next()
    } catch (error) {
      console.error('Socket auth error:', error.message)
      next(new Error('Authentication error'))
    }
  })
}
```

```
setupEventHandlers() {
```

```

this.io.on('connection', (socket) => {

  const username = socket.user?.username || 'Unknown'

  console.log(`👤 User connected: ${username}`)

  // Save socket reference
  this.connectedUsers.set(socket.userId.toString(), socket)

  // Auto-join personal + general rooms
  socket.join(`user:${socket.userId}`)
  socket.join('leaderboard')
  socket.join('notifications')

  socket.on('join_challenge', (challengeId) => {
    socket.join(`challenge:${challengeId}`)
  })

  socket.on('leave_challenge', (challengeId) => {
    socket.leave(`challenge:${challengeId}`)
  })

  socket.on('disconnect', () => {
    this.connectedUsers.delete(socket.userId.toString())

    console.log(`👤 User disconnected: ${username}`)
  })
})

// === Broadcasts ===

```

```
async broadcastLeaderboardUpdate() {  
  const leaderboard = await this.getLeaderboardData()  
  this.io.to('leaderboard').emit('leaderboard_update', leaderboard)  
}
```

```
broadcastChallengeSolve(user, challenge, isFirstBlood = false) {  
  this.io.emit('challenge_solved', {  
    user: { username: user.username, id: user._id },  
    challenge: {  
      title: challenge.title,  
      category: challenge.category,  
      points: challenge.points  
    },  
    isFirstBlood,  
    timestamp: new Date()  
  })  
}
```

```
sendNotification(userId, notification) {  
  this.io.to(`user:${userId}`).emit('notification', notification)  
}
```

```
async getLeaderboardData() {  
  const users = await User.aggregate([  
    { $match: { role: 'participant', isActive: true } },  
    { $sort: { score: -1, lastSolve: 1 } },  
    { $limit: 50 },  
    { $project: { username: 1, score: 1, solvedChallenges: 1 } }  
  ])
```

```

    return users
  }
}

export default new SocketService()
",
validationService.js
"import axios from 'axios'
import { exec } from 'child_process'
import { promisify } from 'util'
import fs from 'fs/promises'
import path from 'path'

const execAsync = promisify(exec)

export class ValidationService {
  async validateWebChallenge(url, expectedOutput) {
    try {
      const response = await axios.get(url, {
        timeout: 10000,
        validateStatus: null // Don't throw on HTTP errors
      })

      // Check if expected output exists in response
      const isValid = response.data.includes(expectedOutput)

      return {
        isValid,
        statusCode: response.status,

```

```

    responseTime: response.headers['response-time'],
    data: isValid ? 'Output found' : 'Output not found'
  }
} catch (error) {
  return {
    isValid: false,
    error: error.message
  }
}
}
}

```

```

async validateBinaryChallenge(binaryPath, testInputs) {
  try {
    const results = []

    for (const test of testInputs) {
      const { stdout, stderr } = await execAsync(
        `${binaryPath} ${test.input}`,
        { timeout: 5000 }
      )

      const isValid = stdout.includes(test.expectedOutput)
      results.push({
        input: test.input,
        expected: test.expectedOutput,
        actual: stdout,
        isValid,
        error: stderr || null
      })
    }
  }
}

```



```

    }

    return {
      overall: results.every(r => r.isValid),
      results
    }
  } catch (error) {
    return {
      overall: false,
      error: error.message
    }
  }
}

```

```

async validateCryptoChallenge(encryptedFile, decryptionKey, expectedPattern) {
  try {
    // This is a simplified example - real implementation would depend on challenge
    const encryptedData = await fs.readFile(encryptedFile, 'utf8')

    // Simple XOR decryption example
    let decrypted = ''
    for (let i = 0; i < encryptedData.length; i++) {
      decrypted += String.fromCharCode(
        encryptedData.charCodeAt(i) ^ decryptionKey.charCodeAt(i % decryptionKey.length)
      )
    }

    const isValid = decrypted.includes(expectedPattern)
  }
}

```

```

return {
  isValid,
  decrypted: isValid ? decrypted : 'Decryption failed',
  length: decrypted.length
}
} catch (error) {
  return {
    isValid: false,
    error: error.message
  }
}
}

```

```

async validateFlagFormat(flag, challenge) {
  const flagRegex = challenge.flagFormat || /^CTF\{.+\\}$/

```

```

  if (!flagRegex.test(flag)) {
    return {
      isValid: false,
      error: 'Flag format is incorrect'
    }
  }

```

```

  return { isValid: true }
}

```

```

async runHealthCheck(challenge) {
  const healthChecks = {
    web: async () => this.validateWebChallenge(challenge.url, challenge.healthCheck),

```

```
    binary: async () => this.validateBinaryChallenge(challenge.binaryPath, challenge.testCases),  
    crypto: async () => this.validateCryptoChallenge(challenge.encryptedFile, challenge.key,  
challenge.expected)  
}
```

```
const check = healthChecks[challenge.type]
```

```
if (check) {  
    return await check()  
}
```

```
return { isValid: true, message: 'No health check configured' }
```

```
}  
}
```

```
export default new ValidationService()
```

```
services/
```

```
faceDetection.js
```

```
"import * as faceapi from 'face-api.js'
```

```
export class FaceDetectionService {
```

```
    constructor() {
```

```
        this.modelsLoaded = false
```

```
        this.detectionOptions = new faceapi.TinyFaceDetectorOptions({
```

```
            inputSize: 512,
```

```
            scoreThreshold: 0.5
```

```
        })
```

```
    }
```

```
async loadModels() {  
  if (this.modelsLoaded) return  
  
  try {  
    await faceapi.nets.tinyFaceDetector.loadFromUri('/models')  
    await faceapi.nets.faceLandmark68TinyNet.loadFromUri('/models')  
    await faceapi.nets.faceExpressionNet.loadFromUri('/models')  
    await faceapi.nets.faceRecognitionNet.loadFromUri('/models')  
  
    this.modelsLoaded = true  
    console.log('Face detection models loaded successfully')  
  } catch (error) {  
    console.error('Error loading face detection models:', error)  
    throw error  
  }  
}  
  
async detectFaces(videoElement) {  
  if (!this.modelsLoaded) {  
    await this.loadModels()  
  }  
  
  try {  
    const detections = await faceapi  
      .detectAllFaces(videoElement, this.detectionOptions)  
      .withFaceLandmarks(true)  
      .withFaceExpressions()  
  
    return this.analyzeDetections(detections)
```

```
    } catch (error) {  
      console.error('Error detecting faces:', error)  
      return { faces: [], alerts: [] }  
    }  
  }  
}
```

```
analyzeDetections(detections) {  
  const result = {  
    faces: detections.length,  
    alerts: [],  
    confidence: 0,  
    expressions: {}  
  }  
}
```

```
if (detections.length === 0) {  
  result.alerts.push({  
    type: 'no_face',  
    confidence: 1.0,  
    message: 'No face detected in frame'  
  })  
} else if (detections.length > 1) {  
  result.alerts.push({  
    type: 'multiple_faces',  
    confidence: 1.0,  
    message: `Multiple faces detected: ${detections.length}`  
  })  
}
```

```
// Analyze each face
```

```

detections.forEach((detection, index) => {
  const { detection: face, expressions } = detection

  result.confidence = Math.max(result.confidence, face.score)

  // Check if face is properly centered and visible
  if (face.box.width < 100 || face.box.height < 100) {
    result.alerts.push({
      type: 'face_too_small',
      confidence: 0.8,
      message: 'Face appears too small in frame'
    })
  }

  // Analyze facial expressions for suspicious activity
  const suspiciousExpressions = this.analyzeExpressions(expressions)
  if (suspiciousExpressions.length > 0) {
    result.alerts.push(...suspiciousExpressions)
  }

  // Store expression data
  result.expressions[`face_${index}`] = expressions
})

return result
}

analyzeExpressions(expressions) {
  const alerts = []

```

```
const threshold = 0.7
```

```
if (expressions.surprised > threshold) {  
  alerts.push({  
    type: 'suspicious_expression',  
    confidence: expressions.surprised,  
    message: 'Suspicious expression detected: surprise'  
  })  
}
```

```
if (expressions.fearful > threshold) {  
  alerts.push({  
    type: 'suspicious_expression',  
    confidence: expressions.fearful,  
    message: 'Suspicious expression detected: fear'  
  })  
}
```

```
if (expressions.angry > threshold) {  
  alerts.push({  
    type: 'suspicious_expression',  
    confidence: expressions.angry,  
    message: 'Suspicious expression detected: anger'  
  })  
}
```

```
return alerts  
}
```

```
async captureFrame(videoElement, quality = 0.7) {  
  return new Promise((resolve) => {  
    const canvas = document.createElement('canvas')  
    canvas.width = videoElement.videoWidth  
    canvas.height = videoElement.videoHeight  
  
    const ctx = canvas.getContext('2d')  
    ctx.drawImage(videoElement, 0, 0, canvas.width, canvas.height)  
  
    const imageData = canvas.toDataURL('image/jpeg', quality)  
    resolve(imageData)  
  })  
}
```

```
validateWebcamStream(stream) {  
  const track = stream.getVideoTracks()[0]  
  if (!track) {  
    throw new Error('No video track found in webcam stream')  
  }  
  
  const settings = track.getSettings()  
  
  // Validate minimum requirements  
  if (settings.width < 640 || settings.height < 480) {  
    throw new Error('Webcam resolution too low. Minimum 640x480 required.')  
  }  
  
  if (settings.frameRate < 15) {  
    throw new Error('Webcam frame rate too low. Minimum 15 FPS required.')  
  }  
}
```



```
    }  
  
    return true  
  }  
}
```

```
export default new FaceDetectionService()",
```

scoringservice.js

```
"import Challenge from '../models/Challenge.js'
```

```
export class ScoringService {  
  calculateDynamicPoints(challenge) {  
    if (!challenge.dynamicScoring) {  
      return challenge.points  
    }  
  
    const { solveCount, basePoints, minPoints, decay } = challenge  
    const calculatedPoints = Math.max(  
      minPoints,  
      Math.floor(basePoints * Math.pow(decay, solveCount))  
    )  
  
    return calculatedPoints  
  }  
}
```

```
async updateChallengePoints(challengeId) {  
  const challenge = await Challenge.findById(challengeId)  
  if (!challenge.dynamicScoring) return
```

```
const newPoints = this.calculateDynamicPoints(challenge)
```

```
if (newPoints !== challenge.points) {  
  await Challenge.findByIdAndUpdate(challengeId, {  
    points: newPoints  
  })  
}
```

```
// Notify about point change
```

```
const socketService = await import('./socketService.js')  
socketService.default.io.emit('points_updated', {  
  challengeId,  
  newPoints,  
  oldPoints: challenge.points  
})  
}  
}
```

```
async initializeDynamicScoring() {
```

```
  // Initialize dynamic scoring for existing challenges
```

```
  const challenges = await Challenge.find({ dynamicScoring: true })
```

```
  for (const challenge of challenges) {
```

```
    const solveCount = await this.getSolveCount(challenge._id)
```

```
    const newPoints = this.calculateDynamicPoints({  
      ...challenge.toObject(),  
      solveCount  
    })  
  }
```

```

    if (newPoints !== challenge.points) {
      await Challenge.findByIdAndUpdate(challenge._id, { points: newPoints })
    }
  }
}

```

```

async getSolveCount(challengeId) {
  const Submission = await import('../models/Submission.js')
  return Submission.default.countDocuments({
    challenge: challengeId,
    isCorrect: true
  })
}
}

```

```

export default new ScoringService()",

```

securitymonitoring.js

```

"import TestSession from '../models/TestSession.js'
import User from '../models/User.js'
import Challenge from '../models/Challenge.js'
import socketService from './socketService.js'

```

```

export class SecurityMonitor {
  constructor() {
    this.activeSessions = new Map()
    this.violationHandlers = new Map()
  }
}

```

```
// Initialize monitoring for a session
async initializeSession(sessionId, userId, challengeId, securityConfig) {
  try {
    const session = await TestSession.findOne({ sessionId })
    if (!session) {
      throw new Error('Session not found')
    }

    // Store session in active sessions
    this.activeSessions.set(sessionId, {
      session,
      startTime: new Date(),
      violations: 0,
      lastScreenshot: null,
      webcamStatus: 'initializing'
    })

    // Setup violation handlers
    this.setupViolationHandlers(sessionId, securityConfig)

    // Start monitoring intervals
    this.startMonitoring(sessionId, securityConfig)

    return session
  } catch (error) {
    console.error('Error initializing security session:', error)
    throw error
  }
}
```

```

setupViolationHandlers(sessionId, config) {
  this.violationHandlers.set(sessionId, {
    handleWebcamViolation: async (violation) => {
      const session = this.activeSessions.get(sessionId)
      if (!session) return

      await session.session.addWebcamAlert(
        violation.type,
        violation.confidence,
        violation.screenshot
      )

      if (violation.type === 'multiple_faces' || violation.type === 'no_face') {
        await this.handleCriticalViolation(sessionId, 'webcam', violation)
      }
    },

    handleTabSwitch: async (data) => {
      const session = this.activeSessions.get(sessionId)
      if (!session) return

      session.session.monitoring.tabSwitches.push({
        timestamp: new Date(),
        count: data.count,
        urls: data.urls || []
      })

      if (data.count > config.browserRestrictions.maxTabSwitches) {

```

```
    await this.handleCriticalViolation(sessionId, 'tab_switch', data)
  }
```

```
    await session.session.save()
  },
```

```
handleFocusLoss: async (duration) => {
  const session = this.activeSessions.get(sessionId)
  if (!session) return
```

```
    session.session.monitoring.focusEvents.push({
      timestamp: new Date(),
      type: 'blur',
      duration
    })
```

```
    if (duration > config.violationThresholds.maxFocusLoss) {
      await this.handleCriticalViolation(sessionId, 'focus_loss', { duration })
    }
  }
```

```
    await session.session.save()
  }
})
}
```

```
async handleCriticalViolation(sessionId, violationType, data) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return
```

```
const config = await this.getSecurityConfig()

let action = 'warning'

switch (violationType) {
  case 'webcam':
    action = config.violationActions.webcamViolation
    break
  case 'tab_switch':
    action = config.violationActions.tabSwitchViolation
    break
  case 'focus_loss':
    action = config.violationActions.focusViolation
    break
}

await session.session.addViolation(
  violationType,
  'high',
  `Automatic detection: ${violationType}`,
  action
)

// Notify via socket
socketService.sendNotification(session.session.user.toString(), {
  type: 'security_violation',
  title: 'Security Violation Detected',
  message: `Violation: ${violationType}. Action: ${action}`,
  severity: 'high'
})
```

```
// Terminate session if required
if (action === 'terminate' || action === 'disqualify') {
  await this.terminateSession(sessionId, `Automatic termination due to ${violationType}`)
}
}
```

```
startMonitoring(sessionId, config) {
  const session = this.activeSessions.get(sessionId)
  if (!session) return
```

```
// Webcam monitoring interval
session.webcamInterval = setInterval(async () => {
  await this.checkWebcamStatus(sessionId)
}, config.monitoring.webcamCheckInterval * 1000)
```

```
// Screenshot interval
session.screenshotInterval = setInterval(async () => {
  await this.captureScreenshot(sessionId)
}, config.monitoring.screenshotInterval * 1000)
```

```
// Auto-terminate timer
if (config.sessionSettings.maxDuration) {
  session.autoTerminateTimer = setTimeout(async () => {
    await this.terminateSession(sessionId, 'Maximum duration reached')
  }, config.sessionSettings.maxDuration * 1000)
}
}
```



```
async checkWebcamStatus(sessionId) {  
  // This would integrate with the frontend webcam monitoring  
  // For now, it's a placeholder for the actual implementation  
  const session = this.activeSessions.get(sessionId)  
  if (!session) return  
  
  // In a real implementation, this would receive status from the frontend  
  console.log(`Checking webcam status for session ${sessionId}`)  
}  
  
async captureScreenshot(sessionId) {  
  const session = this.activeSessions.get(sessionId)  
  if (!session) return  
  
  // This would be triggered by the frontend  
  // The frontend would capture and send screenshots periodically  
  console.log(`Capturing screenshot for session ${sessionId}`)  
}  
  
async terminateSession(sessionId, reason = 'Session terminated') {  
  const session = this.activeSessions.get(sessionId)  
  if (!session) return  
  
  // Clear intervals  
  if (session.webcamInterval) clearInterval(session.webcamInterval)  
  if (session.screenshotInterval) clearInterval(session.screenshotInterval)  
  if (session.autoTerminateTimer) clearTimeout(session.autoTerminateTimer)  
  
  // Update session
```

```
await session.session.terminateSession(reason)
```

```
// Remove from active sessions
```

```
this.activeSessions.delete(sessionId)
```

```
this.violationHandlers.delete(sessionId)
```

```
// Notify user
```

```
socketService.sendNotification(session.session.user.toString(), {
```

```
  type: 'session_terminated',
```

```
  title: 'Session Terminated',
```

```
  message: reason,
```

```
  severity: 'critical'
```

```
})
```

```
return session.session
```

```
}
```

```
async getSessionStatus(sessionId) {
```

```
  const session = this.activeSessions.get(sessionId)
```

```
  if (!session) return null
```

```
  return {
```

```
    sessionId,
```

```
    status: session.session.status,
```

```
    startTime: session.startTime,
```

```
    violations: session.violations,
```

```
    webcamStatus: session.webcamStatus,
```

```
    duration: Math.floor((new Date() - session.startTime) / 1000)
```

```
}
```

```
}
```

```
async getSecurityConfig() {  
  const SecuritySettings = await import('../models/SecuritySettings.js')  
  let config = await SecuritySettings.default.findOne()  
  
  if (!config) {  
    config = new SecuritySettings.default()  
    await config.save()  
  }  
  
  return config  
}  
}
```

```
export default new SecurityMonitor()",
```

```
socketservice.js
```

```
"import { Server } from 'socket.io'  
import jwt from 'jsonwebtoken'  
import User from '../models/User.js'
```

```
class SocketService {  
  constructor() {  
    this.io = null  
    this.connectedUsers = new Map()  
  }
```

```
  async initialize(server) {
```

```

this.io = new Server(server, {
  cors: {
    origin: process.env.FRONTEND_URL || 'http://localhost:3000',
    methods: ['GET', 'POST']
  }
})

// Optional Redis adapter (only if available)
try {
  if (process.env.REDIS_ENABLED === 'true' && process.env.REDIS_URL) {
    try {
      const { createAdapter } = await import('@socket.io/redis-adapter')
      const Redis = (await import('ioredis')).default
      const pubClient = new Redis(process.env.REDIS_URL)
      const subClient = pubClient.duplicate()
      this.io.adapter(createAdapter(pubClient, subClient))
      console.log('✔ Socket.io Redis adapter connected')
    } catch (err) {
      console.warn('⚠ Redis connection failed:', err.message)
    }
  } else {
    console.log('⚠ Redis adapter disabled — using in-memory sockets only')
  }
} catch (err) {
  console.warn('⚠ Redis adapter not available:', err.message)
}

```

```
this.setupMiddleware()  
this.setupEventHandlers()
```

```
return this.io  
}
```

```
setupMiddleware() {  
  this.io.use(async (socket, next) => {  
    try {  
      const token = socket.handshake.auth?.token  
      if (!token) {  
        return next(new Error('Authentication error: missing token'))  
      }  
  
      const decoded = jwt.verify(token, process.env.JWT_SECRET)  
      const user = await User.findById(decoded.userId).select('-password')  
  
      if (!user) {  
        return next(new Error('Authentication error: user not found'))  
      }  
  
      socket.userId = user._id  
      socket.user = user  
      next()  
    } catch (error) {  
      console.error('Socket auth error:', error.message)  
      next(new Error('Authentication error'))  
    }  
  })  
}
```

```
}
```

```
setupEventHandlers() {  
  this.io.on('connection', (socket) => {  
    const username = socket.user?.username || 'Unknown'  
    console.log(`👤 User connected: ${username}`)  
  
    // Save socket reference  
    this.connectedUsers.set(socket.userId.toString(), socket)  
  
    // Auto-join personal + general rooms  
    socket.join(`user:${socket.userId}`)  
    socket.join('leaderboard')  
    socket.join('notifications')  
  
    socket.on('join_challenge', (challengeId) => {  
      socket.join(`challenge:${challengeId}`)  
    })  
  
    socket.on('leave_challenge', (challengeId) => {  
      socket.leave(`challenge:${challengeId}`)  
    })  
  
    socket.on('disconnect', () => {  
      this.connectedUsers.delete(socket.userId.toString())  
      console.log(`👋 User disconnected: ${username}`)  
    })  
  })  
}
```

```
// === Broadcasts ===
```

```
async broadcastLeaderboardUpdate() {  
  const leaderboard = await this.getLeaderboardData()  
  this.io.to('leaderboard').emit('leaderboard_update', leaderboard)  
}
```

```
broadcastChallengeSolve(user, challenge, isFirstBlood = false) {  
  this.io.emit('challenge_solved', {  
    user: { username: user.username, id: user._id },  
    challenge: {  
      title: challenge.title,  
      category: challenge.category,  
      points: challenge.points  
    },  
    isFirstBlood,  
    timestamp: new Date()  
  })  
}
```

```
sendNotification(userId, notification) {  
  this.io.to(`user:${userId}`).emit('notification', notification)  
}
```

```
async getLeaderboardData() {  
  const users = await User.aggregate([  
    { $match: { role: 'participant', isActive: true } },  
    { $sort: { score: -1, lastSolve: 1 } },
```

```
    { $limit: 50 },  
    { $project: { username: 1, score: 1, solvedChallenges: 1 } }  
  ])  
  return users  
}  
}
```

```
export default new SocketService()
```

```
",
```

```
validationService.js
```

```
"import axios from 'axios'
```

```
import { exec } from 'child_process'
```

```
import { promisify } from 'util'
```

```
import fs from 'fs/promises'
```

```
import path from 'path'
```

```
const execAsync = promisify(exec)
```

```
export class ValidationService {
```

```
  async validateWebChallenge(url, expectedOutput) {
```

```
    try {
```

```
      const response = await axios.get(url, {
```

```
        timeout: 10000,
```

```
        validateStatus: null // Don't throw on HTTP errors
```

```
      })
```

```
      // Check if expected output exists in response
```

```
      const isValid = response.data.includes(expectedOutput)
```



```

return {
  isValid,
  statusCode: response.status,
  responseTime: response.headers['response-time'],
  data: isValid ? 'Output found' : 'Output not found'
}
} catch (error) {
  return {
    isValid: false,
    error: error.message
  }
}
}

```

```

async validateBinaryChallenge(binaryPath, testInputs) {
  try {
    const results = []

    for (const test of testInputs) {
      const { stdout, stderr } = await execAsync(
        `"${binaryPath}" ${test.input}`,
        { timeout: 5000 }
      )

      const isValid = stdout.includes(test.expectedOutput)
      results.push({
        input: test.input,
        expected: test.expectedOutput,
        actual: stdout,

```

```

        isValid,
        error: stderr || null
    })
}

return {
    overall: results.every(r => r.isValid),
    results
}
} catch (error) {
    return {
        overall: false,
        error: error.message
    }
}
}

```

```

async validateCryptoChallenge(encryptedFile, decryptionKey, expectedPattern) {
    try {
        // This is a simplified example - real implementation would depend on challenge
        const encryptedData = await fs.readFile(encryptedFile, 'utf8')

        // Simple XOR decryption example
        let decrypted = ''
        for (let i = 0; i < encryptedData.length; i++) {
            decrypted += String.fromCharCode(
                encryptedData.charCodeAt(i) ^ decryptionKey.charCodeAt(i % decryptionKey.length)
            )
        }
    }
}

```

```

const isValid = decrypted.includes(expectedPattern)

return {
  isValid,
  decrypted: isValid ? decrypted : 'Decryption failed',
  length: decrypted.length
}
} catch (error) {
  return {
    isValid: false,
    error: error.message
  }
}
}

async validateFlagFormat(flag, challenge) {
  const flagRegex = challenge.flagFormat || /^CTF\{.+\\}$/

  if (!flagRegex.test(flag)) {
    return {
      isValid: false,
      error: 'Flag format is incorrect'
    }
  }

  return { isValid: true }
}

```

```

async runHealthCheck(challenge) {
  const healthChecks = {
    web: async () => this.validateWebChallenge(challenge.url, challenge.healthCheck),
    binary: async () => this.validateBinaryChallenge(challenge.binaryPath, challenge.testCases),
    crypto: async () => this.validateCryptoChallenge(challenge.encryptedFile, challenge.key,
challenge.expected)
  }

  const check = healthChecks[challenge.type]
  if (check) {
    return await check()
  }

  return { isValid: true, message: 'No health check configured' }
}
}

```

```

export default new ValidationService()"

```

models/

achievements.js

```

"import mongoose from 'mongoose'

```

```

const achievementSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
},

```

```
type: {
  type: String,
  required: true,
  enum: [
    'first_blood',
    'solver',
    'category_master',
    'speed_demon',
    'persistence',
    'streak',
    'point_milestone',
    'team_player'
  ],
},
challenge: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Challenge'
},
category: String,
points: Number,
metadata: mongoose.Schema.Types.Mixed,
awardedAt: {
  type: Date,
  default: Date.now
}
}, {
  timestamps: true
})
```

```
// Index for faster queries
achievementSchema.index({ user: 1, type: 1 })
achievementSchema.index({ awardedAt: -1 })

export default mongoose.model('Achievement', achievementSchema),
```

Badge.js

```
"import mongoose from 'mongoose'

const badgeSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    unique: true
  },
  description: {
    type: String,
    required: true
  },
  icon: {
    type: String,
    required: true
  },
  color: {
    type: String,
    default: '#6B7280'
  },
  criteria: {
    type: {
```

```

    type: String,
    required: true,
    enum: [
      'score_threshold',
      'challenges_solved',
      'category_mastery',
      'streak',
      'first_blood',
      'team_work'
    ],
  },
  threshold: Number,
  category: String,
  days: Number
},
rarity: {
  type: String,
  enum: ['common', 'rare', 'epic', 'legendary'],
  default: 'common'
},
isActive: {
  type: Boolean,
  default: true
}
}, {
  timestamps: true
})

export default mongoose.model('Badge', badgeSchema)",

```

challenge.js

```
"import mongoose from 'mongoose'
```

```
const challengeSchema = new mongoose.Schema({  
  title: {  
    type: String,  
    required: true,  
    trim: true,  
    maxlength: 100  
  },  
  description: {  
    type: String,  
    required: true  
  },  
  category: {  
    type: String,  
    required: true,  
    enum: ['web', 'crypto', 'forensics', 'pwn', 'misc', 'reverse']  
  },  
  difficulty: {  
    type: String,  
    required: true,  
    enum: ['easy', 'medium', 'hard']  
  },  
  points: {  
    type: Number,  
    required: true,  
    min: 1,
```



```
    max: 1000
  },
  flag: {
    type: String,
    required: true
  },
  hint: {
    type: String,
    default: ""
  },
  isActive: {
    type: Boolean,
    default: true
  },
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  // New fields for enhanced features
  dynamicScoring: {
    type: Boolean,
    default: false
  },
  basePoints: {
    type: Number,
    default: function() { return this.points }
  },
  minPoints: {
```

```
    type: Number,
    default: function() { return Math.floor(this.points * 0.3) }
  },
  solveCount: {
    type: Number,
    default: 0
  },
  decay: {
    type: Number,
    default: 0.95
  },
  tags: [String],
  attachments: [{
    name: String,
    url: String,
    size: Number
  }],
  dockerImage: String,
  port: Number,
  healthCheck: {
    type: String,
    default: ""
  },
  validationType: {
    type: String,
    enum: ['static', 'dynamic', 'manual'],
    default: 'static'
  },
  flagFormat: {
```

```

    type: String,
    default: 'CTF\\{.*\\}'
  },
  maxAttempts: {
    type: Number,
    default: 0 // 0 = unlimited
  },
  timeLimit: {
    type: Number,
    default: 0 // in minutes, 0 = no limit
  },
  requiresAttachment: {
    type: Boolean,
    default: false
  },
  metadata: mongoose.Schema.Types.Mixed
}, {
  timestamps: true
})

// Virtual for dynamic points calculation
challengeSchema.virtual('currentPoints').get(function() {
  if (!this.dynamicScoring) return this.points

  return Math.max(
    this.minPoints,
    Math.floor(this.basePoints * Math.pow(this.decay, this.solveCount))
  )
})

```

```

// Indexes for better query performance
challengeSchema.index({ category: 1, difficulty: 1 })
challengeSchema.index({ isActive: 1 })
challengeSchema.index({ createdBy: 1 })
challengeSchema.index({ tags: 1 })
challengeSchema.index({ points: -1 })
challengeSchema.index({ solveCount: -1 })

// Update points when solve count changes
challengeSchema.pre('save', function(next) {
  if (this.dynamicScoring && this.isModified('solveCount')) {
    this.points = this.currentPoints
  }
  next()
})

export default mongoose.model('Challenge', challengeSchema)",

```

Event.js

```

"import mongoose from 'mongoose'

const eventSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  description: {

```

```
    type: String,
    required: true
  },
  startTime: {
    type: Date,
    required: true
  },
  endTime: {
    type: Date,
    required: true
  },
  challenges: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Challenge'
  }],
  isActive: {
    type: Boolean,
    default: false
  },
  maxTeamSize: {
    type: Number,
    default: 4
  },
  rules: mongoose.Schema.Types.Mixed,
  participants: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    },
  }],
```

```

    team: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Team'
    },
    joinedAt: {
      type: Date,
      default: Date.now
    }
  }},
  banner: String,
  isPublic: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true
})

```

```

// Index for active events query

```

```

eventSchema.index({ startTime: 1, endTime: 1 })

```

```

eventSchema.index({ isActive: 1 })

```

```

export default mongoose.model('Event', eventSchema)",

```

```

file.js

```

```

"import mongoose from 'mongoose'

```

```

const fileSchema = new mongoose.Schema({

```

```

  filename: {

```

```
    type: String,
    required: true
  },
  originalName: {
    type: String,
    required: true
  },
  path: {
    type: String,
    required: true
  },
  size: {
    type: Number,
    required: true
  },
  mimetype: {
    type: String,
    required: true
  },
  challenge: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Challenge',
    required: true
  },
  uploadedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
}
```

```
isPublic: {
  type: Boolean,
  default: true
},
downloadCount: {
  type: Number,
  default: 0
}
}, {
  timestamps: true
})
```

```
// Index for challenge files
```

```
fileSchema.index({ challenge: 1, createdAt: -1 })
```

```
fileSchema.index({ uploadedBy: 1 })
```

```
export default mongoose.model('File', fileSchema)",
```

```
HInt.js
```

```
"import mongoose from 'mongoose'
```

```
const hintSchema = new mongoose.Schema({
```

```
  challenge: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'Challenge',
```

```
    required: true
```

```
  },
```

```
  level: {
```

```
    type: Number,
```



```
    required: true,
    min: 1,
    max: 3
  },
  content: {
    type: String,
    required: true
  },
  cost: {
    type: Number,
    required: true,
    min: 0
  },
  cooldown: {
    type: Number,
    default: 300 // 5 minutes in seconds
  },
  isActive: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true
})

// Prevent duplicate hint levels per challenge
hintSchema.index({ challenge: 1, level: 1 }, { unique: true })

export default mongoose.model('Hint', hintSchema),
```

Securitysettings.js

```
"import mongoose from 'mongoose'
```

```
const securitySettingsSchema = new mongoose.Schema({
```

```
  // Webcam monitoring settings
```

```
  webcamMonitoring: {
```

```
    enabled: { type: Boolean, default: true },
```

```
    required: { type: Boolean, default: true },
```

```
    quality: { type: String, enum: ['low', 'medium', 'high'], default: 'medium' },
```

```
    frameRate: { type: Number, default: 1 }, // frames per second
```

```
    detection: {
```

```
      faceDetection: { type: Boolean, default: true },
```

```
      multipleFaces: { type: Boolean, default: true },
```

```
      faceAwayDetection: { type: Boolean, default: true },
```

```
      noFaceTimeout: { type: Number, default: 10 }, // seconds
```

```
      confidenceThreshold: { type: Number, default: 0.7 }
```

```
    }
```

```
  },
```

```
  // Browser restrictions
```

```
  browserRestrictions: {
```

```
    disableDevTools: { type: Boolean, default: true },
```

```
    disableClipboard: { type: Boolean, default: true },
```

```
    disableRightClick: { type: Boolean, default: true },
```

```
    disablePrintScreen: { type: Boolean, default: true },
```

```
    restrictTabSwitching: { type: Boolean, default: true },
```

```
    maxTabSwitches: { type: Number, default: 3 },
```

```
    fullScreenRequired: { type: Boolean, default: true }
```

```
  },
```

```

// Monitoring frequency
monitoring: {
  screenshotInterval: { type: Number, default: 30 }, // seconds
  focusCheckInterval: { type: Number, default: 5 }, // seconds
  webcamCheckInterval: { type: Number, default: 10 } // seconds
},
// Violation thresholds
violationThresholds: {
  maxWebcamAlerts: { type: Number, default: 5 },
  maxTabSwitches: { type: Number, default: 5 },
  maxFocusLoss: { type: Number, default: 60 }, // seconds
  autoTerminate: { type: Boolean, default: true }
},
// Actions on violation
violationActions: {
  webcamViolation: { type: String, enum: ['warning', 'terminate', 'disqualify'], default: 'terminate' },
  tabSwitchViolation: { type: String, enum: ['warning', 'terminate', 'disqualify'], default: 'terminate' },
  focusViolation: { type: String, enum: ['warning', 'terminate', 'disqualify'], default: 'warning' },
  devToolsViolation: { type: String, enum: ['warning', 'terminate', 'disqualify'], default: 'terminate' }
},
// Session settings
sessionSettings: {
  maxDuration: { type: Number, default: 3600 }, // seconds
  autoStart: { type: Boolean, default: true },
  requireVerification: { type: Boolean, default: true }
}
}, {
  timestamps: true
})

```

```
export default mongoose.model('SecuritySettings', securitySettingsSchema),
```

submission.js

```
"import mongoose from 'mongoose'
```

```
const submissionSchema = new mongoose.Schema({
```

```
  user: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'User',
```

```
    required: true
```

```
  },
```

```
  challenge: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'Challenge',
```

```
    required: true
```

```
  },
```

```
  flagSubmitted: {
```

```
    type: String,
```

```
    required: true
```

```
  },
```

```
  isCorrect: {
```

```
    type: Boolean,
```

```
    required: true
```

```
  },
```

```
  ip: String,
```

```
  userAgent: String,
```

```
  // New fields
```

```
  pointsAwarded: {
```

```

    type: Number,
    default: 0
  },
  solveOrder: {
    type: Number,
    default: 0
  },
  timeSpent: {
    type: Number,
    default: 0
  }, // in seconds
  team: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Team'
  },
  metadata: {
    dockerContainerId: String,
    filesSubmitted: [String],
    validationLog: String
  }
}, {
  timestamps: true
})

// Prevent duplicate correct submissions
submissionSchema.index({ user: 1, challenge: 1, isCorrect: 1 }, {
  unique: true,
  partialFilterExpression: { isCorrect: true }
})

```

```

// Indexes for better performance
submissionSchema.index({ user: 1, createdAt: -1 })
submissionSchema.index({ challenge: 1, isCorrect: 1 })
submissionSchema.index({ createdAt: -1 })
submissionSchema.index({ solveOrder: 1 })
submissionSchema.index({ team: 1 })

// Update solve order for first blood tracking
submissionSchema.pre('save', async function(next) {
  if (this.isCorrect && !this.solveOrder) {
    const solveCount = await mongoose.model('Submission').countDocuments({
      challenge: this.challenge,
      isCorrect: true
    })
    this.solveOrder = solveCount + 1
  }
  next()
})

export default mongoose.model('Submission', submissionSchema)

```

team.js

```

"import mongoose from 'mongoose'

const teamSchema = new mongoose.Schema({
  name: {
    type: String,

```

```
    required: true,
    unique: true,
    trim: true,
    maxlength: 50
  },
  description: {
    type: String,
    maxlength: 200
  },
  members: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }],
  captain: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  score: {
    type: Number,
    default: 0
  },
  solvedChallenges: [{
    challenge: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Challenge'
    },
  }],
  solvedAt: {
    type: Date,
```

```

    default: Date.now
  },
  solvedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }
}],
avatar: String,
isActive: {
  type: Boolean,
  default: true
}
}, {
  timestamps: true
})

// Prevent duplicate members
teamSchema.index({ name: 1 }, { unique: true })
teamSchema.index({ members: 1 })
teamSchema.index({ score: -1 })

export default mongoose.model('Team', teamSchema)",

```

Testsession.js

```
"import mongoose from 'mongoose'
```

```

const testSessionSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,

```



```
    ref: 'User',
    required: true
  },
  challenge: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Challenge',
    required: true
  },
  event: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event'
  },
  sessionId: {
    type: String,
    required: true,
    unique: true
  },
  status: {
    type: String,
    enum: ['active', 'completed', 'terminated', 'disqualified'],
    default: 'active'
  },
  // Security monitoring
  security: {
    webcamEnabled: { type: Boolean, default: false },
    screenRecording: { type: Boolean, default: false },
    tabFocus: { type: Boolean, default: true },
    clipboardDisabled: { type: Boolean, default: true },
    devToolsDisabled: { type: Boolean, default: true }
```

```
},
// Monitoring data
monitoring: {
  webcamAlerts: [{
    timestamp: Date,
    type: String, // 'multiple_faces', 'no_face', 'face_away', 'suspicious_activity'
    confidence: Number,
    screenshot: String // Base64 encoded
  }],
  focusEvents: [{
    timestamp: Date,
    type: String, // 'blur', 'focus'
    duration: Number
  }],
  tabSwitches: [{
    timestamp: Date,
    count: Number,
    urls: [String]
  }],
  clipboardEvents: [{
    timestamp: Date,
    action: String // 'copy', 'paste', 'cut'
  }],
  consoleEvents: [{
    timestamp: Date,
    command: String
  }]
},
// Violation tracking
```

```
violations: [{
  timestamp: Date,
  type: String,
  severity: { type: String, enum: ['low', 'medium', 'high', 'critical'] },
  description: String,
  action: { type: String, enum: ['warning', 'terminated', 'disqualified'] }
}],
// Session controls
startTime: {
  type: Date,
  default: Date.now
},
endTime: Date,
duration: Number, // in seconds
autoTerminateAt: Date,
// Screenshot evidence
screenshots: [{
  timestamp: Date,
  image: String, // Base64 encoded
  alertType: String,
  confidence: Number
}],
// System info
userAgent: String,
ipAddress: String,
screenResolution: String,
browserInfo: mongoose.Schema.Types.Mixed
}, {
  timestamps: true
```

```
}}
```

```
// Index for active sessions
```

```
testSessionSchema.index({ user: 1, status: 1 })
```

```
testSessionSchema.index({ sessionId: 1 })
```

```
testSessionSchema.index({ autoTerminateAt: 1 }, { expireAfterSeconds: 0 })
```

```
// Methods
```

```
testSessionSchema.methods.addViolation = function(type, severity, description, action = 'warning') {
```

```
  this.violations.push({
```

```
    timestamp: new Date(),
```

```
    type,
```

```
    severity,
```

```
    description,
```

```
    action
```

```
  })
```

```
  if (action === 'terminated' || action === 'disqualified') {
```

```
    this.status = action
```

```
    this.endTime = new Date()
```

```
  }
```

```
  return this.save()
```

```
}
```

```
testSessionSchema.methods.addWebcamAlert = function(alertType, confidence, screenshot = null) {
```

```
  this.monitoring.webcamAlerts.push({
```

```
    timestamp: new Date(),
```

```
    type: alertType,
```

```

    confidence,
    screenshot
  })

  return this.save()
}

testSessionSchema.methods.terminateSession = function(reason = 'Security violation') {
  this.status = 'terminated'
  this.endTime = new Date()
  this.violations.push({
    timestamp: new Date(),
    type: 'manual_termination',
    severity: 'high',
    description: reason,
    action: 'terminated'
  })

  return this.save()
}

export default mongoose.model('TestSession', testSessionSchema)",

user.js

"import mongoose from 'mongoose'
import bcrypt from 'bcryptjs'

const userSchema = new mongoose.Schema({
  username: {

```

```
    type: String,
    required: true,
    unique: true, // creates index automatically
    trim: true,
    minlength: 3,
    maxlength: 30
  },
  email: {
    type: String,
    required: true,
    unique: true, // creates index automatically
    trim: true,
    lowercase: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  fullName: {
    type: String,
    required: true,
    trim: true
  },
  role: {
    type: String,
    enum: ['participant', 'admin'],
    default: 'participant'
  },
}
```

```
isActive: {
  type: Boolean,
  default: true
},
lastLogin: {
  type: Date,
  default: Date.now
},
solvedChallenges: [{
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Challenge'
}],
score: {
  type: Number,
  default: 0
},
badges: [{
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Badge'
}],
team: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'Team'
},
preferences: {
  emailNotifications: { type: Boolean, default: true },
  showHints: { type: Boolean, default: true },
  theme: { type: String, default: 'dark' }
},
```

```
statistics: {
  totalAttempts: { type: Number, default: 0 },
  correctSubmissions: { type: Number, default: 0 },
  averageTime: { type: Number, default: 0 },
  favoriteCategory: String
},
lastSolve: Date,
streak: {
  current: { type: Number, default: 0 },
  longest: { type: Number, default: 0 },
  lastSolveDate: Date
}
}, {
  timestamps: true
})
```

```
// Hash password before saving
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next()
  try {
    const salt = await bcrypt.genSalt(12)
    this.password = await bcrypt.hash(this.password, salt)
    next()
  } catch (error) {
    next(error)
  }
})
```

```
// Compare password method
```



```
userSchema.methods.comparePassword = async function (candidatePassword) {  
  return await bcrypt.compare(candidatePassword, this.password)  
}
```

```
// Update streak method
```

```
userSchema.methods.updateStreak = function () {  
  const today = new Date().toDateString()  
  const lastSolve = this.streak.lastSolveDate?.toDateString()
```

```
  if (lastSolve === today) return
```

```
  const yesterday = new Date()  
  yesterday.setDate(yesterday.getDate() - 1)
```

```
  if (lastSolve === yesterday.toDateString()) {  
    this.streak.current += 1  
  } else {  
    this.streak.current = 1  
  }
```

```
  if (this.streak.current > this.streak.longest) {  
    this.streak.longest = this.streak.current  
  }
```

```
  this.streak.lastSolveDate = new Date()  
  this.lastSolve = new Date()  
}
```

```
// Remove password from JSON output
```

```
userSchema.methods.toJSON = function () {  
  const user = this.toObject()  
  delete user.password  
  return user  
}
```

```
// ✔ Keep only the performance indexes (no duplicates)
```

```
userSchema.index({ score: -1 })
```

```
userSchema.index({ 'statistics.correctSubmissions': -1 })
```

```
userSchema.index({ lastSolve: -1 })
```

```
export default mongoose.model('User', userSchema)
```

```
",
```

```
userchallengeprogress.js
```

```
"import mongoose from 'mongoose'
```

```
const userChallengeProgressSchema = new mongoose.Schema({
```

```
  user: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'User',
```

```
    required: true
```

```
  },
```

```
  challenge: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'Challenge',
```

```
    required: true
```

```
  },
```

```
  attempts: {
```

```

    type: Number,
    default: 0
  },
  lastAttempt: Date,
  hintsUsed: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Hint'
  }],
  timeSpent: {
    type: Number,
    default: 0
  }, // in seconds
  startedAt: {
    type: Date,
    default: Date.now
  },
  completed: {
    type: Boolean,
    default: false
  },
  completedAt: Date
}, {
  timestamps: true
})

// Unique progress per user per challenge
userChallengeProgressSchema.index({ user: 1, challenge: 1 }, { unique: true })
userChallengeProgressSchema.index({ completed: 1 })

```

```
export default mongoose.model('UserChallengeProgress', userChallengeProgressSchema)",
```

Userhint.js

```
"import mongoose from 'mongoose'
```

```
const userHintSchema = new mongoose.Schema({
```

```
  user: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'User',
```

```
    required: true
```

```
  },
```

```
  hint: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'Hint',
```

```
    required: true
```

```
  },
```

```
  challenge: {
```

```
    type: mongoose.Schema.Types.ObjectId,
```

```
    ref: 'Challenge',
```

```
    required: true
```

```
  },
```

```
  usedAt: {
```

```
    type: Date,
```

```
    default: Date.now
```

```
  },
```

```
  pointsDeducted: {
```

```
    type: Number,
```

```
    required: true
```

```
  }
```

```
}, {  
  timestamps: true  
})  
  
// Prevent duplicate hint usage  
userHintSchema.index({ user: 1, hint: 1 }, { unique: true })  
userHintSchema.index({ usedAt: -1 })  
  
export default mongoose.model('UserHint', userHintSchema),
```

writeup.js

```
"import mongoose from 'mongoose'
```

```
const writeupSchema = new mongoose.Schema({  
  challenge: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'Challenge',  
    required: true  
  },  
  user: {  
    type: mongoose.Schema.Types.ObjectId,  
    ref: 'User',  
    required: true  
  },  
  title: {  
    type: String,  
    required: true,  
    trim: true  
  },  
},
```

```
content: {
  type: String,
  required: true
},
isPublic: {
  type: Boolean,
  default: false
},
rating: {
  type: Number,
  default: 0
},
votes: [{
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  },
  type: {
    type: String,
    enum: ['up', 'down']
  },
  votedAt: {
    type: Date,
    default: Date.now
  }
}],
views: {
  type: Number,
  default: 0
}
```

```
},  
  tags: [String]  
}, {  
  timestamps: true  
})
```

```
// Index for popular writeups
```

```
writeupSchema.index({ challenge: 1, rating: -1 })
```

```
writeupSchema.index({ isPublic: 1, createdAt: -1 })
```

```
writeupSchema.index({ tags: 1 })
```

```
export default mongoose.model('Writeup', writeupSchema)"
```

```
routes/
```

```
achievement.js
```

```
"import express from 'express'
```

```
import Achievement from '../models/Achievement.js'
```

```
import User from '../models/User.js'
```

```
import { authMiddleware } from '../middleware/auth.js'
```

```
import achievementService from '../services/achievementService.js'
```

```
const router = express.Router()
```

```
// Get user's achievements
```

```
router.get('/my', authMiddleware, async (req, res) => {
```

```
  try {
```

```
    const achievements = await Achievement.find({ user: req.userId })
```

```
    .populate('challenge', 'title category points')
```

```
    .sort({ awardedAt: -1 })
```

```
// Group by type for better organization
const groupedAchievements = achievements.reduce((acc, achievement) => {
  const type = achievement.type
  if (!acc[type]) {
    acc[type] = []
  }
  acc[type].push(achievement)
  return acc
}, {})
```

```
// Calculate statistics
const stats = {
  total: achievements.length,
  byType: Object.keys(groupedAchievements).reduce((acc, type) => {
    acc[type] = groupedAchievements[type].length
    return acc
  }, {}),
  recent: achievements.slice(0, 5)
}
```

```
res.json({
  achievements: groupedAchievements,
  statistics: stats
})
} catch (error) {
  console.error('Get achievements error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
```



```
}}
```

```
// Get achievement leaderboard
```

```
router.get('/leaderboard', async (req, res) => {
```

```
  try {
```

```
    const { type, limit = 20 } = req.query
```

```
    const matchStage = {}
```

```
    if (type) {
```

```
      matchStage.type = type
```

```
    }
```

```
    const achievementLeaderboard = await Achievement.aggregate([
```

```
      { $match: matchStage },
```

```
      {
```

```
        $group: {
```

```
          _id: '$user',
```

```
          count: { $sum: 1 },
```

```
          recentAchievement: { $max: '$awardedAt' },
```

```
          achievements: { $push: '$$ROOT' } 
```

```
        }
```

```
      },
```

```
      {
```

```
        $lookup: {
```

```
          from: 'users',
```

```
          localField: '_id',
```

```
          foreignField: '_id',
```

```
          as: 'user'
```

```
        }
```

```

    },
    { $unwind: '$user' },
    {
      $project: {
        'user.password': 0,
        'user.email': 0
      }
    },
    {
      $sort: {
        count: -1,
        recentAchievement: -1
      }
    },
    { $limit: parseInt(limit) }
  ])

```

```

const leaderboardWithRanks = achievementLeaderboard.map((item, index) => ({
  rank: index + 1,
  user: {
    id: item.user._id,
    username: item.user.username,
    fullName: item.user.fullName,
    score: item.user.score
  },
  achievementCount: item.count,
  recentAchievement: item.recentAchievement,
  uniqueTypes: [...new Set(item.achievements.map(a => a.type))].length
}))

```

```
res.json({ leaderboard: leaderboardWithRanks })  
} catch (error) {  
  console.error('Achievement leaderboard error:', error)  
  res.status(500).json({ error: 'Internal server error' })  
}  
})
```

// Get specific achievement types

```
router.get('/types', async (req, res) => {  
  try {  
    const achievementTypes = await Achievement.aggregate([  
      {  
        $group: {  
          _id: '$type',  
          count: { $sum: 1 },  
          description: { $first: '$metadata.description' }  
        }  
      },  
      {  
        $project: {  
          type: '$_id',  
          count: 1,  
          description: 1,  
          _id: 0  
        }  
      },  
      { $sort: { count: -1 } }  
    ])
```

```
// Add human-readable names and descriptions
const typeInfo = {
  first_blood: {
    name: 'First Blood',
    description: 'First to solve a challenge',
    icon: '🩸',
    rarity: 'rare'
  },
  category_master: {
    name: 'Category Master',
    description: 'Master a specific challenge category',
    icon: '🏆',
    rarity: 'common'
  },
  speed_demon: {
    name: 'Speed Demon',
    description: 'Solve challenges quickly',
    icon: '⚡',
    rarity: 'rare'
  },
  point_milestone: {
    name: 'Point Milestone',
    description: 'Reach point milestones',
    icon: '📍',
    rarity: 'common'
  },
  team_player: {
    name: 'Team Player',
```

```

    description: 'Excel in team challenges',
    icon: '👥',
    rarity: 'epic'
  },
  persistence: {
    name: 'Persistence',
    description: 'Keep solving challenges consistently',
    icon: '🔥',
    rarity: 'common'
  },
  streak: {
    name: 'Streak Master',
    description: 'Maintain long solving streaks',
    icon: '💧',
    rarity: 'rare'
  },
  solver: {
    name: 'Solver',
    description: 'Solve multiple challenges',
    icon: '✅',
    rarity: 'common'
  }
}

```

```

const enrichedTypes = achievementTypes.map(type => ({
  ...type,
  ...typeInfo[type.type]
}))

```

```
res.json({ achievementTypes: enrichedTypes })
} catch (error) {
  console.error('Get achievement types error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})
```

```
// Get recent achievements (global feed)
```

```
router.get('/recent', async (req, res) => {
```

```
  try {
```

```
    const { limit = 50 } = req.query
```

```
    const recentAchievements = await Achievement.find()
```

```
      .populate('user', 'username')
```

```
      .populate('challenge', 'title category')
```

```
      .sort({ awardedAt: -1 })
```

```
      .limit(parseInt(limit))
```

```
    const formattedAchievements = recentAchievements.map(achievement => {
```

```
      let message = ''
```

```
      const username = achievement.user.username
```

```
      switch (achievement.type) {
```

```
        case 'first_blood':
```

```
          message = `🩸 ${username} got First Blood on ${achievement.challenge.title}!`
```

```
          break
```

```
        case 'category_master':
```

```
          message = `🏆 ${username} mastered ${achievement.category} category!`
```

```

        break
    case 'point_milestone':
        message = `💰 ${username} reached ${achievement.points} points!`
        break
    case 'streak':
        message = `🔥 ${username} is on a ${achievement.metadata?.streakLength || 0} day streak!`
        break
    default:
        message = `🏆 ${username} earned an achievement!`
}

return {
    id: achievement._id,
    message,
    type: achievement.type,
    user: achievement.user.username,
    challenge: achievement.challenge?.title,
    category: achievement.category,
    points: achievement.points,
    timestamp: achievement.awardedAt,
    metadata: achievement.metadata
}
})

res.json({ achievements: formattedAchievements })
} catch (error) {
    console.error('Get recent achievements error:', error)
    res.status(500).json({ error: 'Internal server error' })
}

```

```
}}
```

```
// Check for new achievements (trigger manual check)
```

```
router.post('/check', authMiddleware, async (req, res) => {
```

```
  try {
```

```
    const userId = req.userId
```

```
    // Trigger achievement checks
```

```
    await achievementService.checkPointMilestones(userId)
```

```
    // Get updated achievements
```

```
    const achievements = await Achievement.find({ user: userId })
```

```
      .sort({ awardedAt: -1 })
```

```
      .limit(5)
```

```
    const newAchievements = achievements.filter(ach =>
```

```
      ach.awardedAt > new Date(Date.now() - 5 * 60 * 1000) // Last 5 minutes
```

```
    )
```

```
    res.json({
```

```
      checked: true,
```

```
      newAchievements: newAchievements.length,
```

```
      recentAchievements: newAchievements
```

```
    })
```

```
  } catch (error) {
```

```
    console.error('Check achievements error:', error)
```

```
    res.status(500).json({ error: 'Internal server error' })
```

```
  }
```

```
}}
```



```

// Get achievement statistics for user
router.get('/stats', authMiddleware, async (req, res) => {
  try {
    const stats = await Achievement.aggregate([
      { $match: { user: req.userId } },
      {
        $group: {
          _id: null,
          totalAchievements: { $sum: 1 },
          uniqueTypes: { $addToSet: '$type' },
          firstAchievement: { $min: '$awardedAt' },
          recentAchievement: { $max: '$awardedAt' },
          byCategory: {
            $push: {
              category: '$category',
              type: '$type'
            }
          }
        },
      },
      {
        $project: {
          totalAchievements: 1,
          uniqueTypes: { $size: '$uniqueTypes' },
          firstAchievement: 1,
          recentAchievement: 1,
          categoryBreakdown: {
            $arrayToObject: {

```

```

$map: {
  input: ['web', 'crypto', 'forensics', 'pwn', 'reverse', 'misc'],
  as: 'cat',
  in: {
    k: '$$cat',
    v: {
      count: {
        $size: {
          $filter: {
            input: '$byCategory',
            as: 'item',
            cond: { $eq: ['$item.category', '$$cat'] }
          }
        }
      }
    }
  }
}
})

```

```

const defaultStats = {
  totalAchievements: 0,
  uniqueTypes: 0,
  firstAchievement: null,
  recentAchievement: null,

```

```
categoryBreakdown: {  
  web: { count: 0 },  
  crypto: { count: 0 },  
  forensics: { count: 0 },  
  pwn: { count: 0 },  
  reverse: { count: 0 },  
  misc: { count: 0 }  
}  
}
```

```
res.json({ stats: stats[0] || defaultStats })  
} catch (error) {  
  console.error('Get achievement stats error:', error)  
  res.status(500).json({ error: 'Internal server error' })  
}  
})
```

```
export default router",
```

```
admin.js
```

```
"import express from 'express'  
import { body, validationResult } from 'express-validator'  
import Challenge from '../models/Challenge.js'  
import User from '../models/User.js'  
import Submission from '../models/Submission.js'  
import Team from '../models/Team.js'  
import { authMiddleware, adminMiddleware } from '../middleware/auth.js'  
import backupManager from '../utils/backupManager.js'  
import dockerService from '../services/dockerService.js'
```

```

import validationService from '../services/validationService.js'

const router = express.Router()

router.use(authMiddleware, adminMiddleware)

// Create challenge with enhanced features
router.post('/challenges', [
  body('title').notEmpty().trim().isLength({ max: 100 }),
  body('description').notEmpty().trim(),
  body('category').isIn(['web', 'crypto', 'forensics', 'pwn', 'misc', 'reverse']),
  body('difficulty').isIn(['easy', 'medium', 'hard']),
  body('points').isInt({ min: 1, max: 1000 }),
  body('flag').notEmpty().trim(),
  body('hint').optional().trim(),
  body('tags').optional().isArray(),
  body('dynamicScoring').optional().isBoolean(),
  body('basePoints').optional().isInt({ min: 1 }),
  body('minPoints').optional().isInt({ min: 1 }),
  body('dockerImage').optional().trim(),
  body('validationType').optional().isIn(['static', 'dynamic', 'manual']),
  body('maxAttempts').optional().isInt({ min: 0 }),
  body('timeLimit').optional().isInt({ min: 0 })
], async (req, res) => {
  try {
    const errors = validationResult(req)

    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }
  }

```

```
const {  
  title,  
  description,  
  category,  
  difficulty,  
  points,  
  flag,  
  hint,  
  tags,  
  dynamicScoring,  
  basePoints,  
  minPoints,  
  dockerImage,  
  validationType,  
  maxAttempts,  
  timeLimit  
} = req.body
```

```
const challenge = new Challenge({  
  title,  
  description,  
  category,  
  difficulty,  
  points: parseInt(points),  
  flag,  
  hint: hint || "",  
  tags: tags || [],  
  dynamicScoring: dynamicScoring || false,
```

```
basePoints: basePoints || points,
minPoints: minPoints || Math.floor(points * 0.3),
dockerImage: dockerImage || '',
validationType: validationType || 'static',
maxAttempts: maxAttempts || 0,
timeLimit: timeLimit || 0,
createdBy: req.userId
})
```

```
await challenge.save()
```

```
// Validate challenge if it has dynamic components
```

```
if (dockerImage) {
  try {
    await validationService.runHealthCheck(challenge)
  } catch (validationError) {
    console.warn(`Challenge validation warning: ${validationError.message}`)
  }
}
```

```
res.status(201).json({
  message: 'Challenge created successfully',
  challenge: {
    id: challenge._id,
    title: challenge.title,
    category: challenge.category,
    difficulty: challenge.difficulty,
    points: challenge.points,
    dynamicScoring: challenge.dynamicScoring,
```

```

    dockerImage: challenge.dockerImage
  }
})

} catch (error) {
  console.error('Create challenge error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Update challenge
router.put('/challenges/:challengeId', [
  body('title').optional().trim().isLength({ max: 100 }),
  body('description').optional().trim(),
  body('category').optional().isIn(['web', 'crypto', 'forensics', 'pwn', 'misc', 'reverse']),
  body('difficulty').optional().isIn(['easy', 'medium', 'hard']),
  body('points').optional().isInt({ min: 1, max: 1000 }),
  body('flag').optional().trim(),
  body('isActive').optional().isBoolean(),
  body('tags').optional().isArray()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const challenge = await Challenge.findById(req.params.challengeId)
    if (!challenge) {

```

```

    return res.status(404).json({ error: 'Challenge not found' })
  }

  const updateFields = { ...req.body }

  // Remove fields that shouldn't be updated directly
  delete updateFields.createdBy
  delete updateFields.solveCount

  const updatedChallenge = await Challenge.findByIdAndUpdate(
    req.params.challenged,
    { $set: updateFields },
    { new: true, runValidators: true }
  )

  res.json({
    message: 'Challenge updated successfully',
    challenge: updatedChallenge
  })

} catch (error) {
  console.error('Update challenge error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Get admin statistics with enhanced analytics
router.get('/statistics', async (req, res) => {
  try {

```



```

const [
  totalUsers,
  totalChallenges,
  totalSubmissions,
  correctSubmissions,
  totalTeams,
  activeUsers
] = await Promise.all([
  User.countDocuments(),
  Challenge.countDocuments(),
  Submission.countDocuments(),
  Submission.countDocuments({ isCorrect: true }),
  Team.countDocuments(),
  User.countDocuments({
    lastLogin: { $gte: new Date(Date.now() - 24 * 60 * 60 * 1000) }
  })
])

```

// Category statistics

```

const categoryStats = {}
const challengesByCategory = await Challenge.aggregate([
  { $group: { _id: '$category', total: { $sum: 1 } } }
])

```

```

const solvedByCategory = await Submission.aggregate([
  { $match: { isCorrect: true } },
  {
    $lookup: {
      from: 'challenges',

```

```

    localField: 'challenge',
    foreignField: '_id',
    as: 'challenge'
  }
},
{ $unwind: '$challenge' },
{ $group: { _id: '$challenge.category', solved: { $sum: 1 } } }
])

```

```

challengesByCategory.forEach(cat => {
  categoryStats[cat._id] = { total: cat.total, solved: 0 }
})

```

```

solvedByCategory.forEach(cat => {
  if (categoryStats[cat._id]) {
    categoryStats[cat._id].solved = cat.solved
  }
})

```

```

// Difficulty statistics
const difficultyStats = await Challenge.aggregate([
  {
    $group: {
      _id: '$difficulty',
      total: { $sum: 1 },
      totalPoints: { $sum: '$points' }
    }
  }
])

```

```
// User activity statistics

const userStats = await User.aggregate([

  {
    $match: { role: 'participant' }
  },

  {
    $group: {
      _id: null,
      avgScore: { $avg: '$score' },
      maxScore: { $max: '$score' },
      totalSolves: { $sum: { $size: '$solvedChallenges' } },
      avgSolves: { $avg: { $size: '$solvedChallenges' } }
    }
  }
])
```

```
// Recent activity

const recentSubmissions = await Submission.find()

  .populate('user', 'username')

  .populate('challenge', 'title category')

  .sort({ createdAt: -1 })

  .limit(10)
```

```
res.json({

  overview: {

    totalUsers,

    totalChallenges,

    totalSubmissions,
```

```

    correctSubmissions,
    totalTeams,
    activeUsers,
    successRate: totalSubmissions > 0 ? (correctSubmissions / totalSubmissions * 100).toFixed(2) : 0
  },
  categoryStats,
  difficultyStats: difficultyStats.reduce((acc, curr) => {
    acc[curr._id] = { total: curr.total, totalPoints: curr.totalPoints }
    return acc
  }, {}),
  userStats: userStats[0] || {},
  recentActivity: recentSubmissions.map(sub => ({
    user: sub.user.username,
    challenge: sub.challenge.title,
    category: sub.challenge.category,
    correct: sub.isCorrect,
    timestamp: sub.createdAt
  })))
})

} catch (error) {
  console.error('Get statistics error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Get detailed challenge analytics
router.get('/analytics/challenges', async (req, res) => {
  try {

```

```
const challengeStats = await Challenge.aggregate([
  {
    $lookup: {
      from: 'submissions',
      localField: '_id',
      foreignField: 'challenge',
      as: 'submissions'
    }
  },
  {
    $project: {
      title: 1,
      category: 1,
      difficulty: 1,
      points: 1,
      solveCount: 1,
      totalAttempts: { $size: '$submissions' },
      correctSubmissions: {
        $size: {
          $filter: {
            input: '$submissions',
            as: 'sub',
            cond: { $eq: ['$sub.isCorrect', true] }
          }
        }
      },
    },
    successRate: {
      $cond: {
        if: { $eq: [{ $size: '$submissions' }, 0] },
```

```

then: 0,
else: {
  $multiply: [
    {
      $divide: [
        { $size: { $filter: { input: '$submissions', as: 'sub', cond: { $eq: ['$sub.isCorrect', true] } } } },
        { $size: '$submissions' }
      ]
    },
    100
  ]
}
},
firstBlood: {
  $arrayElemAt: [
    {
      $filter: {
        input: '$submissions',
        as: 'sub',
        cond: { $and: [{ $eq: ['$sub.isCorrect', true] }, { $eq: ['$sub.solveOrder', 1] } ] }
      }
    },
    0
  ]
}
},
{

```

```

    $lookup: {
      from: 'users',
      localField: 'firstBlood.user',
      foreignField: '_id',
      as: 'firstBloodUser'
    },
    {
      $addFields: {
        firstBloodUser: { $arrayElemAt: ['$firstBloodUser.username', 0] },
        firstBloodTime: '$firstBlood.createdAt'
      }
    },
    {
      $project: {
        firstBlood: 0
      }
    },
    {
      $sort: { solveCount: -1 }
    }
  ])

```

```

    res.json(challengeStats)
  } catch (error) {
    console.error('Challenge analytics error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

```

```
// User management endpoints

router.get('/users', async (req, res) => {

  try {

    const { page = 1, limit = 20, search = '' } = req.query


    const filter = {

      role: 'participant'

    }


    if (search) {

      filter.$or = [

        { username: { $regex: search, $options: 'i' } },

        { email: { $regex: search, $options: 'i' } },

        { fullName: { $regex: search, $options: 'i' } }

      ]

    }

  }


  const users = await User.find(filter)

    .select('-password')

    .populate('team', 'name')

    .sort({ score: -1, createdAt: -1 })

    .limit(limit * 1)

    .skip((page - 1) * limit)


  const total = await User.countDocuments(filter)


  res.json({

    users,
```



```

    totalPages: Math.ceil(total / limit),
    currentPage: page,
    total
  })
} catch (error) {
  console.error('Get users error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Toggle user active status
router.patch('/users/:userId/status', [
  body('isActive').isBoolean()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const { isActive } = req.body
    const user = await User.findByIdAndUpdate(
      req.params.userId,
      { isActive },
      { new: true }
    ).select('-password')

    if (!user) {
      return res.status(404).json({ error: 'User not found' })
    }
  }
})

```

```
}

res.json({
  message: `User ${isActive ? 'activated' : 'deactivated'} successfully`,
  user
})
} catch (error) {
  console.error('Toggle user status error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})
```

```
// Backup management
router.post('/backup', async (req, res) => {
  try {
    const backupPath = await backupManager.createBackup()

    res.json({
      message: 'Backup created successfully',
      path: backupPath,
      timestamp: new Date().toISOString()
    })
  } catch (error) {
    console.error('Backup creation error:', error)
    res.status(500).json({ error: 'Backup failed' })
  }
})
```

```
router.get('/backups', async (req, res) => {
```

```
try {  
  const backups = await backupManager.listBackups()  
  res.json(backups)  
} catch (error) {  
  console.error('List backups error:', error)  
  res.status(500).json({ error: 'Failed to list backups' })  
}  
})
```

// System health check

```
router.get('/system/health', async (req, res) => {
```

```
  try {  
    const health = {  
      database: 'Connected',  
      redis: 'Unknown',  
      docker: 'Unknown',  
      memory: process.memoryUsage(),  
      uptime: process.uptime(),  
      timestamp: new Date().toISOString()  
    }  
  }
```

// Check Docker service

```
  try {  
    // Simple Docker check - you might want to implement actual container checks  
    health.docker = 'Available'  
  } catch {  
    health.docker = 'Unavailable'  
  }
```

```

    res.json(health)
  } catch (error) {
    console.error('System health check error:', error)
    res.status(500).json({ error: 'Health check failed' })
  }
})

// Reset challenge (clear all submissions)
router.post('/challenges/:challengeId/reset', async (req, res) => {
  try {
    const challengeId = req.params.challengeId

    // Clear all submissions for this challenge
    await Submission.deleteMany({ challenge: challengeId })

    // Reset challenge solve count
    await Challenge.findByIdAndUpdate(challengeId, {
      solveCount: 0,
      points: { $ifNull: ['$basePoints', '$points'] }
    })

    // Remove from users' solved challenges and adjust scores
    const challenge = await Challenge.findById(challengeId)
    if (challenge) {
      await User.updateMany(
        { solvedChallenges: challengeId },
        {
          $pull: { solvedChallenges: challengeId },
          $inc: { score: -challenge.points }
        }
      )
    }
  } catch (error) {
    console.error('Error resetting challenge:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

```

```
    }  
  )  
}
```

```
    res.json({ message: 'Challenge reset successfully' })  
  } catch (error) {  
    console.error('Reset challenge error:', error)  
    res.status(500).json({ error: 'Failed to reset challenge' })  
  }  
})
```

```
export default router",
```

```
routes/
```

```
analytics.js
```

```
"import express from 'express'
```

```
import Challenge from '../models/Challenge.js'
```

```
import User from '../models/User.js'
```

```
import Submission from '../models/Submission.js'
```

```
import Team from '../models/Team.js'
```

```
import Achievement from '../models/Achievement.js'
```

```
import { authMiddleware, adminMiddleware } from '../middleware/auth.js'
```

```
const router = express.Router()
```

```
router.use(authMiddleware, adminMiddleware)
```

```
// Get comprehensive platform analytics
```

```
router.get('/overview', async (req, res) => {
```

```
try {  
  const [  
    userStats,  
    challengeStats,  
    submissionStats,  
    teamStats,  
    achievementStats,  
    recentActivity  
  ] = await Promise.all([  
    getUserStatistics(),  
    getChallengeStatistics(),  
    getSubmissionStatistics(),  
    getTeamStatistics(),  
    getAchievementStatistics(),  
    getRecentActivity()  
  ])  
  
  res.json({  
    overview: {  
      timestamp: new Date().toISOString(),  
      ...userStats,  
      ...challengeStats,  
      ...submissionStats,  
      ...teamStats,  
      ...achievementStats  
    },  
    recentActivity  
  })  
} catch (error) {
```

```
    console.error('Get analytics overview error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})
```

```
// User analytics
```

```
router.get('/users', async (req, res) => {
  try {
    const { period = 'all' } = req.query

    const userAnalytics = await User.aggregate([
      {
        $match: getPeriodFilter(period, 'createdAt')
      },
      {
        $facet: {
          // Registration trends
          registrationTrend: [
            {
              $group: {
                _id: {
                  $dateToString: { format: '%Y-%m-%d', date: '$createdAt' }
                },
              },
              count: { $sum: 1 }
            }
          ],
          { $sort: { _id: 1 } }
        ],
        // Score distribution
```

```

scoreDistribution: [
  {
    $bucket: {
      groupBy: '$score',
      boundaries: [0, 100, 500, 1000, 2500, 5000, 10000],
      default: '10000+',
      output: {
        count: { $sum: 1 },
        avgScore: { $avg: '$score' }
      }
    }
  }
],
// Activity levels
activityLevels: [
  {
    $bucket: {
      groupBy: {
        $cond: {
          if: { $gte: ['$lastLogin', new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)] },
          then: 'active',
          else: {
            $cond: {
              if: { $gte: ['$lastLogin', new Date(Date.now() - 30 * 24 * 60 * 60 * 1000)] },
              then: 'inactive',
              else: 'dormant'
            }
          }
        }
      }
    }
  }
]

```



```
    },
    boundaries: ['active', 'inactive', 'dormant'],
    default: 'unknown',
    output: {
      count: { $sum: 1 }
    }
  }
}
],
// Category preferences
categoryPreferences: [
  {
    $lookup: {
      from: 'submissions',
      localField: '_id',
      foreignField: 'user',
      as: 'submissions'
    }
  },
  {
    $unwind: {
      path: '$submissions',
      preserveNullAndEmptyArrays: true
    }
  },
  {
    $match: {
      'submissions.isCorrect': true
    }
  }
]
```

```

    },
    {
      $lookup: {
        from: 'challenges',
        localField: 'submissions.challenge',
        foreignField: '_id',
        as: 'challenge'
      }
    },
    { $unwind: '$challenge' },
    {
      $group: {
        _id: '$challenge.category',
        userCount: { $addToSet: '$_id' },
        solveCount: { $sum: 1 }
      }
    },
    {
      {
        $project: {
          category: '$_id',
          userCount: { $size: '$userCount' },
          solveCount: 1,
          _id: 0
        }
      },
      { $sort: { solveCount: -1 } }
    ]
  }
}

```

```
])
```

```
    res.json({ userAnalytics: userAnalytics[0] })  
  } catch (error) {  
    console.error('Get user analytics error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```
// Challenge analytics
```

```
router.get('/challenges', async (req, res) => {  
  try {  
    const challengeAnalytics = await Challenge.aggregate([  
      {  
        $lookup: {  
          from: 'submissions',  
          localField: '_id',  
          foreignField: 'challenge',  
          as: 'submissions'  
        }  
      },  
      {  
        $facet: {  
          // Difficulty analysis  
          difficultyAnalysis: [  
            {  
              $group: {  
                _id: '$difficulty',  
                count: { $sum: 1 },  
              },  
            },  
          ],  
        },  
      },  
    ],  
  )  
  } catch (error) {  
    console.error('Challenge analytics error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```

    avgPoints: { $avg: '$points' },
    totalPoints: { $sum: '$points' },
    avgSolves: { $avg: '$solveCount' }
  }
}
],
// Category analysis
categoryAnalysis: [
  {
    $group: {
      _id: '$category',
      count: { $sum: 1 },
      avgPoints: { $avg: '$points' },
      totalSolves: { $sum: '$solveCount' },
      avgSuccessRate: {
        $avg: {
          $cond: [
            { $gt: [{ $size: '$submissions' }, 0] },
            {
              $divide: [
                {
                  $size: {
                    $filter: {
                      input: '$submissions',
                      as: 'sub',
                      cond: { $eq: ['$sub.isCorrect', true] }
                    }
                  }
                ]
              }
            },
            {}
          ]
        }
      },
    },
  ],

```

```

        { $size: '$submissions' }
      ]
    },
    0
  ]
}
}
}
},
{ $sort: { totalSolves: -1 } }
],
// Success rate distribution
successRates: [
  {
    $project: {
      title: 1,
      category: 1,
      difficulty: 1,
      points: 1,
      solveCount: 1,
      totalAttempts: { $size: '$submissions' },
      successRate: {
        $cond: [
          { $gt: [{ $size: '$submissions' }, 0] },
          {
            $multiply: [
              {
                $divide: [
                  {

```

```

    $size: {
      $filter: {
        input: '$submissions',
        as: 'sub',
        cond: { $eq: ['$sub.isCorrect', true] }
      }
    },
    { $size: '$submissions' }
  ]
},
100
]
},
0
]
}
}
},
{
  $bucket: {
    groupBy: '$successRate',
    boundaries: [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100],
    default: 'other',
    output: {
      count: { $sum: 1 },
      challenges: { $push: '$title' }
    }
  }
}

```

```

    }
  ],
  // Most popular challenges
  popularChallenges: [
    {
      $project: {
        title: 1,
        category: 1,
        difficulty: 1,
        points: 1,
        solveCount: 1,
        totalAttempts: { $size: '$submissions' },
        successRate: {
          $cond: [
            { $gt: [{ $size: '$submissions' }, 0] },
            {
              $multiply: [
                {
                  $divide: [
                    '$solveCount',
                    { $size: '$submissions' }
                  ]
                },
                100
              ]
            },
            0
          ]
        }
      }
    }
  ]
}

```

```

    }
  },
  { $sort: { solveCount: -1 } },
  { $limit: 10 }
]
}
}
})

```

```

res.json({ challengeAnalytics: challengeAnalytics[0] })
} catch (error) {
  console.error('Get challenge analytics error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

```

```

// Submission analytics
router.get('/submissions', async (req, res) => {
  try {
    const { period = 'all' } = req.query

    const submissionAnalytics = await Submission.aggregate([
      {
        $match: getPeriodFilter(period, 'createdAt')
      },
      {
        $facet: {
          // Submission trends
          submissionTrends: [

```



```

{
  $group: {
    _id: {
      $dateToString: { format: '%Y-%m-%d', date: '$createdAt' }
    },
    total: { $sum: 1 },
    correct: {
      $sum: { $cond: ['$isCorrect', 1, 0] }
    }
  },
  {
    $project: {
      date: '$_id',
      total: 1,
      correct: 1,
      successRate: {
        $cond: [
          { $gt: ['$total', 0] },
          { $multiply: [{ $divide: ['$correct', '$total'] }, 100] },
          0
        ]
      }
    }
  },
  { $sort: { date: 1 } }
],
// Hourly activity
hourlyActivity: [

```

```

{
  $group: {
    _id: { $hour: '$createdAt' },
    count: { $sum: 1 },
    correct: { $sum: { $cond: ['$isCorrect', 1, 0] } }
  }
},
{
  $project: {
    hour: '$_id',
    count: 1,
    correct: 1,
    successRate: {
      $cond: [
        { $gt: ['$count', 0] },
        { $multiply: [{ $divide: ['$correct', '$count'] }, 100] },
        0
      ]
    }
  }
},
{ $sort: { hour: 1 } }
],
// Category performance
categoryPerformance: [
  {
    $lookup: {
      from: 'challenges',
      localField: 'challenge',

```

```

    foreignField: '_id',
    as: 'challenge'
  }
},
{ $unwind: '$challenge' },
{
  $group: {
    _id: '$challenge.category',
    total: { $sum: 1 },
    correct: { $sum: { $cond: ['$isCorrect', 1, 0] } },
    uniqueUsers: { $addToSet: '$user' }
  }
},
{
  $project: {
    category: '$_id',
    totalSubmissions: '$total',
    correctSubmissions: '$correct',
    uniqueUsers: { $size: '$uniqueUsers' },
    successRate: {
      $multiply: [{ $divide: ['$correct', '$total'] }, 100]
    }
  }
},
{ $sort: { totalSubmissions: -1 } }
],
// User submission patterns
userPatterns: [
  {

```

```

$group: {
  _id: '$user',
  totalSubmissions: { $sum: 1 },
  correctSubmissions: { $sum: { $cond: ['$isCorrect', 1, 0] } },
  firstSubmission: { $min: '$createdAt' },
  lastSubmission: { $max: '$createdAt' }
},
{
  $lookup: {
    from: 'users',
    localField: '_id',
    foreignField: '_id',
    as: 'user'
  },
  { $unwind: '$user' },
  {
    $project: {
      username: '$user.username',
      totalSubmissions: 1,
      correctSubmissions: 1,
      successRate: {
        $cond: [
          { $gt: ['$totalSubmissions', 0] },
          { $multiply: [{ $divide: ['$correctSubmissions', '$totalSubmissions'] }, 100] },
          0
        ]
      }
    },
  },

```

```

    activityPeriod: {
      $divide: [
        { $subtract: ['$lastSubmission', '$firstSubmission'] },
        1000 * 60 * 60 * 24 // Convert to days
      ]
    }
  },
  { $sort: { totalSubmissions: -1 } },
  { $limit: 20 }
]
}
}
}))

```

```

    res.json({ submissionAnalytics: submissionAnalytics[0] })
  } catch (error) {
    console.error('Get submission analytics error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

```

```

// Performance analytics
router.get('/performance', async (req, res) => {
  try {
    const performanceAnalytics = await Submission.aggregate([
      {
        $lookup: {
          from: 'challenges',

```

```

    localField: 'challenge',
    foreignField: '_id',
    as: 'challenge'
  }
},
{ $unwind: '$challenge' },
{
  $facet: {
    // Solve time analysis
    solveTimes: [
      {
        $match: {
          isCorrect: true,
          timeSpent: { $gt: 0 }
        }
      },
      {
        $group: {
          _id: '$challenge.difficulty',
          count: { $sum: 1 },
          avgTime: { $avg: '$timeSpent' },
          minTime: { $min: '$timeSpent' },
          maxTime: { $max: '$timeSpent' }
        }
      }
    ],
    // Attempt patterns
    attemptPatterns: [
      {

```

```

$group: {
  _id: {
    user: '$user',
    challenge: '$challenge._id'
  },
  attempts: { $sum: 1 },
  solved: { $max: { $cond: ['$isCorrect', 1, 0] } }
}
},
{
  $group: {
    _id: '$attempts',
    count: { $sum: 1 },
    solvedCount: { $sum: '$solved' }
  }
},
{
  $project: {
    attempts: '$_id',
    count: 1,
    solvedCount: 1,
    successRate: {
      $multiply: [{ $divide: ['$solvedCount', '$count'] }, 100]
    }
  }
},
{ $sort: { attempts: 1 } }
],
// First blood analysis

```

```
firstBloods: [
  {
    $match: {
      isCorrect: true,
      solveOrder: 1
    }
  },
  {
    $lookup: {
      from: 'users',
      localField: 'user',
      foreignField: '_id',
      as: 'user'
    }
  },
  { $unwind: '$user' },
  {
    $group: {
      _id: '$user._id',
      username: { $first: '$user.username' },
      count: { $sum: 1 },
      challenges: { $push: '$challenge.title' }
    }
  },
  { $sort: { count: -1 } },
  { $limit: 10 }
]
}
```



```
)
```

```
    res.json({ performanceAnalytics: performanceAnalytics[0] })  
  } catch (error) {  
    console.error('Get performance analytics error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```
// Helper functions
```

```
async function getUserStatistics() {  
  const [  
    totalUsers,  
    activeUsers,  
    newUsersToday,  
    avgScore,  
    topScore  
  ] = await Promise.all([  
    User.countDocuments(),  
    User.countDocuments({  
      lastLogin: { $gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) }  
    }),  
    User.countDocuments({  
      createdAt: { $gte: new Date().setHours(0, 0, 0, 0) }  
    }),  
    User.aggregate([{$group: { _id: null, avg: { $avg: '$score' } } }]),  
    User.findOne().sort({ score: -1 }).select('score')  
  ])  
}
```

```
return {  
  users: {  
    total: totalUsers,  
    active: activeUsers,  
    newToday: newUsersToday,  
    avgScore: avgScore[0]?.avg || 0,  
    topScore: topScore?.score || 0  
  }  
}  
}
```

```
async function getChallengeStatistics() {  
  const [  
    totalChallenges,  
    activeChallenges,  
    challengesByDifficulty,  
    totalPoints  
  ] = await Promise.all([  
    Challenge.countDocuments(),  
    Challenge.countDocuments({ isActive: true }),  
    Challenge.aggregate([  
      { $group: { _id: '$difficulty', count: { $sum: 1 } } }  
    ]),  
    Challenge.aggregate([  
      { $group: { _id: null, total: { $sum: '$points' } } }  
    ])  
  ])
```

```
return {  
  challenges: {  
    total: totalChallenges,  

```

```

    active: activeChallenges,
    byDifficulty: challengesByDifficulty.reduce((acc, curr) => {
      acc[curr._id] = curr.count
      return acc
    }, {}),
    totalPoints: totalPoints[0]?.total || 0
  }
}
}

```

```

async function getSubmissionStatistics() {
  const [
    totalSubmissions,
    correctSubmissions,
    submissionsToday
  ] = await Promise.all([
    Submission.countDocuments(),
    Submission.countDocuments({ isCorrect: true }),
    Submission.countDocuments({
      createdAt: { $gte: new Date().setHours(0, 0, 0, 0) }
    })
  ])

  return {
    submissions: {
      total: totalSubmissions,
      correct: correctSubmissions,
      today: submissionsToday,
      successRate: totalSubmissions > 0 ? (correctSubmissions / totalSubmissions * 100).toFixed(2) : 0
    }
  }
}

```

```
}  
}  
}
```

```
async function getTeamStatistics() {  
  const [  
    totalTeams,  
    activeTeams,  
    avgTeamSize,  
    largestTeam  
  ] = await Promise.all([  
    Team.countDocuments(),  
    Team.countDocuments({ isActive: true }),  
    Team.aggregate([  
      { $project: { size: { $size: '$members' } } },  
      { $group: { _id: null, avg: { $avg: '$size' } } }  
    ]),  
    Team.findOne().sort({ members: -1 }).populate('members', 'username')  
  ])  
  
  return {  
    teams: {  
      total: totalTeams,  
      active: activeTeams,  
      avgSize: avgTeamSize[0]?.avg || 0,  
      largestTeam: largestTeam ? {  
        name: largestTeam.name,  
        size: largestTeam.members.length  
      } : null  
    }  
  }  
}
```

```
    }  
  }  
}
```

```
async function getAchievementStatistics() {  
  const [  
    totalAchievements,  
    achievementsToday,  
    popularAchievement  
  ] = await Promise.all([  
    Achievement.countDocuments(),  
    Achievement.countDocuments({  
      awardedAt: { $gte: new Date().setHours(0, 0, 0, 0) }  
    }),  
    Achievement.aggregate([  
      { $group: { _id: '$type', count: { $sum: 1 } } },  
      { $sort: { count: -1 } },  
      { $limit: 1 }  
    ])  
  ])  
  
  return {  
    achievements: {  
      total: totalAchievements,  
      today: achievementsToday,  
      mostPopular: popularAchievement[0] || null  
    }  
  }  
}
```

```
async function getRecentActivity() {  
  const recentSubmissions = await Submission.find()  
    .populate('user', 'username')  
    .populate('challenge', 'title category')  
    .sort({ createdAt: -1 })  
    .limit(10)  
  
  const recentAchievements = await Achievement.find()  
    .populate('user', 'username')  
    .populate('challenge', 'title')  
    .sort({ awardedAt: -1 })  
    .limit(5)  
  
  return {  
    submissions: recentSubmissions.map(sub => ({  
      user: sub.user.username,  
      challenge: sub.challenge.title,  
      category: sub.challenge.category,  
      correct: sub.isCorrect,  
      timestamp: sub.createdAt  
    })),  
    achievements: recentAchievements.map(ach => ({  
      user: ach.user.username,  
      type: ach.type,  
      challenge: ach.challenge?.title,  
      timestamp: ach.awardedAt  
    })))  
  }  
}
```

```
}
```

```
function getPeriodFilter(period, field) {
```

```
  const now = new Date()
```

```
  let startDate
```

```
  switch (period) {
```

```
    case 'today':
```

```
      startDate = new Date(now.setHours(0, 0, 0, 0))
```

```
      break
```

```
    case 'week':
```

```
      startDate = new Date(now.setDate(now.getDate() - 7))
```

```
      break
```

```
    case 'month':
```

```
      startDate = new Date(now.setMonth(now.getMonth() - 1))
```

```
      break
```

```
    case 'year':
```

```
      startDate = new Date(now.setFullYear(now.getFullYear() - 1))
```

```
      break
```

```
    case 'all':
```

```
    default:
```

```
      return {}
```

```
  }
```

```
  return { [field]: { $gte: startDate } }
```

```
}
```

```
export default router",
```

auth.js

```
"import express from 'express'
```

```
import { body, validationResult } from 'express-validator'
```

```
import jwt from 'jsonwebtoken'
```

```
import User from '../models/User.js'
```

```
import { authRateLimit } from '../middleware/advancedRateLimit.js'
```

```
const router = express.Router()
```

```
// Generate JWT Token
```

```
const generateToken = (userId) => {
```

```
  return jwt.sign({ userId }, process.env.JWT_SECRET, { expiresIn: '24h' })
```

```
}
```

```
// Register
```

```
router.post('/register', [
```

```
  authRateLimit,
```

```
  body('email').isEmail().normalizeEmail().withMessage('Valid email is required'),
```

```
  body('password')
```

```
    .isLength({ min: 6 })
```

```
    .withMessage('Password must be at least 6 characters')
```

```
    .matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/)
```

```
    .withMessage('Password must contain at least one lowercase letter, one uppercase letter, and one number'),
```

```
  body('username')
```

```
    .isLength({ min: 3, max: 30 })
```

```
    .withMessage('Username must be 3-30 characters')
```

```
    .matches(/^[a-zA-Z0-9_]+$/)
```

```
    .withMessage('Username can only contain letters, numbers, and underscores'),
```



```

body('fullName')
  .notEmpty()
  .trim()
  .withMessage('Full name is required')
  .isLength({ max: 50 })
  .withMessage('Full name too long')
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const { email, password, username, fullName } = req.body

    // Check if user exists
    const existingUser = await User.findOne({
      $or: [{ email }, { username }]
    })

    if (existingUser) {
      return res.status(400).json({
        error: existingUser.email === email ? 'Email already registered' : 'Username already taken'
      })
    }

    // Create user
    const user = new User({
      email,

```

```
password,  
username,  
fullName,  
role: 'participant'  
})
```

```
await user.save()
```

```
// Generate token
```

```
const token = generateToken(user._id)
```

```
// Send welcome notification via socket
```

```
const socketService = await import('../services/socketService.js')
```

```
socketService.default.sendNotification(user._id, {  
  type: 'welcome',  
  title: 'Welcome to CTF Platform!',  
  message: 'Get started by exploring challenges and joining a team!',  
  action: { label: 'View Challenges', url: '/challenges' }  
})
```

```
res.status(201).json({  
  message: 'User created successfully',  
  token,  
  user: {  
    id: user._id,  
    email: user.email,  
    username: user.username,  
    fullName: user.fullName,  
    role: user.role,
```

```

        score: user.score,
        preferences: user.preferences
    }
})

} catch (error) {
    console.error('Registration error:', error)
    res.status(500).json({ error: 'Internal server error' })
}
})

// Login
router.post('/login', [
    authRateLimit,
    body('email').isEmail().normalizeEmail().withMessage('Valid email is required'),
    body('password').exists().withMessage('Password is required')
], async (req, res) => {
    try {
        const errors = validationResult(req)
        if (!errors.isEmpty()) {
            return res.status(400).json({ errors: errors.array() })
        }

        const { email, password } = req.body

        // Find user
        const user = await User.findOne({ email })
        if (!user) {
            return res.status(401).json({ error: 'Invalid credentials' })
        }
    }
})

```

```
}
```

```
// Check if account is active
```

```
if (!user.isActive) {
```

```
  return res.status(401).json({ error: 'Account is deactivated' })
```

```
}
```

```
// Verify password
```

```
const isValidPassword = await user.comparePassword(password)
```

```
if (!isValidPassword) {
```

```
  // Log failed attempt
```

```
  user.statistics.totalAttempts += 1
```

```
  await user.save()
```

```
  return res.status(401).json({ error: 'Invalid credentials' })
```

```
}
```

```
// Update last login and statistics
```

```
user.lastLogin = new Date()
```

```
user.statistics.totalAttempts += 1
```

```
await user.save()
```

```
// Generate token
```

```
const token = generateToken(user._id)
```

```
res.json({
```

```
  message: 'Login successful',
```

```
  token,
```

```
  user: {
```

```
    id: user._id,  
    email: user.email,  
    username: user.username,  
    fullName: user.fullName,  
    role: user.role,  
    score: user.score,  
    preferences: user.preferences,  
    statistics: user.statistics,  
    streak: user.streak,  
    team: user.team  
  }  
})
```

```
  } catch (error) {  
    console.error('Login error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

// Get current user profile

```
router.get('/profile', async (req, res) => {  
  try {  
    const token = req.header('Authorization')?.replace('Bearer ', '')  
  
    if (!token) {  
      return res.status(401).json({ error: 'No token provided' })  
    }  
  
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
```

```
const user = await User.findById(decoded.userId)

    .populate('badges')

    .populate('team')

    .populate('solvedChallenges', 'title category points')

if (!user) {

    return res.status(404).json({ error: 'User not found' })

}

res.json({

    user: {

        id: user._id,

        email: user.email,

        username: user.username,

        fullName: user.fullName,

        role: user.role,

        score: user.score,

        preferences: user.preferences,

        statistics: user.statistics,

        streak: user.streak,

        badges: user.badges,

        team: user.team,

        solvedChallenges: user.solvedChallenges,

        createdAt: user.createdAt

    }

})

} catch (error) {

    console.error('Profile error:', error)
```

```

    res.status(500).json({ error: 'Internal server error' })
  }
})

// Update user preferences
router.put('/preferences', [
  body('emailNotifications').optional().isBoolean(),
  body('showHints').optional().isBoolean(),
  body('theme').optional().isIn(['light', 'dark', 'auto'])
], async (req, res) => {
  try {
    const token = req.header('Authorization')?.replace('Bearer ', '')

    if (!token) {
      return res.status(401).json({ error: 'No token provided' })
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    const user = await User.findById(decoded.userId)

    if (!user) {
      return res.status(404).json({ error: 'User not found' })
    }

    const { emailNotifications, showHints, theme } = req.body

    if (emailNotifications !== undefined) {
      user.preferences.emailNotifications = emailNotifications
    }
  }
})

```

```

    if (showHints !== undefined) {
      user.preferences.showHints = showHints
    }
    if (theme !== undefined) {
      user.preferences.theme = theme
    }

    await user.save()

    res.json({
      message: 'Preferences updated successfully',
      preferences: user.preferences
    })

  } catch (error) {
    console.error('Preferences update error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

export default router"

routes/
challenges.js

"import express from 'express'
import { body, validationResult } from 'express-validator'
import Challenge from '../models/Challenge.js'
import Submission from '../models/Submission.js'
import User from '../models/User.js'

```



```
import { authMiddleware } from '../middleware/auth.js'
import { challengeSpecificRateLimit } from '../middleware/advancedRateLimit.js'
import achievementService from '../services/achievementService.js'
import scoringService from '../services/scoringService.js'
import badgeService from '../services/badgeService.js'
import socketService from '../services/socketService.js'
```

```
const router = express.Router()
```

```
// Get all challenges with enhanced filtering
```

```
router.get('/', authMiddleware, async (req, res) => {
  try {
    const { category, difficulty, status, search } = req.query
```

```
    let filter = { isActive: true }
```

```
    // Apply filters
```

```
    if (category && category !== 'all') {
      filter.category = category
    }
```

```
    if (difficulty && difficulty !== 'all') {
      filter.difficulty = difficulty
    }
```

```
    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } },
```

```
    { tags: { $in: [new RegExp(search, 'i')] } }  
  ]  
}
```

```
const challenges = await Challenge.find(filter)  
  .select('title description category difficulty points tags solveCount dynamicScoring attachments  
dockerImage createdAt')
```

```
  .sort({ category: 1, difficulty: 1, points: 1 })
```

```
// Get user's solved challenges and progress
```

```
const [userSubmissions, userProgress] = await Promise.all([  
  Submission.find({  
    user: req.userId,  
    isCorrect: true  
  }).select('challenge'),
```

```
(await import('../models/UserChallengeProgress.js')).default.find({  
  user: req.userId  
})
```

```
])
```

```
const solvedChallengelds = new Set(userSubmissions.map(sub => sub.challenge.toString()))
```

```
const progressMap = new Map(userProgress.map(p => [p.challenge.toString(), p]))
```

```
const challengesWithStatus = challenges.map(challenge => {  
  const progress = progressMap.get(challenge._id.toString())  
  const solved = solvedChallengelds.has(challenge._id.toString())
```

```
  return {
```

```

    id: challenge._id,
    title: challenge.title,
    description: challenge.description,
    category: challenge.category,
    difficulty: challenge.difficulty,
    points: challenge.points,
    currentPoints: challenge.currentPoints,
    tags: challenge.tags,
    solveCount: challenge.solveCount,
    dynamicScoring: challenge.dynamicScoring,
    attachments: challenge.attachments,
    dockerImage: challenge.dockerImage,
    createdAt: challenge.createdAt,
    solved,
    progress: progress ? {
      attempts: progress.attempts,
      timeSpent: progress.timeSpent,
      hintsUsed: progress.hintsUsed.length,
      startedAt: progress.startedAt
    } : null
  }
})

```

```

// Apply status filter after building the data
let filteredChallenges = challengesWithStatus
if (status === 'solved') {
  filteredChallenges = filteredChallenges.filter(c => c.solved)
} else if (status === 'unsolved') {
  filteredChallenges = filteredChallenges.filter(c => !c.solved)
}

```

```

    }

    res.json(filteredChallenges)
  } catch (error) {
    console.error('Get challenges error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

// Get single challenge details
router.get('/:challengeId', authMiddleware, async (req, res) => {
  try {
    const challenge = await Challenge.findById(req.params.challengeId)
    if (!challenge || !challenge.isActive) {
      return res.status(404).json({ error: 'Challenge not found' })
    }

    // Get user's progress
    const [submission, progress] = await Promise.all([
      Submission.findOne({
        user: req.userId,
        challenge: req.params.challengeId,
        isCorrect: true
      }),
      (await import('../models/UserChallengeProgress.js')).default.findOne({
        user: req.userId,
        challenge: req.params.challengeId
      })
    ])
  }
})

```

```
])
```

```
const solved = !!submission
```

```
// Get hints (without revealing content until purchased)
```

```
const Hint = await import('../models/Hint.js')
```

```
const hints = await Hint.default.find({
```

```
  challenge: req.params.challenged,
```

```
  isActive: true
```

```
}).select('level cost cooldown')
```

```
// Get user's purchased hints
```

```
const UserHint = await import('../models/UserHint.js')
```

```
const userHints = await UserHint.default.find({
```

```
  user: req.userId,
```

```
  challenge: req.params.challenged
```

```
}).populate('hint', 'level content')
```

```
res.json({
```

```
  challenge: {
```

```
    id: challenge._id,
```

```
    title: challenge.title,
```

```
    description: challenge.description,
```

```
    category: challenge.category,
```

```
    difficulty: challenge.difficulty,
```

```
    points: challenge.points,
```

```
    currentPoints: challenge.currentPoints,
```

```
    tags: challenge.tags,
```

```
    attachments: challenge.attachments,
```

```

    dockerImage: challenge.dockerImage,
    validationType: challenge.validationType,
    maxAttempts: challenge.maxAttempts,
    timeLimit: challenge.timeLimit,
    createdAt: challenge.createdAt
  },
  solved,
  submission: solved ? {
    submittedAt: submission.createdAt,
    pointsAwarded: submission.pointsAwarded,
    solveOrder: submission.solveOrder
  } : null,
  progress: progress ? {
    attempts: progress.attempts,
    timeSpent: progress.timeSpent,
    startedAt: progress.startedAt
  } : null,
  hints: hints.map(hint => ({
    level: hint.level,
    cost: hint.cost,
    cooldown: hint.cooldown,
    purchased: userHints.some(uh => uh.hint.level === hint.level),
    content: userHints.find(uh => uh.hint.level === hint.level)?.hint.content
  }))
})

} catch (error) {
  console.error('Get challenge error:', error)
  res.status(500).json({ error: 'Internal server error' })
}

```

```
}  
})
```

```
// Submit flag with enhanced features  
router.post('/:challengeId/submit', [  
  authMiddleware,  
  challengeSpecificRateLimit(10, 60), // 10 attempts per hour  
  body('flag').notEmpty().trim().withMessage('Flag is required')  
], async (req, res) => {  
  try {  
    const errors = validationResult(req)  
    if (!errors.isEmpty()) {  
      return res.status(400).json({ errors: errors.array() })  
    }  
  
    const { challengeId } = req.params  
    const { flag } = req.body  
    const userId = req.userId  
  
    // Check if already solved  
    const existingSubmission = await Submission.findOne({  
      user: userId,  
      challenge: challengeId,  
      isCorrect: true  
    })  
  
    if (existingSubmission) {  
      return res.status(400).json({ error: 'Challenge already solved' })  
    }  
  }  
})
```

```
// Get challenge
const challenge = await Challenge.findById(challengeId)
if (!challenge || !challenge.isActive) {
  return res.status(404).json({ error: 'Challenge not found' })
}

// Check max attempts
if (challenge.maxAttempts > 0) {
  const attemptCount = await Submission.countDocuments({
    user: userId,
    challenge: challengeId
  })

  if (attemptCount >= challenge.maxAttempts) {
    return res.status(400).json({ error: 'Maximum attempts reached for this challenge' })
  }
}

// Verify flag
const isCorrect = flag === challenge.flag

// Get client IP properly
const clientIp = req.headers['x-forwarded-for'] || req.connection.remoteAddress

// Record submission
const submission = new Submission({
  user: userId,
  challenge: challengeId,
```



```
    flagSubmitted: flag,  
    isCorrect,  
    ip: clientIp,  
    userAgent: req.get('User-Agent')  
  })
```

```
  await submission.save()
```

```
  // Update user progress
```

```
  const UserChallengeProgress = await import('../models/UserChallengeProgress.js')
```

```
  let progress = await UserChallengeProgress.default.findOne({  
    user: userId,  
    challenge: challengeld  
  })
```

```
  if (!progress) {  
    progress = new UserChallengeProgress.default({  
      user: userId,  
      challenge: challengeld  
    })  
  }  
}
```

```
progress.attempts += 1
```

```
progress.lastAttempt = new Date()
```

```
if (isCorrect) {  
  const pointsAwarded = challenge.currentPoints
```

```
  // Update challenge solve count
```

```
challenge.solveCount += 1
if (challenge.dynamicScoring) {
  challenge.points = challenge.currentPoints
}
await challenge.save()

// Update submission with points
submission.pointsAwarded = pointsAwarded
await submission.save()

// Update user
const user = await User.findById(userId)
user.solvedChallenges.push(challengeId)
user.score += pointsAwarded
user.updateStreak()
user.statistics.correctSubmissions += 1
user.statistics.totalAttempts += 1

// Update favorite category
const categorySolves = await Submission.aggregate([
  { $match: { user: userId, isCorrect: true } },
  {
    $lookup: {
      from: 'challenges',
      localField: 'challenge',
      foreignField: '_id',
      as: 'challenge'
    }
  },
],
```

```

    { $unwind: '$challenge' },
    { $group: { _id: '$challenge.category', count: { $sum: 1 } } },
    { $sort: { count: -1 } },
    { $limit: 1 }
  ])

  if (categorySolves.length > 0) {
    user.statistics.favoriteCategory = categorySolves[0]._id
  }

  await user.save()

  // Update progress
  progress.completed = true
  progress.completedAt = new Date()
  await progress.save()

  // Check achievements
  await achievementService.checkFirstBlood(challengeId, userId)
  await achievementService.checkCategoryMaster(userId, challenge.category)
  await achievementService.checkPointMilestones(userId)

  // Check badges
  await badgeService.checkAndAwardBadges(userId)

  // Update scoring if dynamic
  if (challenge.dynamicScoring) {
    await scoringService.updateChallengePoints(challengeId)
  }

```

```

// Broadcast solve notification

const isFirstBlood = submission.solveOrder === 1
socketService.broadcastChallengeSolve(user, challenge, isFirstBlood)

res.json({
  correct: true,
  message: isFirstBlood ? '🎯 First Blood! Correct flag!' : '✅ Correct flag! Challenge completed!',
  points: pointsAwarded,
  solveOrder: submission.solveOrder,
  isFirstBlood
})

} else {
  // Incorrect submission
  await progress.save()

  // Update user statistics
  await User.findByIdAndUpdate(userId, {
    $inc: { 'statistics.totalAttempts': 1 }
  })

  res.json({
    correct: false,
    message: '❌ Incorrect flag!',
    attempts: progress.attempts,
    maxAttempts: challenge.maxAttempts
  })
}

```

```

    } catch (error) {
      console.error('Submit flag error:', error)
      res.status(500).json({ error: 'Internal server error' })
    }
  })

// Start challenge (track time)
router.post('/:challengeId/start', authMiddleware, async (req, res) => {
  try {
    const UserChallengeProgress = await import('../models/UserChallengeProgress.js')

    let progress = await UserChallengeProgress.default.findOne({
      user: req.userId,
      challenge: req.params.challengeId
    })

    if (!progress) {
      progress = new UserChallengeProgress.default({
        user: req.userId,
        challenge: req.params.challengeId,
        startedAt: new Date()
      })
      await progress.save()
    }

    res.json({
      message: 'Challenge started',
      progress: {

```

```
    startedAt: progress.startedAt,  
    attempts: progress.attempts  
  }  
})
```

```
  } catch (error) {  
    console.error('Start challenge error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```
export default router",
```

```
events.js
```

```
"import express from 'express'  
import { body, validationResult } from 'express-validator'  
import Event from '../models/Event.js'  
import User from '../models/User.js'  
import Team from '../models/Team.js'  
import Challenge from '../models/Challenge.js'  
import { authMiddleware, adminMiddleware } from '../middleware/auth.js'  
import socketService from '../services/socketService.js'
```

```
const router = express.Router()
```

```
// Get all events
```

```
router.get('/', async (req, res) => {  
  try {  
    const { status = 'all', page = 1, limit = 20 } = req.query
```

```
let filter = { isPublic: true }
```

```
// Filter by status
```

```
const now = new Date()
```

```
switch (status) {
```

```
  case 'active':
```

```
    filter.startTime = { $lte: now }
```

```
    filter.endTime = { $gte: now }
```

```
    filter.isActive = true
```

```
    break
```

```
  case 'upcoming':
```

```
    filter.startTime = { $gt: now }
```

```
    break
```

```
  case 'past':
```

```
    filter.endTime = { $lt: now }
```

```
    break
```

```
  case 'all':
```

```
  default:
```

```
    // No additional filters
```

```
    break
```

```
}
```

```
const events = await Event.find(filter)
```

```
  .populate('challenges', 'title category difficulty points')
```

```
  .select('name description startTime endTime isActive participants challenges maxTeamSize banner')
```

```
  .sort({ startTime: 1 })
```

```
  .limit(limit * 1)
```

```
  .skip((page - 1) * limit)
```

```
const total = await Event.countDocuments(filter)

// Add participant count and status
const eventsWithStats = events.map(event => {
  const participantCount = event.participants.length
  const teamCount = new Set(event.participants.filter(p => p.team).map(p => p.team.toString())).size

  let eventStatus = 'upcoming'
  if (event.startTime <= now && event.endTime >= now && event.isActive) {
    eventStatus = 'active'
  } else if (event.endTime < now) {
    eventStatus = 'past'
  }

  return {
    ...event.toObject(),
    participantCount,
    teamCount,
    eventStatus
  }
})

res.json({
  events: eventsWithStats,
  pagination: {
    total,
    page: parseInt(page),
    limit: parseInt(limit),
```



```

        pages: Math.ceil(total / limit)
    }
})
} catch (error) {
    console.error('Get events error:', error)
    res.status(500).json({ error: 'Internal server error' })
}
})

// Get single event details
router.get('/:eventId', async (req, res) => {
    try {
        const event = await Event.findById(req.params.eventId)
            .populate('challenges', 'title category difficulty points solveCount')
            .populate('participants.user', 'username fullName score')
            .populate('participants.team', 'name score')

        if (!event) {
            return res.status(404).json({ error: 'Event not found' })
        }

        // Calculate event statistics
        const now = new Date()
        let eventStatus = 'upcoming'
        if (event.startTime <= now && event.endTime >= now && event.isActive) {
            eventStatus = 'active'
        } else if (event.endTime < now) {
            eventStatus = 'past'
        }
    }
})

```

```

const participantCount = event.participants.length

const teamCount = new Set(event.participants.filter(p => p.team).map(p => p.team.toString())).size
const individualCount = event.participants.filter(p => !p.team).length

// Get leaderboard for the event
const eventLeaderboard = await getEventLeaderboard(req.params.eventId)

res.json({
  event: {
    ...event.toObject(),
    eventStatus,
    statistics: {
      participantCount,
      teamCount,
      individualCount,
      challengeCount: event.challenges.length,
      totalPoints: event.challenges.reduce((sum, challenge) => sum + challenge.points, 0)
    },
    leaderboard: eventLeaderboard
  }
})
} catch (error) {
  console.error('Get event error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Join event

```

```
router.post('/:eventId/join', [
  authMiddleware,
  body('teamId').optional().isMongoId()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const { eventId } = req.params
    const { teamId } = req.body
    const userId = req.userId

    const event = await Event.findById(eventId)
    if (!event || !event.isActive) {
      return res.status(404).json({ error: 'Event not found or not active' })
    }

    // Check if event has started
    if (event.startTime > new Date()) {
      return res.status(400).json({ error: 'Event has not started yet' })
    }

    // Check if event has ended
    if (event.endTime < new Date()) {
      return res.status(400).json({ error: 'Event has ended' })
    }
  }
})
```

```

// Check if user is already participating
const alreadyParticipating = event.participants.find(p => p.user.toString() === userId)
if (alreadyParticipating) {
  return res.status(400).json({ error: 'Already participating in this event' })
}

// Validate team if provided
if (teamId) {
  const team = await Team.findById(teamId)
  if (!team) {
    return res.status(404).json({ error: 'Team not found' })
  }

  // Check if user is in the team
  if (!team.members.includes(userId)) {
    return res.status(403).json({ error: 'You are not a member of this team' })
  }

  // Check if team is already participating
  const teamParticipating = event.participants.find(p => p.team && p.team.toString() === teamId)
  if (teamParticipating) {
    return res.status(400).json({ error: 'Team is already participating in this event' })
  }

  // Check team size limit
  if (event.maxTeamSize && team.members.length > event.maxTeamSize) {
    return res.status(400).json({ error: `Team exceeds maximum size of ${event.maxTeamSize}` })
  }
}

```

```

// Add participant
event.participants.push({
  user: userId,
  team: teamId || null,
  joinedAt: new Date()
})

await event.save()

res.json({
  message: 'Successfully joined event',
  participation: {
    event: event.name,
    team: teamId ? await Team.findById(teamId).select('name') : null,
    joinedAt: new Date()
  }
})

} catch (error) {
  console.error('Join event error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Leave event
router.post('/:eventId/leave', authMiddleware, async (req, res) => {
  try {
    const event = await Event.findById(req.params.eventId)

```

```

if (!event) {
  return res.status(404).json({ error: 'Event not found' })
}

// Check if event has started
if (event.startTime < new Date()) {
  return res.status(400).json({ error: 'Cannot leave event after it has started' })
}

// Remove participant
const initialLength = event.participants.length
event.participants = event.participants.filter(p => p.user.toString() !== req.userId)

if (event.participants.length === initialLength) {
  return res.status(400).json({ error: 'Not participating in this event' })
}

await event.save()

res.json({ message: 'Successfully left event' })
} catch (error) {
  console.error('Leave event error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Admin: Create event
router.post('/admin', [
  authMiddleware,

```

```

adminMiddleware,
body('name').notEmpty().trim(),
body('description').notEmpty().trim(),
body('startTime').isISO8601(),
body('endTime').isISO8601(),
body('maxTeamSize').optional().isInt({ min: 1, max: 10 }),
body('challenges').optional().isArray(),
body('isPublic').optional().isBoolean()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const {
      name,
      description,
      startTime,
      endTime,
      maxTeamSize = 4,
      challenges = [],
      isPublic = true
    } = req.body

    // Validate dates
    if (new Date(startTime) >= new Date(endTime)) {
      return res.status(400).json({ error: 'End time must be after start time' })
    }
  }
}

```

```
// Validate challenges exist
if (challenges.length > 0) {
  const existingChallenges = await Challenge.countDocuments({ _id: { $in: challenges } })
  if (existingChallenges !== challenges.length) {
    return res.status(400).json({ error: 'One or more challenges not found' })
  }
}
```

```
const event = new Event({
  name,
  description,
  startTime: new Date(startTime),
  endTime: new Date(endTime),
  maxTeamSize,
  challenges,
  isPublic,
  isActive: false // Events are inactive by default
})
```

```
await event.save()
```

```
res.status(201).json({
  message: 'Event created successfully',
  event: {
    id: event._id,
    name: event.name,
    description: event.description,
    startTime: event.startTime,
```



```

        endTime: event.endTime,
        maxTeamSize: event.maxTeamSize,
        challengeCount: event.challenges.length,
        isPublic: event.isPublic
    }
})
} catch (error) {
    console.error('Create event error:', error)
    res.status(500).json({ error: 'Internal server error' })
}
})

// Admin: Update event
router.put('/admin/:eventId', [
    authMiddleware,
    adminMiddleware,
    body('name').optional().trim(),
    body('description').optional().trim(),
    body('startTime').optional().isISO8601(),
    body('endTime').optional().isISO8601(),
    body('maxTeamSize').optional().isInt({ min: 1, max: 10 }),
    body('isActive').optional().isBoolean(),
    body('isPublic').optional().isBoolean()
], async (req, res) => {
    try {
        const errors = validationResult(req)
        if (!errors.isEmpty()) {
            return res.status(400).json({ errors: errors.array() })
        }
    }

```

```
const event = await Event.findById(req.params.eventId)

if (!event) {
  return res.status(404).json({ error: 'Event not found' })
}

const updates = { ...req.body }

// Validate date logic
if (updates.startTime && updates.endTime) {
  if (new Date(updates.startTime) >= new Date(updates.endTime)) {
    return res.status(400).json({ error: 'End time must be after start time' })
  }
}

const updatedEvent = await Event.findByIdAndUpdate(
  req.params.eventId,
  { $set: updates },
  { new: true }
).populate('challenges', 'title category points')

res.json({
  message: 'Event updated successfully',
  event: updatedEvent
})
} catch (error) {
  console.error('Update event error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
```

```
}}
```

```
// Admin: Add challenges to event
```

```
router.post('/admin/:eventId/challenges', [
```

```
  authMiddleware,
```

```
  adminMiddleware,
```

```
  body('challenges').isArray()
```

```
], async (req, res) => {
```

```
  try {
```

```
    const errors = validationResult(req)
```

```
    if (!errors.isEmpty()) {
```

```
      return res.status(400).json({ errors: errors.array() })
```

```
    }
```

```
    const { challenges } = req.body
```

```
    const event = await Event.findById(req.params.eventId)
```

```
    if (!event) {
```

```
      return res.status(404).json({ error: 'Event not found' })
```

```
    }
```

```
// Validate challenges exist
```

```
const existingChallenges = await Challenge.countDocuments({ _id: { $in: challenges } })
```

```
if (existingChallenges !== challenges.length) {
```

```
  return res.status(400).json({ error: 'One or more challenges not found' })
```

```
}
```

```
// Add challenges (avoid duplicates)
```

```
const newChallenges = challenges.filter(id => !event.challenges.includes(id))
```

```

event.challenges.push(...newChallenges)

await event.save()

res.json({
  message: `Added ${newChallenges.length} challenges to event`,
  addedCount: newChallenges.length,
  totalChallenges: event.challenges.length
})
} catch (error) {
  console.error('Add challenges error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Admin: Remove challenge from event
router.delete('/admin/:eventId/challenges/:challengeId', [
  authMiddleware,
  adminMiddleware
], async (req, res) => {
  try {
    const event = await Event.findById(req.params.eventId)
    if (!event) {
      return res.status(404).json({ error: 'Event not found' })
    }

    const initialLength = event.challenges.length

    event.challenges = event.challenges.filter(challenge => challenge.toString() !==
req.params.challengeId)

```

```

    if (event.challenges.length === initialLength) {
      return res.status(400).json({ error: 'Challenge not found in event' })
    }

    await event.save()

    res.json({
      message: 'Challenge removed from event',
      remainingChallenges: event.challenges.length
    })
  } catch (error) {
    console.error('Remove challenge error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

// Get event leaderboard
router.get('/:eventId/leaderboard', async (req, res) => {
  try {
    const leaderboard = await getEventLeaderboard(req.params.eventId)
    res.json({ leaderboard })
  } catch (error) {
    console.error('Get event leaderboard error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

// Helper function to get event leaderboard

```

```

async function getEventLeaderboard(eventId) {
  const event = await Event.findById(eventId)

  .populate('challenges', '_id')
  .populate('participants.user', 'username score')
  .populate('participants.team', 'name score members')

  if (!event) return []

  const challengeIds = event.challenges.map(c => c._id)
  const Submission = await import('../models/Submission.js')

  // Get submissions for event challenges
  const eventSubmissions = await Submission.default.find({
    challenge: { $in: challengeIds },
    isCorrect: true,
    createdAt: { $gte: event.startTime, $lte: event.endTime }
  }).populate('user', 'username')
  .populate('challenge', 'points')

  // Calculate scores
  const scores = new Map()

  event.participants.forEach(participant => {
    const key = participant.team ? `team_${participant.team._id}` : `user_${participant.user._id}`

    if (!scores.has(key)) {
      scores.set(key, {
        id: participant.team ? participant.team._id : participant.user._id,
        name: participant.team ? participant.team.name : participant.user.username,

```

```
    type: participant.team ? 'team' : 'individual',
    score: 0,
    solvedChallenges: new Set(),
    members: participant.team ? participant.team.members : [participant.user._id]
  })
}
})
```

```
// Add scores from submissions
eventSubmissions.forEach(submission => {
  // Find which participant/team this submission belongs to
  for (let [key, entry] of scores) {
    if (entry.members.includes(submission.user._id)) {
      const challengeId = submission.challenge._id.toString()
      if (!entry.solvedChallenges.has(challengeId)) {
        entry.score += submission.challenge.points
        entry.solvedChallenges.add(challengeId)
      }
      break
    }
  }
})
```

```
// Convert to array and sort
const leaderboard = Array.from(scores.values())
  .map(entry => ({
    ...entry,
    solvedCount: entry.solvedChallenges.size
  }))
```

```

.sort((a, b) => b.score - a.score || a.solvedCount - b.solvedCount)

// Add ranks
return leaderboard.map((entry, index) => ({
  rank: index + 1,
  ...entry
}))
}

// Admin: Get event statistics
router.get('/admin/:eventId/statistics', [
  authMiddleware,
  adminMiddleware
], async (req, res) => {
  try {
    const event = await Event.findById(req.params.eventId)
      .populate('participants.user', 'username score')
      .populate('participants.team', 'name score')
      .populate('challenges', 'title category points solveCount')

    if (!event) {
      return res.status(404).json({ error: 'Event not found' })
    }

    const statistics = {
      participants: {
        total: event.participants.length,
        teams: new Set(event.participants.filter(p => p.team).map(p => p.team.toString())).size,
        individuals: event.participants.filter(p => !p.team).length
      }
    }
  } catch (error) {
    return res.status(500).json({ error: 'Internal server error' })
  }
})

```



```

    },
    challenges: {
      total: event.challenges.length,
      byCategory: event.challenges.reduce((acc, challenge) => {
        acc[challenge.category] = (acc[challenge.category] || 0) + 1
        return acc
      }, {}),
      totalPoints: event.challenges.reduce((sum, challenge) => sum + challenge.points, 0)
    },
    submissions: await getEventSubmissionStats(event._id)
  }

  res.json({ statistics })
} catch (error) {
  console.error('Get event statistics error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Helper function to get event submission statistics
async function getEventSubmissionStats(eventId) {
  const event = await Event.findById(eventId).populate('challenges', '_id')
  const challengeIds = event.challenges.map(c => c._id)

  const Submission = await import('../models/Submission.js')
  const submissions = await Submission.default.aggregate([
    {
      $match: {
        challenge: { $in: challengeIds },

```

```

      createdAt: { $gte: event.startTime, $lte: event.endTime }
    }
  },
  {
    $group: {
      _id: '$isCorrect',
      count: { $sum: 1 }
    }
  }
])

```

```

const correct = submissions.find(s => s._id === true)?.count || 0
const incorrect = submissions.find(s => s._id === false)?.count || 0
const total = correct + incorrect

```

```

return {
  total,
  correct,
  incorrect,
  successRate: total > 0 ? (correct / total * 100).toFixed(2) : 0
}
}

```

```
export default router"
```

```
routes/
```

```
files.js
```

```
"import express from 'express'
```

```
import multer from 'multer'
```

```
import path from 'path'
import fs from 'fs/promises'
import { fileURLToPath } from 'url'
import File from '../models/File.js'
import Challenge from '../models/Challenge.js'
import { authMiddleware, adminMiddleware } from '../middleware/auth.js'

const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)

const router = express.Router()

// Configure multer for file uploads
const storage = multer.diskStorage({
  destination: async (req, file, cb) => {
    const challengeId = req.params.challengeId
    const uploadDir = path.join(__dirname, '../uploads/challenges', challengeId)

    try {
      await fs.mkdir(uploadDir, { recursive: true })
      cb(null, uploadDir)
    } catch (error) {
      cb(error, null)
    }
  },
  filename: (req, file, cb) => {
    const timestamp = Date.now()
    const originalName = file.originalname
    const filename = `${timestamp}-${originalName}`
```

```
    cb(null, filename)
  }
})
```

```
const fileFilter = (req, file, cb) => {
  // Check file types for security
  const allowedTypes = [
    'text/plain',
    'application/pdf',
    'image/jpeg',
    'image/png',
    'image/gif',
    'application/zip',
    'application/x-zip-compressed',
    'application/x-rar-compressed',
    'text/x-python',
    'application/x-python-code',
    'text/x-c',
    'text/x-c++',
    'application/javascript',
    'text/html',
    'text/css',
    'application/json'
  ]
```

```
  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true)
  } else {
    cb(new Error('File type not allowed'), false)
```

```
}  
}
```

```
const upload = multer({  
  storage: storage,  
  fileFilter: fileFilter,  
  limits: {  
    fileSize: 50 * 1024 * 1024, // 50MB limit  
    files: 5 // Max 5 files per upload  
  }  
})
```

```
// Upload files for a challenge (admin only)  
router.post('/challenge/:challengeId/upload', [  
  authMiddleware,  
  adminMiddleware,  
  upload.array('files', 5)  
], async (req, res) => {  
  try {  
    const { challengeId } = req.params  
    const files = req.files  
  
    if (!files || files.length === 0) {  
      return res.status(400).json({ error: 'No files uploaded' })  
    }  
  }  
}
```

```
// Check if challenge exists  
const challenge = await Challenge.findById(challengeId)  
if (!challenge) {
```

```
// Clean up uploaded files
await Promise.all(files.map(file => fs.unlink(file.path)))

return res.status(404).json({ error: 'Challenge not found' })
}

const savedFiles = []

for (const file of files) {
  // Create file record
  const fileRecord = new File({
    filename: file.filename,
    originalName: file.originalname,
    path: file.path,
    size: file.size,
    mimetype: file.mimetype,
    challenge: challengeId,
    uploadedBy: req.userId
  })

  await fileRecord.save()
  savedFiles.push(fileRecord)

  // Update challenge attachments
  challenge.attachments.push({
    name: file.originalname,
    url: `/api/files/download/${challengeId}/${fileRecord.filename}`,
    size: file.size
  })
}
```

```
await challenge.save()
```

```
res.status(201).json({  
  message: 'Files uploaded successfully',  
  files: savedFiles.map(file => ({  
    id: file._id,  
    originalName: file.originalName,  
    filename: file.filename,  
    size: file.size,  
    mimetype: file.mimetype,  
    uploadedAt: file.createdAt  
  }))  
})
```

```
} catch (error) {  
  console.error('File upload error:', error)
```

```
// Clean up any uploaded files on error  
if (req.files) {  
  await Promise.all(req.files.map(file => fs.unlink(file.path).catch(() => {})))  
}
```

```
res.status(500).json({ error: 'File upload failed' })  
}  
})
```

```
// Download file
```

```
router.get('/download/:challengeId/:filename', async (req, res) => {
```

```
try {  
  const { challengeId, filename } = req.params  
  
  // Find file record  
  const fileRecord = await File.findOne({  
    filename: filename,  
    challenge: challengeId  
  })  
  
  if (!fileRecord) {  
    return res.status(404).json({ error: 'File not found' })  
  }  
  
  // Check if file exists on disk  
  try {  
    await fs.access(fileRecord.path)  
  } catch {  
    return res.status(404).json({ error: 'File not found on server' })  
  }  
  
  // Increment download count  
  fileRecord.downloadCount += 1  
  await fileRecord.save()  
  
  // Set headers and send file  
  res.setHeader('Content-Disposition', `attachment; filename="${fileRecord.originalName}"`)  
  res.setHeader('Content-Type', fileRecord.mimetype)  
  res.setHeader('Content-Length', fileRecord.size)
```



```
res.sendFile(path.resolve(fileRecord.path))

} catch (error) {
  console.error('File download error:', error)
  res.status(500).json({ error: 'File download failed' })
}
})

// Get files for a challenge
router.get('/challenge/:challengeId', async (req, res) => {
  try {
    const { challengeId } = req.params
    const { publicOnly = 'true' } = req.query

    const filter = { challenge: challengeId }
    if (publicOnly === 'true') {
      filter.isPublic = true
    }

    const files = await File.find(filter)
      .populate('uploadedBy', 'username')
      .select('originalName filename size mimetype downloadCount uploadedBy createdAt')
      .sort({ createdAt: -1 })

    res.json({ files })
  } catch (error) {
    console.error('Get files error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})
```

```
}}
```

```
// Admin: Get all files
```

```
router.get('/admin', [
```

```
  authMiddleware,
```

```
  adminMiddleware
```

```
], async (req, res) => {
```

```
  try {
```

```
    const { page = 1, limit = 50, challengeId } = req.query
```

```
    const filter = {}
```

```
    if (challengeId) {
```

```
      filter.challenge = challengeId
```

```
    }
```

```
    const files = await File.find(filter)
```

```
      .populate('challenge', 'title category')
```

```
      .populate('uploadedBy', 'username')
```

```
      .sort({ createdAt: -1 })
```

```
      .limit(limit * 1)
```

```
      .skip((page - 1) * limit)
```

```
    const total = await File.countDocuments(filter)
```

```
    res.json({
```

```
      files,
```

```
      pagination: {
```

```
        total,
```

```
        page: parseInt(page),
```

```

        limit: parseInt(limit),
        pages: Math.ceil(total / limit)
    }
})
} catch (error) {
    console.error('Get admin files error:', error)
    res.status(500).json({ error: 'Internal server error' })
}
})

// Admin: Update file visibility
router.patch('/admin/:fileId', [
    authMiddleware,
    adminMiddleware,
    upload.none()
], async (req, res) => {
    try {
        const { isPublic } = req.body
        const file = await File.findById(req.params.fileId)

        if (!file) {
            return res.status(404).json({ error: 'File not found' })
        }

        file.isPublic = isPublic === 'true'
        await file.save()

        res.json({
            message: 'File updated successfully',

```

```
    file: {
      id: file._id,
      originalName: file.originalName,
      isPublic: file.isPublic
    }
  })
} catch (error) {
  console.error('Update file error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})
```

// Admin: Delete file

```
router.delete('/admin/:fileId', [
  authMiddleware,
  adminMiddleware
], async (req, res) => {
  try {
    const file = await File.findById(req.params.fileId)

    if (!file) {
      return res.status(404).json({ error: 'File not found' })
    }
  }
```

// Delete physical file

```
  try {
    await fs.unlink(file.path)
  } catch (error) {
    console.warn('Could not delete physical file:', error.message)
  }
```

```
}
```

```
// Remove from challenge attachments
```

```
await Challenge.findByIdAndUpdate(file.challenge, {
```

```
  $pull: {
```

```
    attachments: { name: file.originalName }
```

```
  }
```

```
})
```

```
// Delete file record
```

```
await File.findByIdAndDelete(req.params.fileId)
```

```
res.json({ message: 'File deleted successfully' })
```

```
} catch (error) {
```

```
  console.error('Delete file error:', error)
```

```
  res.status(500).json({ error: 'File deletion failed' })
```

```
}
```

```
})
```

```
// Get file statistics
```

```
router.get('/stats', [
```

```
  authMiddleware,
```

```
  adminMiddleware
```

```
], async (req, res) => {
```

```
  try {
```

```
    const stats = await File.aggregate([
```

```
      {
```

```
        $group: {
```

```
          _id: null,
```

```

totalFiles: { $sum: 1 },
totalSize: { $sum: '$size' },
totalDownloads: { $sum: '$downloadCount' },
filesByType: {
  $push: {
    mimetype: '$mimetype',
    size: '$size'
  }
}
},
{
  $project: {
    totalFiles: 1,
    totalSize: 1,
    totalDownloads: 1,
    averageSize: { $round: [{ $divide: ['$totalSize', '$totalFiles'] }, 2] },
    typeBreakdown: {
      $arrayToObject: {
        $map: {
          input: [
            { type: 'images', mimes: ['image/jpeg', 'image/png', 'image/gif'] },
            { type: 'documents', mimes: ['text/plain', 'application/pdf'] },
            { type: 'archives', mimes: ['application/zip', 'application/x-zip-compressed', 'application/x-rar-compressed'] },
            { type: 'code', mimes: ['text/x-python', 'application/x-python-code', 'text/x-c', 'text/x-c++', 'application/javascript', 'text/html', 'text/css', 'application/json'] }
          ],
          as: 'category',

```

```

in: {
  k: '$$category.type',
  v: {
    count: {
      $size: {
        $filter: {
          input: '$filesByType',
          as: 'file',
          cond: { $in: ['$file.mimetype', '$$category.mimes'] }
        }
      }
    },
    size: {
      $sum: {
        $filter: {
          input: '$filesByType',
          as: 'file',
          cond: { $in: ['$file.mimetype', '$$category.mimes'] }
        }.size
      }
    }
  }
}
})

```

```

const defaultStats = {
  totalFiles: 0,
  totalSize: 0,
  totalDownloads: 0,
  averageSize: 0,
  typeBreakdown: {
    images: { count: 0, size: 0 },
    documents: { count: 0, size: 0 },
    archives: { count: 0, size: 0 },
    code: { count: 0, size: 0 }
  }
}

res.json({ stats: stats[0] || defaultStats })
} catch (error) {
  console.error('Get file stats error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Clean up orphaned files (admin utility)
router.post('/admin/cleanup', [
  authMiddleware,
  adminMiddleware
], async (req, res) => {
  try {
    const allFiles = await File.find({})

    let cleanedCount = 0

```



```

for (const file of allFiles) {
  try {
    await fs.access(file.path)
  } catch {
    // File doesn't exist on disk, remove record
    await File.findByIdAndDelete(file._id)
    cleanedCount++
  }
}

res.json({
  message: 'Cleanup completed',
  cleanedCount,
  remainingFiles: allFiles.length - cleanedCount
})
} catch (error) {
  console.error('File cleanup error:', error)
  res.status(500).json({ error: 'Cleanup failed' })
}
})

```

export default router",

hints.js

```

"import express from 'express'
import { body, validationResult } from 'express-validator'
import Hint from '../models/Hint.js'
import UserHint from '../models/UserHint.js'

```

```
import User from '../models/User.js'
import Challenge from '../models/Challenge.js'
import { authMiddleware, adminMiddleware } from '../middleware/auth.js'

const router = express.Router()

// Get available hints for a challenge
router.get('/challenge/:challengeId', authMiddleware, async (req, res) => {
  try {
    const { challengeId } = req.params
    const userId = req.userId

    // Get all hints for the challenge
    const hints = await Hint.find({
      challenge: challengeId,
      isActive: true
    }).sort({ level: 1 })

    // Get user's purchased hints
    const userHints = await UserHint.find({
      user: userId,
      challenge: challengeId
    }).populate('hint')

    // Format response
    const hintsWithPurchaseStatus = hints.map(hint => {
      const purchased = userHints.find(uh => uh.hint._id.toString() === hint._id.toString())
      return {
        id: hint._id,
```

```
    level: hint.level,  
    cost: hint.cost,  
    cooldown: hint.cooldown,  
    purchased: !!purchased,  
    content: purchased ? hint.content : null,  
    purchasedAt: purchased ? purchased.usedAt : null  
  }  
})
```

```
    res.json({ hints: hintsWithPurchaseStatus })  
  } catch (error) {  
    console.error('Get hints error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```
// Purchase a hint  
router.post('/purchase', [  
  authMiddleware,  
  body('hintId').isMongoid(),  
  body('challengeId').isMongoid()  
], async (req, res) => {  
  try {  
    const errors = validationResult(req)  
    if (!errors.isEmpty()) {  
      return res.status(400).json({ errors: errors.array() })  
    }  
  
    const { hintId, challengeId } = req.body
```

```
const userId = req.userId
```

```
// Get hint details
```

```
const hint = await Hint.findOne({  
  _id: hintId,  
  challenge: challengId,  
  isActive: true  
})
```

```
if (!hint) {  
  return res.status(404).json({ error: 'Hint not found' })  
}
```

```
// Check if user already purchased this hint
```

```
const existingPurchase = await UserHint.findOne({  
  user: userId,  
  hint: hintId  
})
```

```
if (existingPurchase) {  
  return res.status(400).json({ error: 'Hint already purchased' })  
}
```

```
// Check cooldown for previous hint purchases
```

```
const lastPurchase = await UserHint.findOne({  
  user: userId,  
  challenge: challengId  
}).sort({ usedAt: -1 })
```

```
if (lastPurchase) {  
  const timeSinceLastPurchase = Date.now() - lastPurchase.usedAt.getTime()  
  const cooldownRemaining = hint.cooldown * 1000 - timeSinceLastPurchase  
  
  if (cooldownRemaining > 0) {  
    const minutes = Math.ceil(cooldownRemaining / (60 * 1000))  
    return res.status(400).json({  
      error: `Hint cooldown active. Please wait ${minutes} minute(s) before purchasing another hint.`  
    })  
  }  
}
```

```
// Check if user has enough points  
const user = await User.findById(userId)  
if (user.score < hint.cost) {  
  return res.status(400).json({  
    error: `Insufficient points. Hint costs ${hint.cost} but you have ${user.score}.`  
  })  
}
```

```
// Check if user has solved the challenge  
const Submission = await import('../models/Submission.js')  
const isSolved = await Submission.default.exists({  
  user: userId,  
  challenge: challengId,  
  isCorrect: true  
})
```

```
if (isSolved) {
```

```
    return res.status(400).json({ error: 'Cannot purchase hints for solved challenges' })  
  }
```

```
  // Purchase the hint  
  const userHint = new UserHint({  
    user: userId,  
    hint: hintId,  
    challenge: challengeId,  
    pointsDeducted: hint.cost  
  })
```

```
  await userHint.save()
```

```
  // Deduct points from user  
  user.score -= hint.cost  
  await user.save()
```

```
  res.json({  
    message: 'Hint purchased successfully',  
    hint: {  
      level: hint.level,  
      content: hint.content,  
      cost: hint.cost,  
      pointsRemaining: user.score  
    }  
  })
```

```
} catch (error) {  
  console.error('Purchase hint error:', error)
```

```
    res.status(500).json({ error: 'Internal server error' })
  }
})
```

```
// Admin: Create hint
```

```
router.post('/admin', [
  authMiddleware,
  adminMiddleware,
  body('challengeId').isMongoId(),
  body('level').isInt({ min: 1, max: 3 }),
  body('content').notEmpty().trim(),
  body('cost').isInt({ min: 0 }),
  body('cooldown').optional().isInt({ min: 0 })
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const { challengeId, level, content, cost, cooldown = 300 } = req.body

    // Check if challenge exists
    const challenge = await Challenge.findById(challengeId)
    if (!challenge) {
      return res.status(404).json({ error: 'Challenge not found' })
    }

    // Check if hint level already exists for this challenge
```

```
const existingHint = await Hint.findOne({
  challenge: challengeld,
  level: level
})
```

```
if (existingHint) {
  return res.status(400).json({ error: `Hint level ${level} already exists for this challenge` })
}
```

```
// Create hint
```

```
const hint = new Hint({
  challenge: challengeld,
  level,
  content,
  cost,
  cooldown
})
```

```
await hint.save()
```

```
res.status(201).json({
  message: 'Hint created successfully',
  hint: {
    id: hint._id,
    challenge: hint.challenge,
    level: hint.level,
    cost: hint.cost,
    cooldown: hint.cooldown
  }
})
```



```

    })

    } catch (error) {
      console.error('Create hint error:', error)
      res.status(500).json({ error: 'Internal server error' })
    }
  })

// Admin: Update hint
router.put('/admin/:hintId', [
  authMiddleware,
  adminMiddleware,
  body('content').optional().trim(),
  body('cost').optional().isInt({ min: 0 }),
  body('cooldown').optional().isInt({ min: 0 }),
  body('isActive').optional().isBoolean()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const hint = await Hint.findById(req.params.hintId)
    if (!hint) {
      return res.status(404).json({ error: 'Hint not found' })
    }

    const { content, cost, cooldown, isActive } = req.body

```

```
const updates = {}

if (content !== undefined) updates.content = content
if (cost !== undefined) updates.cost = cost
if (cooldown !== undefined) updates.cooldown = cooldown
if (isActive !== undefined) updates.isActive = isActive

const updatedHint = await Hint.findByIdAndUpdate(
  req.params.hintId,
  { $set: updates },
  { new: true }
)

res.json({
  message: 'Hint updated successfully',
  hint: updatedHint
})

} catch (error) {
  console.error('Update hint error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Admin: Get all hints for a challenge
router.get('/admin/challenge/:challengeId', [
  authMiddleware,
  adminMiddleware
], async (req, res) => {
```

```

try {
  const hints = await Hint.find({ challenge: req.params.challengeId })
    .populate('challenge', 'title category')
    .sort({ level: 1 })

  // Get purchase statistics for each hint
  const hintsWithStats = await Promise.all(
    hints.map(async (hint) => {
      const purchaseCount = await UserHint.countDocuments({ hint: hint._id })
      const totalRevenue = purchaseCount * hint.cost

      return {
        ...hint.toObject(),
        purchaseCount,
        totalRevenue
      }
    })
  )

  res.json({ hints: hintsWithStats })
} catch (error) {
  console.error('Get admin hints error:', error)
  res.status(500).json({ error: 'Internal server error' })
}

// Get user's hint purchase history
router.get('/history', authMiddleware, async (req, res) => {
  try {

```

```
const { page = 1, limit = 20 } = req.query
```

```
const userHints = await UserHint.find({ user: req.userId })
```

```
  .populate('hint', 'level cost content')
```

```
  .populate('challenge', 'title category points')
```

```
  .sort({ usedAt: -1 })
```

```
  .limit(limit * 1)
```

```
  .skip((page - 1) * limit)
```

```
const total = await UserHint.countDocuments({ user: req.userId })
```

```
const history = userHints.map(uh => ({
```

```
  id: uh._id,
```

```
  challenge: {
```

```
    title: uh.challenge.title,
```

```
    category: uh.challenge.category,
```

```
    points: uh.challenge.points
```

```
  },
```

```
  hint: {
```

```
    level: uh.hint.level,
```

```
    cost: uh.hint.cost,
```

```
    content: uh.hint.content
```

```
  },
```

```
  purchasedAt: uh.usedAt,
```

```
  pointsDeducted: uh.pointsDeducted
```

```
}))
```

```
res.json({
```

```
  history,
```

```

    pagination: {
      total,
      page: parseInt(page),
      limit: parseInt(limit),
      pages: Math.ceil(total / limit)
    }
  })
} catch (error) {
  console.error('Get hint history error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Get hint statistics for user
router.get('/stats', authMiddleware, async (req, res) => {
  try {
    const userId = req.userId

    const stats = await UserHint.aggregate([
      { $match: { user: userId } },
      {
        $lookup: {
          from: 'hints',
          localField: 'hint',
          foreignField: '_id',
          as: 'hint'
        }
      },
      { $unwind: '$hint' },

```

```

{
  $group: {
    _id: null,
    totalHintsPurchased: { $sum: 1 },
    totalPointsSpent: { $sum: '$pointsDeducted' },
    hintsByLevel: {
      $push: {
        level: '$hint.level',
        cost: '$hint.cost'
      }
    }
  },
  {
    $project: {
      totalHintsPurchased: 1,
      totalPointsSpent: 1,
      averageCost: { $round: [{ $divide: ['$totalPointsSpent', '$totalHintsPurchased'] }, 2] },
      levelBreakdown: {
        $arrayToObject: {
          $map: {
            input: [1, 2, 3],
            as: 'level',
            in: {
              k: { $toString: '$$level' },
              v: {
                count: {
                  $size: {
                    $filter: {

```

```

        input: '$hintsByLevel',
        as: 'h',
        cond: { $eq: ['$h.level', '$$level'] }
      }
    }
  }
}
}
}
}
}
}
}
}
}
})

```

```

const defaultStats = {
  totalHintsPurchased: 0,
  totalPointsSpent: 0,
  averageCost: 0,
  levelBreakdown: { '1': { count: 0 }, '2': { count: 0 }, '3': { count: 0 } }
}

```

```

res.json({ stats: stats[0] || defaultStats })
} catch (error) {
  console.error('Get hint stats error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

```

```
export default router",
```

```
leaderboard.js
```

```
"import express from 'express'
```

```
import User from '../models/User.js'
```

```
import Team from '../models/Team.js'
```

```
const router = express.Router()
```

```
// Get user leaderboard with enhanced data
```

```
router.get('/users', async (req, res) => {
```

```
  try {
```

```
    const { limit = 50, offset = 0 } = req.query
```

```
    const leaderboard = await User.aggregate([
```

```
      {
```

```
        $match: {
```

```
          role: 'participant',
```

```
          isActive: true
```

```
        }
```

```
      },
```

```
      {
```

```
        $lookup: {
```

```
          from: 'submissions',
```

```
          let: { userId: '$_id' },
```

```
          pipeline: [
```

```
            { $match: { $expr: { $eq: ['$user', '$$userId'] }, isCorrect: true } },
```

```
            { $count: 'count' }
```

```
          ],
```



```

      as: 'submissionCount'
    }
  },
  {
    $lookup: {
      from: 'achievements',
      let: { userId: '$_id' },
      pipeline: [
        { $match: { $expr: { $eq: ['$user', '$$userId'] } } },
        { $count: 'count' }
      ],
      as: 'achievementCount'
    }
  },
  {
    $lookup: {
      from: 'teams',
      localField: 'team',
      foreignField: '_id',
      as: 'teamInfo'
    }
  },
  {
    $project: {
      username: 1,
      fullName: 1,
      score: 1,
      solvedChallenges: 1,
      solvedCount: { $size: '$solvedChallenges' },

```

```

    submissionCount: { $arrayElemAt: ['$submissionCount.count', 0] },
    achievementCount: { $arrayElemAt: ['$achievementCount.count', 0] },
    team: { $arrayElemAt: ['$teamInfo.name', null] },
    lastSolve: 1,
    streak: 1,
    statistics: 1,
    createdAt: 1
  }
},
{
  $sort: {
    score: -1,
    lastSolve: 1,
    createdAt: 1
  }
},
{
  $skip: parseInt(offset)
},
{
  $limit: parseInt(limit)
}
])

```

// Add ranks

```

const leaderboardWithRank = leaderboard.map((user, index) => ({
  rank: parseInt(offset) + index + 1,
  username: user.username,
  fullName: user.fullName,

```

```
    score: user.score || 0,  
    solvedCount: user.solvedCount || 0,  
    submissionCount: user.submissionCount || 0,  
    achievementCount: user.achievementCount || 0,  
    team: user.team,  
    lastSolve: user.lastSolve,  
    streak: user.streak?.current || 0,  
    favoriteCategory: user.statistics?.favoriteCategory,  
    joinDate: user.createdAt  
  )))
```

```
// Get total count for pagination
```

```
const totalUsers = await User.countDocuments({  
  role: 'participant',  
  isActive: true  
})
```

```
res.json({  
  leaderboard: leaderboardWithRank,  
  pagination: {  
    total: totalUsers,  
    limit: parseInt(limit),  
    offset: parseInt(offset),  
    hasMore: (parseInt(offset) + leaderboard.length) < totalUsers  
  }  
})
```

```
} catch (error) {  
  console.error('User leaderboard error:', error)  
  res.status(500).json({ error: 'Internal server error' })  
}
```

```
}  
})
```

```
// Get team leaderboard
```

```
router.get('/teams', async (req, res) => {  
  try {  
    const teams = await Team.aggregate([  
      {  
        $match: { isActive: true }  
      },  
      {  
        $lookup: {  
          from: 'users',  
          localField: 'members',  
          foreignField: '_id',  
          as: 'memberDetails'  
        }  
      },  
      {  
        $addFields: {  
          memberCount: { $size: '$members' },  
          totalSolves: { $size: '$solvedChallenges' },  
          averageScore: { $divide: ['$score', { $size: '$members' }] }  
        }  
      },  
      {  
        $project: {  
          name: 1,  
          description: 1,
```

```

    score: 1,
    memberCount: 1,
    totalSolves: 1,
    averageScore: 1,
    captain: 1,
    members: {
      $map: {
        input: '$memberDetails',
        as: 'member',
        in: {
          username: '$$member.username',
          score: '$$member.score'
        }
      }
    },
    createdAt: 1
  }
},
{
  $sort: {
    score: -1,
    averageScore: -1,
    createdAt: 1
  }
}
])

```

```

const teamsWithRank = teams.map((team, index) => ({
  rank: index + 1,

```

```
    name: team.name,  
    description: team.description,  
    score: team.score,  
    memberCount: team.memberCount,  
    totalSolves: team.totalSolves,  
    averageScore: Math.round(team.averageScore),  
    members: team.members,  
    createdAt: team.createdAt  
  )))
```

```
    res.json(teamsWithRank)  
  } catch (error) {  
    console.error('Team leaderboard error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

// Get category-based leaderboard

```
router.get('/categories/:category', async (req, res) => {  
  try {  
    const { category } = req.params  
    const { limit = 20 } = req.query  
  
    const categoryLeaderboard = await User.aggregate([  
      {  
        $match: {  
          role: 'participant',  
          isActive: true  
        }  
      }  
    ])
```

```

},
{
  $lookup: {
    from: 'submissions',
    let: { userId: '$_id' },
    pipeline: [
      { $match: { $expr: { $eq: ['$user', '$$userId'] }, isCorrect: true } },
      {
        $lookup: {
          from: 'challenges',
          localField: 'challenge',
          foreignField: '_id',
          as: 'challenge'
        }
      },
      { $unwind: '$challenge' },
      { $match: { 'challenge.category': category } }
    ],
    as: 'categorySubmissions'
  }
},
{
  $addFields: {
    categoryScore: {
      $sum: '$categorySubmissions.challenge.points'
    },
    categorySolves: {
      $size: '$categorySubmissions'
    }
  }
}

```

```

    }
  },
  {
    $match: {
      categorySolves: { $gt: 0 }
    }
  },
  {
    $project: {
      username: 1,
      fullName: 1,
      categoryScore: 1,
      categorySolves: 1,
      totalScore: '$score'
    }
  },
  {
    $sort: {
      categoryScore: -1,
      categorySolves: -1
    }
  },
  {
    $limit: parseInt(limit)
  }
])

```

```

const leaderboardWithRank = categoryLeaderboard.map((user, index) => ({
  rank: index + 1,

```



```
    username: user.username,  
    fullName: user.fullName,  
    categoryScore: user.categoryScore,  
    categorySolves: user.categorySolves,  
    totalScore: user.totalScore  
  )))
```

```
    res.json(leaderboardWithRank)  
  } catch (error) {  
    console.error('Category leaderboard error:', error)  
    res.status(500).json({ error: 'Internal server error' })  
  }  
})
```

```
// Get badge leaderboard  
router.get('/badges', async (req, res) => {  
  try {  
    const badgeLeaderboard = await User.aggregate([  
      {  
        $match: {  
          role: 'participant',  
          isActive: true  
        }  
      },  
      {  
        $project: {  
          username: 1,  
          fullName: 1,  
          score: 1,
```

```

        badgeCount: { $size: '$badges' },
        badges: 1
    }
},
{
    $sort: {
        badgeCount: -1,
        score: -1
    }
},
{
    $limit: 20
}
])

```

```

const leaderboardWithRank = badgeLeaderboard.map((user, index) => ({
    rank: index + 1,
    username: user.username,
    fullName: user.fullName,
    badgeCount: user.badgeCount,
    score: user.score
}))

```

```

res.json(leaderboardWithRank)
} catch (error) {
    console.error('Badge leaderboard error:', error)
    res.status(500).json({ error: 'Internal server error' })
}
})

```

```
export default router"
```

```
routes/
```

```
seuresession.js
```

```
"import express from 'express'
```

```
import { body, validationResult } from 'express-validator'
```

```
import TestSession from '../models/TestSession.js'
```

```
import Challenge from '../models/Challenge.js'
```

```
import User from '../models/User.js'
```

```
import { authMiddleware } from '../middleware/auth.js'
```

```
import securityMonitor from '../services/securityMonitor.js'
```

```
import { randomBytes } from 'crypto'
```

```
const router = express.Router()
```

```
// Initialize secure test session
```

```
router.post('/initialize', [
```

```
  authMiddleware,
```

```
  body('challengeId').isMongoid(),
```

```
  body('eventId').optional().isMongoid()
```

```
], async (req, res) => {
```

```
  try {
```

```
    const errors = validationResult(req)
```

```
    if (!errors.isEmpty()) {
```

```
      return res.status(400).json({ errors: errors.array() })
```

```
    }
```

```
    const { challengeId, eventId } = req.body
```

```
const userId = req.userId
```

```
// Check if challenge exists and is active
```

```
const challenge = await Challenge.findById(challengeId)
```

```
if (!challenge || !challenge.isActive) {
```

```
  return res.status(404).json({ error: 'Challenge not found or inactive' })
```

```
}
```

```
// Check if user has active session
```

```
const existingSession = await TestSession.findOne({
```

```
  user: userId,
```

```
  status: 'active'
```

```
})
```

```
if (existingSession) {
```

```
  return res.status(400).json({ error: 'You already have an active test session' })
```

```
}
```

```
// Generate unique session ID
```

```
const sessionId = randomBytes(16).toString('hex')
```

```
// Create test session
```

```
const testSession = new TestSession({
```

```
  user: userId,
```

```
  challenge: challengeId,
```

```
  event: eventId,
```

```
  sessionId,
```

```
  userAgent: req.get('User-Agent'),
```

```
  ipAddress: req.ip,
```

```
security: {  
  webcamEnabled: true,  
  screenRecording: true,  
  tabFocus: true,  
  clipboardDisabled: true,  
  devToolsDisabled: true  
}  
})
```

```
await testSession.save()
```

```
// Get security configuration
```

```
const securityConfig = await securityMonitor.getSecurityConfig()
```

```
// Initialize security monitoring
```

```
await securityMonitor.initializeSession(sessionId, userId, challengeId, securityConfig)
```

```
res.json({  
  message: 'Secure session initialized successfully',  
  session: {  
    sessionId,  
    challenge: {  
      title: challenge.title,  
      category: challenge.category,  
      points: challenge.points  
    },  
    security: testSession.security,  
    monitoring: securityConfig.monitoring,  
    requirements: {
```

```

        webcam: securityConfig.webcamMonitoring.required,
        fullscreen: securityConfig.browserRestrictions.fullScreenRequired,
        noTabSwitch: securityConfig.browserRestrictions.restrictTabSwitching
    }
}
})

} catch (error) {
    console.error('Initialize secure session error:', error)
    res.status(500).json({ error: 'Failed to initialize secure session' })
}
})

// Verify webcam access and start monitoring
router.post('/:sessionId/verify-webcam', [
    authMiddleware,
    body('streamData').optional()
], async (req, res) => {
    try {
        const { sessionId } = req.params
        const { streamData } = req.body

        const session = await TestSession.findOne({ sessionId, user: req.userId })
        if (!session) {
            return res.status(404).json({ error: 'Session not found' })
        }

        if (session.status !== 'active') {
            return res.status(400).json({ error: 'Session is not active' })
        }
    }
})

```

```

    }

    // Update webcam status
    session.security.webcamEnabled = true
    await session.save()

    res.json({
      message: 'Webcam verified successfully',
      webcamStatus: 'active',
      monitoring: {
        faceDetection: true,
        multipleFaceDetection: true,
        frameRate: 1
      }
    })

  } catch (error) {
    console.error('Webcam verification error:', error)
    res.status(500).json({ error: 'Webcam verification failed' })
  }
})

// Report security violation from frontend
router.post('/:sessionId/violation', [
  authMiddleware,
  body('type').isIn(['tab_switch', 'focus_loss', 'clipboard', 'dev_tools', 'webcam_issue']),
  body('data').isObject(),
  body('screenshot').optional().isString()
], async (req, res) => {

```

```
try {  
  const errors = validationResult(req)  
  if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() })  
  }  
  
  const { sessionId } = req.params  
  const { type, data, screenshot } = req.body  
  
  const session = await TestSession.findOne({ sessionId, user: req.userId })  
  if (!session) {  
    return res.status(404).json({ error: 'Session not found' })  
  }  
  
  // Handle different violation types  
  switch (type) {  
    case 'tab_switch':  
      await session.monitoring.tabSwitches.push({  
        timestamp: new Date(),  
        count: data.count || 1,  
        urls: data.urls || []  
      })  
      break  
  
    case 'focus_loss':  
      await session.monitoring.focusEvents.push({  
        timestamp: new Date(),  
        type: 'blur',  
        duration: data.duration || 0
```



```

    })
    break

    case 'webcam_issue':
        await session.monitoring.webcamAlerts.push({
            timestamp: new Date(),
            type: data.alertType || 'unknown',
            confidence: data.confidence || 0.5,
            screenshot: screenshot
        })
        break
    }

    await session.save()

    // Check if violation requires immediate action
    const securityConfig = await securityMonitor.getSecurityConfig()
    await checkViolationThreshold(session, securityConfig)

    res.json({
        message: 'Violation recorded',
        action: 'recorded'
    })

} catch (error) {
    console.error('Violation reporting error:', error)
    res.status(500).json({ error: 'Failed to record violation' })
}
})

```

```
// Get session status

router.get('/:sessionId/status', authMiddleware, async (req, res) => {

  try {

    const { sessionId } = req.params

    const session = await TestSession.findOne({ sessionId, user: req.userId })

    if (!session) {

      return res.status(404).json({ error: 'Session not found' })

    }

    const status = await securityMonitor.getSessionStatus(sessionId)

    res.json({

      session: {

        id: session._id,

        sessionId: session.sessionId,

        status: session.status,

        startTime: session.startTime,

        duration: session.duration,

        violations: session.violations.length,

        security: session.security

      },

      monitoring: status

    })

  } catch (error) {

    console.error('Get session status error:', error)

    res.status(500).json({ error: 'Failed to get session status' })

  }

})
```

```
}  
})
```

```
// Terminate session
```

```
router.post('/:sessionId/terminate', authMiddleware, async (req, res) => {
```

```
  try {
```

```
    const { sessionId } = req.params
```

```
    const { reason } = req.body
```

```
    const session = await securityMonitor.terminateSession(sessionId, reason || 'Manual termination')
```

```
    res.json({
```

```
      message: 'Session terminated successfully',
```

```
      session: {
```

```
        status: session.status,
```

```
        endTime: session.endTime,
```

```
        duration: session.duration
```

```
      }  
    })
```

```
  } catch (error) {
```

```
    console.error('Terminate session error:', error)
```

```
    res.status(500).json({ error: 'Failed to terminate session' })
```

```
  }  
})
```

```
// Admin: Get all active sessions
```

```
router.get('/admin/active-sessions', [  
  authMiddleware,
```

```

async (req, res) => {
  try {
    const activeSessions = await TestSession.find({ status: 'active' })
      .populate('user', 'username fullName email')
      .populate('challenge', 'title category')
      .populate('event', 'name')
      .sort({ startTime: -1 })

    const sessionsWithDetails = activeSessions.map(session => ({
      id: session._id,
      sessionId: session.sessionId,
      user: session.user,
      challenge: session.challenge,
      event: session.event,
      startTime: session.startTime,
      duration: Math.floor((new Date() - session.startTime) / 1000),
      violations: session.violations.length,
      webcamAlerts: session.monitoring.webcamAlerts.length,
      tabSwitches: session.monitoring.tabSwitches.reduce((sum, ts) => sum + ts.count, 0),
      ipAddress: session.ipAddress
    }))

    res.json({ sessions: sessionsWithDetails })
  } catch (error) {
    console.error('Get active sessions error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
}
})

```

```

// Admin: Force terminate session
router.post('/admin/:sessionId/force-terminate', [
  authMiddleware,
  body('reason').optional().trim()
], async (req, res) => {
  try {
    const { sessionId } = req.params
    const { reason } = req.body

    const session = await TestSession.findOne({ sessionId })
    if (!session) {
      return res.status(404).json({ error: 'Session not found' })
    }

    const terminatedSession = await securityMonitor.terminateSession(
      sessionId,
      reason || 'Admin forced termination'
    )

    res.json({
      message: 'Session force terminated',
      session: {
        user: terminatedSession.user,
        status: terminatedSession.status,
        endTime: terminatedSession.endTime
      }
    })
  }
})

```

```

    } catch (error) {
      console.error('Force terminate session error:', error)
      res.status(500).json({ error: 'Failed to force terminate session' })
    }
  })

// Helper function to check violation thresholds
async function checkViolationThreshold(session, config) {
  const webcamAlerts = session.monitoring.webcamAlerts.length
  const tabSwitches = session.monitoring.tabSwitches.reduce((sum, ts) => sum + ts.count, 0)

  let shouldTerminate = false
  let reason = ''

  if (webcamAlerts >= config.violationThresholds.maxWebcamAlerts) {
    shouldTerminate = true
    reason = 'Exceeded maximum webcam alerts'
  }

  if (tabSwitches >= config.violationThresholds.maxTabSwitches) {
    shouldTerminate = true
    reason = 'Exceeded maximum tab switches'
  }

  if (shouldTerminate) {
    await securityMonitor.terminateSession(session.sessionId, reason)
  }
}

```

```
export default router",
```

```
teams.js
```

```
"import express from 'express'
```

```
import { body, validationResult } from 'express-validator'
```

```
import Team from '../models/Team.js'
```

```
import User from '../models/User.js'
```

```
import { authMiddleware } from '../middleware/auth.js'
```

```
import socketService from '../services/socketService.js'
```

```
const router = express.Router()
```

```
// Get all teams
```

```
router.get('/', async (req, res) => {
```

```
  try {
```

```
    const { search = "", page = 1, limit = 20 } = req.query
```

```
    const filter = { isActive: true }
```

```
    if (search) {
```

```
      filter.name = { $regex: search, $options: 'i' }
```

```
    }
```

```
    const teams = await Team.find(filter)
```

```
      .populate('captain', 'username fullName')
```

```
      .populate('members', 'username fullName score')
```

```
      .select('name description score members captain solvedChallenges createdAt')
```

```
      .sort({ score: -1, createdAt: 1 })
```

```
      .limit(limit * 1)
```

```
      .skip((page - 1) * limit)
```

```
const total = await Team.countDocuments(filter)
```

```
// Add member count and statistics
```

```
const teamsWithStats = teams.map(team => ({  
  id: team._id,  
  name: team.name,  
  description: team.description,  
  score: team.score,  
  memberCount: team.members.length,  
  captain: team.captain,  
  members: team.members,  
  solvedCount: team.solvedChallenges.length,  
  createdAt: team.createdAt,  
  averageScore: team.members.length > 0 ? Math.round(team.score / team.members.length) : 0  
}))
```

```
res.json({  
  teams: teamsWithStats,  
  pagination: {  
    total,  
    page: parseInt(page),  
    limit: parseInt(limit),  
    pages: Math.ceil(total / limit)  
  }  
})
```

```
} catch (error) {  
  console.error('Get teams error:', error)  
  res.status(500).json({ error: 'Internal server error' })  
}
```



```
}  
})
```

```
// Get single team details
```

```
router.get('/:teamId', async (req, res) => {  
  try {  
    const team = await Team.findById(req.params.teamId)  
      .populate('captain', 'username fullName email score')  
      .populate('members', 'username fullName score solvedChallenges lastSolve')  
      .populate('solvedChallenges.challenge', 'title category points')  
  
    if (!team || !team.isActive) {  
      return res.status(404).json({ error: 'Team not found' })  
    }  
  }  
}
```

```
// Calculate team statistics
```

```
const memberStats = {  
  totalMembers: team.members.length,  
  totalScore: team.score,  
  averageScore: Math.round(team.score / team.members.length),  
  totalSolves: team.solvedChallenges.length,  
  activeMembers: team.members.filter(member =>  
    member.lastSolve > new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)  
  ).length  
}
```

```
// Category breakdown
```

```
const categoryStats = {}  
team.solvedChallenges.forEach(solve => {
```

```

    const category = solve.challenge.category
    categoryStats[category] = (categoryStats[category] || 0) + 1
  })

  res.json({
    team: {
      id: team._id,
      name: team.name,
      description: team.description,
      captain: team.captain,
      members: team.members,
      solvedChallenges: team.solvedChallenges,
      createdAt: team.createdAt,
      statistics: memberStats,
      categoryStats
    }
  })
} catch (error) {
  console.error('Get team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Create new team
router.post('/', [
  authMiddleware,
  body('name').notEmpty().trim().isLength({ min: 3, max: 50 }),
  body('description').optional().trim().isLength({ max: 200 })
], async (req, res) => {

```

```
try {  
  const errors = validationResult(req)  
  if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() })  
  }  
  
  const { name, description } = req.body  
  const userId = req.userId  
  
  // Check if user already in a team  
  const existingTeam = await Team.findOne({ members: userId })  
  if (existingTeam) {  
    return res.status(400).json({ error: 'You are already in a team' })  
  }  
  
  // Check if team name exists  
  const nameExists = await Team.findOne({ name: { $regex: new RegExp(`^${name}$`, 'i') } })  
  if (nameExists) {  
    return res.status(400).json({ error: 'Team name already exists' })  
  }  
  
  // Create team  
  const team = new Team({  
    name,  
    description: description || '',  
    captain: userId,  
    members: [userId]  
  })
```

```
await team.save()

// Update user's team reference
await User.findByIdAndUpdate(userId, { team: team._id })

// Populate for response
await team.populate('captain', 'username fullName')
await team.populate('members', 'username fullName')

res.status(201).json({
  message: 'Team created successfully',
  team: {
    id: team._id,
    name: team.name,
    description: team.description,
    captain: team.captain,
    members: team.members,
    score: team.score,
    createdAt: team.createdAt
  }
})
} catch (error) {
  console.error('Create team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Update team
router.put('/:teamId', [
```

```

authMiddleware,
body('description').optional().trim().isLength({ max: 200 }),
body('name').optional().trim().isLength({ min: 3, max: 50 })
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const team = await Team.findById(req.params.teamId)
    if (!team) {
      return res.status(404).json({ error: 'Team not found' })
    }

    // Check if user is captain
    if (team.captain.toString() !== req.userId) {
      return res.status(403).json({ error: 'Only team captain can update team' })
    }

    const { name, description } = req.body
    const updates = {}

    if (name && name !== team.name) {
      // Check if new name is available
      const nameExists = await Team.findOne({
        name: { $regex: new RegExp(`^${name}$`, 'i') },
        _id: { $ne: team._id }
      })
    }

```

```
    if (nameExists) {
      return res.status(400).json({ error: 'Team name already exists' })
    }
    updates.name = name
  }

  if (description !== undefined) {
    updates.description = description
  }

  const updatedTeam = await Team.findByIdAndUpdate(
    req.params.teamId,
    { $set: updates },
    { new: true }
  ).populate('captain', 'username fullName')
    .populate('members', 'username fullName score')

  res.json({
    message: 'Team updated successfully',
    team: updatedTeam
  })
} catch (error) {
  console.error('Update team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Join team
router.post('/:teamId/join', authMiddleware, async (req, res) => {
```

```
try {  
  const team = await Team.findById(req.params.teamId)  
  if (!team || !team.isActive) {  
    return res.status(404).json({ error: 'Team not found' })  
  }  
  
  // Check if team is full (max 4 members)  
  if (team.members.length >= 4) {  
    return res.status(400).json({ error: 'Team is full' })  
  }  
  
  // Check if user already in a team  
  const user = await User.findById(req.userId)  
  if (user.team) {  
    return res.status(400).json({ error: 'You are already in a team' })  
  }  
  
  // Add user to team  
  team.members.push(req.userId)  
  await team.save()  
  
  // Update user's team reference  
  await User.findByIdAndUpdate(req.userId, { team: team._id })  
  
  // Notify team members  
  socketService.sendNotification(team.captain.toString(), {  
    type: 'team_join',  
    title: 'New Team Member',  
    message: `${user.username} has joined your team`,
```

```

    team: team.name
  })

  res.json({
    message: 'Successfully joined team',
    team: {
      id: team._id,
      name: team.name,
      memberCount: team.members.length
    }
  })
} catch (error) {
  console.error('Join team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Leave team
router.post('/:teamId/leave', authMiddleware, async (req, res) => {
  try {
    const team = await Team.findById(req.params.teamId)
    if (!team) {
      return res.status(404).json({ error: 'Team not found' })
    }

    // Check if user is in the team
    if (!team.members.includes(req.userId)) {
      return res.status(400).json({ error: 'You are not in this team' })
    }
  }
})

```



```

// Check if user is captain
if (team.captain.toString() === req.userId) {
  return res.status(400).json({ error: 'Captain cannot leave team. Transfer ownership first or disband team.' })
}

// Remove user from team
team.members = team.members.filter(member => member.toString() !== req.userId)
await team.save()

// Remove user's team reference
await User.findByIdAndUpdate(req.userId, { $unset: { team: 1 } })

res.json({ message: 'Successfully left team' })
} catch (error) {
  console.error('Leave team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Transfer captaincy
router.post('/:teamId/transfer', [
  authMiddleware,
  body('newCaptainId').isMongoid()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {

```

```
    return res.status(400).json({ errors: errors.array() })
  }

  const { newCaptainId } = req.body
  const team = await Team.findById(req.params.teamId)

  if (!team) {
    return res.status(404).json({ error: 'Team not found' })
  }

  // Check if current user is captain
  if (team.captain.toString() !== req.userId) {
    return res.status(403).json({ error: 'Only captain can transfer ownership' })
  }

  // Check if new captain is in the team
  if (!team.members.includes(newCaptainId)) {
    return res.status(400).json({ error: 'New captain must be a team member' })
  }

  // Transfer captaincy
  team.captain = newCaptainId
  await team.save()

  // Notify new captain
  socketService.sendNotification(newCaptainId, {
    type: 'team_promotion',
    title: 'Team Captain',
    message: `You are now the captain of ${team.name}`,
```

```

    team: team.name
  })

  res.json({ message: 'Captaincy transferred successfully' })
} catch (error) {
  console.error('Transfer captaincy error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Disband team (captain only)
router.delete('/:teamId', authMiddleware, async (req, res) => {
  try {
    const team = await Team.findById(req.params.teamId)
    if (!team) {
      return res.status(404).json({ error: 'Team not found' })
    }

    // Check if user is captain
    if (team.captain.toString() !== req.userId) {
      return res.status(403).json({ error: 'Only captain can disband team' })
    }

    // Remove team reference from all members
    await User.updateMany(
      { _id: { $in: team.members } },
      { $unset: { team: 1 } }
    )
  }
})

```

```
// Delete team
await Team.findByIdAndDelete(req.params.teamId)

res.json({ message: 'Team disbanded successfully' })
} catch (error) {
  console.error('Disband team error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Get team members
router.get('/:teamId/members', async (req, res) => {
  try {
    const team = await Team.findById(req.params.teamId)
      .populate('members', 'username fullName score solvedChallenges lastSolve createdAt')
      .select('members')

    if (!team) {
      return res.status(404).json({ error: 'Team not found' })
    }

    // Add member statistics
    const membersWithStats = team.members.map(member => ({
      id: member._id,
      username: member.username,
      fullName: member.fullName,
      score: member.score,
      solvedCount: member.solvedChallenges.length,
      lastSolve: member.lastSolve,
```

```

      joinDate: member.createdAt,
      isCaptain: team.captain.toString() === member._id.toString()
    ))

    res.json({ members: membersWithStats })
  } catch (error) {
    console.error('Get team members error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})

```

```
export default router",
```

```
terminal.js
```

```

"import express from 'express'
import { body, validationResult } from 'express-validator'
import Challenge from '../models/Challenge.js'
import User from '../models/User.js'
import { executeCommand } from '../utils/commandExecutor.js'
import { authMiddleware } from '../middleware/auth.js'

```

```
const router = express.Router()
```

```
// Terminal command execution
```

```

router.post('/command', [
  authMiddleware,
  body('command').notEmpty().trim(),
  body('challengeId').isMongoid()
], async (req, res) => {

```

```
try {  
  const errors = validationResult(req)  
  if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() })  
  }  
  
  const { command, challengeId } = req.body  
  const userId = req.userId  
  
  // Get challenge details  
  const challenge = await Challenge.findById(challengeId)  
  if (!challenge) {  
    return res.status(404).json({ error: 'Challenge not found' })  
  }  
  
  // Get user details  
  const user = await User.findById(userId)  
  if (!user) {  
    return res.status(404).json({ error: 'User not found' })  
  }  
  
  // Execute command  
  const result = await executeCommand(command, challenge, user)  
  
  res.json({  
    success: true,  
    output: result.output,  
    files: result.files,  
    solved: result.solved || false,  
  })  
}
```

```

    points: result.points || 0
  })

} catch (error) {
  console.error('Terminal command error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

export default router",

routes/writeup.js
"import express from 'express'
import { body, validationResult } from 'express-validator'
import Writeup from '../models/Writeup.js'
import Challenge from '../models/Challenge.js'
import User from '../models/User.js'
import { authMiddleware } from '../middleware/auth.js'

const router = express.Router()

// Get writeups for a challenge
router.get('/challenge/:challengeId', async (req, res) => {
  try {
    const { challengeId } = req.params
    const { page = 1, limit = 20, sort = 'rating' } = req.query

    // Check if challenge exists
    const challenge = await Challenge.findById(challengeId)

```

```
if (!challenge) {  
  return res.status(404).json({ error: 'Challenge not found' })  
}
```

```
const sortOptions = {  
  rating: { rating: -1, createdAt: -1 },  
  recent: { createdAt: -1 },  
  views: { views: -1 }  
}
```

```
const writeups = await Writeup.find({  
  challenge: challengeld,  
  isPublic: true  
})  
  .populate('user', 'username fullName')  
  .select('title content rating views votes tags createdAt')  
  .sort(sortOptions[sort] || sortOptions.rating)  
  .limit(limit * 1)  
  .skip((page - 1) * limit)
```

```
const total = await Writeup.countDocuments({  
  challenge: challengeld,  
  isPublic: true  
})
```

```
// Calculate vote counts for each writeup
```

```
const writeupsWithVotes = writeups.map(writeup => {  
  const upvotes = writeup.votes.filter(v => v.type === 'up').length  
  const downvotes = writeup.votes.filter(v => v.type === 'down').length
```



```

return {
  ...writeup.toObject(),
  upvotes,
  downvotes,
  userVote: null // Will be set if user is authenticated
}
})

res.json({
  writeups: writeupsWithVotes,
  challenge: {
    title: challenge.title,
    category: challenge.category,
    difficulty: challenge.difficulty
  },
  pagination: {
    total,
    page: parseInt(page),
    limit: parseInt(limit),
    pages: Math.ceil(total / limit)
  }
})

} catch (error) {
  console.error('Get writeups error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

```

```

// Get single writeup
router.get('/:writeupId', async (req, res) => {
  try {
    const writeup = await Writeup.findById(req.params.writeupId)
      .populate('user', 'username fullName')
      .populate('challenge', 'title category difficulty points')

    if (!writeup) {
      return res.status(404).json({ error: 'Writeup not found' })
    }

    // Increment view count
    writeup.views += 1
    await writeup.save()

    const upvotes = writeup.votes.filter(v => v.type === 'up').length
    const downvotes = writeup.votes.filter(v => v.type === 'down').length

    // Check if user has voted (if authenticated)
    let userVote = null
    if (req.userId) {
      const userVoteObj = writeup.votes.find(v => v.user.toString() === req.userId)
      userVote = userVoteObj ? userVoteObj.type : null
    }

    res.json({
      writeup: {
        id: writeup._id,
        title: writeup.title,

```

```

    content: writeup.content,
    user: writeup.user,
    challenge: writeup.challenge,
    rating: writeup.rating,
    views: writeup.views,
    upvotes,
    downvotes,
    userVote,
    tags: writeup.tags,
    isPublic: writeup.isPublic,
    createdAt: writeup.createdAt,
    updatedAt: writeup.updatedAt
  }
})
} catch (error) {
  console.error('Get writeup error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Create writeup
router.post('/', [
  authMiddleware,
  body('challengeId').isMongoId(),
  body('title').notEmpty().trim().isLength({ max: 100 }),
  body('content').notEmpty().trim().isLength({ min: 100 }),
  body('tags').optional().isArray(),
  body('isPublic').optional().isBoolean()
], async (req, res) => {

```

```
try {  
  const errors = validationResult(req)  
  if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() })  
  }  
  
  const { challengeId, title, content, tags = [], isPublic = false } = req.body  
  const userId = req.userId  
  
  // Check if challenge exists  
  const challenge = await Challenge.findById(challengeId)  
  if (!challenge) {  
    return res.status(404).json({ error: 'Challenge not found' })  
  }  
  
  // Check if user has solved the challenge  
  const Submission = await import('../models/Submission.js')  
  const hasSolved = await Submission.default.exists({  
    user: userId,  
    challenge: challengeId,  
    isCorrect: true  
  })  
  
  if (!hasSolved) {  
    return res.status(403).json({ error: 'You must solve the challenge before writing a writeup' })  
  }  
  
  // Check if user already has a writeup for this challenge  
  const existingWriteup = await Writeup.findOne({
```

```
    user: userId,  
    challenge: challengeId  
  })
```

```
  if (existingWriteup) {  
    return res.status(400).json({ error: 'You already have a writeup for this challenge' })  
  }
```

```
  // Create writeup  
  const writeup = new Writeup({  
    challenge: challengeId,  
    user: userId,  
    title,  
    content,  
    tags,  
    isPublic  
  })
```

```
  await writeup.save()
```

```
  // Populate for response  
  await writeup.populate('user', 'username fullName')  
  await writeup.populate('challenge', 'title category')
```

```
  res.status(201).json({  
    message: 'Writeup created successfully',  
    writeup: {  
      id: writeup._id,  
      title: writeup.title,
```

```

    content: writeup.content,
    user: writeup.user,
    challenge: writeup.challenge,
    tags: writeup.tags,
    isPublic: writeup.isPublic,
    createdAt: writeup.createdAt
  }
})
} catch (error) {
  console.error('Create writeup error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Update writeup
router.put('/:writeupId', [
  authMiddleware,
  body('title').optional().trim().isLength({ max: 100 }),
  body('content').optional().trim().isLength({ min: 100 }),
  body('tags').optional().isArray(),
  body('isPublic').optional().isBoolean()
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const writeup = await Writeup.findById(req.params.writeupId)

```

```
if (!writeup) {
  return res.status(404).json({ error: 'Writeup not found' })
}

// Check if user owns the writeup
if (writeup.user.toString() !== req.userId) {
  return res.status(403).json({ error: 'You can only edit your own writeups' })
}

const { title, content, tags, isPublic } = req.body
const updates = {}

if (title !== undefined) updates.title = title
if (content !== undefined) updates.content = content
if (tags !== undefined) updates.tags = tags
if (isPublic !== undefined) updates.isPublic = isPublic

const updatedWriteup = await Writeup.findByIdAndUpdate(
  req.params.writeupId,
  { $set: updates },
  { new: true }
).populate('user', 'username fullName')
  .populate('challenge', 'title category')

res.json({
  message: 'Writeup updated successfully',
  writeup: updatedWriteup
})
} catch (error) {
```

```
    console.error('Update writeup error:', error)
    res.status(500).json({ error: 'Internal server error' })
  }
})
```

```
// Delete writeup
```

```
router.delete('/:writeupId', authMiddleware, async (req, res) => {
  try {
    const writeup = await Writeup.findById(req.params.writeupId)
    if (!writeup) {
      return res.status(404).json({ error: 'Writeup not found' })
    }
  }
```

```
  // Check if user owns the writeup
```

```
  if (writeup.user.toString() !== req.userId) {
    return res.status(403).json({ error: 'You can only delete your own writeups' })
  }
```

```
  await Writeup.findByIdAndDelete(req.params.writeupId)
```

```
  res.json({ message: 'Writeup deleted successfully' })
```

```
} catch (error) {
```

```
  console.error('Delete writeup error:', error)
```

```
  res.status(500).json({ error: 'Internal server error' })
```

```
}
```

```
})
```

```
// Vote on writeup
```

```
router.post('/:writeupId/vote', [
```



```
authMiddleware,
body('type').isIn(['up', 'down']))
], async (req, res) => {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() })
    }

    const { type } = req.body
    const writeup = await Writeup.findById(req.params.writeupId)

    if (!writeup) {
      return res.status(404).json({ error: 'Writeup not found' })
    }

    // Check if user owns the writeup
    if (writeup.user.toString() === req.userId) {
      return res.status(400).json({ error: 'You cannot vote on your own writeup' })
    }

    // Remove existing vote
    writeup.votes = writeup.votes.filter(vote => vote.user.toString() !== req.userId)

    // Add new vote
    writeup.votes.push({
      user: req.userId,
      type: type
    })
  }
}
```

```

// Recalculate rating
const upvotes = writeup.votes.filter(v => v.type === 'up').length
const downvotes = writeup.votes.filter(v => v.type === 'down').length
writeup.rating = upvotes - downvotes

await writeup.save()

res.json({
  message: 'Vote recorded successfully',
  rating: writeup.rating,
  upvotes,
  downvotes,
  userVote: type
})
} catch (error) {
  console.error('Vote error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})

// Remove vote
router.delete('/:writeupId/vote', authMiddleware, async (req, res) => {
  try {
    const writeup = await Writeup.findById(req.params.writeupId)

    if (!writeup) {
      return res.status(404).json({ error: 'Writeup not found' })
    }
  }

```

```
// Remove user's vote

const initialLength = writeup.votes.length
writeup.votes = writeup.votes.filter(vote => vote.user.toString() !== req.userId)

if (writeup.votes.length === initialLength) {
  return res.status(400).json({ error: 'No vote to remove' })
}

// Recalculate rating
const upvotes = writeup.votes.filter(v => v.type === 'up').length
const downvotes = writeup.votes.filter(v => v.type === 'down').length
writeup.rating = upvotes - downvotes

await writeup.save()

res.json({
  message: 'Vote removed successfully',
  rating: writeup.rating,
  upvotes,
  downvotes,
  userVote: null
})
} catch (error) {
  console.error('Remove vote error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})
```

```

// Get user's writeups
router.get('/user/:userId', async (req, res) => {
  try {
    const { userId } = req.params
    const { page = 1, limit = 20, publicOnly = 'true' } = req.query

    const filter = { user: userId }
    if (publicOnly === 'true') {
      filter.isPublic = true
    }

    const writeups = await Writeup.find(filter)
      .populate('challenge', 'title category difficulty points')
      .select('title rating views tags isPublic createdAt')
      .sort({ createdAt: -1 })
      .limit(limit * 1)
      .skip((page - 1) * limit)

    const total = await Writeup.countDocuments(filter)

    res.json({
      writeups,
      pagination: {
        total,
        page: parseInt(page),
        limit: parseInt(limit),
        pages: Math.ceil(total / limit)
      }
    })
  }
})

```

```

    } catch (error) {
      console.error('Get user writeups error:', error)
      res.status(500).json({ error: 'Internal server error' })
    }
  })

// Search writeups
router.get('/search', async (req, res) => {
  try {
    const { q, category, tags, page = 1, limit = 20 } = req.query

    const filter = { isPublic: true }

    if (q) {
      filter.$or = [
        { title: { $regex: q, $options: 'i' } },
        { content: { $regex: q, $options: 'i' } },
        { tags: { $in: [new RegExp(q, 'i')] } }
      ]
    }

    if (category) {
      filter['challenge.category'] = category
    }

    if (tags) {
      const tagArray = Array.isArray(tags) ? tags : [tags]
      filter.tags = { $in: tagArray }
    }
  }
})

```

```
const writeups = await Writeup.find(filter)

  .populate('user', 'username fullName')
  .populate('challenge', 'title category difficulty')
  .select('title content rating views tags createdAt')
  .sort({ rating: -1, views: -1 })
  .limit(limit * 1)
  .skip((page - 1) * limit)
```

```
const total = await Writeup.countDocuments(filter)
```

```
res.json({
  writeups,
  pagination: {
    total,
    page: parseInt(page),
    limit: parseInt(limit),
    pages: Math.ceil(total / limit)
  }
})
} catch (error) {
  console.error('Search writeups error:', error)
  res.status(500).json({ error: 'Internal server error' })
}
})
```

```
// Get popular writeups
```

```
router.get('/popular', async (req, res) => {
  try {
```

```
const { limit = 10 } = req.query
```

```
const writeups = await Writeup.find({ isPublic: true })  
  .populate('user', 'username fullName')  
  .populate('challenge', 'title category difficulty')  
  .select('title rating views tags createdAt')  
  .sort({ rating: -1, views: -1 })  
  .limit(parseInt(limit))
```

```
res.json({ writeups })  
} catch (error) {  
  console.error('Get popular writeups error:', error)  
  res.status(500).json({ error: 'Internal server error' })  
}  
})
```

```
export default router"
```

```
server.js
```

```
"import dotenv from 'dotenv'  
import express from 'express'  
import cors from 'cors'  
import helmet from 'helmet'  
import rateLimit from 'express-rate-limit'  
import morgan from 'morgan'  
import { createServer } from 'http'
```

```
// Load environment variables
```

```
dotenv.config()
```

```
// Validate required environment variables

const requiredEnvVars = ['JWT_SECRET', 'MONGODB_URI']
const missingEnvVars = requiredEnvVars.filter(envVar => !process.env[envVar])

if (missingEnvVars.length > 0) {
  console.error('✖ Missing required environment variables:', missingEnvVars.join(', '))
  process.exit(1)
}

// Import database connection
import connectDB from './config/database.js'

// Import services
import socketService from './services/socketService.js'
import scoringService from './services/scoringService.js'
import badgeService from './services/badgeService.js'
import dockerService from './services/dockerService.js'
import backupManager from './utils/backupManager.js'

// Import routes
import authRoutes from './routes/auth.js'
import challengeRoutes from './routes/challenges.js'
import adminRoutes from './routes/admin.js'
import leaderboardRoutes from './routes/leaderboard.js'
import terminalRoutes from './routes/terminal.js'
import teamRoutes from './routes/teams.js'
import hintRoutes from './routes/hints.js'
import achievementRoutes from './routes/achievements.js'
```



```
import fileRoutes from './routes/files.js'
import writeupRoutes from './routes/writeups.js'
import eventRoutes from './routes/events.js'
import analyticsRoutes from './routes/analytics.js'

const app = express()
const server = createServer(app)

// Initialize Socket.IO
socketService.initialize(server)

// Connect to MongoDB
await connectDB()

// Initialize services
await scoringService.initializeDynamicScoring()
await badgeService.initializeDefaultBadges()

// Security Middleware
app.use(helmet({
  crossOriginResourcePolicy: { policy: "cross-origin" },
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"]
    }
  }
})
})
```

```
}}
```

```
app.use(cors({  
  origin: process.env.FRONTEND_URL || 'http://localhost:3000',  
  credentials: true  
}))
```

```
// Trust proxy for rate limiting and IP detection  
app.set('trust proxy', 1)
```

```
// Body parsing middleware  
app.use(express.json({ limit: '50mb' }))  
app.use(express.urlencoded({ extended: true, limit: '50mb' }))
```

```
// Logging  
app.use(morgan('combined'))
```

```
// Rate Limiting  
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 500,  
  message: { error: 'Too many requests from this IP' }  
})  
app.use('/api/', limiter)
```

```
// Auth rate limiting  
const authLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 20,
```

```
    message: { error: 'Too many authentication attempts' }  
  })  
  app.use('/api/auth', authLimiter)
```

```
// File upload rate limiting  
const uploadLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 50,  
  message: { error: 'Too many file upload attempts' }  
})
```

```
// Routes  
app.use('/api/auth', authRoutes)  
app.use('/api/challenges', challengeRoutes)  
app.use('/api/admin', adminRoutes)  
app.use('/api/leaderboard', leaderboardRoutes)  
app.use('/api/terminal', terminalRoutes)  
app.use('/api/teams', teamRoutes)  
app.use('/api/hints', hintRoutes)  
app.use('/api/achievements', achievementRoutes)  
app.use('/api/files', uploadLimiter, fileRoutes)  
app.use('/api/writeups', writeupRoutes)  
app.use('/api/events', eventRoutes)  
app.use('/api/analytics', analyticsRoutes)
```

```
// Health check with system status  
app.get('/api/health', async (req, res) => {  
  const status = {  
    status: 'OK',
```

```

timestamp: new Date().toISOString(),
environment: process.env.NODE_ENV || 'development',
database: 'MongoDB',
services: {
  socket: socketService.io ? 'Running' : 'Stopped',
  scoring: 'Active',
  badges: 'Active',
  docker: dockerService ? 'Available' : 'Unavailable'
},
system: {
  memory: process.memoryUsage(),
  uptime: process.uptime()
}
}

res.json(status)
})

// Backup endpoint (admin only)
app.get('/api/admin/backup', async (req, res) => {
  // This would be protected with admin middleware in production
  try {
    const backupPath = await backupManager.createBackup()
    res.json({
      message: 'Backup created successfully',
      path: backupPath
    })
  } catch (error) {
    res.status(500).json({ error: 'Backup failed' })
  }
})

```

```

    }
  })

  // 404 handler
  app.use('/api/*', (req, res) => {
    res.status(404).json({ error: 'Endpoint not found' })
  })

  // Error handling middleware
  app.use((err, req, res, next) => {
    console.error('Error:', err.stack)

    let error = { message: 'Internal server error', status: 500 }

    if (err.name === 'JsonWebTokenError') {
      error = { message: 'Invalid token', status: 401 }
    } else if (err.name === 'TokenExpiredError') {
      error = { message: 'Token expired', status: 401 }
    } else if (err.name === 'ValidationError') {
      error = { message: err.message, status: 400 }
    } else if (err.code === 11000) {
      error = { message: 'Duplicate entry', status: 400 }
    } else if (err.type === 'entity.parse.failed') {
      error = { message: 'Invalid JSON in request body', status: 400 }
    }

    res.status(error.status).json({ error: error.message })
  })

```

```
const PORT = process.env.PORT || 5000
```

```
server.listen(PORT, () => {  
  console.log(`🚀 Enhanced CTF Hackathon Portal Started`)  
  console.log('=====')  
  console.log(`📍 Port: ${PORT}`)  
  console.log(`🌐 Environment: ${process.env.NODE_ENV || 'development'}`)  
  console.log(`🗄 Database: MongoDB`)  
  console.log(`🌐 Frontend: ${process.env.FRONTEND_URL || 'http://localhost:3000'}`)  
  console.log(`🔌 WebSocket: Enabled`)  
  console.log(`🎯 Features: Dynamic Scoring, Teams, Achievements, Docker`)  
  console.log(`✅ Health: http://localhost:${PORT}/api/health`)  
  console.log('=====')  
})
```

```
// Cleanup on exit
```

```
process.on('SIGINT', async () => {  
  console.log(`\n🛑 Shutting down gracefully...`)
```

```
// Cleanup Docker containers
```

```
await dockerService.cleanupExpiredContainers()
```

```
console.log(`✅ Cleanup completed`)
```

```
process.exit(0)
```

```
})"
```

Frontend files

frontend files

```
src/components/challenges/challengecard.jsx
```

```
"/ components/challenges/ChallengeCard.jsx (updated)
```

```
import React, { useState } from 'react';
```

```
import { Terminal, Flag, Clock, Award, Zap, Eye } from 'lucide-react';
```

```
const ChallengeCard = ({ challenge, onSolve, viewMode }) => {
```

```
  const [showTerminal, setShowTerminal] = useState(false);
```

```
  const getDifficultyColor = (difficulty) => {
```

```
    switch (difficulty) {
```

```
      case 'easy': return 'text-green-400 border-green-500 bg-green-500 bg-opacity-10';
```

```
      case 'medium': return 'text-yellow-400 border-yellow-500 bg-yellow-500 bg-opacity-10';
```

```
      case 'hard': return 'text-red-400 border-red-500 bg-red-500 bg-opacity-10';
```

```
      default: return 'text-gray-400 border-gray-500 bg-gray-500 bg-opacity-10';
```

```
    }
```

```
  };
```

```
  const handleChallengeSolved = (challengeId, points, flag) => {
```

```
    onSolve(challengeId, points);
```

```
    setShowTerminal(false);
```

```
  };
```

```
  if (viewMode === 'list') {
```

```
    return (
```

```
      <>
```

```
      <div className="bg-black border border-green-500 rounded-lg p-6 hover:border-cyan-400  
transition-all group">
```

```
        <div className="flex items-center justify-between">
```

```

<div className="flex items-center space-x-4 flex-1">
  <div className="flex-shrink-0">
    <div className="w-12 h-12 bg-gradient-to-r from-green-600 to-cyan-600 rounded-lg flex
items-center justify-center">
      <Zap className="w-6 h-6 text-white" />
    </div>
  </div>

```

```

<div className="flex-1">
  <h3 className="text-white font-bold text-lg group-hover:text-cyan-300 transition-colors">
    {challenge.title}
  </h3>
  <p className="text-green-300 text-sm mt-1">{challenge.description}</p>
  <div className="flex items-center space-x-4 mt-2">
    <span className={`px-3 py-1 rounded-full border text-xs font-bold
${getDifficultyColor(challenge.difficulty)}}`>
      {challenge.difficulty.toUpperCase()}
    </span>
    <span className="text-yellow-400 text-sm flex items-center">
      <Award className="w-4 h-4 mr-1" />
      {challenge.points} POINTS
    </span>
  </div>
</div>
</div>

```

```

<div className="flex items-center space-x-3">
  {challenge.solved ? (
    <span className="text-green-400 font-bold text-sm bg-green-500 bg-opacity-20 px-3 py-2
rounded-lg border border-green-500">

```


SOLVED ✓

): (

<button

onClick={() => setShowTerminal(true)}

className="bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold px-6 py-2 rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all border border-cyan-400 flex items-center space-x-2 group"

>

<Terminal className="w-4 h-4" />

LAUNCH_TERMINAL

</button>

}}

</div>

</div>

</div>

{/* Terminal Modal */}

{showTerminal && (

<CTFTerminal

challenge={challenge}

isOpen={showTerminal}

onClose={() => setShowTerminal(false)}

onChallengeSolved={handleChallengeSolved}

/>

}}

</>

);

}

```
// Grid view (your existing code with terminal button added)

return (
  <>

    <div className="bg-black border border-green-500 rounded-lg p-6 hover:border-cyan-400
transition-all group">

      <div className="flex items-center justify-between mb-4">

        <div className="flex items-center space-x-3">

          <div className="w-10 h-10 bg-gradient-to-r from-green-600 to-cyan-600 rounded-lg flex items-
center justify-center">

            <Zap className="w-5 h-5 text-white" />

          </div>

          <div>

            <h3 className="text-white font-bold group-hover:text-cyan-300 transition-colors">

              {challenge.title}

            </h3>

            <p className="text-green-300 text-sm">{challenge.category}</p>

          </div>

        </div>

        <span className={`px-3 py-1 rounded-full border text-xs font-bold
${getDifficultyColor(challenge.difficulty)}`}>

          {challenge.difficulty.toUpperCase()}

        </span>

      </div>

      <p className="text-green-300 text-sm mb-4 line-clamp-2">{challenge.description}</p>

      <div className="flex items-center justify-between mb-4">

        <span className="text-yellow-400 text-sm flex items-center">

          <Award className="w-4 h-4 mr-1" />
```

```

    {challenge.points} POINTS
  </span>

  {challenge.solved && (
    <span className="text-green-400 text-sm flex items-center">
      <Flag className="w-4 h-4 mr-1" />
      SOLVED
    </span>
  )}
</div>

```

```

{!challenge.solved ? (
  <button
    onClick={() => setShowTerminal(true)}
    className="w-full bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold py-3
rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all border border-cyan-400 flex items-
center justify-center space-x-2 group"
    >
    <Terminal className="w-4 h-4" />
    <span>ACCESS_TERMINAL</span>
  </button>
) : (
  <button
    onClick={() => setShowTerminal(true)}
    className="w-full bg-gray-700 text-gray-300 font-bold py-3 rounded-lg border border-gray-600
flex items-center justify-center space-x-2 hover:bg-gray-600 transition-all"
    >
    <Eye className="w-4 h-4" />
    <span>VIEW_TERMINAL</span>
  </button>
)}

```

```
</div>
```

```
{/* Terminal Modal */}
```

```
{showTerminal && (
```

```
  <CTFTerminal
```

```
    challenge={challenge}
```

```
    isOpen={showTerminal}
```

```
    onClose={() => setShowTerminal(false)}
```

```
    onChallengeSolved={handleChallengeSolved}
```

```
  />
```

```
  )}
```

```
</>
```

```
);
```

```
};
```

```
export default ChallengeCard;"
```

```
createchallengeform.jsx
```

```
"import React, { useState } from 'react'
```

```
import {
```

```
  Terminal,
```

```
  Plus,
```

```
  Code,
```

```
  FileText,
```

```
  Image,
```

```
  Link,
```

```
  Upload,
```

```
  Server,
```

```
  Shield,
```

```
Cpu,  
Network,  
Binary,  
FileSearch,  
Key,  
Bug,  
Zap,  
Lock,  
Eye,  
EyeOff,  
Copy,  
Download,  
AlertTriangle,  
CheckCircle,  
XCircle  
} from 'lucide-react'  
import axios from 'axios'
```

```
const CreateChallengeForm = ({ onChallengeCreated }) => {  
  const [formData, setFormData] = useState({  
    title: "",  
    description: "",  
    category: 'web',  
    difficulty: 'easy',  
    points: 100,  
    flag: "",  
    hint: "",  
    files: []  
  })  
}
```

```

const [loading, setLoading] = useState(false)
const [message, setMessage] = useState('')
const [activeTab, setActiveTab] = useState('basic')
const [terminalOutput, setTerminalOutput] = useState([])
const [showFlag, setShowFlag] = useState(false)

const categories = [
  { value: 'web', label: 'WEB_EXPLOITATION', icon: Network, color: 'from-blue-500 to-cyan-500' },
  { value: 'crypto', label: 'CRYPTOGRAPHY', icon: Key, color: 'from-yellow-500 to-amber-500' },
  { value: 'forensics', label: 'DIGITAL_FORENSICS', icon: FileSearch, color: 'from-orange-500 to-red-500' },
  { value: 'pwn', label: 'BINARY_PWN', icon: Binary, color: 'from-purple-500 to-pink-500' },
  { value: 'reverse', label: 'REVERSE_ENGINEERING', icon: Cpu, color: 'from-green-500 to-emerald-500' },
  { value: 'misc', label: 'MISCELLANEOUS', icon: Bug, color: 'from-indigo-500 to-purple-500' }
]

const difficulties = [
  { value: 'easy', label: 'BEGINNER', points: 100, color: 'text-green-400' },
  { value: 'medium', label: 'INTERMEDIATE', points: 250, color: 'text-yellow-400' },
  { value: 'hard', label: 'EXPERT', points: 500, color: 'text-red-400' }
]

const addTerminalLine = (text, type = 'info') => {
  setTerminalOutput(prev => [...prev, {
    text,
    type,
    timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })
  }])
}

```

```
const handleChange = (e) => {  
  const { name, value } = e.target  
  setFormData(prev => ({  
    ...prev,  
    [name]: value  
  })))
```

```
// Auto-adjust points based on difficulty  
if (name === 'difficulty') {  
  const difficulty = difficulties.find(d => d.value === value)  
  if (difficulty) {  
    setFormData(prev => ({  
      ...prev,  
      points: difficulty.points  
    })))  
  }  
}
```

```
const handleSubmit = async (e) => {  
  e.preventDefault()  
  setLoading(true)  
  setMessage('')  
  addTerminalLine('Initializing challenge deployment protocol...', 'system')  
  
  try {  
    await axios.post('/admin/challenges', formData)  
    const successMsg = 'TARGET_SUCCESSFULLY_DEPLOYED'  
    setMessage(successMsg)
```

```
addTerminalLine(successMsg, 'success')

addTerminalLine(`Challenge "${formData.title}" added to active targets`, 'system')


setFormData({
  title: "",
  description: "",
  category: 'web',
  difficulty: 'easy',
  points: 100,
  flag: "",
  hint: "",
  files: []
})

onChallengeCreated?.()
} catch (error) {
  const errorMsg = error.response?.data?.error || 'DEPLOYMENT_FAILED: System error'
  setMessage(errorMsg)
  addTerminalLine(errorMsg, 'error')
} finally {
  setLoading(false)
}
}

const tabs = [
  { id: 'basic', label: 'TARGET_SPECS', icon: FileText, command: './config_basic' },
  { id: 'advanced', label: 'SECURITY_PROTOCOLS', icon: Shield, command: './config_advanced' },
  { id: 'files', label: 'RESOURCE_FILES', icon: Download, command: './manage_files' }
]
```



```

const generateFlag = () => {
  const flag = `CTF{${Math.random().toString(36).substr(2,
12).toUpperCase()}_${Math.random().toString(36).substr(2, 8).toUpperCase()}}`
  setFormData(prev => ({ ...prev, flag }))
  addTerminalLine('Generated secure flag template', 'system')
}

const copyToClipboard = (text) => {
  navigator.clipboard.writeText(text)
  addTerminalLine('Flag copied to clipboard', 'success')
}

const simulateFileUpload = () => {
  addTerminalLine('File upload simulation initiated...', 'system')
  setTimeout(() => addTerminalLine('File validation passed', 'success'), 1000)
  setTimeout(() => addTerminalLine('File encrypted and stored', 'success'), 1500)
}

return (
  <div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20 p-6
font-mono">
    {/* Terminal Header */}
    <div className="flex items-center space-x-2 text-green-300 mb-6">
      <Terminal className="w-5 h-5" />
      <span className="text-sm">TARGET_DEPLOYMENT_SYSTEM</span>
    </div>

    <div className="flex items-center space-x-3 mb-6">
      <div className="w-10 h-10 bg-gradient-to-r from-green-500 to-cyan-500 rounded-lg flex items-
center justify-center">

```

```

    <Plus className="w-6 h-6 text-white" />
  </div>

  <div>

    <h2 className="text-2xl font-bold text-white glitch" data-text="DEPLOY_NEW_TARGET">
      DEPLOY_NEW_TARGET
    </h2>

    <p className="text-green-300 text-sm">Create new challenge for operators</p>
  </div>
</div>

```

```

{/* Terminal Output */}

<div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-24 overflow-y-auto">
  {terminalOutput.map((line, index) => (
    <div key={index} className={`text-sm ${
      line.type === 'error' ? 'text-red-400' :
      line.type === 'success' ? 'text-green-400' :
      line.type === 'system' ? 'text-cyan-400' : 'text-green-300'
    }`}>
      <span className="text-gray-500">[{line.timestamp}]</span> {line.text}
    </div>
  ))}
</div>

```

```

{/* Tabs */}

<div className="border-b border-green-500 mb-6">
  <nav className="flex flex-wrap gap-2">
    {tabs.map(tab => {
      const Icon = tab.icon
      return (

```

```

<button
  key={tab.id}
  onClick={() => setActiveTab(tab.id)}
  className={`flex items-center space-x-2 py-3 px-4 border-b-2 font-bold text-sm transition-all
group relative ${
  activeTab === tab.id
    ? 'border-cyan-400 text-cyan-400 bg-cyan-500 bg-opacity-10'
    : 'border-transparent text-green-400 hover:text-cyan-400 hover:border-gray-300'
  }}
>
  <Icon className="w-4 h-4" />
  <span>{tab.label}</span>
  <div className="hidden group-hover:block absolute top-full mt-2 bg-black border border-
green-500 rounded px-2 py-1 text-xs text-green-400">
    {tab.command}
  </div>
</button>
)
}}
</nav>
</div>

```

```

<form onSubmit={handleSubmit} className="space-y-6">
  {message && (
    <div className={`p-4 rounded-lg border font-mono text-sm ${
      message.includes('FAILED')
        ? 'bg-red-500 bg-opacity-10 border-red-500 text-red-400'
        : 'bg-green-500 bg-opacity-10 border-green-500 text-green-400'
      }`}>

```

```

<div className="flex items-center space-x-2">
  {message.includes('FAILED') ?
    <XCircle className="w-4 h-4" /> :
    <CheckCircle className="w-4 h-4" />
  }
  <span className="font-bold">{message}</span>
</div>
</div>
)}

```

```

{/* Basic Info Tab */}
{activeTab === 'basic' && (
  <div className="space-y-6">
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
      <div>
        <label className="block text-sm font-bold text-green-400 mb-2">
          [TARGET] CHALLENGE_TITLE
        </label>
        <input
          type="text"
          name="title"
          value={formData.title}
          onChange={handleChange}
          required
          className="w-full px-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-none
focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-green-600 transition-all
font-mono"
          placeholder="ENTER_TARGET_NAME"
        />

```

```
</div>
```

```
<div>
```

```
  <label className="block text-sm font-bold text-green-400 mb-2">
```

```
    [CATEGORY] OPERATION_TYPE
```

```
  </label>
```

```
  <div className="grid grid-cols-2 gap-2">
```

```
    {categories.map(category => {
```

```
      const Icon = category.icon
```

```
      return (
```

```
        <button
```

```
          key={category.value}
```

```
          type="button"
```

```
          onClick={() => setFormData(prev => ({ ...prev, category: category.value })))}
```

```
          className={`p-3 rounded-lg border text-left transition-all ${
```

```
            formData.category === category.value
```

```
            ? 'border-cyan-400 bg-cyan-500 bg-opacity-10 text-cyan-400'
```

```
            : 'border-green-500 text-green-400 hover:border-cyan-400 hover:bg-cyan-500 hover:bg-
```

```
opacity-5'
```

```
          `}
```

```
        >
```

```
        <div className="flex items-center space-x-3">
```

```
          <div className={`w-8 h-8 rounded bg-gradient-to-r ${category.color} flex items-center justify-center`}>
```

```
            <Icon className="w-4 h-4 text-white" />
```

```
          </div>
```

```
          <span className="text-sm font-bold">{category.label}</span>
```

```
        </div>
```

```
      </button>
```

```

    )
  }}}
</div>
</div>
</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
  <div>
    <label className="block text-sm font-bold text-green-400 mb-2">
      [DIFFICULTY] THREAT_LEVEL
    </label>
    <div className="space-y-2">
      {difficulties.map(diff => (
        <button
          key={diff.value}
          type="button"
          onClick={() => setFormData(prev => ({
            ...prev,
            difficulty: diff.value,
            points: diff.points
          })))}
          className={`w-full p-3 rounded-lg border transition-all text-left ${
            formData.difficulty === diff.value
              ? 'border-cyan-400 bg-cyan-500 bg-opacity-10 text-cyan-400'
              : `${diff.color} border-green-500 hover:border-cyan-400`
            }`}
        >
          <div className="flex justify-between items-center">
            <span className="font-bold">{diff.label}</span>

```

```

        <span className="text-white">{diff.points} PTS</span>
      </div>
    </button>
  )}
</div>
</div>

<div>
  <label className="block text-sm font-bold text-green-400 mb-2">
    [REWARD] POINT_VALUE
  </label>
  <input
    type="number"
    name="points"
    value={formData.points}
    onChange={handleChange}
    required
    min="1"
    max="1000"
    className="w-full px-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-none
focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white font-mono"
  />
  <div className="flex space-x-2 mt-2">
    {[100, 250, 500].map(points => (
      <button
        key={points}
        type="button"
        onClick={() => setFormData(prev => ({ ...prev, points })))}

```

```

        className="px-2 py-1 text-xs border border-green-500 text-green-400 rounded
hover:border-cyan-400 hover:text-cyan-400"

        >

        {points}

    </button>

    )})

</div>

</div>

</div>

<div>

    <label className="block text-sm font-bold text-green-400 mb-2">

        [BRIEFING] MISSION_DESCRIPTION

    </label>

    <textarea

        name="description"

        value={formData.description}

        onChange={handleChange}

        required

        rows="6"

        className="w-full px-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-none
focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-green-600 resize-vertical
font-mono"

        placeholder="DESCRIBE_THE_OPERATION_OBJECTIVES_AND_REQUIREMENTS..."

    />

</div>

</div>

)}

{ /* Advanced Tab */ }

```



```

{activeTab === 'advanced' && (
  <div className="space-y-6">
    <div>
      <div className="flex items-center justify-between mb-2">
        <label className="block text-sm font-bold text-green-400">
          [SECURITY] ENCRYPTION_FLAG
        </label>
        <div className="flex items-center space-x-2">
          <button
            type="button"
            onClick={generateFlag}
            className="px-3 py-1 border border-yellow-500 text-yellow-400 rounded text-sm
            hover:border-yellow-400 hover:text-yellow-300 transition-all"
          >
            GENERATE
          </button>
          <button
            type="button"
            onClick={() => setShowFlag(!showFlag)}
            className="p-1 text-green-400 hover:text-cyan-400 transition-colors"
          >
            {showFlag ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
          </button>
        </div>
      </div>
      {formData.flag && (
        <button
          type="button"
          onClick={() => copyToClipboard(formData.flag)}
          className="p-1 text-green-400 hover:text-cyan-400 transition-colors"
        >

```

```

        <Copy className="w-4 h-4" />
    </button>
  })
</div>
</div>
<input
  type={showFlag ? "text" : "password"}
  name="flag"
  value={formData.flag}
  onChange={handleChange}
  required

  className="w-full px-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-none
focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-green-600 font-mono
transition-all"

  placeholder="CTF{SECURE_FLAG_HERE}"
/>
<p className="text-sm text-green-600 mt-2">
  Primary authentication token for target validation
</p>
</div>

<div>
  <label className="block text-sm font-bold text-green-400 mb-2">
    [INTEL] OPERATION_HINT
  </label>
  <textarea
    name="hint"
    value={formData.hint}
    onChange={handleChange}

```

```
        rows="3"

        className="w-full px-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-none
focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-green-600 resize-vertical
font-mono"

        placeholder="PROVIDE_TACTICAL_INTELLIGENCE_FOR_OPERATORS..."
    />
</div>
```

```
<div className="bg-yellow-500 bg-opacity-10 border border-yellow-500 rounded-lg p-4">
  <div className="flex items-center space-x-2 text-yellow-400 mb-3">
    <AlertTriangle className="w-5 h-5" />
    <h4 className="font-bold">DEPLOYMENT_PROTOCOLS</h4>
  </div>
  <ul className="text-sm text-yellow-300 space-y-2 font-mono">
    <li className="flex items-center space-x-2">
      <Shield className="w-4 h-4" />
      <span>Ensure flags are cryptographically secure</span>
    </li>
    <li className="flex items-center space-x-2">
      <Zap className="w-4 h-4" />
      <span>Test all challenge components before deployment</span>
    </li>
    <li className="flex items-center space-x-2">
      <Lock className="w-4 h-4" />
      <span>Use CTF{'{'...'{'}} format for consistency</span>
    </li>
    <li className="flex items-center space-x-2">
      <Server className="w-4 h-4" />
      <span>Validate resource file integrity</span>
    </li>
  </ul>
</div>
```

```
    </li>
  </ul>
</div>
</div>
}}
```

```
{/* Files Tab */}
{activeTab === 'files' && (
  <div className="space-y-6">
    <div className="border-2 border-dashed border-green-500 rounded-lg p-8 text-center
hover:border-cyan-400 transition-all group">
      <Download className="w-12 h-12 text-green-400 mx-auto mb-4 group-hover:text-cyan-400
transition-colors" />
      <p className="text-green-300 mb-4">Upload mission resource files</p>
      <button
        type="button"
        onClick={simulateFileUpload}
        className="px-6 py-3 bg-gradient-to-r from-green-600 to-cyan-600 text-white rounded-lg
hover:from-green-700 hover:to-cyan-700 transition-all border border-cyan-400 font-bold"
      >
        <Upload className="w-4 h-4 inline mr-2" />
        SELECT_FILES
      </button>
      <p className="text-sm text-green-600 mt-3">
        MAXIMUM_PAYLOAD: 50MB PER FILE
      </p>
    </div>
```

```
<div className="bg-cyan-500 bg-opacity-10 border border-cyan-500 rounded-lg p-4">
  <div className="flex items-center space-x-2 text-cyan-400 mb-3">
```

```

        <FileText className="w-5 h-5" />

        <h4 className="font-bold">FILE_REQUIREMENTS</h4>

    </div>

    <ul className="text-sm text-cyan-300 space-y-2 font-mono">

        <li>• SUPPORTED_FORMATS: ZIP, PDF, EXE, PNG, JPG, TXT</li>

        <li>• MAXIMUM_TOTAL_PAYLOAD: 100MB</li>

        <li>• SCAN_FILES_FOR_MALWARE_BEFORE_UPLOAD</li>

        <li>• PROVIDE_CLEAR_USAGE_INSTRUCTIONS</li>

        <li>• ENCRYPT_SENSITIVE_FILES</li>

    </ul>

</div>


{ /* File List Preview */ }

<div className="bg-black border border-green-500 rounded-lg p-4">

    <h4 className="text-green-400 font-bold mb-3">DEPLOYED_RESOURCES</h4>

    <div className="text-green-600 text-sm text-center py-4">

        NO_FILES_DEPLOYED

    </div>

</div>

</div>

)}


{ /* Submit Buttons */ }

<div className="flex flex-col sm:flex-row justify-end space-y-4 sm:space-y-0 sm:space-x-4 pt-6
border-t border-green-500 border-opacity-30">

    <button

        type="button"

        className="px-6 py-3 border border-green-500 text-green-400 rounded-lg hover:border-cyan-
400 hover:text-cyan-400 transition-all font-bold"

```

```

    >
    SAVE_DRAFT
  </button>
  <button
    type="submit"
    disabled={loading}

    className="px-6 py-3 bg-gradient-to-r from-green-600 to-cyan-600 text-white rounded-lg
    hover:from-green-700 hover:to-cyan-700 focus:outline-none disabled:opacity-50 disabled:cursor-not-
    allowed transition-all font-bold border border-cyan-400 shadow-lg shadow-cyan-500/25 flex items-
    center justify-center space-x-2"

    >
    {loading ? (
      <>
        <div className="w-5 h-5 border-2 border-white border-t-transparent rounded-full animate-
        spin"></div>
        <span>DEPLOYING_TARGET...</span>
      </>
    ) : (
      <>
        <Zap className="w-5 h-5" />
        <span>DEPLOY_CHALLENGE</span>
      </>
    )}
  </button>
</div>
</form>

<style jsx>{`
  .glitch {
    position: relative;

```

```
}
```

```
.glitch::before,
```

```
.glitch::after {
```

```
  content: attr(data-text);
```

```
  position: absolute;
```

```
  top: 0;
```

```
  left: 0;
```

```
  width: 100%;
```

```
  height: 100%;
```

```
}
```

```
.glitch::before {
```

```
  animation: glitch-1 0.5s infinite linear alternate-reverse;
```

```
  color: #ff00ff;
```

```
  z-index: -1;
```

```
}
```

```
.glitch::after {
```

```
  animation: glitch-2 0.5s infinite linear alternate-reverse;
```

```
  color: #00ffff;
```

```
  z-index: -2;
```

```
}
```

```
@keyframes glitch-1 {
```

```
  0% { transform: translate(0); }
```

```
  20% { transform: translate(-1px, 1px); }
```

```
  40% { transform: translate(-1px, -1px); }
```

```
  60% { transform: translate(1px, 1px); }
```

```
80% { transform: translate(1px, -1px); }  
100% { transform: translate(0); }  
}
```

```
@keyframes glitch-2 {  
  0% { transform: translate(0); }  
  20% { transform: translate(1px, -1px); }  
  40% { transform: translate(1px, 1px); }  
  60% { transform: translate(-1px, -1px); }  
  80% { transform: translate(-1px, 1px); }  
  100% { transform: translate(0); }  
}
```

```
`</style>
```

```
</div>
```

```
)
```

```
}
```

```
export default CreateChallengeForm",
```

```
src/components/navigation/adminsidebar.jsx
```

```
"// src/components/navigation/AdminSidebar.jsx
```

```
import React, { useState } from 'react'
```

```
import { Link, useLocation, useNavigate } from 'react-router-dom'
```

```
import {
```

```
  Shield,
```

```
  Users,
```

```
  Settings,
```

```
  BarChart3,
```

```
  Terminal,
```



```
    Logout,
    ChevronLeft,
    ChevronRight,
    Home,
    Database,
    Server
  } from 'lucide-react'

import { useAuth } from '../contexts/AuthContext'

const AdminSidebar = () => {
  const { user, logout } = useAuth()
  const location = useLocation()
  const navigate = useNavigate()
  const [isCollapsed, setIsCollapsed] = useState(false)

  const adminMenu = [
    {
      category: 'Overview',
      items: [
        { path: '/admin', label: 'Dashboard', icon: BarChart3, description: 'System overview' },
        { path: '/admin/analytics', label: 'Analytics', icon: BarChart3, description: 'Performance metrics' },
      ]
    },
    {
      category: 'Management',
      items: [
        { path: '/admin/users', label: 'User Management', icon: Users, description: 'Manage users' },
        { path: '/admin/challenges', label: 'Challenges', icon: Terminal, description: 'Manage challenges' },
        { path: '/admin/content', label: 'Content', icon: Database, description: 'Manage content' },
      ]
    }
  ]
}
```

```

    ]
  },
  {
    category: 'System',
    items: [
      { path: '/admin/servers', label: 'Servers', icon: Server, description: 'Server status' },
      { path: '/admin/settings', label: 'Settings', icon: Settings, description: 'System configuration' },
    ]
  }
]

```

```
const isActive = (path) => location.pathname === path
```

```

const handleLogout = () => {
  logout()
  navigate('/')
}

```

```

return (
  <div className={`flex flex-col bg-gray-900 border-r border-gray-700 transition-all duration-300 ${
    isCollapsed ? 'w-20' : 'w-64'
  }`} >
    { /* Header */ }
    <div className="flex items-center justify-between p-4 border-b border-gray-700">
      { !isCollapsed && (
        <div className="flex items-center space-x-3">
          <div className="flex items-center justify-center w-8 h-8 bg-gradient-to-br from-yellow-600 to-orange-600 rounded-lg">
            <Shield className="w-4 h-4 text-white" />

```

```

</div>

<div className="flex flex-col">

  <span className="text-sm font-bold text-white">Admin Panel</span>

  <span className="text-xs text-gray-400">v2.1.4</span>

</div>

</div>

)}

```

```

{isCollapsed && (
  <div className="flex justify-center w-full">

    <div className="flex items-center justify-center w-8 h-8 bg-gradient-to-br from-yellow-600 to-orange-600 rounded-lg">

      <Shield className="w-4 h-4 text-white" />

    </div>

  </div>

)}

```

```

<button
  onClick={() => setIsCollapsed(!isCollapsed)}
  className="p-1 rounded-lg text-gray-400 hover:text-white hover:bg-gray-800 transition-all"
>
  {isCollapsed ? <ChevronRight className="w-4 h-4" /> : <ChevronLeft className="w-4 h-4" />}
</button>

</div>

```

```

{/* User Info */}

{!isCollapsed && (
  <div className="p-4 border-b border-gray-700">

    <div className="flex items-center space-x-3">

```

```
<div className="flex items-center justify-center w-10 h-10 bg-gradient-to-br from-blue-500 to-purple-500 rounded-lg">
```

```
<Users className="w-5 h-5 text-white" />
```

```
</div>
```

```
<div className="flex flex-col flex-1 min-w-0">
```

```
<span className="text-sm font-medium text-white truncate">{user?.username}</span>
```

```
<span className="text-xs text-gray-400">Administrator</span>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
}}
```

```
{/* Navigation */}
```

```
<div className="flex-1 overflow-y-auto py-4">
```

```
{adminMenu.map((section, index) => (
```

```
<div key={index} className="mb-6">
```

```
{!isCollapsed && (
```

```
<div className="px-4 mb-2">
```

```
<span className="text-xs font-semibold text-gray-500 uppercase tracking-wider">
```

```
{section.category}
```

```
</span>
```

```
</div>
```

```
}}
```

```
<div className="space-y-1">
```

```
{section.items.map((item) => {
```

```
const Icon = item.icon
```

```
return (
```

```
<Link
```

```

      key={item.path}
      to={item.path}
      className={`flex items-center mx-2 p-3 rounded-lg transition-all group ${
        isActive(item.path)
          ? 'bg-blue-600 text-white shadow-lg shadow-blue-500/25'
          : 'text-gray-300 hover:text-white hover:bg-gray-800'
      }`}
      title={isCollapsed ? item.label : ''}
    >
      <Icon className={`flex-shrink-0 ${isCollapsed ? 'w-5 h-5' : 'w-4 h-4'}`} />

      {!isCollapsed && (
        <div className="ml-3 flex-1 min-w-0">
          <span className="text-sm font-medium">{item.label}</span>
          <p className="text-xs text-gray-400 truncate">{item.description}</p>
        </div>
      )}
    </Link>
  )
  }}}
</div>
</div>
)}}
</div>

{/* Footer Actions */}
<div className="p-4 border-t border-gray-700 space-y-2">
  <Link
    to="/"

```

```

        className="flex items-center p-3 rounded-lg text-gray-300 hover:text-white hover:bg-gray-800
transition-all"
        title={isCollapsed ? 'Back to Site' : ''}
    >
    <Home className="w-4 h-4" />
    {!isCollapsed && <span className="ml-3 text-sm font-medium">Back to Site</span>}
</Link>

<button
    onClick={handleLogout}
    className="flex items-center w-full p-3 rounded-lg text-red-400 hover:text-red-300 hover:bg-red-
600 hover:bg-opacity-10 transition-all"
    title={isCollapsed ? 'Sign Out' : ''}
    >
    <LogOut className="w-4 h-4" />
    {!isCollapsed && <span className="ml-3 text-sm font-medium">Sign Out</span>}
</button>
</div>
</div>
)
}

```

```
export default AdminSidebar",
```

```
mainnavbar.jsx
```

```
"// src/components/navigation/MainNavbar.jsx
```

```
import React, { useState, useEffect } from 'react'
```

```
import { Link, useLocation } from 'react-router-dom'
```

```
import {
```

```
Terminal,  
Cpu,  
Trophy,  
User,  
LogOut,  
Shield,  
Menu,  
X  
} from 'lucide-react'  
import { useAuth } from '../contexts/AuthContext'  
  
const MainNavbar = () => {  
  const { user, logout, isAdmin } = useAuth()  
  const location = useLocation()  
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false)  
  const [scrollY, setScrollY] = useState(0)  
  
  useEffect(() => {  
    const handleScroll = () => setScrollY(window.scrollY)  
    window.addEventListener('scroll', handleScroll)  
    return () => window.removeEventListener('scroll', handleScroll)  
  }, [])  
  
  const navigation = [  
    { path: '/', label: 'Home', icon: Terminal },  
    { path: '/challenges', label: 'Challenges', icon: Cpu },  
    { path: '/leaderboard', label: 'Leaderboard', icon: Trophy },  
  ]
```

```
const isActive = (path) => location.pathname === path
```

```
return (
```

```
<nav className={`fixed top-0 left-0 w-full z-50 transition-all duration-300 ${
  scrolly > 50
```

```
  ? 'bg-gray-900/95 backdrop-blur-lg border-b border-gray-700'
```

```
  : 'bg-transparent'
```

```
}`}>
```

```
<div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
```

```
<div className="flex justify-between items-center h-16">
```

```
  {/* Logo */}
```

```
<Link to="/" className="flex items-center space-x-3 group">
```

```
  <div className="flex items-center justify-center w-10 h-10 bg-gradient-to-br from-blue-600 to-
purple-600 rounded-lg">
```

```
    <Terminal className="w-6 h-6 text-white" />
```

```
  </div>
```

```
<div className="flex flex-col">
```

```
  <span className="text-xl font-bold text-white tracking-tight">
```

```
    CyberArena
```

```
  </span>
```

```
  <span className="text-xs text-gray-400">Professional Edition</span>
```

```
</div>
```

```
</Link>
```

```
  {/* Desktop Navigation */}
```

```
<div className="hidden lg:flex space-x-1">
```

```
  {navigation.map((item) => {
```

```
    const Icon = item.icon
```

```
    return (
```



```

<Link
  key={item.path}
  to={item.path}
  className={`relative flex items-center space-x-2 px-4 py-2 rounded-lg transition-all duration-
200 ${
    isActive(item.path)
      ? 'text-white bg-blue-600 shadow-lg shadow-blue-500/25'
      : 'text-gray-300 hover:text-white hover:bg-gray-800/50'
  }}
>
  <Icon className="w-4 h-4" />
  <span className="font-medium text-sm">{item.label}</span>
</Link>
)
}}
</div>

```

```

{/* Right Section */}
<div className="flex items-center space-x-4">
  {user ? (
    <>
      {/* User Actions */}
      <div className="hidden md:flex items-center space-x-3">
        <Link
          to="/dashboard"
          className="px-4 py-2 text-sm font-medium text-white bg-gradient-to-r from-blue-600 to-
purple-600 rounded-lg hover:from-blue-700 hover:to-purple-700 transition-all shadow-lg shadow-blue-
500/25"
        >
          Go to Dashboard

```

```
</Link>
```

```
{isAdmin && (
```

```
<Link
```

```
to="/admin"
```

```
className="flex items-center space-x-2 px-4 py-2 rounded-lg border border-yellow-500  
text-yellow-400 hover:bg-yellow-600 hover:bg-opacity-10 transition-all"
```

```
>
```

```
<Shield className="w-4 h-4" />
```

```
<span className="text-sm font-medium">Admin</span>
```

```
</Link>
```

```
})
```

```
</div>
```

```
{/* Mobile Menu Button */}
```

```
<button
```

```
onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
```

```
className="lg:hidden p-2 rounded-lg text-gray-400 hover:text-white hover:bg-gray-800  
transition-all"
```

```
>
```

```
{isMobileMenuOpen ? <X className="w-6 h-6" /> : <Menu className="w-6 h-6" />}
```

```
</button>
```

```
</>
```

```
): (
```

```
<div className="flex space-x-3">
```

```
<Link
```

```
to="/login"
```

```
className="px-4 py-2 text-sm font-medium text-gray-300 hover:text-white transition-colors"
```

```
>
```

```

        Sign In
      </Link>

      <Link
        to="/register"

        className="px-6 py-2 text-sm font-medium text-white bg-gradient-to-r from-blue-600 to-
purple-600 rounded-lg hover:from-blue-700 hover:to-purple-700 transition-all shadow-lg shadow-blue-
500/25"

      >

        Get Started
      </Link>
    </div>
  )}
</div>
</div>

```

```

{/* Mobile Navigation */}

{isMobileMenuOpen && user && (
  <div className="lg:hidden border-t border-gray-700 py-4 space-y-2 bg-gray-900/95 backdrop-
blur-lg">
    {navigation.map((item) => {
      const Icon = item.icon
      return (
        <Link
          key={item.path}
          to={item.path}
          onClick={() => setIsMobileMenuOpen(false)}
          className={`flex items-center space-x-3 px-4 py-3 rounded-lg transition-all ${
            isActive(item.path)
              ? 'text-white bg-blue-600'
              : 'text-gray-300 hover:text-white hover:bg-gray-800'
          }`}
        >

```

```

    }}
  >
    <Icon className="w-5 h-5" />
    <span className="font-medium">{item.label}</span>
  </Link>
)
}}

```

```

<div className="pt-4 border-t border-gray-700 space-y-2">
  <Link
    to="/dashboard"
    onClick={() => setIsMobileMenuOpen(false)}
    className="flex items-center space-x-3 px-4 py-3 rounded-lg bg-gradient-to-r from-blue-600
to-purple-600 text-white font-medium"
  >
    <User className="w-5 h-5" />
    <span>Go to Dashboard</span>
  </Link>

```

```

{isAdmin && (
  <Link
    to="/admin"
    onClick={() => setIsMobileMenuOpen(false)}
    className="flex items-center space-x-3 px-4 py-3 rounded-lg border border-yellow-500 text-
yellow-400"
  >
    <Shield className="w-5 h-5" />
    <span>Admin Panel</span>
  </Link>

```

```
    }}  
  </div>  
</div>  
  })  
</div>  
</nav>  
)  
}
```

```
export default MainNavbar",
```

```
usersidebar.jsx
```

```
"// src/components/navigation/UserSidebar.jsx
```

```
import React, { useState } from 'react'
```

```
import { Link, useLocation, useNavigate } from 'react-router-dom'
```

```
import {
```

```
  User,
```

```
  Settings,
```

```
  Award,
```

```
  LogOut,
```

```
  Terminal,
```

```
  BarChart3,
```

```
  Users,
```

```
  ChevronLeft,
```

```
  ChevronRight,
```

```
  Home,
```

```
  Shield,
```

```
  Bell,
```

```
  BookOpen
```

```
} from 'lucide-react'

import { useAuth } from '../contexts/AuthContext'

const UserSidebar = () => {
  const { user, logout, isAdmin } = useAuth()
  const location = useLocation()
  const navigate = useNavigate()
  const [isCollapsed, setIsCollapsed] = useState(false)

  const userMenu = [
    {
      category: 'Dashboard',
      items: [
        { path: '/dashboard', label: 'Overview', icon: BarChart3, description: 'Your progress' },
        { path: '/dashboard/profile', label: 'Profile', icon: User, description: 'Personal settings' },
      ]
    },
    {
      category: 'Training',
      items: [
        { path: '/dashboard/challenges', label: 'Challenges', icon: Terminal, description: 'Practice exercises' },
        { path: '/dashboard/achievements', label: 'Achievements', icon: Award, description: 'Your badges' },
      ]
    }
  ]

  const isActive = (path) => location.pathname === path

  const handleLogout = () => {
```

```

logout()
navigate('/')
}

return (
  <div className={`flex flex-col bg-gray-900 border-r border-gray-700 transition-all duration-300 ${
    isCollapsed ? 'w-20' : 'w-64'
  }`} >
    { /* Header */ }
    <div className="flex items-center justify-between p-4 border-b border-gray-700">
      { !isCollapsed && (
        <div className="flex items-center space-x-3">
          <div className="flex items-center justify-center w-8 h-8 bg-gradient-to-br from-green-600 to-blue-600 rounded-lg">
            <User className="w-4 h-4 text-white" />
          </div>
          <div className="flex flex-col">
            <span className="text-sm font-bold text-white">Dashboard</span>
            <span className="text-xs text-gray-400">Welcome back</span>
          </div>
        </div>
      ) }
    </div>
  ) }

  { isCollapsed && (
    <div className="flex justify-center w-full">
      <div className="flex items-center justify-center w-8 h-8 bg-gradient-to-br from-green-600 to-blue-600 rounded-lg">
        <User className="w-4 h-4 text-white" />
      </div>
    </div>
  ) }

```

```
</div>
```

```
}}
```

```
<button
```

```
  onClick={() => setIsCollapsed(!isCollapsed)}
```

```
  className="p-1 rounded-lg text-gray-400 hover:text-white hover:bg-gray-800 transition-all"
```

```
>
```

```
  {isCollapsed ? <ChevronRight className="w-4 h-4" /> : <ChevronLeft className="w-4 h-4" />}
```

```
</button>
```

```
</div>
```

```
{/* User Info */}
```

```
{!isCollapsed && (
```

```
  <div className="p-4 border-b border-gray-700">
```

```
    <div className="flex items-center space-x-3">
```

```
      <div className="flex items-center justify-center w-10 h-10 bg-gradient-to-br from-blue-500 to-purple-500 rounded-lg">
```

```
        <User className="w-5 h-5 text-white" />
```

```
      </div>
```

```
    <div className="flex flex-col flex-1 min-w-0">
```

```
      <span className="text-sm font-medium text-white truncate">{user?.username}</span>
```

```
      <span className="text-xs text-gray-400">
```

```
        Level {user?.level || 1} • {user?.points || 0} points
```

```
      </span>
```

```
    </div>
```

```
</div>
```

```
</div>
```

```
}}
```



```

{/* Navigation */}

<div className="flex-1 overflow-y-auto py-4">

  {userMenu.map((section, index) => (

    <div key={index} className="mb-6">

      {!isCollapsed && (

        <div className="px-4 mb-2">

          <span className="text-xs font-semibold text-gray-500 uppercase tracking-wider">

            {section.category}

          </span>

        </div>

      )}

    <div className="space-y-1">

      {section.items.map((item) => {

        const Icon = item.icon

        return (

          <Link

            key={item.path}

            to={item.path}

            className={`flex items-center mx-2 p-3 rounded-lg transition-all group ${

              isActive(item.path)

                ? 'bg-green-600 text-white shadow-lg shadow-green-500/25'

                : 'text-gray-300 hover:text-white hover:bg-gray-800'

            }`}

          >

            <Icon className={`flex-shrink-0 ${isCollapsed ? 'w-5 h-5' : 'w-4 h-4'}`} />

            {!isCollapsed && (

```

```

        <div className="ml-3 flex-1 min-w-0">
            <span className="text-sm font-medium">{item.label}</span>
            <p className="text-xs text-gray-400 truncate">{item.description}</p>
        </div>
    )}
</Link>
)
}}
</div>
</div>
)}}
</div>

```

```

{/* Footer Actions */}
<div className="p-4 border-t border-gray-700 space-y-2">
    <Link
        to="/"
        className="flex items-center p-3 rounded-lg text-gray-300 hover:text-white hover:bg-gray-800
transition-all"
        title={isCollapsed ? 'Back to Home' : ''}
    >
        <Home className="w-4 h-4" />
        {!isCollapsed && <span className="ml-3 text-sm font-medium">Back to Home</span>}
    </Link>

    {isAdmin && (
        <Link
            to="/admin"

```

```
      className="flex items-center p-3 rounded-lg text-yellow-400 hover:text-yellow-300 hover:bg-yellow-600 hover:bg-opacity-10 transition-all"
```

```
      title={isCollapsed ? 'Admin Panel' : ''}
```

```
>
```

```
<Shield className="w-4 h-4" />
```

```
{!isCollapsed && <span className="ml-3 text-sm font-medium">Admin Panel</span>}
```

```
</Link>
```

```
}}
```

```
<button
```

```
  onClick={handleLogout}
```

```
  className="flex items-center w-full p-3 rounded-lg text-red-400 hover:text-red-300 hover:bg-red-600 hover:bg-opacity-10 transition-all"
```

```
  title={isCollapsed ? 'Sign Out' : ''}
```

```
>
```

```
<LogOut className="w-4 h-4" />
```

```
{!isCollapsed && <span className="ml-3 text-sm font-medium">Sign Out</span>}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
)
```

```
}
```

```
export default UserSidebar",
```

```
usermanu.jsx
```

```
"import React, { useState, useRef, useEffect } from 'react'
```

```
import {
```

```
  Terminal,
```

```
    LogOut,  
    User,  
    Settings,  
    Award,  
    Shield,  
    Cpu,  
    Network,  
    Binary,  
    Key,  
    Scan,  
    Zap,  
    Eye,  
    Server,  
    Clock  
  } from 'lucide-react'  
  
import { useAuth } from '../contexts/AuthContext'  
  
const UserMenu = () => {  
  const [isOpen, setIsOpen] = useState(false)  
  const [currentTime, setCurrentTime] = useState("")  
  const { user, logout, isAdmin } = useAuth()  
  const menuRef = useRef(null)  
  
  useEffect(() => {  
    const handleClickOutside = (event) => {  
      if (menuRef.current && !menuRef.current.contains(event.target)) {  
        setIsOpen(false)  
      }  
    }  
  })  
}
```

```

// Update current time
const updateTime = () => {
  setCurrentTime(new Date().toLocaleTimeString('en-US', {
    hour12: false,
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  )))
}

updateTime()
const interval = setInterval(updateTime, 1000)
document.addEventListener('mousedown', handleClickOutside)

return () => {
  document.removeEventListener('mousedown', handleClickOutside)
  clearInterval(interval)
}
}, [])

const menuItems = [
  {
    icon: User,
    label: 'USER_PROFILE',
    command: 'whoami',
    onClick: () => console.log('Profile'),
    color: 'text-cyan-400'
  },

```

```

{
  icon: Award,
  label: 'ACHIEVEMENTS',
  command: 'cat /var/log/achievements',
  onClick: () => console.log('Achievements'),
  color: 'text-yellow-400'
},
{
  icon: Shield,
  label: 'SECURITY',
  command: './security_check',
  onClick: () => console.log('Security'),
  color: 'text-green-400'
},
{
  icon: Settings,
  label: 'SYSTEM_CONFIG',
  command: 'nano ~/.config',
  onClick: () => console.log('Settings'),
  color: 'text-blue-400'
},
]

```

```

const getRoleBadge = (role) => {
  switch (role) {
    case 'admin':
      return { text: 'ROOT_ACCESS', color: 'text-red-400 border-red-500 bg-red-500 bg-opacity-10' }
    case 'user':

```

```
    return { text: 'STANDARD_ACCESS', color: 'text-green-400 border-green-500 bg-green-500 bg-opacity-10' }
```

```
    default:
```

```
    return { text: 'GUEST_ACCESS', color: 'text-gray-400 border-gray-500 bg-gray-500 bg-opacity-10' }  
  }  
}
```

```
const roleBadge = getRoleBadge(user?.role)
```

```
return (
```

```
  <div className="relative font-mono" ref={menuRef}>
```

```
    <button
```

```
      onClick={() => setIsOpen(!isOpen)}
```

```
      className="flex items-center space-x-3 p-2 rounded-lg border border-green-500 hover:border-cyan-400 hover:bg-cyan-500 hover:bg-opacity-5 transition-all group"
```

```
    >
```

```
      <div className="relative">
```

```
        <div className="w-10 h-10 bg-gradient-to-r from-green-500 to-cyan-500 rounded-full flex items-center justify-center text-white font-bold text-sm border-2 border-white border-opacity-20">
```

```
          {user?.username?.charAt(0).toUpperCase()}
```

```
        </div>
```

```
        <div className="absolute -bottom-1 -right-1 w-4 h-4 bg-green-500 rounded-full border-2 border-black"></div>
```

```
      </div>
```

```
    <div className="text-left hidden md:block">
```

```
      <p className="text-sm font-bold text-white">{user?.username}</p>
```

```
      <p className="text-xs text-green-400 flex items-center space-x-1">
```

```
        <span className={`px-1 py-0.5 rounded text-xs border ${roleBadge.color}`}>
```

```
          {roleBadge.text}
```


</p>

</div>

<div className={`transform transition-transform \${isOpen ? 'rotate-180' : ''}`>

<div className="text-green-400 text-xs">▼</div>

</div>

</button>

{isOpen && (

<div className="absolute right-0 mt-2 w-80 bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20 py-2 z-50 overflow-hidden">

{/* Terminal Header */}

<div className="flex items-center space-x-2 p-3 border-b border-green-500/30 bg-black">

<div className="w-2 h-2 bg-red-500 rounded-full"></div>

<div className="w-2 h-2 bg-yellow-500 rounded-full"></div>

<div className="w-2 h-2 bg-green-500 rounded-full"></div>

<div className="text-green-400 text-xs ml-2">user@cyber-arena:~</div>

</div>

{/* User Info Section */}

<div className="p-4 border-b border-green-500/30">

<div className="flex items-center space-x-3 mb-3">

<div className="w-12 h-12 bg-gradient-to-r from-green-500 to-cyan-500 rounded-full flex items-center justify-center text-white font-bold border-2 border-white border-opacity-20">

{user?.username?.charAt(0).toUpperCase()}

</div>

<div className="flex-1">

<p className="text-white font-bold text-sm">{user?.username}</p>


```

<p className="text-green-300 text-xs">{user?.email}</p>
<div className="flex items-center space-x-2 mt-1">
  <span className={`px-2 py-0.5 rounded text-xs border ${roleBadge.color}`}>
    {roleBadge.text}
  </span>
  <span className="text-cyan-400 text-xs bg-cyan-500 bg-opacity-10 px-1 rounded border
border-cyan-500">
    ACTIVE
  </span>
</div>
</div>
</div>

```

```

{/* System Status */}
<div className="grid grid-cols-2 gap-3 text-xs">
  <div className="bg-black border border-green-500 rounded p-2 text-center">
    <div className="text-green-300">SESSION</div>
    <div className="text-white font-bold">SECURE</div>
  </div>
  <div className="bg-black border border-green-500 rounded p-2 text-center">
    <div className="text-green-300">TIME</div>
    <div className="text-white font-bold">{currentTime}</div>
  </div>
</div>
</div>

```

```

{/* Quick Stats */}
<div className="p-3 border-b border-green-500/30 bg-black bg-opacity-50">
  <div className="grid grid-cols-3 gap-2 text-center text-xs">

```

```

<div>
  <div className="text-cyan-400 font-bold">12</div>
  <div className="text-green-300">TARGETS</div>
</div>
<div>
  <div className="text-yellow-400 font-bold">850</div>
  <div className="text-green-300">POINTS</div>
</div>
<div>
  <div className="text-green-400 font-bold">#24</div>
  <div className="text-green-300">RANK</div>
</div>
</div>
</div>

```

```

{/* Menu Items */}
<div className="py-2">
  {menuItems.map((item, index) => {
    const Icon = item.icon
    return (
      <button
        key={index}
        onClick={() => {
          item.onClick()
          setIsOpen(false)
        }}
        className="w-full flex items-center justify-between px-4 py-3 text-sm hover:bg-green-500
        hover:bg-opacity-10 transition-all group"
      >

```

```

<div className="flex items-center space-x-3">

  <Icon className={`w-4 h-4 ${item.color}`} />

  <span className="text-white font-bold">{item.label}</span>

</div>

<div className="text-green-400 text-xs opacity-0 group-hover:opacity-100 transition-
opacity">
  {item.command}
</div>
</button>
)
}}}

{/* Admin Access */}
{isAdmin && (
  <button
    onClick={() => {
      console.log('Admin Panel')
      setIsOpen(false)
    }}
    className="w-full flex items-center justify-between px-4 py-3 text-sm hover:bg-yellow-500
hover:bg-opacity-10 transition-all group border-t border-yellow-500 border-opacity-30"
    >

    <div className="flex items-center space-x-3">

      <Shield className="w-4 h-4 text-yellow-400" />

      <span className="text-yellow-400 font-bold">ADMIN_PANEL</span>

    </div>

    <div className="text-yellow-400 text-xs opacity-0 group-hover:opacity-100 transition-
opacity">
      sudo ./admin_panel
    </div>
  </button>
)
}

```

```
</button>
```

```
}}
```

```
</div>
```

```
{/* Logout Section */}
```

```
<div className="border-t border-red-500 border-opacity-30 pt-2">
```

```
<button
```

```
  onClick={() => {
```

```
    logout()
```

```
    setIsOpen(false)
```

```
  }}
```

```
  className="w-full flex items-center justify-between px-4 py-3 text-sm hover:bg-red-500  
  hover:bg-opacity-10 transition-all group"
```

```
>
```

```
<div className="flex items-center space-x-3">
```

```
<LogOut className="w-4 h-4 text-red-400" />
```

```
<span className="text-red-400 font-bold">TERMINATE_SESSION</span>
```

```
</div>
```

```
<div className="text-red-400 text-xs opacity-0 group-hover:opacity-100 transition-opacity">
```

```
  kill -9 $$
```

```
</div>
```

```
</button>
```

```
</div>
```

```
{/* Footer */}
```

```
<div className="px-4 py-2 border-t border-green-500 border-opacity-30 bg-black bg-opacity-50">
```

```
<div className="flex items-center justify-between text-xs text-green-600">
```

```
<span>SESSION_ACTIVE</span>
```

```
<span>v2.1.4</span>
```

```
        </div>
      </div>
    </div>
  )}
</div>
)
}
```

```
export default UserMenu"
```

```
src/components/security/securechallengestart.jsx
```

```
"import React, { useState } from 'react'
```

```
const SecureChallengeStart = ({ challenge, onStart, onCancel }) => {
  const [acceptedTerms, setAcceptedTerms] = useState(false)
  const [webcamTested, setWebcamTested] = useState(false)
  const [isTestingWebcam, setIsTestingWebcam] = useState(false)
```

```
const testWebcam = async () => {
  setIsTestingWebcam(true)
  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: { width: 1280, height: 720 }
    })
    // Stop the stream after testing
    stream.getTracks().forEach(track => track.stop())
    setWebcamTested(true)
  } catch (error) {
    alert('Webcam access is required to continue with the test.')
```

```
    } finally {  
      setIsTestingWebcam(false)  
    }  
  }  
}
```

```
const handleStart = () => {  
  if (!acceptedTerms || !webcamTested) return  
  onStart()  
}
```

```
return (  
  <div className="min-h-screen bg-gradient-to-br from-gray-900 to-gray-800 flex items-center justify-center p-4">  
    <div className="max-w-2xl w-full bg-gray-800 rounded-2xl shadow-2xl overflow-hidden">  
      {/* Header */}  
      <div className="bg-red-600 p-6 text-center">  
        <h1 className="text-3xl font-bold text-white mb-2">Secure Test Environment</h1>  
        <p className="text-red-100">Enhanced security monitoring enabled</p>  
      </div>  
  
      {/* Content */}  
      <div className="p-8 space-y-6">  
        {/* Challenge Info */}  
        <div className="bg-gray-700 rounded-lg p-6">  
          <h2 className="text-xl font-semibold text-white mb-2">{challenge.title}</h2>  
          <div className="flex flex-wrap gap-4 text-sm text-gray-300">  
            <span className="flex items-center">  
              <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20">
```

```
<path fillRule="evenodd" d="M12 7a1 1 0 110-2h5a1 1 0 011 1v5a1 1 0 11-2 0V8.414l-4.293
4.293a1 1 0 01-1.414 0L8 10.414l-4.293 4.293a1 1 0 01-1.414-1.414l5-5a1 1 0 011.414 0L11 10.586
14.586 7H12z" clipRule="evenodd" />
```

```
</svg>
```

```
{challenge.points} Points
```

```
</span>
```

```
<span className="flex items-center">
```

```
<svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20">
```

```
<path fillRule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zm1-11a1 1 0 10-2 0v2H7a1 1
0 100 2h2v2a1 1 0 102 0v-2h2a1 1 0 100-2h-2V7z" clipRule="evenodd" />
```

```
</svg>
```

```
{challenge.category}
```

```
</span>
```

```
<span className="flex items-center">
```

```
<svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20">
```

```
<path fillRule="evenodd" d="M10 18a8 8 0 100-16 8 8 0 00 16zm1-12a1 1 0 10-2 0v4a1 1 0
00.293.707l2.828 2.829a1 1 0 101.415-1.415L11 9.586V6z" clipRule="evenodd" />
```

```
</svg>
```

```
{challenge.difficulty}
```

```
</span>
```

```
</div>
```

```
</div>
```

```
{/* Security Requirements */}
```

```
<div className="bg-yellow-900/30 border border-yellow-600 rounded-lg p-6">
```

```
<h3 className="text-lg font-semibold text-yellow-300 mb-4 flex items-center">
```

```
<svg className="w-5 h-5 mr-2" fill="currentColor" viewBox="0 0 20 20">
```

```
<path fillRule="evenodd" d="M8.257 3.099c.765-1.36 2.722-1.36 3.486 0l5.58 9.92c.75 1.334-
.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-2.98l5.58-9.92zM11 13a1 1 0 11-2 0 1 1 0 012
0zm-1-8a1 1 0 00-1 1v3a1 1 0 002 0V6a1 1 0 00-1-1z" clipRule="evenodd" />
```

```
</svg>
```

Security Requirements

</h3>

<ul className="space-y-3 text-yellow-200">

<li className="flex items-center">

<svg className="w-4 h-4 mr-3 text-yellow-400" fill="currentColor" viewBox="0 0 20 20">

<path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clipRule="evenodd" />

</svg>

Webcam access for facial recognition

<li className="flex items-center">

<svg className="w-4 h-4 mr-3 text-yellow-400" fill="currentColor" viewBox="0 0 20 20">

<path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clipRule="evenodd" />

</svg>

Fullscreen mode throughout the test

<li className="flex items-center">

<svg className="w-4 h-4 mr-3 text-yellow-400" fill="currentColor" viewBox="0 0 20 20">

<path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clipRule="evenodd" />

</svg>

No tab switching allowed

<li className="flex items-center">

<svg className="w-4 h-4 mr-3 text-yellow-400" fill="currentColor" viewBox="0 0 20 20">

<path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clipRule="evenodd" />

</svg>

Clipboard and right-click disabled


```

</li>
<li className="flex items-center">
  <svg className="w-4 h-4 mr-3 text-yellow-400" fill="currentColor" viewBox="0 0 20 20">
    <path fillRule="evenodd" d="M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-7.293a1 1 0 011.414 0z" clipRule="evenodd" />
  </svg>
  Real-time activity monitoring
</li>
</ul>
</div>

```

```

{/* Webcam Test */}
<div className="bg-gray-700 rounded-lg p-6">
  <h3 className="text-lg font-semibold text-white mb-4">Webcam Setup</h3>
  <div className="flex items-center justify-between">
    <div>
      <p className="text-gray-300 mb-2">
        Test your webcam to ensure it's working properly
      </p>
      <p className={`text-sm ${webcamTested ? 'text-green-400' : 'text-yellow-400'}`}>
        {webcamTested ? 'Webcam test successful!' : 'Webcam not tested yet'}
      </p>
    </div>
    <button
      onClick={testWebcam}
      disabled={isTestingWebcam || webcamTested}
      className={`px-6 py-2 rounded-lg font-medium transition-colors ${
        webcamTested
          ? 'bg-green-600 text-white cursor-not-allowed'

```

```

      : isTestingWebcam
      ? 'bg-gray-600 text-gray-400 cursor-not-allowed'
      : 'bg-blue-600 hover:bg-blue-700 text-white'
    }}
  >
  {isTestingWebcam ? 'Testing...' : webcamTested ? 'Tested ✓' : 'Test Webcam'}
</button>
</div>
</div>

```

```

{/* Terms Agreement */}
<div className="bg-gray-700 rounded-lg p-6">
  <label className="flex items-start space-x-3 cursor-pointer">
    <input
      type="checkbox"
      checked={acceptedTerms}
      onChange={(e) => setAcceptedTerms(e.target.checked)}
      className="mt-1 w-4 h-4 text-blue-600 bg-gray-800 border-gray-600 rounded focus:ring-blue-
500 focus:ring-2"
    />
    <div className="text-sm text-gray-300">
      <span className="font-medium">I understand and agree to the following:</span>
      <ul className="mt-2 space-y-1 text-gray-400">
        <li>• I will not switch tabs or windows during the test</li>
        <li>• I will keep my face visible to the webcam at all times</li>
        <li>• I will not use any prohibited keyboard shortcuts</li>
        <li>• I understand that violations may result in test termination</li>
        <li>• I consent to real-time monitoring and recording</li>
      </ul>
    </div>
  </label>

```

```

    </div>
  </label>
</div>

  { /* Action Buttons */ }
  <div className="flex space-x-4 pt-4">
    <button
      onClick={onCancel}
      className="flex-1 px-6 py-3 border border-gray-600 text-gray-300 rounded-lg hover:bg-gray-700 transition-colors font-medium"
    >
      Cancel
    </button>
    <button
      onClick={handleStart}
      disabled={!acceptedTerms || !webcamTested}
      className={`flex-1 px-6 py-3 rounded-lg font-medium transition-colors ${
        acceptedTerms && webcamTested
          ? 'bg-green-600 hover:bg-green-700 text-white'
          : 'bg-gray-600 text-gray-400 cursor-not-allowed'
      }`}
    >
      Start Secure Test
    </button>
  </div>
</div>
</div>
</div>
)

```

```
}
```

```
export default SecureChallengeStart",
```

```
SecureTestEnvironment.jsx
```

```
"import React, { useState, useRef, useEffect, useCallback } from 'react'
```

```
import { toast } from 'react-toastify'
```

```
const SecureTestEnvironment = ({ sessionId, challengeId, onViolation, onTerminate }) => {
```

```
  const [webcamActive, setWebcamActive] = useState(false)
```

```
  const [fullscreen, setFullscreen] = useState(false)
```

```
  const [violations, setViolations] = useState(0)
```

```
  const [sessionStatus, setSessionStatus] = useState('initializing')
```

```
  const videoRef = useRef(null)
```

```
  const canvasRef = useRef(null)
```

```
  const streamRef = useRef(null)
```

```
  const detectionInterval = useRef(null)
```

```
  const focusMonitor = useRef(null)
```

```
  const tabSwitchCount = useRef(0)
```

```
// Initialize secure environment
```

```
useEffect(() => {
```

```
  initializeSecureEnvironment()
```

```
  return () => cleanup()
```

```
}, [])
```

```
const initializeSecureEnvironment = async () => {
```

```
  try {
```

```
// Request fullscreen
await requestFullscreen()

// Initialize webcam
await initializeWebcam()

// Start security monitoring
startSecurityMonitoring()

// Setup event listeners
setupEventListeners()

setSessionStatus('active')
toast.success('Secure test environment activated')

} catch (error) {
  console.error('Failed to initialize secure environment:', error)
  toast.error('Failed to initialize secure environment')
  onTerminate?.(`Initialization failed: ${error.message}`)
}
}

const requestFullscreen = async () => {
  try {
    const element = document.documentElement

    if (element.requestFullscreen) {
      await element.requestFullscreen()
    } else if (element.mozRequestFullScreen) {
```

```

    await element.mozRequestFullscreen()
  } else if (element.webkitRequestFullscreen) {
    await element.webkitRequestFullscreen()
  } else if (element.msRequestFullscreen) {
    await element.msRequestFullscreen()
  }

  setFullscreen(true)

  // Lock orientation if on mobile
  if (screen.orientation && screen.orientation.lock) {
    try {
      await screen.orientation.lock('portrait')
    } catch (e) {
      console.warn('Orientation lock not supported')
    }
  }

} catch (error) {
  throw new Error('Fullscreen access denied')
}
}

```

```

const initializeWebcam = async () => {
  try {
    const constraints = {
      video: {
        width: { ideal: 1280 },
        height: { ideal: 720 },

```

```
    frameRate: { ideal: 15 },  
    facingMode: 'user'  
  },  
  audio: false  
}
```

```
const stream = await navigator.mediaDevices.getUserMedia(constraints)  
streamRef.current = stream
```

```
if (videoRef.current) {  
  videoRef.current.srcObject = stream  
  await videoRef.current.play()  
}
```

```
setWebcamActive(true)
```

```
// Verify webcam with backend  
await verifyWebcamWithBackend()
```

```
} catch (error) {  
  throw new Error(`Webcam access denied: ${error.message}`)  
}  
}
```

```
const verifyWebcamWithBackend = async () => {  
  try {  
    const response = await fetch(`/api/secure-session/${sessionId}/verify-webcam`, {  
      method: 'POST',  
      headers: {
```

```

    'Content-Type': 'application/json',
    'Authorization': `Bearer ${localStorage.getItem('token')}`
  },
  body: JSON.stringify({
    streamData: 'verified'
  })
})

if (!response.ok) {
  throw new Error('Webcam verification failed')
}
} catch (error) {
  throw new Error('Failed to verify webcam with server')
}
}

const startSecurityMonitoring = () => {
  // Start face detection interval
  detectionInterval.current = setInterval(() => {
    performFaceDetection()
  }, 5000) // Check every 5 seconds

  // Start focus monitoring
  focusMonitor.current = setInterval(() => {
    checkFocusStatus()
  }, 1000)
}

const performFaceDetection = async () => {

```



```
if (!videoRef.current || !canvasRef.current) return

try {
  // This would integrate with face-api.js or similar
  // For now, we'll simulate basic checks
  const video = videoRef.current
  const canvas = canvasRef.current
  const context = canvas.getContext('2d')

  // Draw current frame to canvas
  canvas.width = video.videoWidth
  canvas.height = video.videoHeight
  context.drawImage(video, 0, 0, canvas.width, canvas.height)

  // Capture frame for evidence
  const screenshot = canvas.toDataURL('image/jpeg', 0.7)

  // Basic face detection simulation
  const detectionResult = await simulateFaceDetection(video)

  if (detectionResult.alerts.length > 0) {
    detectionResult.alerts.forEach(alert => {
      reportViolation('webcam_issue', {
        alertType: alert.type,
        confidence: alert.confidence
      }, screenshot)
    })
  }
}
```

```
    } catch (error) {  
      console.error('Face detection error:', error)  
    }  
  }  
}
```

```
const simulateFaceDetection = async (videoElement) => {  
  // This is a simulation - in real implementation, use face-api.js  
  return new Promise((resolve) => {  
    // Simulate detection logic  
    const alerts = []  
  
    // Simulate random face detection issues (for demo)  
    if (Math.random() < 0.05) { // 5% chance of fake alert  
      alerts.push({  
        type: Math.random() > 0.5 ? 'multiple_faces' : 'no_face',  
        confidence: 0.8 + Math.random() * 0.2  
      })  
    }  
  
    resolve({  
      faces: Math.random() > 0.1 ? 1 : 0, // 90% chance of one face  
      alerts,  
      confidence: 0.9  
    })  
  })  
}
```

```
const checkFocusStatus = () => {  
  if (!document.hasFocus()) {
```

```
    reportViolation('focus_loss', {  
      duration: 1000 // 1 second  
    })  
  }  
}
```

```
const setupEventListeners = () => {  
  // Visibility change (tab switch)  
  document.addEventListener('visibilitychange', handleVisibilityChange)  
  
  // Fullscreen change  
  document.addEventListener('fullscreenchange', handleFullscreenChange)  
  
  // Context menu (right click)  
  document.addEventListener('contextmenu', handleContextMenu)  
  
  // Keyboard shortcuts  
  document.addEventListener('keydown', handleKeyDown)  
  
  // Copy/paste events  
  document.addEventListener('copy', handleClipboard)  
  document.addEventListener('paste', handleClipboard)  
  document.addEventListener('cut', handleClipboard)  
  
  // Before unload  
  window.addEventListener('beforeunload', handleBeforeUnload)  
}
```

```
const handleVisibilityChange = () => {
```

```
if (document.hidden) {  
  tabSwitchCount.current++  
  reportViolation('tab_switch', {  
    count: tabSwitchCount.current,  
    urls: [] // Can't access actual URLs for security  
  })  
  
  toast.warning('Please return to the test window immediately!')  
}  
}
```

```
const handleFullscreenChange = () => {  
  const isFullscreen = document.fullscreenElement ||  
    document.webkitFullscreenElement ||  
    document.mozFullScreenElement ||  
    document.msFullscreenElement
```

```
if (!isFullscreen) {  
  reportViolation('focus_loss', { type: 'fullscreen_exit' })  
  toast.error('Fullscreen mode is required!')
```

```
  // Attempt to re-enter fullscreen  
  setTimeout(requestFullscreen, 1000)  
}  
}
```

```
const handleContextMenu = (e) => {  
  e.preventDefault()  
  reportViolation('dev_tools', { action: 'right_click' })
```

```
    return false
  }
```

```
const handleKeyDown = (e) => {
  // Block F12, Ctrl+Shift+I, Ctrl+Shift+J, Ctrl+U
  const blockedCombinations = [
    e.key === 'F12',
    (e.ctrlKey && e.shiftKey && e.key === 'I'),
    (e.ctrlKey && e.shiftKey && e.key === 'J'),
    (e.ctrlKey && e.key === 'u'),
    (e.ctrlKey && e.key === 'U'),
    (e.metaKey && e.altKey && e.key === 'I') // Mac
  ]
```

```
  if (blockedCombinations.some(Boolean)) {
    e.preventDefault()
    reportViolation('dev_tools', { action: 'keyboard_shortcut', keys: e.key })
  }
}
```

```
const handleClipboard = (e) => {
  e.preventDefault()
  reportViolation('clipboard', { action: e.type })
  toast.warning('Clipboard operations are disabled during the test')
  return false
}
```

```
const handleBeforeUnload = (e) => {
  e.preventDefault()
```

```
e.returnValue = 'Are you sure you want to leave? This may terminate your test session.'  
return e.returnValue  
}
```

```
const reportViolation = async (type, data, screenshot = null) => {  
  setViolations(prev => prev + 1)  
  
  try {  
    const response = await fetch(`/api/secure-session/${sessionId}/violation`, {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
        'Authorization': `Bearer ${localStorage.getItem('token')}`  
      },  
      body: JSON.stringify({  
        type,  
        data,  
        screenshot  
      })  
    })  
  
    if (!response.ok) {  
      throw new Error('Failed to report violation')  
    }  
  
    const result = await response.json()  
  
    // Notify parent component  
    onViolation?.({
```

```
    type,
    data,
    action: result.action
  })

  // Show warning toast
  if (result.action === 'warning') {
    toast.warning(`Security violation detected: ${type}`)
  }

} catch (error) {
  console.error('Error reporting violation:', error)
}
}

const cleanup = () => {
  // Stop all intervals
  if (detectionInterval.current) {
    clearInterval(detectionInterval.current)
  }

  if (focusMonitor.current) {
    clearInterval(focusMonitor.current)
  }

  // Stop webcam stream
  if (streamRef.current) {
    streamRef.current.getTracks().forEach(track => track.stop())
  }
}
```

```

// Remove event listeners
document.removeEventListener('visibilitychange', handleVisibilityChange)
document.removeEventListener('fullscreenchange', handleFullscreenChange)
document.removeEventListener('contextmenu', handleContextMenu)
document.removeEventListener('keydown', handleKeyDown)
document.removeEventListener('copy', handleClipboard)
document.removeEventListener('paste', handleClipboard)
document.removeEventListener('cut', handleClipboard)
window.removeEventListener('beforeunload', handleBeforeUnload)

// Exit fullscreen
if (document.fullscreenElement) {
  document.exitFullscreen().catch(() => {})
}
}

if (sessionStatus === 'terminated') {
  return (
    <div className="fixed inset-0 bg-gray-900 flex flex-col items-center justify-center text-white p-6">
      <div className="bg-red-900/50 border border-red-500 rounded-lg p-8 max-w-md text-center">
        <div className="w-16 h-16 bg-red-500 rounded-full flex items-center justify-center mx-auto mb-4">
          <svg className="w-8 h-8 text-white" fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-2.5L13.732 4c-.77-.833-1.964-.833-2.732 0L4.082 16.5c-.77.833.192 2.5 1.732 2.5z" />
          </svg>
        </div>
        <h2 className="text-2xl font-bold mb-2">Test Session Terminated</h2>
        <p className="text-red-200 mb-6">

```


Your test session has been terminated due to security violations.

</p>

<button

onClick={() => window.location.reload()}

className="bg-blue-600 hover:bg-blue-700 text-white px-6 py-3 rounded-lg font-medium transition-colors"

>

Return to Dashboard

</button>

</div>

</div>

)

}

return (

<div className="fixed inset-0 bg-gray-900 text-white select-none overflow-hidden">

{/* Security Status Bar */}

<div className="absolute top-0 left-0 right-0 bg-gray-800 border-b border-red-500 px-6 py-3 flex justify-between items-center z-50">

<div className="flex items-center space-x-6">

<div className="flex items-center space-x-2">

<div className={`w-3 h-3 rounded-full \${webcamActive ? 'bg-green-500 animate-pulse' : 'bg-red-500'} `}></div>

Webcam: {webcamActive ? 'Active' : 'Inactive'}

</div>

<div className="flex items-center space-x-2">

<div className={`w-3 h-3 rounded-full \${fullscreen ? 'bg-green-500' : 'bg-red-500'} `}></div>


```

        Fullscreen: {fullscreen ? 'Active' : 'Inactive'}
    </span>
</div>
</div>
<div className="flex items-center space-x-4">
    <div className="text-sm font-medium bg-red-600 px-3 py-1 rounded-full">
        Violations: {violations}
    </div>
</div>
</div>

{/* Hidden video and canvas for face detection */}
<video
    ref={videoRef}
    autoPlay
    muted
    playsInline
    className="hidden"
/>
<canvas
    ref={canvasRef}
    className="hidden"
/>

{/* Main test content */}
<div className="pt-16 h-full overflow-y-auto">
    <div className="max-w-4xl mx-auto p-6">
        <div className="bg-gray-800 rounded-lg p-8 mb-6">
            <h1 className="text-3xl font-bold mb-4 text-center">Secure Test Environment</h1>

```

<p className="text-gray-300 text-center mb-6">

You are being monitored for security purposes. Please follow all instructions carefully.

</p>

<div className="bg-red-900/30 border-l-4 border-red-500 p-4 rounded">

<div className="flex items-start space-x-3">

<svg className="w-5 h-5 text-red-400 mt-0.5 flex-shrink-0" fill="currentColor" viewBox="0 0 20 20">

<path fillRule="evenodd" d="M8.257 3.099c.765-1.36 2.722-1.36 3.486 0l5.58 9.92c.75 1.334-.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-2.98l5.58-9.92zM11 13a1 1 0 11-2 0 1 1 0 0 12 0zm-1-8a1 1 0 00-1 1v3a1 1 0 002 0V6a1 1 0 00-1-1z" clipRule="evenodd" />

</svg>

<div className="space-y-2">

<p className="text-red-200 font-medium">⚠ Do not switch tabs or windows</p>

<p className="text-red-200 font-medium">⚠ Keep your face visible to the webcam</p>

<p className="text-red-200 font-medium">⚠ Do not use keyboard shortcuts</p>

<p className="text-red-200 font-medium">⚠ Fullscreen mode is required</p>

<p className="text-red-200 font-medium">⚠ Do not copy/paste any content</p>

</div>

</div>

</div>

</div>

{/* Challenge content area */}

<div className="bg-gray-800 rounded-lg p-6">

<h2 className="text-xl font-semibold mb-4">Challenge: {challengeId}</h2>

<div className="prose prose-invert max-w-none">

{/* Your challenge content will be rendered here */}

<div className="border-2 border-dashed border-gray-600 rounded-lg p-8 text-center">

```
    <p className="text-gray-400">Challenge content will appear here</p>
  </div>
</div>
</div>
</div>
</div>
```

```
{/* Webcam preview */}
{webcamActive && (
  <div className="fixed bottom-6 right-6 bg-black/80 backdrop-blur-sm rounded-lg p-4 border-2
border-blue-500 z-40">
    <div className="text-center mb-2">
      <span className="text-blue-400 text-sm font-medium flex items-center justify-center">
        <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20">
          <path d="M2 6a2 2 0 0 1 2-2h6a2 2 0 0 1 2 2v8a2 2 0 0 1 2 2h4a2 2 0 0 1 2 2v6z" data-bbox="111 478 878 511"/>
        </svg>
        Webcam Active
      </span>
    </div>
    <video
      ref={videoRef}
      autoPlay
      muted
      playsInline
      className="w-48 h-36 rounded-lg border border-blue-400"
    />
    <p className="text-blue-300 text-xs mt-2 text-center">You are being monitored</p>
  </div>
```

```

    })

    {/* Violation flash effect */}
    {violations > 0 && (
      <div className="absolute inset-0 border-4 border-red-500 animate-pulse pointer-events-none z-30"></div>
    )}
  </div>
)
}

```

```
export default SecureTestEnvironment"
```

```
src/components/stats/adminstats.js
```

```
"// src/components/stats/AdminStats.jsx
```

```
import React from 'react'
```

```
import { Users, Flag, BarChart3, Server } from 'lucide-react'
```

```

const AdminStats = () => {
  const stats = [
    {
      label: 'Total Users',
      value: '1,247',
      change: '+12%',
      icon: Users,
      color: 'text-blue-600 bg-blue-50 border-blue-200'
    },
    {
      label: 'Active Challenges',

```

```

    value: '24',
    change: '+3',
    icon: Flag,
    color: 'text-green-600 bg-green-50 border-green-200'
  },
  {
    label: 'Submissions Today',
    value: '156',
    change: '+23%',
    icon: BarChart3,
    color: 'text-purple-600 bg-purple-50 border-purple-200'
  },
  {
    label: 'System Uptime',
    value: '99.97%',
    change: 'Stable',
    icon: Server,
    color: 'text-yellow-600 bg-yellow-50 border-yellow-200'
  }
]

```

```

return (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
    {stats.map((stat, index) => {
      const Icon = stat.icon
      return (
        <div key={index} className={`bg-white border rounded-lg p-6 shadow-sm ${stat.color}`}>
          <div className="flex items-center justify-between mb-4">
            <div className="p-2 bg-white rounded-lg shadow-sm">

```

```

        <Icon className="w-5 h-5" />
      </div>

      <span className={`text-sm font-medium ${
        stat.change.includes('+') ? 'text-green-600' :
        stat.change === 'Stable' ? 'text-yellow-600' : 'text-red-600'
      }`}>
        {stat.change}
      </span>
    </div>
    <div>
      <p className="text-2xl font-bold text-gray-900">{stat.value}</p>
      <p className="text-sm text-gray-600 mt-1">{stat.label}</p>
    </div>
  </div>
)
}}
</div>
)
}

```

```
export default AdminStats",
```

```
userstats.js
```

```
"import React, { useState, useEffect } from 'react'
```

```
import {
```

```
  Trophy,
```

```
  Target,
```

```
  Zap,
```

```
  TrendingUp,
```

```
Terminal,  
Shield,  
Cpu,  
Network,  
Binary,  
FileSearch,  
Key,  
Bug,  
Server,  
Clock,  
Activity,  
Award,  
Star,  
User,  
TrendingDown,  
AlertTriangle  
} from 'lucide-react'
```

```
const UserStats = ({ user, stats }) => {  
  const [animatedStats, setAnimatedStats] = useState({  
    totalPoints: 0,  
    solvedChallenges: 0,  
    rank: 0,  
    successRate: 0  
  })
```

```
  const [isVisible, setIsVisible] = useState(false)
```

```
  useEffect(() => {
```



```

// Trigger animation when component comes into view
setIsVisible(true)
}, [])

useEffect(() => {
  if (!isVisible) return

  // Animate stats counting up
  const animateStats = () => {
    const duration = 1500 // 1.5 seconds
    const steps = 60
    const stepDuration = duration / steps

    let currentStep = 0

    const interval = setInterval(() => {
      currentStep++
      const progress = currentStep / steps

      setAnimatedStats({
        totalPoints: Math.floor((stats?.totalPoints || 0) * progress),
        solvedChallenges: Math.floor((stats?.solvedChallenges || 0) * progress),
        rank: stats?.rank ? Math.max(1, Math.floor(stats.rank - (stats.rank - 1) * (1 - progress))) : 0,
        successRate: Math.floor((stats?.successRate || 0) * progress)
      })

      if (currentStep >= steps) {
        clearInterval(interval)
        setAnimatedStats({

```

```

        totalPoints: stats?.totalPoints || 0,
        solvedChallenges: stats?.solvedChallenges || 0,
        rank: stats?.rank || 0,
        successRate: stats?.successRate || 0
      })
    }
  }, stepDuration)

  return () => clearInterval(interval)
}

animateStats()
}, [stats, isVisible])

const getCategoryStats = () => {
  // This would typically come from props or API
  return [
    { category: 'web', solved: 3, total: 8, icon: Network, color: 'from-blue-500 to-cyan-500', command:
'scan_web_targets' },
    { category: 'crypto', solved: 2, total: 6, icon: Key, color: 'from-yellow-500 to-amber-500', command:
'decrypt_targets' },
    { category: 'forensics', solved: 1, total: 5, icon: FileSearch, color: 'from-orange-500 to-red-500',
command: 'analyze_evidence' },
    { category: 'pwn', solved: 0, total: 4, icon: Binary, color: 'from-purple-500 to-pink-500', command:
'exploit_binaries' },
    { category: 'reverse', solved: 2, total: 5, icon: Cpu, color: 'from-green-500 to-emerald-500', command:
'reverse_engineering' },
    { category: 'misc', solved: 1, total: 3, icon: Bug, color: 'from-indigo-500 to-purple-500', command:
'misc_operations' }
  ]
}

```

```
const statCards = [  
  {  
    icon: Trophy,  
    label: 'TOTAL_POINTS',  
    value: animatedStats.totalPoints,  
    suffix: 'PTS',  
    color: 'from-yellow-500 to-orange-500',  
    command: 'cat /var/log/points',  
    description: 'Accumulated mission rewards',  
    gradient: 'bg-gradient-to-r from-yellow-500 to-orange-500'  
  },  
  {  
    icon: Target,  
    label: 'SUCCESSFUL_BREACHES',  
    value: `${animatedStats.solvedChallenges}/${stats?.totalChallenges || 0}`,  
    suffix: 'TARGETS',  
    color: 'from-green-500 to-emerald-500',  
    command: 'grep "SUCCESS" /var/log/breaches',  
    description: 'Completed system infiltrations',  
    gradient: 'bg-gradient-to-r from-green-500 to-emerald-500'  
  },  
  {  
    icon: Zap,  
    label: 'GLOBAL_RANK',  
    value: `#${animatedStats.rank || '--}'`,  
    suffix: 'POSITION',  
    color: 'from-cyan-500 to-blue-500',  
    command: './rank_checker --user current',  
  }  
]
```

```

    description: 'Operator standing worldwide',
    gradient: 'bg-gradient-to-r from-cyan-500 to-blue-500'
  },
  {
    icon: TrendingUp,
    label: 'SUCCESS_RATE',
    value: `${animatedStats.successRate}%`,
    suffix: 'EFFICIENCY',
    color: 'from-purple-500 to-pink-500',
    command: 'calc success_ratio',
    description: 'Mission completion ratio',
    gradient: 'bg-gradient-to-r from-purple-500 to-pink-500'
  }
]

```

```
const categoryStats = getCategoryStats()
```

```

const recentActivity = [
  { action: 'Breached Web-200', time: '2m ago', points: '+200', status: 'success', category: 'web' },
  { action: 'Solved Crypto-100', time: '15m ago', points: '+100', status: 'success', category: 'crypto' },
  { action: 'Completed Forensics-150', time: '1h ago', points: '+150', status: 'success', category: 'forensics' },
  { action: 'Failed Pwn-300', time: '2h ago', points: '+0', status: 'failed', category: 'pwn' }
]

```

```

const getStatusIcon = (status) => {
  return status === 'success' ? '√' : 'X'
}

```

```

const getStatusColor = (status) => {
  return status === 'success'
    ? 'bg-green-500 bg-opacity-20 text-green-400 border-green-500'
    : 'bg-red-500 bg-opacity-20 text-red-400 border-red-500'
}

return (
  <div className="space-y-6 animate-fade-in">
    {/* Terminal Header */}

    <div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20
transform transition-all duration-300 hover:shadow-green-500/30">

      <div className="flex items-center justify-between p-4 border-b border-green-500/30">

        <div className="flex items-center space-x-2">

          <div className="w-3 h-3 bg-red-500 rounded-full cursor-pointer hover:bg-red-400 transition-
colors"></div>

          <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>

          <div className="w-3 h-3 bg-green-500 rounded-full"></div>

          <div className="text-green-400 text-sm ml-2 font-mono">cyber-arena.ctf --
operator_stats</div>

        </div>

        <div className="flex items-center space-x-2 text-green-400 text-sm">

          <User className="w-4 h-4" />

          <span className="font-mono">{user?.username || 'OPERATOR'}</span>

        </div>

      </div>

      <div className="p-6">

        <div className="flex items-center space-x-2 text-green-300 mb-6">

          <Terminal className="w-5 h-5" />

          <span className="text-sm font-mono">OPERATOR_PERFORMANCE_METRICS</span>

```

</div>

{/* Main Stats Grid */}

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4 mb-8">

{statCards.map((stat, index) => {

const Icon = stat.icon

return (

<div

key={index}

className="bg-black border border-green-500 rounded-lg p-4 hover:border-cyan-400
transition-all duration-300 group hover:transform hover:scale-105 relative overflow-hidden"

style={{ animationDelay: `\${index \* 100}ms` }}

>

{/* Animated background effect */}

<div className="absolute inset-0 bg-gradient-to-r from-green-500 to-cyan-500 opacity-0
group-hover:opacity-5 transition-opacity duration-300"></div>

<div className="flex items-center justify-between mb-3 relative z-10">

<div className={`p-3 rounded-lg \${stat.gradient} border border-white border-opacity-20
shadow-lg`}>

<Icon className="w-5 h-5 text-white" />

</div>

<div className="text-green-400 text-xs font-bold bg-green-500 bg-opacity-10 px-2 py-1
rounded border border-green-500 font-mono">

{stat.suffix}

</div>

</div>

<div className="space-y-2 relative z-10">

<div className="text-2xl font-bold text-white glitch font-mono" data-text={stat.value}>

```

        {stat.value}
    </div>

    <div className="text-sm text-green-300 font-bold font-mono">{stat.label}</div>
    <div className="text-green-600 text-xs">{stat.description}</div>
</div>

    {/* Hover command */}

    <div className="absolute bottom-3 left-4 right-4 opacity-0 group-hover:opacity-100
transition-opacity duration-300">

        <div className="text-cyan-400 text-xs font-mono bg-black bg-opacity-50 p-2 rounded
border border-cyan-500">

            $ {stat.command}

        </div>

    </div>

</div>

)

}}

</div>

    {/* Category Breakdown */}

    <div className="bg-black border border-cyan-500 rounded-lg p-6 mb-6 transform transition-all
duration-300 hover:border-cyan-400">

        <div className="flex items-center justify-between mb-6">

            <div className="flex items-center space-x-2 text-cyan-400">

                <Activity className="w-5 h-5" />

                <span className="text-sm font-bold font-mono">CATEGORY_BREAKDOWN</span>

            </div>

            <div className="text-green-400 text-xs font-mono bg-green-500 bg-opacity-10 px-3 py-1
rounded border border-green-500">

                ./analyze_performance.sh

            </div>

        </div>

```

```
</div>
```

```
</div>
```

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {categoryStats.map((cat, index) => {
    const Icon = cat.icon

    const percentage = cat.total > 0 ? Math.round((cat.solved / cat.total) * 100) : 0

    return (
      <div
        key={index}
        className="bg-black border border-green-500 rounded-lg p-4 hover:border-cyan-400
        transition-all duration-300 group hover:transform hover:scale-105 relative overflow-hidden"
        style={{ animationDelay: `${index * 150}ms` }}
      >
        <div className="flex items-center justify-between mb-3">
          <div className="flex items-center space-x-3">
            <div className={`w-10 h-10 rounded-lg bg-gradient-to-r ${cat.color} flex items-center
            justify-center shadow-lg`}>
              <Icon className="w-5 h-5 text-white" />
            </div>
            <div>
              <div className="text-white font-bold text-sm uppercase font-
              mono">{cat.category}</div>
              <div className="text-green-300 text-xs font-mono">
                {cat.solved}/{cat.total} BREACHED
              </div>
            </div>
          </div>
          <div className="text-right">
```



```

    <div className="text-cyan-400 font-bold text-lg">{percentage}%</div>

    <div className="text-green-400 text-xs font-mono">SUCCESS</div>

  </div>
</div>

<div className="w-full bg-gray-800 rounded-full h-2 mb-2 overflow-hidden">

  <div

    className={`h-2 rounded-full bg-gradient-to-r ${cat.color} transition-all duration-1000
ease-out`}

    style={{ width: `${percentage}%` }}

  ></div>

</div>

<div className="flex justify-between text-xs text-green-400 font-mono">

  <span>PROGRESS</span>

  <span>{percentage}%</span>

</div>

  { /* Hover command */ }

  <div className="absolute bottom-2 left-4 right-4 opacity-0 group-hover:opacity-100
transition-opacity duration-300">

    <div className="text-cyan-400 text-xs font-mono bg-black bg-opacity-70 p-1 rounded text-
center border border-cyan-500">

      $ {cat.command}

    </div>

  </div>

</div>

)

}}

</div>

```

```
</div>
```

```
{/* Performance Metrics */}
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-6">
```

```
  {/* Recent Activity */}
```

```
    <div className="bg-black border border-green-500 rounded-lg p-4 transform transition-all duration-300 hover:border-cyan-400">
```

```
      <div className="flex items-center justify-between mb-4">
```

```
        <div className="flex items-center space-x-2 text-green-400">
```

```
          <Clock className="w-4 h-4" />
```

```
          <span className="text-sm font-bold font-mono">RECENT_ACTIVITY</span>
```

```
        </div>
```

```
      <div className="text-green-400 text-xs font-mono bg-green-500 bg-opacity-10 px-2 py-1 rounded border border-green-500">
```

```
        tail -f /var/log/activity
```

```
      </div>
```

```
</div>
```

```
<div className="space-y-3">
```

```
  {recentActivity.map((activity, index) => (
```

```
    <div
```

```
      key={index}
```

```
      className="flex items-center justify-between p-3 hover:bg-green-500 hover:bg-opacity-5 rounded transition-all duration-200 group"
```

```
    >
```

```
      <div className="flex items-center space-x-3">
```

```
        <div className={`w-8 h-8 rounded-full flex items-center justify-center border ${getStatusColor(activity.status)} transition-all duration-300 group-hover:scale-110`}>
```

```
          {getStatusIcon(activity.status)}
```

```
        </div>
```

```
      <div>
```

```

        <div className="text-white text-sm font-bold font-mono">{activity.action}</div>
        <div className="text-green-300 text-xs font-mono">{activity.time}</div>
    </div>
</div>
<div className={`text-sm font-bold font-mono transition-all duration-300 group-hover:scale-
110 ${
    activity.status === 'success' ? 'text-yellow-400' : 'text-red-400'
}`}>
    {activity.points}
</div>
</div>
)}}
</div>
</div>

```

```

{/* System Status */}
<div className="bg-black border border-cyan-500 rounded-lg p-4 transform transition-all
duration-300 hover:border-cyan-400">
    <div className="flex items-center justify-between mb-4">
        <div className="flex items-center space-x-2 text-cyan-400">
            <Server className="w-4 h-4" />
            <span className="text-sm font-bold font-mono">OPERATOR_STATUS</span>
        </div>
        <div className="text-cyan-400 text-xs font-mono bg-cyan-500 bg-opacity-10 px-2 py-1
rounded border border-cyan-500">
            systemctl status operator
        </div>
    </div>
    <div className="space-y-4">
        <div className="grid grid-cols-2 gap-3 text-center">

```

```
<div className="bg-black border border-green-500 rounded p-3 hover:border-cyan-400 transition-colors">
```

```
<div className="text-green-300 text-xs font-mono">SESSION_TIME</div>
```

```
<div className="text-white font-bold text-lg font-mono">02:34:17</div>
```

```
</div>
```

```
<div className="bg-black border border-green-500 rounded p-3 hover:border-cyan-400 transition-colors">
```

```
<div className="text-green-300 text-xs font-mono">AVG_SCORE</div>
```

```
<div className="text-white font-bold text-lg font-mono">87.5</div>
```

```
</div>
```

```
</div>
```

```
<div className="bg-black border border-yellow-500 rounded p-3 hover:border-yellow-400 transition-colors">
```

```
<div className="flex items-center justify-between text-sm mb-2">
```

```
<span className="text-yellow-300 font-mono">NEXT_RANK</span>
```

```
<span className="text-white font-bold font-mono">#18</span>
```

```
</div>
```

```
<div className="w-full bg-gray-800 rounded-full h-2 mb-1 overflow-hidden">
```

```
<div
```

```
  className="h-2 rounded-full bg-gradient-to-r from-yellow-500 to-orange-500 transition-all duration-1000 ease-out"
```

```
    style={{ width: '72%' }}
```

```
></div>
```

```
</div>
```

```
<div className="text-yellow-400 text-xs text-center font-mono">
```

```
  215 POINTS TO NEXT TIER
```

```
</div>
```

```
</div>
```

```
<div className="flex items-center justify-between p-2 bg-black border border-purple-500 rounded hover:border-purple-400 transition-colors">
```

```
<div className="flex items-center space-x-2 text-purple-400">
```

```
<Award className="w-4 h-4" />
```

```
<span className="text-xs font-mono">ACHIEVEMENTS</span>
```

```
</div>
```

```
<span className="text-white text-sm font-bold font-mono">8/15</span>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<style jsx>{`
```

```
.animate-fade-in {
```

```
  animation: fadeIn 0.8s ease-out;
```

```
}
```

```
@keyframes fadeIn {
```

```
  from {
```

```
    opacity: 0;
```

```
    transform: translateY(20px);
```

```
  }
```

```
  to {
```

```
    opacity: 1;
```

```
    transform: translateY(0);
```

```
  }
```

```
}
```

```
.glitch {  
  position: relative;  
  display: inline-block;  
}
```

```
.glitch::before,  
.glitch::after {  
  content: attr(data-text);  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background: transparent;  
}
```

```
.glitch::before {  
  left: 2px;  
  text-shadow: -2px 0 #ff00ff;  
  animation: glitch-1 2s infinite linear alternate-reverse;  
  clip-path: polygon(0 0, 100% 0, 100% 35%, 0 35%);  
}
```

```
.glitch::after {  
  left: -2px;  
  text-shadow: 2px 0 #00ffff;  
  animation: glitch-2 3s infinite linear alternate-reverse;  
  clip-path: polygon(0 65%, 100% 65%, 100% 100%, 0 100%);  
}
```

```
}
```

```
@keyframes glitch-1 {
```

```
0%, 14%, 15%, 49%, 50%, 99%, 100% {
```

```
    opacity: 0;
```

```
}
```

```
15%, 49% {
```

```
    opacity: 0.1;
```

```
}
```

```
}
```

```
@keyframes glitch-2 {
```

```
0%, 24%, 25%, 59%, 60%, 99%, 100% {
```

```
    opacity: 0;
```

```
}
```

```
25%, 59% {
```

```
    opacity: 0.1;
```

```
}
```

```
}
```

```
/* Hover animations */
```

```
.hover-scale:hover {
```

```
    transform: scale(1.05);
```

```
}
```

```
`</style>
```

```
</div>
```

```
)
```

```
}
```

```
export default UserStats"
```

```
src/components/terminal/ctfterminal.jsx
```

```
"import React, { useState, useRef, useEffect } from 'react';
```

```
import {
```

```
  X,
```

```
  Terminal as TerminalIcon,
```

```
  Send,
```

```
  Flag,
```

```
  Clock,
```

```
  Cpu,
```

```
  Network,
```

```
  Key,
```

```
  FileSearch,
```

```
  Binary,
```

```
  Bug,
```

```
  Zap,
```

```
  Shield,
```

```
  AlertTriangle,
```

```
  CheckCircle,
```

```
  Play,
```

```
  Square
```

```
} from 'lucide-react';
```

```
const CTFTerminal = ({ challenge, isOpen, onClose, onChallengeSolved }) => {
```

```
  const [input, setInput] = useState("");
```

```
  const [output, setOutput] = useState([]);
```

```
  const [isProcessing, setIsProcessing] = useState(false);
```

```
  const [sessionActive, setSessionActive] = useState(false);
```



```
const terminalEndRef = useRef(null);
```

```
const inputRef = useRef(null);
```

```
const categories = {
```

```
  web: { icon: Network, color: 'text-blue-400', label: 'WEB EXPLOITATION' },
```

```
  crypto: { icon: Key, color: 'text-yellow-400', label: 'CRYPTOGRAPHY' },
```

```
  forensics: { icon: FileSearch, color: 'text-orange-400', label: 'DIGITAL FORENSICS' },
```

```
  pwn: { icon: Binary, color: 'text-purple-400', label: 'BINARY EXPLOITATION' },
```

```
  reverse: { icon: Cpu, color: 'text-green-400', label: 'REVERSE ENGINEERING' },
```

```
  misc: { icon: Bug, color: 'text-indigo-400', label: 'MISCELLANEOUS' }
```

```
};
```

```
const CategoryIcon = categories[challenge.category]?.icon || TerminalIcon;
```

```
const categoryColor = categories[challenge.category]?.color || 'text-gray-400';
```

```
// Initialize terminal
```

```
useEffect(() => {
```

```
  if (isOpen) {
```

```
    initializeTerminal();
```

```
    inputRef.current?.focus();
```

```
  }
```

```
}, [isOpen]);
```

```
// Scroll to bottom when output changes
```

```
useEffect(() => {
```

```
  terminalEndRef.current?.scrollIntoView({ behavior: 'smooth' });
```

```
}, [output]);
```

```
const initializeTerminal = () => {
```

```

setOutput([
  { type: 'system', text: 'Initializing CTF Terminal v2.1.4...' },
  { type: 'system', text: 'Establishing secure connection to challenge server...' },
  { type: 'success', text: 'Connection established successfully' },
  { type: 'info', text: `Target: ${challenge.title}` },
  { type: 'info', text: `Category: ${categories[challenge.category]?.label}` },
  { type: 'info', text: `Difficulty: ${challenge.difficulty.toUpperCase()}` },
  { type: 'info', text: `Points: ${challenge.points}` },
  { type: 'system', text: 'Type "help" for available commands' },
  { type: 'prompt', text: 'Ready for commands...' }
]);

setSessionActive(true);

};

const addOutput = (text, type = 'info') => {
  setOutput(prev => [...prev, { type, text, timestamp: new Date().toLocaleTimeString() }]);
};

const executeCommand = async (command) => {
  if (!command.trim()) return;

  setIsProcessing(true);

  addOutput(`${command}`, 'command');

  const args = command.trim().split(' ');
  const cmd = args[0].toLowerCase();

  try {
    switch (cmd) {

```

```
case 'help':  
    addOutput(`
```

Available Commands:

Command	Description
help	Show this help message
info	Show challenge information
hint	Get a challenge hint
check <flag>	Submit flag for verification
ls	List directory contents
cat <file>	Display file content
connect	Connect to challenge service
status	Show current progress
clear	Clear terminal screen

```
    `.trim(), 'info');
```

```
    break;
```

```
case 'info':
```

```
    addOutput(`
```

Challenge Information:

Title	\${challenge.title.padEnd(30)}
Category	\${categories[challenge.category]?.label.padEnd(30)}
Difficulty	\${challenge.difficulty.toUpperCase().padEnd(30)}

Points	\${challenge.points.toString().padEnd(30)}
--------	--

--	--

Description	\${challenge.description.padEnd(30)}
-------------	--------------------------------------

--	--

```
`trim(), 'info');
```

```
break;
```

```
case 'hint':
```

```
if (challenge.hint) {
```

```
  addOutput(`💡 Hint: ${challenge.hint}`, 'hint');
```

```
} else {
```

```
  addOutput('No hints available for this challenge. Try harder!', 'warning');
```

```
}
```

```
break;
```

```
case 'check':
```

```
if (args.length < 2) {
```

```
  addOutput('Usage: check <flag>', 'error');
```

```
} else {
```

```
  const submittedFlag = args.slice(1).join(' ');
```

```
  addOutput('🔎 Verifying flag...', 'system');
```

```
  // Simulate verification delay
```

```
  await new Promise(resolve => setTimeout(resolve, 1500));
```

```
  if (submittedFlag === challenge.flag) {
```

```
    addOutput('🎉 FLAG VERIFIED! Challenge completed!', 'success');
```

```
    addOutput(`🏆 Points awarded: ${challenge.points}`, 'success');
```

```

        onChallengeSolved(challenge.id, challenge.points, submittedFlag);
    } else {
        addOutput('✗ Incorrect flag. Keep searching!', 'error');
    }
}

break;

case 'ls':
    addOutput('Directory contents:', 'info');
    addOutput('drwxr-xr-x 2 root root 4096 flag.txt', 'file');
    addOutput('-rw-r--r-- 1 root root 123 readme.md', 'file');
    addOutput('-rwxr-xr-x 1 root root 2048 challenge.bin', 'file');
    addOutput('drwxr-xr-x 2 root root 4096 src/', 'file');
    break;

case 'cat':
    if (args.length < 2) {
        addOutput('Usage: cat <filename>', 'error');
    } else {
        const filename = args[1];
        switch (filename) {
            case 'readme.md':
                addOutput(`
# ${challenge.title}

## Challenge Description

${challenge.description}

## Instructions

```

- Analyze the provided files and services
- Look for vulnerabilities and misconfigurations
- Find the flag in the format: CTF{...}
- Submit using: check CTF{your_flag_here}

Tips

- Read all documentation carefully
- Check file permissions and configurations
- Look for hidden data and metadata

```

        `.trim(), 'file-content');
        break;
    case 'flag.txt':
        addOutput('The flag is not in this file. Look elsewhere!', 'warning');
        break;
    default:
        addOutput(`cat: ${filename}: No such file or directory`, 'error');
    }
}
break;

case 'connect':
    addOutput('🌐 Connecting to challenge service...', 'system');
    await new Promise(resolve => setTimeout(resolve, 2000));
    addOutput('✅ Connected to service on port 1337', 'success');
    addOutput('Service is running. Interact with it to find the flag.', 'info');
    break;

case 'status':
    addOutput(`
```

Current Session Status:

Challenge		$\${challenge.solved ? 'SOLVED \checkmark' : 'ACTIVE \times'}$
Time Elapsed		$\${Math.floor(Math.random() * 60)} \text{ minutes}$
Attempts		$\${Math.floor(Math.random() * 10) + 1}$
Last Activity		Just now

```
` .trim(), 'info');
```

```
break;
```

```
case 'clear':
```

```
  setOutput([]);
```

```
  break;
```

```
default:
```

```
  addOutput(`Command not found: ${cmd}. Type "help" for available commands.` , 'error');
```

```
}
```

```
} catch (error) {
```

```
  addOutput(`Error executing command: ${error.message}`, 'error');
```

```
} finally {
```

```
  setIsProcessing(false);
```

```
  addOutput('Ready for next command...', 'prompt');
```

```
}
```

```
};
```

```
const handleSubmit = (e) => {
```

```
e.preventDefault();  
if (input.trim() && !isProcessing) {  
  executeCommand(input);  
  setInput("");  
}  
};
```

```
const handleKeyPress = (e) => {  
  if (e.key === 'Enter' && !e.shiftKey) {  
    handleSubmit(e);  
  }  
};
```

```
const getOutputColor = (type) => {  
  switch (type) {  
    case 'error': return 'text-red-400';  
    case 'success': return 'text-green-400';  
    case 'warning': return 'text-yellow-400';  
    case 'system': return 'text-cyan-400';  
    case 'command': return 'text-purple-400';  
    case 'hint': return 'text-yellow-300';  
    case 'file': return 'text-blue-300';  
    case 'file-content': return 'text-gray-300';  
    case 'prompt': return 'text-green-300';  
    default: return 'text-gray-400';  
  }  
};
```

```
if (!isOpen) return null;
```



```

return (
  <div className="fixed inset-0 bg-black bg-opacity-75 flex items-center justify-center p-4 z-50">
    <div className="bg-gray-900 border border-green-500 rounded-lg w-full max-w-4xl h-[80vh] flex
flex-col shadow-2xl shadow-green-500/20">
      {/* Terminal Header */}

      <div className="flex items-center justify-between p-4 border-b border-green-500/30 bg-gray-800
rounded-t-lg">
        <div className="flex items-center space-x-3">
          <div className="flex items-center space-x-2">
            <div className="w-3 h-3 bg-red-500 rounded-full cursor-pointer hover:bg-red-400"
onClick={onClose}></div>
            <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
            <div className="w-3 h-3 bg-green-500 rounded-full"></div>
          </div>
          <div className="flex items-center space-x-2 text-green-400">
            <CategoryIcon className={`w-5 h-5 ${categoryColor}`} />
            <span className="font-mono text-sm">CTF_TERMINAL — {challenge.title}</span>
          </div>
        </div>
        <div className="flex items-center space-x-2">
          {sessionActive && (
            <div className="flex items-center space-x-1 text-green-400 text-sm">
              <div className="w-2 h-2 bg-green-500 rounded-full animate-pulse"></div>
              <span>ACTIVE</span>
            </div>
          )}
        <button
          onClick={onClose}
          className="text-gray-400 hover:text-white transition-colors"

```

```

    >
    <X className="w-5 h-5" />
  </button>
</div>
</div>

```

```

  { /* Terminal Body */ }

  <div className="flex-1 p-4 overflow-hidden bg-black">
    <div className="h-full overflow-y-auto font-mono text-sm">
      { /* Welcome Banner */ }

      <div className="text-center mb-4 p-4 border border-cyan-500 rounded-lg bg-cyan-500 bg-opacity-10">
        <div className="flex items-center justify-center space-x-2 text-cyan-400 mb-2">
          <Shield className="w-5 h-5" />
          <span className="font-bold">CYBER ARENA CTF TERMINAL</span>
        </div>
        <div className="text-gray-300 text-xs">
          Secure challenge interface for {challenge.title}
        </div>
      </div>
    </div>
  </div>

```

```

  { /* Terminal Output */ }

  <div className="space-y-1 mb-4">
    {output.map((line, index) => (
      <div key={index} className={` ${getOutputColor(line.type)} break-words`} >
        {line.type === 'prompt' ? (
          <span className="flex items-center">
            <span className="text-green-500 mr-2">└─[cyber@arena]</span>
            <span className="text-cyan-500 mr-2">[~/challenges/{challenge.category}]</span>

```

```

        <span className="text-yellow-500">${</span>
    </span>
  ) : (
    <>
      {line.timestamp && (
        <span className="text-gray-600 text-xs mr-2">[{line.timestamp}]</span>
      )}
      {line.text}
    </>
  )}
</div>
)}}
</div>

```

```

{/* Input Area */}
<form onSubmit={handleSubmit} className="flex items-center space-x-2">
  <div className="flex items-center text-green-500 flex-shrink-0">
    <span className="mr-2">└─</span>
    <span className="text-yellow-500">${</span>
  </div>
  <input
    ref={inputRef}
    type="text"
    value={input}
    onChange={(e) => setInput(e.target.value)}
    onKeyPress={handleKeyPress}
    disabled={isProcessing}
    placeholder={isProcessing ? "Processing..." : "Type your command..."}
  >

```

```
        className="flex-1 bg-transparent border-none outline-none text-green-400 placeholder-
green-600 font-mono"
```

```
        autoFocus
```

```
    />
```

```
    {isProcessing && (
```

```
        <div className="w-4 h-4 border-2 border-green-500 border-t-transparent rounded-full
animate-spin"></div>
```

```
    )}
```

```
</form>
```

```
    <div ref={terminalEndRef} />
```

```
</div>
```

```
</div>
```

```
{/* Terminal Footer */}
```

```
<div className="p-3 border-t border-green-500/30 bg-gray-800 rounded-b-lg">
```

```
    <div className="flex items-center justify-between text-xs text-gray-400">
```

```
        <div className="flex items-center space-x-4">
```

```
            <span>🐞 {challenge.category.toUpperCase()}</span>
```

```
            <span>🔪 {challenge.difficulty.toUpperCase()}</span>
```

```
            <span>🏆 {challenge.points} POINTS</span>
```

```
        </div>
```

```
    <div className="flex items-center space-x-2">
```

```
        <span>TERMINAL v2.1.4</span>
```

```
        <div className="w-2 h-2 bg-green-500 rounded-full animate-pulse"></div>
```

```
    </div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
    </div>
  );
};

export default CTFTerminal;"
```

src/components/ui/errorboundary.jsx

```
"import React from 'react'
import {
  Terminal,
  Server,
  AlertTriangle,
  RefreshCw,
  Home,
  Cpu,
  Shield
} from 'lucide-react'

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props)
    this.state = {
      hasError: false,
      error: null,
      errorCode: this.generateErrorCode(),
      timestamp: null
    }
  }
}
```

```
static getDerivedStateFromError(error) {  
  return {  
    hasError: true,  
    error,  
    timestamp: new Date().toISOString()  
  }  
}
```

```
componentDidCatch(error, errorInfo) {  
  console.error('Error caught by boundary:', error, errorInfo)  
  // In a real app, you might want to send this to an error reporting service  
  this.logErrorToConsole(error, errorInfo)  
}
```

```
generateErrorCode() {  
  const codes = [  
    '0xDEADBEEF',  
    '0xCAFEFABE',  
    '0xBADC0DE1',  
    '0xSYSTEM32',  
    '0xKERNELP',  
    '0xSEGFAULT',  
    '0xNULLPTR',  
    '0xSTACKOVF'  
  ]  
  return codes[Math.floor(Math.random() * codes.length)]  
}
```

```
logErrorToConsole(error, errorInfo) {
```

```
const styles = [  
  'color: #ff0000; font-weight: bold; font-size: 14px;',  
  'color: #ffffff;',  
  'color: #00ff00;' ]
```

```
console.log(  
  `%c🚨 SYSTEM FAILURE DETECTED 🚨\n%cError: ${error.message}\n%cStack:  
${errorInfo.componentStack.split('\n')[1]}`,  
  ...styles  
)  
}
```

```
handleReset = () => {  
  this.setState({  
    hasError: false,  
    error: null,  
    errorCode: this.generateErrorCode(),  
    timestamp: null  
  })  
}
```

```
handleReload = () => {  
  window.location.reload()  
}
```

```
handleGoHome = () => {  
  window.location.href = '/'  
}
```

```

render() {
  if (this.state.hasError) {
    return (
      <div className="min-h-screen bg-gray-950 text-green-400 font-mono relative">
        {/* Matrix Background */}
        <div className="fixed inset-0 bg-black opacity-90 z-0">
          <div className="matrix-rain"></div>
        </div>

        <div className="relative z-10 flex items-center justify-center min-h-screen p-4">
          <div className="max-w-2xl w-full">
            {/* Terminal Header */}
            <div className="bg-black border border-red-500 rounded-lg shadow-2xl shadow-red-500/20
mb-6">
              <div className="flex items-center space-x-2 p-4 border-b border-red-500/30">
                <div className="w-3 h-3 bg-red-500 rounded-full"></div>
                <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
                <div className="w-3 h-3 bg-green-500 rounded-full"></div>
                <div className="text-red-400 text-sm ml-2">cyber-arena.ctf -- system_monitor</div>
              </div>

              <div className="p-6">
                {/* System Header */}
                <div className="flex items-center space-x-2 text-red-400 mb-6">
                  <Terminal className="w-5 h-5" />
                  <span className="text-sm">CRITICAL_SYSTEM_FAILURE</span>
                </div>

```



```
{/* Error Display */}
```

```
<div className="space-y-4">
```

```
  <div className="flex items-center space-x-3 text-white mb-4">
```

```
    <AlertTriangle className="w-8 h-8 text-red-400" />
```

```
    <h1 className="text-3xl font-bold glitch" data-text="SYSTEM_FAILURE">
```

```
      SYSTEM_FAILURE
```

```
    </h1>
```

```
  </div>
```

```
{/* Error Details */}
```

```
<div className="bg-black border border-red-500 rounded-lg p-4 space-y-3">
```

```
  <div className="grid grid-cols-2 gap-4 text-sm">
```

```
    <div>
```

```
      <span className="text-gray-500">ERROR_CODE:</span>
```

```
      <div className="text-red-400 font-bold">{this.state.errorCode}</div>
```

```
    </div>
```

```
    <div>
```

```
      <span className="text-gray-500">TIMESTAMP:</span>
```

```
      <div className="text-green-400">
```

```
        {this.state.timestamp ? new Date(this.state.timestamp).toLocaleTimeString() : 'N/A'}
```

```
      </div>
```

```
    </div>
```

```
  </div>
```

```
<div>
```

```
  <span className="text-gray-500">DESCRIPTION:</span>
```

```
  <div className="text-yellow-400 mt-1 font-mono text-sm">
```

```
    {this.state.error?.message || 'Unknown system malfunction'}
```

```
  </div>
```

</div>

<div>

COMPONENT:

<div className="text-cyan-400 mt-1 font-mono text-sm">

{this.state.error?.componentStack?.split('\n')[1]?.trim() || 'Unknown module'}

</div>

</div>

</div>

{/* System Status */}

<div className="bg-black border border-yellow-500 rounded-lg p-4">

<div className="flex items-center space-x-2 text-yellow-400 mb-3">

<Server className="w-4 h-4" />

SYSTEM_STATUS

</div>

<div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-center text-xs">

<div>

<div className="text-green-300">FRONTEND</div>

<div className="text-red-400 font-bold">FAILED</div>

</div>

<div>

<div className="text-green-300">BACKEND</div>

<div className="text-green-400 font-bold">ONLINE</div>

</div>

<div>

<div className="text-green-300">DATABASE</div>

<div className="text-green-400 font-bold">ONLINE</div>

</div>

```
<div>

  <div className="text-green-300">NETWORK</div>

  <div className="text-green-400 font-bold">STABLE</div>

</div>

</div>

</div>
```

```
{/* Recovery Instructions */}

<div className="bg-black border border-cyan-500 rounded-lg p-4">

  <div className="flex items-center space-x-2 text-cyan-400 mb-3">

    <Cpu className="w-4 h-4" />

    <span className="text-sm font-bold">RECOVERY_PROTOCOL</span>

  </div>

  <div className="text-green-300 text-sm space-y-2">

    <div className="flex items-center space-x-2">

      <span className="text-cyan-400">[1]</span>

      <span>Attempt component reload</span>

    </div>

    <div className="flex items-center space-x-2">

      <span className="text-cyan-400">[2]</span>

      <span>Return to system main</span>

    </div>

    <div className="flex items-center space-x-2">

      <span className="text-cyan-400">[3]</span>

      <span>Full system restart</span>

    </div>

  </div>

</div>

</div>
```

```

    { /* Action Buttons */ }

    <div className="grid grid-cols-1 md:grid-cols-3 gap-3 pt-4">

        <button

            onClick={this.handleReset}

            className="flex items-center justify-center space-x-2 px-4 py-3 bg-gradient-to-r from-
green-600 to-cyan-600 text-white rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all
border border-cyan-400 font-bold"

            >

                <RefreshCw className="w-4 h-4" />

                <span>RELOAD_COMPONENT</span>

            </button>

            <button

                onClick={this.handleGoHome}

                className="flex items-center justify-center space-x-2 px-4 py-3 border border-green-500
text-green-400 rounded-lg hover:border-cyan-400 hover:text-cyan-400 transition-all font-bold"

                >

                    <Home className="w-4 h-4" />

                    <span>SYSTEM_MAIN</span>

                </button>

                <button

                    onClick={this.handleReload}

                    className="flex items-center justify-center space-x-2 px-4 py-3 border border-yellow-500
text-yellow-400 rounded-lg hover:border-yellow-400 hover:text-yellow-300 transition-all font-bold"

                    >

                        <Shield className="w-4 h-4" />

                        <span>FULL_RESTART</span>

                    </button>

                </div>

```

```

    { /* Debug Info */ }

    { process.env.NODE_ENV === 'development' && this.state.error && (
      <div className="mt-6 p-4 bg-black border border-gray-500 rounded-lg">
        <div className="text-gray-400 text-sm mb-2">DEBUG_INFO:</div>
        <pre className="text-gray-500 text-xs overflow-auto max-h-32">
          {this.state.error.stack}
        </pre>
      </div>
    ) }
  </div>
</div>
</div>

```

```

    { /* Footer Note */ }

    <div className="text-center text-green-600 text-sm">
      <div className="flex items-center justify-center space-x-2">
        <div className="w-2 h-2 bg-red-500 rounded-full animate-pulse"></div>
        <span>SYSTEM_MONITOR_ACTIVE</span>
      </div>
    </div>
  </div>
</div>

```

```

<style jsx>{`
  .matrix-rain {
    position: absolute;
    top: 0;
    left: 0;

```

```
width: 100%;  
height: 100%;  
background: linear-gradient(180deg, rgba(255,0,0,0.1) 0%, transparent 50%);  
opacity: 0.1;  
pointer-events: none;  
}
```

```
.glitch {  
  position: relative;  
}
```

```
.glitch::before,  
.glitch::after {  
  content: attr(data-text);  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
}
```

```
.glitch::before {  
  animation: glitch-1 0.5s infinite linear alternate-reverse;  
  color: #ff00ff;  
  z-index: -1;  
}
```

```
.glitch::after {  
  animation: glitch-2 0.5s infinite linear alternate-reverse;
```

```
color: #00ffff;
z-index: -2;
}
```

```
@keyframes glitch-1 {
  0% { transform: translate(0); }
  20% { transform: translate(-2px, 2px); }
  40% { transform: translate(-2px, -2px); }
  60% { transform: translate(2px, 2px); }
  80% { transform: translate(2px, -2px); }
  100% { transform: translate(0); }
}
```

```
@keyframes glitch-2 {
  0% { transform: translate(0); }
  20% { transform: translate(2px, -2px); }
  40% { transform: translate(2px, 2px); }
  60% { transform: translate(-2px, -2px); }
  80% { transform: translate(-2px, 2px); }
  100% { transform: translate(0); }
}
```

```
.blinking-cursor {
  animation: blink 1s infinite;
  color: #00ff00;
}
```

```
@keyframes blink {
  0%, 50% { opacity: 1; }
```

```
        51%, 100% { opacity: 0; }  
    }  
    `}</style>  
</div>  
)  
}
```

```
    return this.props.children  
  }  
}
```

```
export default ErrorBoundary",
```

```
loadingspinner.jsx
```

```
"/ src/components/ui/LoadingSpinner.jsx  
import React, { useState, useEffect } from 'react'  
import {  
  Terminal,  
  Cpu,  
  Server,  
  Database,  
  Shield,  
  Zap,  
  Activity,  
  Binary,  
  Network  
} from 'lucide-react'
```

```
const LoadingSpinner = ({ message = "Initializing system..." }) => {
```



```
const [currentStep, setCurrentStep] = useState(0)
const [progress, setProgress] = useState(0)
const [terminalLines, setTerminalLines] = useState([])
const [currentLine, setCurrentLine] = useState("")
```

```
const loadingSteps = [
  {
    text: "BOOTING_SYSTEM_KERNEL",
    icon: Cpu,
    command: "init 0x1A3F",
    duration: 800
  },
  {
    text: "LOADING_SECURITY_PROTOCOLS",
    icon: Shield,
    command: "secure_boot --verify",
    duration: 1000
  },
  {
    text: "MOUNTING_CHALLENGE_DATABASE",
    icon: Database,
    command: "mount /dev/sda1 /challenges",
    duration: 1200
  },
  {
    text: "ESTABLISHING_NETWORK_SOCKETS",
    icon: Network,
    command: "netstat -tulpn | grep :1337",
    duration: 900
  }
]
```

```

},
{
  text: "INITIALIZING_CRYPTOSYSTEMS",
  icon: Binary,
  command: "openssl rand -base64 32",
  duration: 1100
},
{
  text: "STARTING_AUTHENTICATION_DAEMON",
  icon: Shield,
  command: "./auth_daemon --start",
  duration: 800
},
{
  text: "SYSTEM_READY",
  icon: Zap,
  command: "systemctl status cyber-arena",
  duration: 500
}
]

```

```

useEffect(() => {
  const totalDuration = loadingSteps.reduce((sum, step) => sum + step.duration, 0)
  let currentProgress = 0

  const progressInterval = setInterval(() => {
    currentProgress += 1
    setProgress(Math.min(currentProgress, 100))
  }, totalDuration / 100)

```

```
// Simulate terminal output

let stepIndex = 0

const processSteps = () => {
  if (stepIndex < loadingSteps.length) {
    const step = loadingSteps[stepIndex]

    // Add new terminal line
    setTerminalLines(prev => [...prev, {
      text: step.text,
      command: step.command,
      timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })
    }])

    setCurrentStep(stepIndex)

    // Typewriter effect for current line
    let charIndex = 0

    const typeInterval = setInterval(() => {
      if (charIndex <= step.command.length) {
        setCurrentLine(step.command.slice(0, charIndex))
        charIndex++
      } else {
        clearInterval(typeInterval)
        setTimeout(() => {
          setCurrentLine("")
          stepIndex++
          processSteps()
        }, 500)
      }
    }, 500)
  }
}
```

```

    }
  }, 30)

  return () => clearInterval(typeInterval)
}
}

processSteps()

return () => {
  clearInterval(progressInterval)
}
}, [])

const CurrentIcon = loadingSteps[currentStep]?.icon || Activity

return (
  <div className="min-h-screen bg-gray-950 text-green-400 font-mono relative overflow-hidden">
    {/* Matrix Background Effect */}
    <div className="absolute inset-0 bg-black opacity-90 z-0">
      <div
        className="w-full h-full opacity-10"
        style={{
          background: 'linear-gradient(180deg, rgba(0,255,0,0.1) 0%, transparent 50%)'
        }}
      />
    </div>

    <div className="relative z-10 flex items-center justify-center min-h-screen p-4">

```

```

<div className="max-w-2xl w-full">

  {/* Terminal Header */}

  <div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20
mb-6">

    <div className="flex items-center space-x-2 p-4 border-b border-green-500/30">

      <div className="w-3 h-3 bg-red-500 rounded-full"></div>

      <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>

      <div className="w-3 h-3 bg-green-500 rounded-full"></div>

      <div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- system_boot</div>

    </div>

    <div className="p-6">

      {/* System Header */}

      <div className="flex items-center space-x-2 text-green-300 mb-6">

        <Terminal className="w-5 h-5" />

        <span className="text-sm">SYSTEM_INITIALIZATION</span>

      </div>

      {/* Terminal Output */}

      <div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-48 overflow-y-
auto">

        {terminalLines.map((line, index) => (

          <div key={index} className="text-sm text-green-300 mb-2">

            <span className="text-gray-500">[{line.timestamp}]</span>{' '}

            <span className="text-cyan-400">[</span>

            <span className="text-yellow-400">OK</span>

            <span className="text-cyan-400">]</span>{' '}

            {line.text}

            <div className="text-green-400 ml-8 text-xs">

```

```

    <span className="text-gray-500">$ </span>
    {line.command}
  </div>
</div>
)}}

```

```

{ /* Current Typing Line */
{currentLine && (
  <div className="text-sm text-green-400">
    <span className="text-gray-500">[{new Date().toLocaleTimeString('en-US', { hour12: false
    })]}</span>{' '}
    <span className="text-cyan-400"></span>
    <span className="text-yellow-400">EXEC</span>
    <span className="text-cyan-400">]</span>{' '}
    {loadingSteps[currentStep]?.text}
    <div className="text-green-400 ml-8 text-xs">
      <span className="text-gray-500">$ </span>
      {currentLine}
      <span className="animate-pulse">█</span>
    </div>
  </div>
)}}
</div>

```

```

{ /* Progress Section */
<div className="space-y-4">
  { /* Current Step */
  <div className="flex items-center justify-between">
    <div className="flex items-center space-x-3">

```

```
<div className="w-10 h-10 bg-gradient-to-r from-green-500 to-cyan-500 rounded-lg flex
items-center justify-center">
```

```
<CurrentIcon className="w-5 h-5 text-white" />
```

```
</div>
```

```
<div>
```

```
<div className="text-white font-bold text-sm">
```

```
{loadingSteps[currentStep]?.text || "SYSTEM_READY"}
```

```
</div>
```

```
<div className="text-green-300 text-xs">
```

```
Step {currentStep + 1} of {loadingSteps.length}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className="text-cyan-400 font-bold text-lg">
```

```
{progress}%
```

```
</div>
```

```
</div>
```

```
{/* Progress Bar */}
```

```
<div className="w-full bg-gray-700 rounded-full h-3">
```

```
<div
```

```
className="h-3 rounded-full bg-gradient-to-r from-green-500 to-cyan-500 transition-all
duration-300"
```

```
style={{ width: `${progress}%` }}
```

```
></div>
```

```
</div>
```

```
{/* Progress Indicators */}
```

```
<div className="grid grid-cols-4 gap-4 text-center text-xs">
```

```
<div>

  <div className="text-green-300">KERNEL</div>

  <div className="text-white font-bold">

    {progress > 15 ? 'LOADED' : '...'}

  </div>

</div>

<div>

  <div className="text-green-300">SECURITY</div>

  <div className="text-white font-bold">

    {progress > 35 ? 'ACTIVE' : '...'}

  </div>

</div>

<div>

  <div className="text-green-300">NETWORK</div>

  <div className="text-white font-bold">

    {progress > 65 ? 'ONLINE' : '...'}

  </div>

</div>

<div>

  <div className="text-green-300">AUTH</div>

  <div className="text-white font-bold">

    {progress > 85 ? 'READY' : '...'}

  </div>

</div>

</div>

</div>

</div>
```



```

    { /* System Status Footer */ }

    <div className="bg-black border border-cyan-500 rounded-lg p-4">

        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-center text-sm">

            <div>

                <div className="text-green-300">BOOT_SEQUENCE</div>

                <div className="text-white font-bold">ACTIVE</div>

            </div>

            <div>

                <div className="text-green-300">MEMORY_USAGE</div>

                <div className="text-white font-bold">{Math.floor(progress * 2.56)}MB</div>

            </div>

            <div>

                <div className="text-green-300">CPU_LOAD</div>

                <div className="text-white font-bold">{Math.floor(progress * 0.8)}%</div>

            </div>

            <div>

                <div className="text-green-300">TIME_ELAPSED</div>

                <div className="text-white font-bold">0:{Math.floor(progress * 0.03).toString().padStart(2,
'0')}}s</div>

            </div>

        </div>

    </div>

    { /* Loading Message */ }

    <div className="text-center mt-6">

        <div className="text-green-300 text-sm mb-2">{message}</div>

        <div className="flex items-center justify-center space-x-2 text-green-600 text-xs">

            <div className="w-2 h-2 bg-green-500 rounded-full animate-pulse"></div>

            <span>SECURE_CONNECTION_ESTABLISHED</span>

        </div>

    </div>

```

```
        </div>
      </div>
    </div>
  </div>
</div>
)
}
```

```
export default LoadingSpinner"
```

```
src/components/enhancedterminal.jsx
```

```
"// components/EnhancedTerminal.jsx
```

```
import React, { useState, useRef, useEffect } from 'react';
```

```
const EnhancedTerminal = ({ challengeId, userId, challengeTitle, onChallengeSolved }) => {
```

```
  const [input, setInput] = useState("");
```

```
  const [output, setOutput] = useState([]);
```

```
  const [isLoading, setIsLoading] = useState(false);
```

```
  const [commandHistory, setCommandHistory] = useState([]);
```

```
  const [historyIndex, setHistoryIndex] = useState(-1);
```

```
  const terminalEndRef = useRef(null);
```

```
  const inputRef = useRef(null);
```

```
// Auto-scroll and focus management
```

```
useEffect(() => {
```

```
  scrollToBottom();
```

```
  if (!isLoading) {
```

```
    inputRef.current?.focus();
```

```
  }
```

```

}, [output, isLoading]);

const scrollToBottom = () => {
  terminalEndRef.current?.scrollIntoView({ behavior: "smooth" });
};

// Initialize terminal
useEffect(() => {
  setOutput([
    { type: 'system', content: '=== CTF Terminal ===' },
    { type: 'system', content: 'Welcome to the hacking environment!' },
    { type: 'system', content: `Challenge: ${challengeTitle}` },
    { type: 'system', content: 'Type "help" to see available commands' },
    { type: 'system', content: '' }
  ]);
}, [challengeTitle]);

const handleCommand = async (cmd) => {
  const command = cmd.trim();
  if (!command) return;

  // Add to history
  setCommandHistory(prev => [command, ...prev]);
  setHistoryIndex(-1);

  // Add user input to output
  setOutput(prev => [...prev, { type: 'input', content: command }]);
  setInput("");
  setIsLoading(true);

```

```
try {  
  const response = await fetch('/api/terminal/command', {  
    method: 'POST',  
    headers: {  
      'Content-Type': 'application/json',  
    },  
    body: JSON.stringify({  
      command,  
      challengeId,  
      userId  
    })  
  });  
  
  const data = await response.json();  
  
  if (data.success) {  
    const newOutput = [{ type: 'output', content: data.output }];  
  
    if (data.files && data.files.length > 0) {  
      newOutput.push({  
        type: 'files',  
        content: 'Files in directory:',  
        files: data.files  
      });  
    }  
  
    setOutput(prev => [...prev, ...newOutput]);  
  }  
}
```

```

    if (data.solved) {
      setOutput(prev => [...prev,
        { type: 'success', content: `🚩 Challenge Solved!` },
        { type: 'success', content: `Flag: ${data.flag}` },
        { type: 'success', content: `Points: +${data.points}` }
      ]);
      onChallengeSolved?.(data.flag, data.points);
    }
  } else {
    setOutput(prev => [...prev,
      { type: 'error', content: data.error || 'Command execution failed' }
    ]);
  }
} catch (error) {
  setOutput(prev => [...prev,
    { type: 'error', content: `Network error: ${error.message}` }
  ]);
} finally {
  setIsLoading(false);
}
};

```

```

const handleSubmit = (e) => {
  e.preventDefault();
  handleCommand(input);
};

```

```

const handleKeyDown = (e) => {
  // Command history with up/down arrows

```

```

if (e.key === 'ArrowUp') {
  e.preventDefault();
  if (commandHistory.length > 0) {
    const newIndex = historyIndex < commandHistory.length - 1 ? historyIndex + 1 : historyIndex;
    setHistoryIndex(newIndex);
    setInput(commandHistory[newIndex] || '');
  }
} else if (e.key === 'ArrowDown') {
  e.preventDefault();
  if (historyIndex > 0) {
    const newIndex = historyIndex - 1;
    setHistoryIndex(newIndex);
    setInput(commandHistory[newIndex] || '');
  } else if (historyIndex === 0) {
    setHistoryIndex(-1);
    setInput('');
  }
} else if (e.key === 'Enter') {
  handleCommand(input);
} else if (e.key === 'Tab') {
  e.preventDefault();
  // Basic tab completion could be added here
}
};

const clearTerminal = () => {
  setOutput([{ type: 'system', content: 'Terminal cleared' }]);
};

```

```

const getOutputClass = (type) => {
  const baseClasses = "font-mono text-sm mb-1 break-words leading-relaxed";
  switch (type) {
    case 'input':
      return `${baseClasses} text-green-400 font-medium`;
    case 'output':
      return `${baseClasses} text-gray-100`;
    case 'system':
      return `${baseClasses} text-blue-400`;
    case 'success':
      return `${baseClasses} text-green-300 font-bold animate-pulse`;
    case 'error':
      return `${baseClasses} text-red-400`;
    case 'files':
      return `${baseClasses} text-cyan-300`;
    default:
      return `${baseClasses} text-gray-400`;
  }
};

return (
  <div className="w-full max-w-6xl mx-auto bg-gray-900 rounded-xl shadow-2xl overflow-hidden border border-gray-600">
    {/* Terminal Header */}
    <div className="bg-gray-800 px-6 py-4 flex items-center justify-between border-b border-gray-600">
      <div className="flex items-center space-x-4">
        <div className="flex space-x-2">
          <div className="w-3 h-3 bg-red-500 rounded-full hover:bg-red-400 cursor-pointer"></div>

```

```
<div className="w-3 h-3 bg-yellow-500 rounded-full hover:bg-yellow-400 cursor-
pointer"></div>
```

```
<div className="w-3 h-3 bg-green-500 rounded-full hover:bg-green-400 cursor-
pointer"></div>
```

```
</div>
```

```
<div className="flex items-center space-x-3">
```

```
<span className="text-gray-200 font-semibold text-lg">
```

```
🔪 CTF Terminal
```

```
</span>
```

```
<span className="text-gray-400 text-sm">|</span>
```

```
<span className="text-cyan-300 text-sm font-medium">
```

```
{challengeTitle}
```

```
</span>
```

```
</div>
```

```
</div>
```

```
<div className="flex items-center space-x-3">
```

```
<button
```

```
onClick={clearTerminal}
```

```
className="px-3 py-1 text-xs bg-gray-700 text-gray-300 rounded hover:bg-gray-600
transition-colors"
```

```
>
```

```
Clear
```

```
</button>
```

```
<span className="text-gray-400 text-sm font-mono">
```

```
{challengeId}
```

```
</span>
```

```
</div>
```

```
</div>
```

```
{/* Terminal Body */}
```



```

<div className="p-6 h-[500px] flex flex-col bg-gray-900">
  {/* Output Area */}
  <div className="flex-1 overflow-y-auto mb-4 pr-2">
    <div className="space-y-2">
      {output.map((item, index) => (
        <div key={index} className={getOutputClass(item.type)}>
          {item.type === 'input' && (
            <span className="text-green-400 mr-3 font-bold">→</span>
          )}
          {item.type === 'system' && (
            <span className="text-blue-400 mr-3">●</span>
          )}
          {item.type === 'error' && (
            <span className="text-red-400 mr-3">X</span>
          )}
          {item.type === 'success' && (
            <span className="text-green-300 mr-3">✓</span>
          )}
          {item.content}
          {item.files && (
            <div className="ml-8 mt-2 space-y-1">
              {item.files.map((file, idx) => (
                <div key={idx} className="text-cyan-300 flex items-center">
                  <span className="mr-2">■</span>
                  <span className="font-mono">{file}</span>
                </div>
              ))}
            </div>
          )}
        </div>
      )}
    </div>
  )}

```

```

        </div>
    )))
    {isLoading && (
        <div className="text-blue-400 font-mono text-sm flex items-center">
            <span className="mr-3">🔄</span>
            <span className="animate-pulse">Executing command...</span>
        </div>
    )}
</div>

<div ref={terminalEndRef} />
</div>

{/* Input Area */}

<form onSubmit={handleSubmit} className="flex items-center bg-gray-800 rounded-lg px-4
py-3 border border-gray-600">
    <span className="text-green-400 font-mono text-sm font-bold mr-3 whitespace-nowrap">
        ctf@{challengeId}:~$
    </span>
    <input
        ref={inputRef}
        type="text"
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={handleKeyDown}
        disabled={isLoading}
        className="flex-1 bg-transparent text-green-400 font-mono text-sm outline-none border-
none focus:ring-0 placeholder-gray-500"
        placeholder={isLoading ? "Processing command..." : "Type command and press Enter..."}
        spellCheck="false"
    />

```

```

      autoComplete="off"
      autoCapitalize="off"
    />
    {isLoading && (
      <div className="ml-3">
        <div className="animate-spin rounded-full h-4 w-4 border-b-2 border-green-400"></div>
      </div>
    )}
  </form>

  { /* Quick Help */ }
  <div className="mt-3 text-xs text-gray-500 font-mono">
    💡 Tip: Use ↑↓ arrows for command history • Type 'help' for commands
  </div>
</div>
</div>
);
};

```

```
export default EnhancedTerminal;"
```

```
terminal.jsx
```

```
"// components/Terminal.jsx
```

```
import React, { useState, useRef, useEffect } from 'react';
```

```

const Terminal = ({ challengeId, userId, challengeTitle = "Challenge" }) => {
  const [input, setInput] = useState("");
  const [output, setOutput] = useState([]);

```

```
const [isLoading, setIsLoading] = useState(false);

const terminalEndRef = useRef(null);

const inputRef = useRef(null);

const scrollToBottom = () => {
  terminalEndRef.current?.scrollIntoView({ behavior: "smooth" });
};

useEffect(() => {
  scrollToBottom();
}, [output]);

// Focus input on mount and when loading finishes
useEffect(() => {
  if (!isLoading) {
    inputRef.current?.focus();
  }
}, [isLoading]);

// Initial welcome message
useEffect(() => {
  setOutput([
    { type: 'system', content: 'Welcome to CTF Terminal! 🚀' },
    { type: 'system', content: 'Type "help" for available commands' },
    { type: 'system', content: `Connected to: ${challengeTitle}` }
  ]);
}, [challengeTitle]);

const handleCommand = async (cmd) => {
```

```
const command = cmd.trim();
if (!command) return;

// Add user input to output
setOutput(prev => [...prev, { type: 'input', content: command }]);
setInput('');
setIsLoading(true);

try {
  const response = await fetch('/api/terminal/command', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      command,
      challengeId,
      userId
    })
  });

  const data = await response.json();

  if (data.success) {
    const newOutput = [{ type: 'output', content: data.output }];

    if (data.files) {
      newOutput.push({ type: 'files', content: data.files, files: data.files });
    }
  }
}
```

```

    setOutput(prev => [...prev, ...newOutput]);

    // Check if challenge is solved
    if (data.solved) {
      setOutput(prev => [...prev,
        { type: 'success', content: `🚩 Challenge Solved! Flag: ${data.flag}` }
      ]);
    }
  } else {
    setOutput(prev => [...prev,
      { type: 'error', content: data.error || 'Command failed' }
    ]);
  }
} catch (error) {
  setOutput(prev => [...prev,
    { type: 'error', content: `Connection error: ${error.message}` }
  ]);
} finally {
  setIsLoading(false);
}
};

```

```

const handleSubmit = (e) => {
  e.preventDefault();
  handleCommand(input);
};

```

```

const handleKeyPress = (e) => {

```

```
    if (e.key === 'Enter') {  
      handleCommand(input);  
    }  
  };  
};
```

```
const getOutputClass = (type) => {  
  const baseClasses = "font-mono text-sm mb-1 break-words";  
  switch (type) {  
    case 'input':  
      return `${baseClasses} text-green-400`;  
    case 'output':  
      return `${baseClasses} text-white`;  
    case 'system':  
      return `${baseClasses} text-blue-300`;  
    case 'success':  
      return `${baseClasses} text-green-300 font-bold`;  
    case 'error':  
      return `${baseClasses} text-red-400`;  
    default:  
      return `${baseClasses} text-gray-300`;  
  }  
};
```

```
return (  
  <div className="w-full max-w-4xl mx-auto bg-gray-900 rounded-lg shadow-2xl overflow-hidden  
border border-gray-700">  
    {/* Terminal Header */}  
    <div className="bg-gray-800 px-4 py-3 flex items-center justify-between border-b border-gray-  
700">
```

```

<div className="flex items-center space-x-2">
  <div className="flex space-x-2">
    <div className="w-3 h-3 bg-red-500 rounded-full"></div>
    <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
    <div className="w-3 h-3 bg-green-500 rounded-full"></div>
  </div>
  <span className="text-gray-300 text-sm font-medium ml-2">
    CTF Terminal - {challengeTitle}
  </span>
</div>

<div className="text-gray-400 text-xs">
  {challengeId}
</div>
</div>

{/* Terminal Body */}
<div className="p-4 h-96 flex flex-col bg-gray-900">
  {/* Output Area */}
  <div className="flex-1 overflow-y-auto mb-3 font-mono text-sm">
    <div className="space-y-1">
      {output.map((item, index) => (
        <div key={index} className={getOutputClass(item.type)}>
          {item.type === 'input' && (
            <span className="text-green-400 mr-2">ctf@challenge:~$</span>
          )}
          {item.content}
          {item.files && (
            <div className="ml-6 mt-1 space-y-1">
              {item.files.map((file, idx) => (

```



```

        <div key={idx} className="text-cyan-300">
             {file}
        </div>

    )}}
</div>

    }}
</div>

)}}
{isLoading && (
    <div className="text-blue-300 font-mono text-sm mb-1">
        <span className="animate-pulse">Processing command...</span>
    </div>
    )}
</div>

<div ref={terminalEndRef} />
</div>

{/* Input Area */}
<form onSubmit={handleSubmit} className="flex items-center">
    <span className="text-green-400 font-mono text-sm mr-2 whitespace-nowrap">
        ctf@challenge:~$
    </span>
    <input
        ref={inputRef}
        type="text"
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyPress={handleKeyPress}
        disabled={isLoading}
    />
</form>

```

```
        className="flex-1 bg-transparent text-green-400 font-mono text-sm outline-none border-
none focus:ring-0"
```

```
        placeholder={isLoading ? "Processing..." : "Type your command..."}
```

```
    />
```

```
  </form>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default Terminal;"
```

```
src/contexts/AuthContext.js
```

```
"import React, { createContext, useState, useContext, useEffect } from 'react'
```

```
import axios from 'axios'
```

```
const AuthContext = createContext()
```

```
export const useAuth = () => {
```

```
  const context = useContext(AuthContext)
```

```
  if (!context) {
```

```
    throw new Error('useAuth must be used within an AuthProvider')
```

```
  }
```

```
  return context
```

```
}
```

```
export const AuthProvider = ({ children }) => {
```

```
  const [user, setUser] = useState(null)
```

```
  const [loading, setLoading] = useState(true)
```

```
// Configure axios base URL - FIXED: use import.meta.env instead of process.env
axios.defaults.baseURL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api'
```

```
useEffect(() => {
  checkAuthStatus()
}, [])
```

```
const checkAuthStatus = async () => {
  try {
    const token = localStorage.getItem('token')
    if (token) {
      axios.defaults.headers.common['Authorization'] = `Bearer ${token}`
      const userData = JSON.parse(localStorage.getItem('userData'))
      setUser(userData)
    }
  } catch (error) {
    console.error('Auth check failed:', error)
    logout()
  } finally {
    setLoading(false)
  }
}
```

```
const login = async (email, password) => {
  try {
    const response = await axios.post('/auth/login', { email, password })
    const { token, user } = response.data
```

```

localStorage.setItem('token', token)
localStorage.setItem('userData', JSON.stringify(user))
axios.defaults.headers.common['Authorization'] = `Bearer ${token}`

setUser(user)
return { success: true }
} catch (error) {
return {
  success: false,
  error: error.response?.data?.error || 'Login failed'
}
}
}

```

```

const register = async (userData) => {
  try {
    const response = await axios.post('/auth/register', userData)
    const { token, user } = response.data

    localStorage.setItem('token', token)
    localStorage.setItem('userData', JSON.stringify(user))
    axios.defaults.headers.common['Authorization'] = `Bearer ${token}`

    setUser(user)
    return { success: true }
  } catch (error) {
    return {
      success: false,
      error: error.response?.data?.error || 'Registration failed'
    }
  }
}

```

```
    }  
  }  
}
```

```
const logout = () => {  
  localStorage.removeItem('token')  
  localStorage.removeItem('userData')  
  delete axios.defaults.headers.common['Authorization']  
  setUser(null)  
}
```

```
const value = {  
  user,  
  login,  
  register,  
  logout,  
  loading,  
  isAdmin: user?.role === 'admin'  
}
```

```
return (  
  <AuthContext.Provider value={value}>  
    {children}  
  </AuthContext.Provider>  
)  
}";
```

src/layouts/adminlayout.jsx

```
"import React from 'react'
```

```
import { Outlet } from 'react-router-dom'

import AdminSidebar from '../components/navigation/AdminSidebar.jsx'
```

```
const AdminLayout = () => {
  return (
    <div className="flex h-screen bg-gray-800">
      <AdminSidebar />
      <main className="flex-1 overflow-auto bg-gray-50">
        <div className="p-6">
          <Outlet />
        </div>
      </main>
    </div>
  )
}
```

```
export default AdminLayout"
```

userlayout.jsx

```
"import React from 'react'

import { Outlet } from 'react-router-dom'

import UserSidebar from '../components/navigation/UserSidebar.jsx'
```

```
const UserLayout = () => {
  return (
    <div className="flex h-screen bg-gray-800">
      <UserSidebar />
      <main className="flex-1 overflow-auto bg-gray-50">
        <div className="p-6">
```

```
        <Outlet />
      </div>
    </main>
  </div>
)
}
```

```
export default UserLayout"
```

```
mainlayout.jsx
```

```
"import React from 'react'
```

```
import { Outlet } from 'react-router-dom'
```

```
import MainNavbar from '../components/navigation/MainNavbar.jsx'
```

```
const MainLayout = () => {
```

```
  return (
```

```
    <div className="min-h-screen bg-gradient-to-br from-gray-900 to-gray-800">
```

```
      <MainNavbar />
```

```
      <main className="pt-16">
```

```
        <Outlet />
```

```
      </main>
```

```
    { /* Global System Status Footer */ }
```

```
    <footer className="border-t border-green-500 border-opacity-20 mt-16">
```

```
      <div className="container mx-auto px-4 py-6">
```

```
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-center text-sm text-green-400 font-mono">
```

```
          <div>
```

```
            <div className="text-green-300">SYSTEM_STATUS</div>
```

```
    <div className="text-white font-bold">OPERATIONAL</div>
  </div>

  <div>

    <div className="text-green-300">NETWORK</div>

    <div className="text-white font-bold">SECURE</div>
  </div>

  <div>

    <div className="text-green-300">VERSION</div>

    <div className="text-white font-bold">v2.1.4</div>
  </div>

  <div>

    <div className="text-green-300">UPTIME</div>

    <div className="text-white font-bold">99.97%</div>
  </div>
</div>

<div className="text-center text-green-600 text-xs mt-4">
  CYBER_ARENA_CTF © 2024 | SECURE_TRAINING_PLATFORM
</div>
</div>
</footer>
</div>
)
}
```

```
export default MainLayout"
```

```
src/pages/admin/AdminSecurityDashboard.jsx
```

```
"import React, { useState, useEffect } from 'react'
```



```

const AdminSecurityDashboard = () => {
  const [activeSessions, setActiveSessions] = useState([])
  const [selectedSession, setSelectedSession] = useState(null)
  const [isLoading, setIsLoading] = useState(true)

  useEffect(() => {
    fetchActiveSessions()
    const interval = setInterval(fetchActiveSessions, 5000) // Refresh every 5 seconds
    return () => clearInterval(interval)
  }, [])

  const fetchActiveSessions = async () => {
    try {
      const response = await fetch('/api/secure-session/admin/active-sessions', {
        headers: {
          'Authorization': `Bearer ${localStorage.getItem('token')}`
        }
      })
      const data = await response.json()
      setActiveSessions(data.sessions)
    } catch (error) {
      console.error('Error fetching active sessions:', error)
    } finally {
      setIsLoading(false)
    }
  }

  const terminateSession = async (sessionId, reason) => {
    try {

```

```
const response = await fetch(`/api/secure-session/admin/${sessionId}/force-terminate`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${localStorage.getItem('token')}`
  },
  body: JSON.stringify({ reason })
})
```

```
if (response.ok) {
  fetchActiveSessions()
  setSelectedSession(null)
}
} catch (error) {
  console.error('Error terminating session:', error)
}
}
```

```
const getSeverityColor = (violations) => {
  if (violations === 0) return 'bg-green-500'
  if (violations <= 2) return 'bg-yellow-500'
  if (violations <= 5) return 'bg-orange-500'
  return 'bg-red-500'
}
```

```
if (isLoading) {
  return (
    <div className="flex items-center justify-center p-8">
      <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-blue-600"></div>
    </div>
  )
}
```

```

    </div>
  )
}

return (
  <div className="p-6 space-y-6">
    { /* Header */ }
    <div className="bg-white rounded-lg shadow-sm border p-6">
      <h1 className="text-2xl font-bold text-gray-900 mb-2">Security Monitoring Dashboard</h1>
      <p className="text-gray-600">Monitor active test sessions and security violations</p>

      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mt-6">
        <div className="bg-blue-50 border border-blue-200 rounded-lg p-4">
          <div className="text-2xl font-bold text-blue-600">{activeSessions.length}</div>
          <div className="text-sm text-blue-600 font-medium">Active Sessions</div>
        </div>
        <div className="bg-yellow-50 border border-yellow-200 rounded-lg p-4">
          <div className="text-2xl font-bold text-yellow-600">
            {activeSessions.reduce((sum, session) => sum + session.violations, 0)}
          </div>
          <div className="text-sm text-yellow-600 font-medium">Total Violations</div>
        </div>
        <div className="bg-green-50 border border-green-200 rounded-lg p-4">
          <div className="text-2xl font-bold text-green-600">
            {activeSessions.filter(s => s.violations === 0).length}
          </div>
          <div className="text-sm text-green-600 font-medium">Clean Sessions</div>
        </div>
        <div className="bg-red-50 border border-red-200 rounded-lg p-4">

```

```
<div className="text-2xl font-bold text-red-600">
  {activeSessions.filter(s => s.violations > 5).length}
</div>

<div className="text-sm text-red-600 font-medium">High Risk</div>
</div>
</div>
</div>
```

```
{/* Active Sessions Table */}
```

```
<div className="bg-white rounded-lg shadow-sm border overflow-hidden">
  <div className="px-6 py-4 border-b">
    <h2 className="text-lg font-semibold text-gray-900">Active Test Sessions</h2>
  </div>
```

```
  <div className="overflow-x-auto">
    <table className="min-w-full divide-y divide-gray-200">
      <thead className="bg-gray-50">
        <tr>
          <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
            User
          </th>
          <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
            Challenge
          </th>
          <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
            Duration
          </th>
```

```

        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-
wider">
            Violations
        </th>

        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-
wider">
            Status
        </th>

        <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-
wider">
            Actions
        </th>
    </tr>
</thead>
<tbody className="bg-white divide-y divide-gray-200">
    {activeSessions.map((session) => (
        <tr key={session.id} className="hover:bg-gray-50">
            <td className="px-6 py-4 whitespace-nowrap">
                <div className="flex items-center">
                    <div className="flex-shrink-0 h-10 w-10 bg-gray-300 rounded-full flex items-center justify-
center">
                        <span className="text-gray-600 font-medium">
                            {session.user.username.charAt(0).toUpperCase()}
                        </span>
                    </div>
                    <div className="ml-4">
                        <div className="text-sm font-medium text-gray-900">
                            {session.user.username}
                        </div>
                        <div className="text-sm text-gray-500">

```

```

        {session.user.email}
      </div>
    </div>
  </div>
</td>
<td className="px-6 py-4 whitespace-nowrap">
  <div className="text-sm text-gray-900">{session.challenge.title}</div>
  <div className="text-sm text-gray-500 capitalize">{session.challenge.category}</div>
</td>
<td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500">
  {Math.floor(session.duration / 60)}m {session.duration % 60}s
</td>
<td className="px-6 py-4 whitespace-nowrap">
  <span className={`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium
    ${getSeverityColor(session.violations)} text-white`}>
    {session.violations} violations
  </span>
</td>
<td className="px-6 py-4 whitespace-nowrap">
  <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium
    bg-green-100 text-green-800">
    <span className="w-2 h-2 bg-green-400 rounded-full mr-1"></span>
    Active
  </span>
</td>
<td className="px-6 py-4 whitespace-nowrap text-sm font-medium">
  <button
    onClick={() => setSelectedSession(session)}
    className="text-blue-600 hover:text-blue-900 mr-3"
  >

```

```

    >
    View Details
  </button>
  <button
    onClick={() => terminateSession(session.sessionId, 'Admin manual termination')}
    className="text-red-600 hover:text-red-900"
  >
    Terminate
  </button>
</td>
</tr>
)}}
</tbody>
</table>

```

```

{activeSessions.length === 0 && (
  <div className="text-center py-12">
    <svg className="mx-auto h-12 w-12 text-gray-400" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9.75 17L9 20l-1
1h8l-1-1-.75-3M3 13h18M5 17h14a2 2 0 002-2V5a2 2 0 00-2-2H5a2 2 0 00-2 2v10a2 2 0 002 2z" />
    </svg>
    <h3 className="mt-2 text-sm font-medium text-gray-900">No active sessions</h3>
    <p className="mt-1 text-sm text-gray-500">There are no active test sessions to monitor.</p>
  </div>
)}
</div>
</div>

```

```

{ /* Session Detail Modal */}

{selectedSession && (

  <div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center p-4 z-50">
    <div className="bg-white rounded-lg max-w-2xl w-full max-h-[90vh] overflow-y-auto">
      <div className="px-6 py-4 border-b flex justify-between items-center">
        <h3 className="text-lg font-semibold">Session Details</h3>
        <button
          onClick={() => setSelectedSession(null)}
          className="text-gray-400 hover:text-gray-600"
        >
          <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18 6M6 6l12
12" />
          </svg>
        </button>
      </div>

      <div className="p-6 space-y-4">
        <div className="grid grid-cols-2 gap-4">
          <div>
            <label className="text-sm font-medium text-gray-500">User</label>
            <p className="text-sm text-gray-900">{selectedSession.user.username}</p>
          </div>
          <div>
            <label className="text-sm font-medium text-gray-500">Challenge</label>
            <p className="text-sm text-gray-900">{selectedSession.challenge.title}</p>
          </div>
          <div>
            <label className="text-sm font-medium text-gray-500">Duration</label>

```



```
<p className="text-sm text-gray-900">
  {Math.floor(selectedSession.duration / 60)}m {selectedSession.duration % 60}s
</p>
</div>
<div>
  <label className="text-sm font-medium text-gray-500">Violations</label>
  <p className="text-sm text-gray-900">{selectedSession.violations}</p>
</div>
</div>

<div>
  <label className="text-sm font-medium text-gray-500">IP Address</label>
  <p className="text-sm text-gray-900">{selectedSession.ipAddress}</p>
</div>

<div className="pt-4 border-t">
  <div className="flex justify-end space-x-3">
    <button
      onClick={() => setSelectedSession(null)}
      className="px-4 py-2 border border-gray-300 rounded-md text-sm font-medium text-gray-
700 hover:bg-gray-50"
    >
      Close
    </button>
    <button
      onClick={() => terminateSession(selectedSession.sessionId, 'Admin review termination')}
      className="px-4 py-2 bg-red-600 text-white rounded-md text-sm font-medium hover:bg-
red-700"
    >
```

```
        Terminate Session
      </button>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
  )}
</div>
)
}
```

```
export default AdminSecurityDashboard",
```

```
analytics.jsx
```

```
"// src/pages/admin/Analytics.jsx
```

```
import React from 'react'
```

```
const Analytics = () => {
```

```
  return (
```

```
    <div className="p-6">
```

```
      <h1 className="text-3xl font-bold text-gray-900 mb-6">Analytics</h1>
```

```
      <div className="bg-white rounded-lg shadow-lg p-6">
```

```
        <p className="text-gray-600">Analytics content will go here...</p>
```

```
      </div>
```

```
    </div>
```

```
  )
```

```
}
```

```
export default Analytics",
```

ChallengeManagement.jsx

```
"// src/pages/admin/ChallengeManagement.jsx
```

```
import React from 'react'
```

```
const ChallengeManagement = () => {
```

```
  return (
```

```
    <div className="p-6">
```

```
      <h1 className="text-3xl font-bold text-gray-900 mb-6">Challenge Management</h1>
```

```
      <div className="bg-white rounded-lg shadow-lg p-6">
```

```
        <p className="text-gray-600">Challenge management content will go here...</p>
```

```
      </div>
```

```
    </div>
```

```
  )
```

```
}
```

```
export default ChallengeManagement",
```

Dashboard.jsx

```
"// src/pages/admin/Dashboard.jsx
```

```
import React from 'react'
```

```
const AdminDashboard = () => {
```

```
  return (
```

```
    <div className="p-6">
```

```
      <h1 className="text-3xl font-bold text-gray-900 mb-6">Admin Dashboard</h1>
```

```
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
```

```
        <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-blue-500">
```

```

    <h3 className="text-lg font-semibold text-gray-800">System Status</h3>
    <p className="text-gray-600">All systems operational</p>
  </div>

  <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-green-500">
    <h3 className="text-lg font-semibold text-gray-800">Users Online</h3>
    <p className="text-gray-600">24 active users</p>
  </div>

  <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-purple-500">
    <h3 className="text-lg font-semibold text-gray-800">Challenges</h3>
    <p className="text-gray-600">15 active challenges</p>
  </div>
</div>
</div>
)
}

```

```
export default AdminDashboard",
```

```
SystemSettings.jsx
```

```
"// src/pages/admin/SystemSettings.jsx
```

```
import React from 'react'
```

```

const SystemSettings = () => {
  return (
    <div className="p-6">
      <h1 className="text-3xl font-bold text-gray-900 mb-6">System Settings</h1>
      <div className="bg-white rounded-lg shadow-lg p-6">
        <p className="text-gray-600">System settings content will go here...</p>
      </div>
    </div>
  )
}

```

```
    </div>
  )
}
```

```
export default SystemSettings",
```

```
UserManagement.jsx
```

```
"// src/pages/admin/UserManagement.jsx
```

```
import React from 'react'
```

```
const UserManagement = () => {
  return (
    <div className="p-6">
      <h1 className="text-3xl font-bold text-gray-900 mb-6">User Management</h1>
      <div className="bg-white rounded-lg shadow-lg p-6">
        <p className="text-gray-600">User management content will go here...</p>
      </div>
    </div>
  )
}
```

```
export default UserManagement"
```

```
src/pages/user/achievements.jsx
```

```
"// src/pages/user/Achievements.jsx
```

```
import React from 'react'
```

```
const Achievements = () => {
  return (
```

```
<div className="p-6">
  <h1 className="text-3xl font-bold text-gray-900 mb-6">Achievements</h1>
  <div className="bg-white rounded-lg shadow-lg p-6">
    <p className="text-gray-600">Achievements content will go here...</p>
  </div>
</div>
)
```

```
export default Achievements",
```

```
challenges.jsx
```

```
"// src/pages/user/Challenges.jsx
```

```
import React from 'react'
```

```
const UserChallenges = () => {
  return (
    <div className="p-6">
      <h1 className="text-3xl font-bold text-gray-900 mb-6">My Challenges</h1>
      <div className="bg-white rounded-lg shadow-lg p-6">
        <p className="text-gray-600">User challenges content will go here...</p>
      </div>
    </div>
  )
}
```

```
export default UserChallenges",
```

```
Dashboard.jsx
```

```
// src/pages/user/Dashboard.jsx
```

```
import React from 'react'
```

```
const UserDashboard = () => {
```

```
  return (
```

```
    <div className="p-6">
```

```
      <h1 className="text-3xl font-bold text-gray-900 mb-6">User Dashboard</h1>
```

```
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
```

```
        <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-blue-500">
```

```
          <h3 className="text-lg font-semibold text-gray-800">Completed Challenges</h3>
```

```
          <p className="text-2xl font-bold text-gray-900">12</p>
```

```
        </div>
```

```
        <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-green-500">
```

```
          <h3 className="text-lg font-semibold text-gray-800">Current Rank</h3>
```

```
          <p className="text-2xl font-bold text-gray-900">#24</p>
```

```
        </div>
```

```
        <div className="bg-white p-6 rounded-lg shadow-lg border-l-4 border-purple-500">
```

```
          <h3 className="text-lg font-semibold text-gray-800">Points</h3>
```

```
          <p className="text-2xl font-bold text-gray-900">1,250</p>
```

```
        </div>
```

```
      </div>
```

```
    </div>
```

```
  )
```

```
}
```

```
export default UserDashboard",
```

```
profile.jsx
```

```
// src/pages/user/Profile.jsx
```

```
import React from 'react'
```

```
const Profile = () => {
```

```
  return (
```

```
    <div className="p-6">
```

```
      <h1 className="text-3xl font-bold text-gray-900 mb-6">Profile</h1>
```

```
      <div className="bg-white rounded-lg shadow-lg p-6">
```

```
        <p className="text-gray-600">Profile content will go here...</p>
```

```
      </div>
```

```
    </div>
```

```
  )
```

```
}
```

```
export default Profile"
```

```
src/pages/admin.jsx
```

```
"// src/pages/Admin.jsx
```

```
import React, { useState, useEffect } from 'react'
```

```
import { useLocation, useNavigate } from 'react-router-dom'
```

```
import {
```

```
  BarChart3,
```

```
  Flag,
```

```
  Users,
```

```
  Settings as SettingsIcon,
```

```
  Download,
```

```
  Eye,
```

```
  Edit3,
```

```
  Trash2,
```

```
  Shield,
```



```

Zap,
AlertTriangle,
Terminal
} from 'lucide-react'

import CreateChallengeForm from '../components/challenges/CreateChallengeForm'
import AdminStats from '../components/stats/AdminStats'
import axios from 'axios'

const Admin = () => {
  const location = useLocation()
  const navigate = useNavigate()
  const [activeTab, setActiveTab] = useState('stats')
  const [challenges, setChallenges] = useState([])
  const [users, setUsers] = useState([])
  const [loading, setLoading] = useState(true)
  const [terminalOutput, setTerminalOutput] = useState([])
  const [systemStatus, setSystemStatus] = useState({
    database: 'ONLINE',
    auth: 'SECURE',
    challenges: 'ACTIVE',
    submissions: 'LIVE'
  })

  // Determine active tab from URL
  useEffect(() => {
    const path = location.pathname
    if (path.includes('/admin/users')) setActiveTab('users')
    else if (path.includes('/admin/challenges')) setActiveTab('challenges')
    else if (path.includes('/admin/settings')) setActiveTab('settings')
  })
}

```

```
    else setActiveTab('stats')
  }, [location])
```

```
const addTerminalLine = (text, type = 'info') => {
  setTerminalOutput(prev => [...prev, {
    text,
    type,
    timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })
  }])
}
```

```
useEffect(() => {
  fetchData()
  simulateSystemCheck()
}, [])
```

```
const simulateSystemCheck = () => {
  addTerminalLine('Initializing system diagnostics...', 'system')
  setTimeout(() => addTerminalLine('Database connection: ESTABLISHED', 'success'), 500)
  setTimeout(() => addTerminalLine('Authentication system: SECURE', 'success'), 800)
  setTimeout(() => addTerminalLine('Challenge service: ACTIVE', 'success'), 1100)
  setTimeout(() => addTerminalLine('Submission handler: ONLINE', 'success'), 1400)
}
```

```
const fetchData = async () => {
  try {
    addTerminalLine('Accessing admin control panel...', 'system')
```

```
    // Use mock data since API endpoints don't exist yet
```

```
const mockChallenges = [  
  {  
    id: 1,  
    title: 'SQL Injection Challenge',  
    description: 'Exploit SQL injection vulnerability to retrieve the flag',  
    category: 'web',  
    difficulty: 'easy',  
    points: 100,  
    solveCount: 15,  
    solved: false  
  },  
  {  
    id: 2,  
    title: 'Buffer Overflow',  
    description: 'Exploit buffer overflow to gain code execution',  
    category: 'pwn',  
    difficulty: 'hard',  
    points: 500,  
    solveCount: 3,  
    solved: false  
  },  
  {  
    id: 3,  
    title: 'RSA Challenge',  
    description: 'Break weak RSA implementation',  
    category: 'crypto',  
    difficulty: 'medium',  
    points: 250,  
    solveCount: 8,  
  }  
]
```

```
    solved: false
  }
]
```

```
const mockUsers = [
  {
    id: 1,
    username: 'admin',
    email: 'admin@cyberarena.com',
    role: 'admin',
    lastActive: new Date().toISOString(),
    createdAt: new Date('2024-01-01').toISOString()
  },
  {
    id: 2,
    username: 'user1',
    email: 'user1@example.com',
    role: 'user',
    lastActive: new Date(Date.now() - 86400000).toISOString(),
    createdAt: new Date('2024-01-15').toISOString()
  },
  {
    id: 3,
    username: 'hacker42',
    email: 'hacker42@protonmail.com',
    role: 'user',
    lastActive: new Date(Date.now() - 3600000).toISOString(),
    createdAt: new Date('2024-02-01').toISOString()
  }
]
```

```
]
```

```
setChallenges(mockChallenges)
```

```
setUsers(mockUsers)
```

```
addTerminalLine(`Loaded ${mockChallenges.length} targets and ${mockUsers.length} users`,  
'success')
```

```
} catch (error) {
```

```
addTerminalLine('WARNING: Using mock data - API endpoints not available', 'warning')
```

```
console.warn('API endpoints not available, using mock data:', error)
```

```
// Fallback to mock data
```

```
const mockChallenges = [
```

```
{
```

```
  id: 1,
```

```
  title: 'SQL Injection Challenge',
```

```
  description: 'Exploit SQL injection vulnerability to retrieve the flag',
```

```
  category: 'web',
```

```
  difficulty: 'easy',
```

```
  points: 100,
```

```
  solveCount: 15
```

```
}
```

```
]
```

```
const mockUsers = [
```

```
{
```

```
  id: 1,
```

```
  username: 'admin',
```

```
  email: 'admin@cyberarena.com',
```

```
    role: 'admin',  
    lastActive: new Date().toISOString(),  
    createdAt: new Date().toISOString()  
  }  
]
```

```
    setChallenges(mockChallenges)  
    setUsers(mockUsers)  
  } finally {  
    setLoading(false)  
  }  
}
```

```
const handleChallengeCreated = () => {  
  addTerminalLine('New target deployed successfully', 'success')  
  fetchData()  
  navigate('/admin/challenges')  
}
```

```
const handleExportData = () => {  
  addTerminalLine('Exporting system data...', 'system')  
  const data = {  
    challenges,  
    users,  
    exportedAt: new Date().toISOString()  
  }  
  const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' })  
  const url = URL.createObjectURL(blob)  
  const link = document.createElement('a')
```

```

link.href = url
link.download = `ctf-export-${new Date().toISOString().split('T')[0]}.json`
link.click()
addTerminalLine('Data export completed', 'success')
}

```

```

const handleDeleteChallenge = (challengeId) => {
  addTerminalLine(`Initiating target deletion: ${challengeId}`, 'warning')
  setChallenges(prev => prev.filter(c => c.id !== challengeId))
  addTerminalLine('Target successfully removed', 'success')
}

```

```

const getCategoryIcon = (category) => {
  const icons = {
    web: '🌐',
    crypto: '🔑',
    forensics: '🔍',
    pwn: '💣',
    reverse: '⚙️',
    misc: '🦋'
  }
  return icons[category] || '▶️'
}

```

```

const getDifficultyColor = (difficulty) => {
  switch (difficulty) {
    case 'easy': return 'text-green-600 bg-green-50 border-green-200'
    case 'medium': return 'text-yellow-600 bg-yellow-50 border-yellow-200'
  }
}

```

```
case 'hard': return 'text-red-600 bg-red-50 border-red-200'
default: return 'text-gray-600 bg-gray-50 border-gray-200'
}
}
```

```
const renderTabContent = () => {
  switch (activeTab) {
    case 'stats':
      return (
        <div className="space-y-6">
          <div className="flex items-center justify-between">
            <h1 className="text-2xl font-bold text-gray-900">System Overview</h1>
            <div className="flex items-center space-x-2 bg-gradient-to-r from-blue-600 to-purple-600 text-white px-4 py-2 rounded-lg">
              <Shield className="w-4 h-4" />
              <span className="font-bold">Admin Access</span>
            </div>
          </div>
          <AdminStats />

          { /* Terminal Output */ }
          <div className="bg-gray-900 border border-gray-700 rounded-lg p-4">
            <div className="flex items-center space-x-2 text-green-400 mb-3">
              <Terminal className="w-4 h-4" />
              <span className="text-sm font-medium">SYSTEM_LOG</span>
            </div>
            <div className="bg-black rounded p-3 h-32 overflow-y-auto">
              {terminalOutput.map((line, index) => (
                <div key={index} className={`text-sm ${`
```



```

        line.type === 'error' ? 'text-red-400' :
        line.type === 'success' ? 'text-green-400' :
        line.type === 'warning' ? 'text-yellow-400' :
        line.type === 'system' ? 'text-blue-400' : 'text-gray-400'
      `}>
      <span className="text-gray-500">[{line.timestamp}]</span> {line.text}
    </div>
  )}
</div>
</div>
</div>
)

```

case 'challenges':

```

return (
  <div className="space-y-6">
    <div className="flex flex-col lg:flex-row justify-between items-start lg:items-center gap-4">
      <div>
        <h1 className="text-2xl font-bold text-gray-900">Challenge Management</h1>
        <p className="text-gray-600">Manage CTF challenges and targets</p>
      </div>
      <button
        onClick={handleExportData}
        className="flex items-center space-x-2 px-4 py-2 border border-blue-500 text-blue-600
rounded-lg hover:bg-blue-50 transition-all"
      >
        <Download className="w-4 h-4" />
        <span className="font-medium">Export Data</span>
      </button>
    </div>
  </div>
)

```

</div>

<CreateChallengeForm onChallengeCreated={handleChallengeCreated} />

{/* Challenges List */}

<div className="bg-white border border-gray-200 rounded-lg shadow-sm">

<div className="px-6 py-4 border-b border-gray-200">

<h3 className="text-lg font-semibold text-gray-900">Deployed Challenges</h3>

<p className="text-gray-600 text-sm">Active challenge instances</p>

</div>

<div className="divide-y divide-gray-200">

{challenges.map((challenge) => (

<div key={challenge.id} className="px-6 py-4 flex items-center justify-between hover:bg-gray-50 transition-all group">

<div className="flex-1">

<div className="flex items-center space-x-4">

<div className="text-2xl">

{getCategoryIcon(challenge.category)}

</div>

<div className="flex-1">

<div className="flex items-center space-x-3 mb-2">

<h4 className="font-semibold text-gray-900 text-lg">{challenge.title}</h4>

{challenge.difficulty.toUpperCase()}

{challenge.points} PTS


```

    </div>

    <p className="text-gray-600 text-sm">{challenge.description}</p>

    <div className="flex items-center space-x-4 mt-2">

        <span className="text-gray-500 text-xs font-medium bg-gray-100 px-2 py-1 rounded
border border-gray-200">

            {challenge.category.toUpperCase()}

        </span>

        <span className="text-yellow-600 text-xs">

            Solves: {challenge.solveCount || 0}

        </span>

    </div>

</div>

</div>

</div>

<div className="flex items-center space-x-2 opacity-0 group-hover:opacity-100 transition-
opacity">

    <button className="p-2 text-gray-400 hover:text-blue-600 transition-colors border border-
gray-300 rounded hover:border-blue-400">

        <Eye className="w-4 h-4" />

    </button>

    <button className="p-2 text-gray-400 hover:text-yellow-600 transition-colors border
border-gray-300 rounded hover:border-yellow-400">

        <Edit3 className="w-4 h-4" />

    </button>

    <button

        onClick={() => handleDeleteChallenge(challenge.id)}

        className="p-2 text-gray-400 hover:text-red-600 transition-colors border border-gray-300
rounded hover:border-red-400"

    >

        <Trash2 className="w-4 h-4" />

```

```

        </button>
      </div>
    </div>
  )}
</div>
</div>
</div>
)

```

```
case 'users':
```

```
return (
```

```
  <div className="space-y-6">
```

```
    <div className="flex flex-col lg:flex-row justify-between items-start lg:items-center gap-4">
```

```
      <div>
```

```
        <h1 className="text-2xl font-bold text-gray-900">User Management</h1>
```

```
        <p className="text-gray-600">Manage system users and permissions</p>
```

```
      </div>
```

```
      <div className="flex space-x-3">
```

```
        <button className="flex items-center space-x-2 px-4 py-2 border border-blue-500 text-blue-600 rounded-lg hover:bg-blue-50 transition-all">
```

```
          <Download className="w-4 h-4" />
```

```
          <span className="font-medium">Export Users</span>
```

```
        </button>
```

```
      </div>
```

```
    </div>
```

```
  <div className="bg-white border border-gray-200 rounded-lg shadow-sm">
```

```
    <div className="px-6 py-4 border-b border-gray-200">
```

```
      <h3 className="text-lg font-semibold text-gray-900">Registered Users</h3>
```

```

    <p className="text-gray-600 text-sm">System access management</p>
  </div>

  <div className="divide-y divide-gray-200">
    {users.length > 0 ? (
      users.map((user) => (
        <div key={user.id} className="px-6 py-4 flex items-center justify-between hover:bg-gray-50
transition-all">
          <div className="flex items-center space-x-4">
            <div className="w-12 h-12 bg-gradient-to-r from-blue-500 to-purple-500 rounded-full flex
items-center justify-center text-white font-bold border-2 border-white shadow-sm">
              {user.username?.charAt(0).toUpperCase()}
            </div>
            <div>
              <h4 className="font-semibold text-gray-900 text-lg">{user.username}</h4>
              <p className="text-gray-600 text-sm">{user.email}</p>
              <div className="flex items-center space-x-2 mt-1">
                <span className={`px-2 py-1 rounded text-xs font-medium border ${
                  user.role === 'admin'
                    ? 'text-purple-600 bg-purple-50 border-purple-200'
                    : 'text-green-600 bg-green-50 border-green-200'
                }`}>
                  {user.role.toUpperCase()}
                </span>
                <span className="text-gray-500 text-xs">
                  Last active: {user.lastActive ? new Date(user.lastActive).toLocaleDateString() : 'N/A'}
                </span>
              </div>
            </div>
          </div>
        </div>
      )
    )}
  </div>

```

```

    <div className="flex items-center space-x-4">
      <span className="text-gray-500 text-sm">
        Joined {new Date(user.createdAt).toLocaleDateString()}
      </span>
      <div className="flex items-center space-x-2">
        <button className="p-2 text-gray-400 hover:text-blue-600 transition-colors border
border-gray-300 rounded hover:border-blue-400">
          <Eye className="w-4 h-4" />
        </button>
        <button className="p-2 text-gray-400 hover:text-yellow-600 transition-colors border
border-gray-300 rounded hover:border-yellow-400">
          <Edit3 className="w-4 h-4" />
        </button>
      </div>
    </div>
  </div>
</div>
))
):(
  <div className="px-6 py-12 text-center">
    <Users className="w-16 h-16 mx-auto mb-4 text-gray-400" />
    <p className="text-gray-500 text-lg">No Users Registered</p>
    <p className="text-gray-400 text-sm">System awaiting user registration</p>
  </div>
  </div>
  </div>
  </div>
  )

```

```
case 'settings':

return (

  <div className="space-y-6">

    <div>

      <h1 className="text-2xl font-bold text-gray-900">System Configuration</h1>

      <p className="text-gray-600">Manage platform settings and security</p>

    </div>


    <div className="grid gap-6">

      { /* Competition Settings */ }

      <div className="bg-white border border-gray-200 rounded-lg p-6 shadow-sm">

        <h3 className="text-lg font-semibold text-gray-900 mb-4 flex items-center space-x-2">

          <BarChart3 className="w-5 h-5 text-blue-600" />

          <span>Competition Settings</span>

        </h3>

        <div className="space-y-4">

          <div>

            <label className="block text-sm font-medium text-gray-700 mb-2">

              Competition Name

            </label>

            <input

              type="text"

              className="w-full px-4 py-2 bg-white border border-gray-300 rounded-lg focus:outline-
none focus:ring-2 focus:ring-blue-500 focus:border-blue-500 text-gray-900 transition-all"

              placeholder="CTF Competition 2024"

            />

          </div>

          <div>

            <label className="block text-sm font-medium text-gray-700 mb-2">
```

```

        Competition Description
    </label>

    <textarea

        className="w-full px-4 py-2 bg-white border border-gray-300 rounded-lg focus:outline-
none focus:ring-2 focus:ring-blue-500 focus:border-blue-500 text-gray-900 transition-all"

        rows="3"

        placeholder="Describe the competition objectives and rules..."

    />
</div>
</div>
</div>

{/* Security Settings */}
<div className="bg-white border border-gray-200 rounded-lg p-6 shadow-sm">
    <h3 className="text-lg font-semibold text-gray-900 mb-4 flex items-center space-x-2">
        <Shield className="w-5 h-5 text-green-600" />
        <span>Security Settings</span>
    </h3>
    <div className="space-y-3">
        <label className="flex items-center space-x-3 p-3 border border-gray-200 rounded-lg
hover:border-blue-400 transition-all cursor-pointer">
            <input type="checkbox" className="rounded text-blue-600 border-gray-300" />
            <span className="text-gray-700 font-medium">Require Email Verification</span>
        </label>

        <label className="flex items-center space-x-3 p-3 border border-gray-200 rounded-lg
hover:border-blue-400 transition-all cursor-pointer">
            <input type="checkbox" className="rounded text-blue-600 border-gray-300" />
            <span className="text-gray-700 font-medium">Enable Rate Limiting</span>
        </label>
    </div>
</div>

```



```
<label className="flex items-center space-x-3 p-3 border border-gray-200 rounded-lg
hover:border-blue-400 transition-all cursor-pointer">
```

```
<input type="checkbox" className="rounded text-blue-600 border-gray-300" />
```

```
<span className="text-gray-700 font-medium">Log All Submissions</span>
```

```
</label>
```

```
</div>
```

```
</div>
```

```
{/* System Controls */}
```

```
<div className="bg-white border border-gray-200 rounded-lg p-6 shadow-sm">
```

```
<h3 className="text-lg font-semibold text-gray-900 mb-4 flex items-center space-x-2">
```

```
<Zap className="w-5 h-5 text-yellow-600" />
```

```
<span>System Controls</span>
```

```
</h3>
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
```

```
<button className="p-4 border border-green-200 text-green-700 rounded-lg hover:border-
green-400 hover:bg-green-50 transition-all text-left">
```

```
<div className="font-semibold">Backup System</div>
```

```
<div className="text-sm text-green-600">Create system backup</div>
```

```
</button>
```

```
<button className="p-4 border border-blue-200 text-blue-700 rounded-lg hover:border-blue-
400 hover:bg-blue-50 transition-all text-left">
```

```
<div className="font-semibold">Update System</div>
```

```
<div className="text-sm text-blue-600">Check for updates</div>
```

```
</button>
```

```
</div>
```

```
</div>
```

```
{/* Danger Zone */}
```

```
<div className="bg-red-50 border border-red-200 rounded-lg p-6">
```

```

<h3 className="text-lg font-semibold text-red-700 mb-4 flex items-center space-x-2">
  <AlertTriangle className="w-5 h-5 text-red-600" />
  <span>Danger Zone</span>
</h3>

<p className="text-red-600 mb-4">Irreversible actions. Proceed with caution.</p>

<div className="grid grid-cols-1 md:grid-cols-2 gap-3">
  <button className="px-4 py-3 bg-red-600 text-white rounded-lg hover:bg-red-700 transition-
colors font-semibold border border-red-600">
    Reset Competition
  </button>
  <button className="px-4 py-3 border border-red-600 text-red-600 rounded-lg hover:bg-red-
50 transition-colors font-semibold">
    Purge All Data
  </button>
</div>
</div>
</div>
</div>
)

```

default:

```

  return null
}
}

```

if (loading) {

```

  return (
    <div className="flex items-center justify-center h-64">
      <div className="text-center">

```

```
    <div className="w-12 h-12 border-4 border-blue-500 border-t-transparent rounded-full animate-spin mx-auto mb-4"></div>
```

```
    <div className="text-gray-600 font-medium">Loading Admin Interface...</div>
```

```
  </div>
```

```
</div>
```

```
)
```

```
}
```

```
return (
```

```
  <div className="space-y-6">
```

```
    {renderTabContent()}</div>
```

```
  </div>
```

```
)
```

```
}
```

```
export default Admin",
```

```
src/pages/challenges.jsx
```

```
"import React, { useState, useEffect } from 'react'
```

```
import { useAuth } from '../contexts/AuthContext'
```

```
import ChallengeCard from '../components/challenges/ChallengeCard'
```

```
import UserStats from '../components/stats/UserStats'
```

```
import CTFTerminal from '../components/terminal/CTFTerminal';
```

```
import SecureChallengeStart from '../components/security/SecureChallengeStart';
```

```
import {
```

```
  Filter,
```

```
  Search,
```

```
  Grid,
```

```
List,  
Trophy,  
Terminal,  
Cpu,  
Network,  
Binary,  
FileSearch,  
Key,  
Bug,  
Zap,  
Eye,  
EyeOff,  
Server,  
Clock,  
Award,  
Shield,  
AlertTriangle,  
CheckCircle,  
XCircle,  
Loader  
} from 'lucide-react'  
  
import axios from 'axios'
```

```
const Challenges = () => {  
  const [challenges, setChallenges] = useState([])  
  const [filteredChallenges, setFilteredChallenges] = useState([])  
  const [loading, setLoading] = useState(true)  
  const [searchTerm, setSearchTerm] = useState('')  
  const [selectedCategory, setSelectedCategory] = useState('all')
```

```
const [selectedDifficulty, setSelectedDifficulty] = useState('all')
```

```
const [viewMode, setViewMode] = useState('grid')
```

```
const [userStats, setUserStats] = useState({})
```

```
const [showFilters, setShowFilters] = useState(false)
```

```
const [terminalOutput, setTerminalOutput] = useState([])
```

```
const [secureSession, setSecureSession] = useState(null)
```

```
const [sessionData, setSessionData] = useState(null)
```

```
const [autoRefresh, setAutoRefresh] = useState(true)
```

```
const { user } = useAuth()
```

```
const categories = [
```

```
  { value: 'all', label: 'ALL SYSTEMS', icon: Terminal, color: 'from-gray-500 to-gray-700', bgColor: 'bg-gradient-to-r from-gray-500 to-gray-700' },
```

```
  { value: 'web', label: 'WEB EXPLOITATION', icon: Network, color: 'from-blue-500 to-cyan-500', bgColor: 'bg-gradient-to-r from-blue-500 to-cyan-500' },
```

```
  { value: 'crypto', label: 'CRYPTOGRAPHY', icon: Key, color: 'from-yellow-500 to-amber-500', bgColor: 'bg-gradient-to-r from-yellow-500 to-amber-500' },
```

```
  { value: 'forensics', label: 'DIGITAL FORENSICS', icon: FileSearch, color: 'from-orange-500 to-red-500', bgColor: 'bg-gradient-to-r from-orange-500 to-red-500' },
```

```
  { value: 'pwn', label: 'BINARY EXPLOITATION', icon: Binary, color: 'from-purple-500 to-pink-500', bgColor: 'bg-gradient-to-r from-purple-500 to-pink-500' },
```

```
  { value: 'reverse', label: 'REVERSE ENGINEERING', icon: Cpu, color: 'from-green-500 to-emerald-500', bgColor: 'bg-gradient-to-r from-green-500 to-emerald-500' },
```

```
  { value: 'misc', label: 'MISCELLANEOUS', icon: Bug, color: 'from-indigo-500 to-purple-500', bgColor: 'bg-gradient-to-r from-indigo-500 to-purple-500' }
```

```
]
```

```
const difficulties = [
```

```
  { value: 'all', label: 'ALL DIFFICULTIES', color: 'text-gray-400', badgeColor: 'bg-gray-500' },
```

```
  { value: 'easy', label: 'BEGINNER', color: 'text-green-400', badgeColor: 'bg-green-500' },
```

```
    { value: 'medium', label: 'INTERMEDIATE', color: 'text-yellow-400', badgeColor: 'bg-yellow-500' },  
    { value: 'hard', label: 'EXPERT', color: 'text-red-400', badgeColor: 'bg-red-500' }  
  ]
```

```
const addTerminalLine = (text, type = 'info') => {  
  setTerminalOutput(prev => [...prev.slice(-19), {  
    text,  
    type,  
    timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })  
  }])  
}
```

```
// Auto-refresh challenges  
useEffect(() => {  
  if (autoRefresh) {  
    const interval = setInterval(() => {  
      fetchChallenges()  
    }, 30000) // Refresh every 30 seconds  
    return () => clearInterval(interval)  
  }  
}, [autoRefresh])
```

```
useEffect(() => {  
  fetchChallenges()  
  fetchUserStats()  
}, [])
```

```
useEffect(() => {  
  filterChallenges()
```

```
}, [challenges, searchTerm, selectedCategory, selectedDifficulty])
```

```
const fetchChallenges = async () => {  
  try {  
    if (terminalOutput.length === 0) {  
      addTerminalLine('Initializing challenge database...', 'system')  
    }  
    const response = await axios.get('/api/challenges')  
    setChallenges(response.data)  
    if (terminalOutput.length > 0) {  
      addTerminalLine(`Challenge database synchronized - ${response.data.length} targets`, 'success')  
    }  
  } catch (error) {  
    addTerminalLine('ERROR: Failed to load challenges - ' + error.message, 'error')  
    console.error('Error fetching challenges:', error)  
  } finally {  
    setLoading(false)  
  }  
}
```

```
const fetchUserStats = async () => {  
  try {  
    const response = await axios.get('/api/challenges')  
    const solved = response.data.filter(c => c.solved).length  
    const total = response.data.length  
    const totalPoints = response.data  
      .filter(c => c.solved)  
      .reduce((sum, c) => sum + c.points, 0)
```

```
setUserStats({
  solvedChallenges: solved,
  totalChallenges: total,
  totalPoints: totalPoints,
  successRate: total > 0 ? Math.round((solved / total) * 100) : 0,
  averagePoints: solved > 0 ? Math.round(totalPoints / solved) : 0
})
} catch (error) {
  console.error('Error fetching user stats:', error)
  addTerminalLine('WARNING: Failed to update user statistics', 'error')
}
}
```

```
const filterChallenges = () => {
```

```
  let filtered = challenges
```

```
  if (searchTerm) {
```

```
    filtered = filtered.filter(challenge =>
```

```
      challenge.title.toLowerCase().includes(searchTerm.toLowerCase()) ||
```

```
      challenge.description.toLowerCase().includes(searchTerm.toLowerCase()) ||
```

```
      (challenge.tags && challenge.tags.some(tag =>
```

```
        tag.toLowerCase().includes(searchTerm.toLowerCase())
```

```
      ))
```

```
  )
```

```
}
```

```
if (selectedCategory !== 'all') {
```

```
  filtered = filtered.filter(challenge => challenge.category === selectedCategory)
```

```
}
```



```

    if (selectedDifficulty !== 'all') {
      filtered = filtered.filter(challenge => challenge.difficulty === selectedDifficulty)
    }

    setFilteredChallenges(filtered)

    // Add terminal feedback for filtering
    if (searchTerm || selectedCategory !== 'all' || selectedDifficulty !== 'all') {
      addTerminalLine(`Filter applied: ${filtered.length} targets match criteria`, 'system')
    }
  }

  const startSecureSession = async (challengeId) => {
    try {
      addTerminalLine(`Initializing secure environment for challenge ${challengeId}...`, 'system')
      const response = await axios.post('/api/secure-session/initialize', {
        challengeId: challengeId
      })

      const data = await response.data
      setSessionData(data.session)
      setSecureSession('active')
      addTerminalLine('Secure session initialized - Webcam monitoring activated', 'success')
    } catch (error) {
      addTerminalLine('ERROR: Failed to start secure session - ' + error.message, 'error')
      console.error('Failed to start secure session:', error)
    }
  }
}

```

```

const handleChallengeSolve = (challengeId, points) => {
  setChallenges(prev =>
    prev.map(challenge =>
      challenge.id === challengeId
        ? { ...challenge, solved: true }
        : challenge
    )
  )

  addTerminalLine(`🎯 TARGET NEUTRALIZED: Challenge ${challengeId} completed! +${points} points`,
'success')

  fetchUserStats()

  // Add celebration effect
  setTimeout(() => {
    addTerminalLine('🎉 System access granted - Proceeding to next objective', 'system')
  }, 1000)
}

const handleViolation = (violation) => {
  addTerminalLine(`🚫 SECURITY VIOLATION: ${violation.type} detected`, 'error')
}

const handleTerminate = (reason) => {
  addTerminalLine(`💀 SESSION TERMINATED: ${reason}`, 'error')

  setSecureSession(null)

  setSessionData(null)
}

```

```
const getCategoryIcon = (category) => {  
  const cat = categories.find(c => c.value === category)  
  return cat ? cat.icon : Terminal  
}
```

```
const getDifficultyColor = (difficulty) => {  
  switch (difficulty) {  
    case 'easy': return 'text-green-400 border-green-500 bg-green-500 bg-opacity-10'  
    case 'medium': return 'text-yellow-400 border-yellow-500 bg-yellow-500 bg-opacity-10'  
    case 'hard': return 'text-red-400 border-red-500 bg-red-500 bg-opacity-10'  
    default: return 'text-gray-400 border-gray-500 bg-gray-500 bg-opacity-10'  
  }  
}
```

```
const clearAllFilters = () => {  
  setSearchTerm("")  
  setSelectedCategory('all')  
  setSelectedDifficulty('all')  
  addTerminalLine('All filters cleared - Showing complete target list', 'system')  
}
```

```
if (secureSession === 'active' && sessionData) {  
  return (  
    <SecureChallengeStart  
      sessionId={sessionData.sessionId}  
      challengeId={sessionData.challenge.id}  
      onViolation={handleViolation}  
      onTerminate={handleTerminate}  
      challenge={challenges.find(c => c.id === sessionData.challenge.id)}
```

```

    />
  )
}

if (loading) {
  return (
    <div className="min-h-screen bg-gray-950 flex items-center justify-center">
      <div className="text-center">
        <div className="w-16 h-16 border-4 border-green-500 border-t-transparent rounded-full
animate-spin mx-auto mb-4"></div>
        <div className="text-green-400 font-mono text-lg">INITIALIZING_CHALLENGE_DATABASE</div>
        <div className="text-green-600 text-sm mt-2 animate-pulse">Establishing secure
connection...</div>
        <div className="mt-4 flex space-x-2 justify-center">
          <div className="w-2 h-2 bg-green-500 rounded-full animate-bounce"></div>
          <div className="w-2 h-2 bg-green-500 rounded-full animate-bounce" style={{animationDelay:
'0.1s'}}></div>
          <div className="w-2 h-2 bg-green-500 rounded-full animate-bounce" style={{animationDelay:
'0.2s'}}></div>
        </div>
      </div>
    </div>
  )
}

return (
  <div className="min-h-screen bg-gray-950 text-green-400 font-mono">
    <div className="max-w-7xl mx-auto px-4 py-8">
      {/* Terminal Header */}

```

```
<div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20 mb-8">
```

```
<div className="flex items-center justify-between p-4 border-b border-green-500/30">
```

```
<div className="flex items-center space-x-2">
```

```
<div className="w-3 h-3 bg-red-500 rounded-full animate-pulse"></div>
```

```
<div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
```

```
<div className="w-3 h-3 bg-green-500 rounded-full"></div>
```

```
<div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- challenge_interface</div>
```

```
</div>
```

```
<div className="flex items-center space-x-4">
```

```
<button
```

```
onClick={() => setAutoRefresh(!autoRefresh)}
```

```
className={`text-xs px-3 py-1 rounded border ${
```

```
autoRefresh
```

```
? 'bg-green-500 bg-opacity-20 border-green-500 text-green-400'
```

```
: 'bg-gray-500 bg-opacity-20 border-gray-500 text-gray-400'
```

```
}`}>
```

```
>
```

```
AUTO: {autoRefresh ? 'ON' : 'OFF'}
```

```
</button>
```

```
<button
```

```
onClick={fetchChallenges}
```

```
className="text-xs px-3 py-1 rounded border border-cyan-500 bg-cyan-500 bg-opacity-10 text-cyan-400 hover:bg-cyan-500 hover:bg-opacity-20"
```

```
>
```

```
SYNC
```

```
</button>
```

```
</div>
```

```
</div>
```

```
<div className="p-6">

  <div className="flex items-center space-x-2 text-green-300 mb-4">

    <Terminal className="w-5 h-5" />

    <span className="text-sm">CHALLENGE_MANAGEMENT_SYSTEM v2.1.4</span>

  </div>
```

```
{/* Terminal Output */}
```

```
<div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-32 overflow-y-auto">
```

```
{terminalOutput.length === 0 ? (
```

```
  <div className="text-green-600 text-sm">
```

```
    <div>$ Initializing terminal...</div>
```

```
    <div>$ Loading challenge database...</div>
```

```
    <div>$ Establishing secure connection...</div>
```

```
    <div>$ Ready for operations</div>
```

```
  </div>
```

```
): (
```

```
  terminalOutput.map((line, index) => (
```

```
    <div key={index} className={`text-sm ${
```

```
      line.type === 'error' ? 'text-red-400' :
```

```
      line.type === 'success' ? 'text-green-400' :
```

```
      line.type === 'system' ? 'text-cyan-400' : 'text-green-300'
```

```
    `}>
```

```
    <span className="text-gray-500">[{line.timestamp}]</span> {line.text}
```

```
  </div>
```

```
))
```

```
})
```

```
</div>
```

```

    { /* User Stats */ }

    <UserStats user={user} stats={userStats} />

  </div>

</div>

{ /* Main Content */ }

<div className="flex flex-col lg:flex-row gap-6">

  { /* Sidebar Filters */ }

  <div className="lg:w-1/4 space-y-6">

    { /* Search Box */ }

    <div className="bg-black border border-green-500 rounded-lg p-4">

      <label className="block text-sm font-bold text-green-400 mb-2">

        <Search className="w-4 h-4 inline mr-2" />

        [SEARCH] TARGET_SCAN

      </label>

      <div className="relative">

        <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"

/>

        <input

          type="text"

          placeholder="grep -r 'flag' ./challenges"

          value={searchTerm}

          onChange={(e) => setSearchTerm(e.target.value)}

          className="w-full pl-10 pr-4 py-3 bg-black border border-green-500 rounded-lg focus:outline-
none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-green-600
transition-all font-mono text-sm"

        />

      </div>

    </div>

  </div>

```

```

    { /* Category Filter */ }

    <div className="bg-black border border-green-500 rounded-lg p-4">

      <div className="flex items-center justify-between mb-4">

        <label className="block text-sm font-bold text-green-400">

          <Filter className="w-4 h-4 inline mr-2" />

          [FILTER] OPERATION_TYPES

        </label>

        <div className="text-green-400 text-xs">

          {categories.find(c => c.value === selectedCategory)?.label.split(' ')[0]}

        </div>

      </div>

      <div className="space-y-2 max-h-96 overflow-y-auto">

        {categories.map(category => {

          const Icon = category.icon

          return (

            <button

              key={category.value}

              onClick={() => setSelectedCategory(category.value)}

              className={`w-full flex items-center space-x-3 p-3 rounded-lg border transition-all text-left
group ${
                selectedCategory === category.value
                  ? 'border-cyan-400 bg-cyan-500 bg-opacity-10 text-cyan-400 shadow-lg shadow-cyan-
500/20'
                  : 'border-green-500 text-green-400 hover:border-cyan-400 hover:bg-cyan-500 hover:bg-
opacity-5 hover:shadow-lg hover:shadow-cyan-500/10'
              }`}

            >

              <div className={`w-8 h-8 rounded-lg ${category.bgColor} flex items-center justify-center
shadow-md group-hover:scale-110 transition-transform`}>

                <Icon className="w-4 h-4 text-white" />

```



```

</div>

<span className="text-sm font-semibold flex-1">{category.label}</span>

{selectedCategory === category.value && (
  <CheckCircle className="w-4 h-4 text-cyan-400" />
)}
</button>
)
}}
</div>
</div>

```

```

{/* Difficulty Filter */}
<div className="bg-black border border-green-500 rounded-lg p-4">
  <label className="block text-sm font-bold text-green-400 mb-4">
    <Zap className="w-4 h-4 inline mr-2" />
    [DIFFICULTY] THREAT_LEVEL
  </label>
  <div className="space-y-2">
    {difficulties.map(difficulty => (
      <button
        key={difficulty.value}
        onClick={() => setSelectedDifficulty(difficulty.value)}
        className={`w-full p-3 rounded-lg border transition-all text-left group relative overflow-
hidden ${
          selectedDifficulty === difficulty.value
            ? 'border-cyan-400 bg-cyan-500 bg-opacity-10 text-cyan-400 shadow-lg shadow-cyan-
500/20'
            : `${difficulty.color} border-green-500 hover:border-cyan-400 hover:bg-cyan-500 hover:bg-
opacity-5`
        }`}
      >

```

```

>
<span className="text-sm font-semibold relative z-10">{difficulty.label}</span>
{selectedDifficulty === difficulty.value && (
  <div className="absolute right-3 top-1/2 transform -translate-y-1/2">
    <CheckCircle className="w-4 h-4 text-cyan-400" />
  </div>
)}
</button>
)}}
</div>
</div>

```

```

{/* Quick Stats */}
<div className="bg-black border border-cyan-500 rounded-lg p-4">
  <label className="block text-sm font-bold text-cyan-400 mb-4">
    <Server className="w-4 h-4 inline mr-2" />
    [STATUS] SYSTEM_OVERVIEW
  </label>
  <div className="space-y-3">
    <div className="flex justify-between items-center">
      <span className="text-green-300 text-sm">ACTIVE_TARGETS</span>
      <span className="text-white font-bold">{challenges.length}</span>
    </div>
    <div className="flex justify-between items-center">
      <span className="text-green-300 text-sm">FILTERED_TARGETS</span>
      <span className="text-white font-bold">{filteredChallenges.length}</span>
    </div>
    <div className="flex justify-between items-center">
      <span className="text-green-300 text-sm">NEUTRALIZED</span>

```

```

    <span className="text-green-400 font-bold">{userStats.solvedChallenges || 0}</span>
  </div>

  <div className="flex justify-between items-center">
    <span className="text-green-300 text-sm">SUCCESS_RATE</span>
    <span className="text-yellow-400 font-bold">{userStats.successRate || 0}%</span>
  </div>

  <div className="flex justify-between items-center">
    <span className="text-green-300 text-sm">TOTAL_POINTS</span>
    <span className="text-cyan-400 font-bold">{userStats.totalPoints || 0}</span>
  </div>
</div>
</div>
</div>
</div>

```

```

{/* Main Challenges Area */}
<div className="lg:w-3/4 space-y-6">
  {/* Controls Bar */}
  <div className="bg-black border border-green-500 rounded-lg p-4">
    <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center space-y-4 sm:space-y-0">
      <div className="flex items-center space-x-4">
        <span className="text-green-300 text-sm font-bold">
          DISPLAYING {filteredChallenges.length} OF {challenges.length} TARGETS
        </span>
        {(searchTerm || selectedCategory !== 'all' || selectedDifficulty !== 'all') && (
          <button
            onClick={clearAllFilters}
            className="text-cyan-400 hover:text-cyan-300 text-sm font-bold border border-cyan-400
            px-3 py-1 rounded hover:bg-cyan-500 hover:bg-opacity-10 transition-all"

```

```

    >
    [CLEAR_FILTERS]
  </button>
)}
</div>

```

```

<div className="flex items-center space-x-4">
  <span className="text-green-300 text-sm">VIEW_MODE:</span>
  <div className="flex border border-green-500 rounded-lg overflow-hidden">
    <button
      onClick={() => setViewMode('grid')}
      className={`p-2 transition-all ${
        viewMode === 'grid'
          ? 'bg-cyan-500 text-white'
          : 'bg-black text-green-400 hover:bg-cyan-500 hover:bg-opacity-10'
      }`}
      title="Grid View"
    >
      <Grid className="w-4 h-4" />
    </button>
    <button
      onClick={() => setViewMode('list')}
      className={`p-2 transition-all ${
        viewMode === 'list'
          ? 'bg-cyan-500 text-white'
          : 'bg-black text-green-400 hover:bg-cyan-500 hover:bg-opacity-10'
      }`}
      title="List View"
    >

```

```

        <List className="w-4 h-4" />
      </button>
    </div>
  </div>
</div>
</div>
</div>

```

```

{ /* Challenges Grid/List */
{filteredChallenges.length === 0 ? (
  <div className="bg-black border border-green-500 rounded-lg p-12 text-center">
    <Filter className="w-16 h-16 text-green-400 mx-auto mb-4 opacity-50" />
    <h3 className="text-xl font-bold text-white mb-2">NO TARGETS FOUND</h3>
    <p className="text-green-300 mb-4">Adjust your search parameters or clear filters</p>
    <button
      onClick={clearAllFilters}
      className="px-6 py-3 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold
rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all border border-cyan-400 shadow-lg
shadow-cyan-500/20"
    >
      RESET_FILTERS
    </button>
  </div>
) : (
  <div className={viewMode === 'grid'
    ? 'grid md:grid-cols-2 xl:grid-cols-3 gap-6'
    : 'space-y-4'}>
    <div>
      {filteredChallenges.map(challenge => (
        <ChallengeCard

```

```

        key={challenge.id}
        challenge={challenge}
        onSolve={handleChallengeSolve}
        onStartSecureSession={startSecureSession}
        viewMode={viewMode}
      />
    )}
  </div>
})

```

```

{/* System Status Footer */}
<div className="bg-black border border-green-500 rounded-lg p-4">
  <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-center">
    <div className="text-green-400">
      <div className="text-xs uppercase tracking-wider">CHALLENGE_DB</div>
      <div className="text-white font-bold text-lg flex items-center justify-center">
        <div className="w-2 h-2 bg-green-500 rounded-full mr-2 animate-pulse"></div>
        ONLINE
      </div>
    </div>
    <div className="text-cyan-400">
      <div className="text-xs uppercase tracking-wider">AUTH_SYSTEM</div>
      <div className="text-white font-bold text-lg flex items-center justify-center">
        <Shield className="w-4 h-4 mr-2" />
        SECURE
      </div>
    </div>
    <div className="text-yellow-400">
      <div className="text-xs uppercase tracking-wider">FLAG_SYSTEM</div>

```

```

    <div className="text-white font-bold text-lg flex items-center justify-center">
      <Key className="w-4 h-4 mr-2" />
      ACTIVE
    </div>
  </div>
  <div className="text-green-400">
    <div className="text-xs uppercase tracking-wider">SCOREBOARD</div>
    <div className="text-white font-bold text-lg flex items-center justify-center">
      <Trophy className="w-4 h-4 mr-2" />
      LIVE
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
)
}

```

```

export default Challenges"
src/pages/Home.jsx
"import React, { useState, useEffect } from 'react'
import { Link } from 'react-router-dom'
import { useAuth } from '../contexts/AuthContext'
import {
  Terminal,
  Cpu,

```

Binary,
Network,
Server,
Key,
FileSearch,
Code2,
Shield,
Zap,
Lock,
Scan,
Users,
Clock,
Award,
Flag,
Github,
Twitter,
Linkedin,
Mail,
ShieldCheck,
Cpu as Chip,
Database,
Globe,
Code,
Bug,
Eye
} from 'lucide-react'

```
const Home = () => {  
  const { user } = useAuth()
```



```

const [terminalText, setTerminalText] = useState("")
const [isTyping, setIsTyping] = useState(true)
const [activeUsers, setActiveUsers] = useState(0)
const [challengesSolved, setChallengesSolved] = useState(0)
const [currentTime, setCurrentTime] = useState("")

// Terminal typing effect
useEffect(() => {
  const fullText = "cyber-arena@ctf:~$ initialize_system --mode=competition"
  let currentIndex = 0

  const typingInterval = setInterval(() => {
    if (currentIndex <= fullText.length) {
      setTerminalText(fullText.slice(0, currentIndex))
      currentIndex++
    } else {
      setIsTyping(false)
      clearInterval(typingInterval)
    }
  }, 50)

  return () => clearInterval(typingInterval)
}, [])

// Live stats simulation
useEffect(() => {
  const interval = setInterval(() => {
    setActiveUsers(prev => {
      const change = Math.floor(Math.random() * 5) - 2

```

```
    return Math.max(142, prev + change)
  })
  setChallengesSolved(prev => prev + Math.floor(Math.random() * 3))
}, 3000)
```

```
// Update time
const timeInterval = setInterval(() => {
  setCurrentTime(new Date().toLocaleTimeString('en-US', {
    hour12: false,
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  }))
}, 1000)
```

```
return () => {
  clearInterval(interval)
  clearInterval(timeInterval)
}
}, [])
```

```
const features = [
  {
    icon: Network,
    title: 'NETWORK PENETRATION',
    description: 'Infiltrate secure networks and bypass enterprise security systems.',
    color: 'from-green-500 to-cyan-500',
    command: 'nmap --stealth',
    difficulty: 'MEDIUM-HARD',
```

```
points: 200
},
{
  icon: Binary,
  title: 'BINARY REVERSING',
  description: 'Decompile and analyze malicious software for vulnerabilities.',
  color: 'from-purple-500 to-pink-500',
  command: 'objdump -M intel',
  difficulty: 'HARD',
  points: 300
},
{
  icon: FileSearch,
  title: 'DIGITAL FORENSICS',
  description: 'Recover and analyze digital evidence from compromised systems.',
  color: 'from-orange-500 to-red-500',
  command: 'foremost -t all',
  difficulty: 'MEDIUM',
  points: 150
},
{
  icon: Key,
  title: 'CRYPTOGRAPHY',
  description: 'Break encryption schemes and decrypt classified communications.',
  color: 'from-yellow-500 to-amber-500',
  command: 'openssl rsautl -decrypt',
  difficulty: 'EASY-MEDIUM',
  points: 100
},
```

```

{
  icon: Bug,
  title: 'WEB EXPLOITATION',
  description: 'Find and exploit vulnerabilities in web applications and APIs.',
  color: 'from-blue-500 to-indigo-500',
  command: 'sqlmap -u target',
  difficulty: 'MEDIUM',
  points: 250
},
{
  icon: Chip,
  title: 'SYSTEM HACKING',
  description: 'Gain root access and maintain persistence on target systems.',
  color: 'from-red-500 to-pink-500',
  command: 'metasploit exploit',
  difficulty: 'HARD',
  points: 400
}
]

```

```

const stats = [
  { number: '127.0.0.1', label: 'ACTIVE CONNECTIONS', icon: Server },
  { number: '0x3A7', label: 'LINES OF CODE', icon: Code2 },
  { number: '64-BIT', label: 'SYSTEM ARCH', icon: Cpu },
  { number: 'ROOT', label: 'ACCESS LEVEL', icon: Shield }
]

```

```

const howItWorks = [
  {

```

```

    step: '01',
    title: 'ESTABLISH CONNECTION',
    description: 'Authenticate and gain system access',
    command: 'ssh ctf@cyber-arena.io'
  },
  {
    step: '02',
    title: 'ANALYZE TARGETS',
    description: 'Select and recon challenge objectives',
    command: './recon --target all'
  },
  {
    step: '03',
    title: 'EXPLOIT VULNERABILITIES',
    description: 'Execute precision attacks on systems',
    command: 'msfconsole -q'
  },
  {
    step: '04',
    title: 'MAINTAIN ACCESS',
    description: 'Submit flags and track persistence',
    command: 'echo $FLAG > /tmp/persistence'
  }
]

```

```

const liveStats = [
  { icon: Users, value: activeUsers, label: 'ACTIVE HACKERS', change: '+2.4%' },
  { icon: Flag, value: challengesSolved, label: 'CHALLENGES SOLVED', change: '+12%' },
  { icon: Award, value: '24/7', label: 'UPTIME', change: '100%' },

```

```
{ icon: Clock, value: currentTime, label: 'SERVER TIME', change: 'UTC+0' }  
]
```

```
const recentActivity = [  
  { user: 'ne0n', action: 'solved Crypto-100', time: '2 min ago', points: '+100' },  
  { user: 'ph0b0s', action: 'solved Web-200', time: '5 min ago', points: '+200' },  
  { user: 'zer0c00l', action: 'solved Forensics-150', time: '8 min ago', points: '+150' },  
  { user: 'dark_side', action: 'solved Binary-300', time: '12 min ago', points: '+300' }  
]
```

```
const sponsors = [  
  { name: 'CYBER_SEC INC', tier: 'PLATINUM' },  
  { name: 'SECURE_TECH', tier: 'GOLD' },  
  { name: 'HACK_LABS', tier: 'SILVER' },  
  { name: 'BYTE_GUARD', tier: 'BRONZE' }  
]
```

```
return (  
  <div className="min-h-screen bg-gray-950 text-green-400 font-mono relative">  
    { /* Matrix Background */ }  
    <div className="fixed inset-0 bg-black opacity-90 z-0 pt-16">  
      <div className="matrix-rain"></div>  
    </div>  
  
    <div className="relative z-10 space-y-16 lg:space-y-24 pt-16">  
      { /* Hero Section */ }  
      <section className="px-4">  
        <div className="max-w-6xl mx-auto">  
          { /* Terminal Header */ }
```

```
<div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20 mb-8">
```

```
<div className="flex items-center space-x-2 p-4 border-b border-green-500/30">
```

```
<div className="w-3 h-3 bg-red-500 rounded-full"></div>
```

```
<div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
```

```
<div className="w-3 h-3 bg-green-500 rounded-full"></div>
```

```
<div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- bash</div>
```

```
</div>
```

```
<div className="p-6">
```

```
<div className="flex items-center space-x-2 text-green-300 mb-4">
```

```
<Terminal className="w-5 h-5" />
```

```
<span className="text-sm">SYSTEM INITIALIZED</span>
```

```
</div>
```

```
<div className="space-y-2">
```

```
<div className="text-green-400">
```

```
<span className="text-cyan-400">└─[</span>
```

```
<span className="text-yellow-400">root@cyber-arena</span>
```

```
<span className="text-cyan-400">]─[</span>
```

```
<span className="text-white">/home/ctf</span>
```

```
<span className="text-cyan-400">]</span>
```

```
</div>
```

```
<div className="text-green-400">
```

```
<span className="text-cyan-400">└─</span>
```

```
<span className="text-white">$ </span>
```

```
<span className="terminal-text">{terminalText}</span>
```

```
{isTyping && <span className="blinking-cursor">█</span>}
```

```
</div>
```

```
</div>
```


 <<

Advanced cybersecurity challenges for elite operators

</p>

{!user ? (

<div className="flex flex-col sm:flex-row gap-4 justify-center mt-8">

<Link

to="/register"

className="inline-flex items-center justify-center px-8 py-4 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all transform hover:scale-105 border border-green-400 shadow-lg shadow-green-500/25"

>

<Shield className="w-5 h-5 mr-2" />

INITIATE_SYSTEM_ACCESS

</Link>

<Link

to="/login"

className="inline-flex items-center justify-center px-8 py-4 border-2 border-green-500 text-green-400 font-bold rounded-lg hover:bg-green-500 hover:bg-opacity-10 transition-all"

>

AUTHENTICATE

</Link>

</div>

): (

<div className="flex flex-col sm:flex-row gap-4 justify-center mt-8">

<Link

to="/challenges"

className="inline-flex items-center justify-center px-8 py-4 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all transform hover:scale-105 border border-green-400 shadow-lg shadow-green-500/25"

```

    >
    <Zap className="w-5 h-5 mr-2" />
    ACCESS_CHALLENGES
  </Link>
  <Link
    to="/leaderboard"
    className="inline-flex items-center justify-center px-8 py-4 border-2 border-cyan-500 text-
cyan-400 font-bold rounded-lg hover:bg-cyan-500 hover:bg-opacity-10 transition-all"
  >
    <Scan className="w-5 h-5 mr-2" />
    VIEW_RANKINGS
  </Link>
</div>
)}
</div>
</div>
</section>

```

```

{/* Live Stats Section */}
<section className="px-4">
  <div className="max-w-6xl mx-auto">
    <div className="grid grid-cols-2 lg:grid-cols-4 gap-4">
      {liveStats.map((stat, index) => {
        const Icon = stat.icon
        return (
          <div key={index} className="bg-black border border-green-500 rounded-lg p-4 text-center
hover:border-cyan-400 transition-all duration-300 group">
            <Icon className="w-8 h-8 text-cyan-400 mx-auto mb-2 group-hover:scale-110 transition-
transform" />
            <div className="text-2xl font-bold text-white mb-1">{stat.value}</div>

```

```

        <div className="text-green-300 text-sm font-semibold mb-1">{stat.label}</div>

        <div className="text-cyan-400 text-xs">{stat.change}</div>

    </div>

    )

    }}}

</div>

</div>

</section>

```

```

{/* Features Section */}

<section className="px-4">

    <div className="max-w-6xl mx-auto">

        <div className="text-center mb-12">

            <h2 className="text-2xl lg:text-4xl font-bold text-white mb-4 glitch" data-
text="OPERATION_MODULES">

                OPERATION_MODULES

            </h2>

            <p className="text-green-300 max-w-2xl mx-auto">

                Specialized cyber operation units ready for deployment

            </p>

        </div>

    </div>

    <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-6">

        {features.map((feature, index) => {

            const Icon = feature.icon

            return (

                <div key={index} className="group">

                    <div className="bg-black border border-green-500 rounded-lg p-6 hover:border-cyan-400
hover:shadow-2xl hover:shadow-cyan-500/10 transition-all duration-300 transform hover:-translate-y-1
h-full">

```

```
<div className="flex items-start justify-between mb-4">

  <div className={`inline-flex items-center justify-center w-12 h-12 rounded-lg bg-gradient-
to-r ${feature.color} border border-white border-opacity-20`}>

    <Icon className="w-6 h-6 text-white" />

  </div>

  <div className="text-right space-y-1">

    <div className="text-green-400 text-xs font-mono bg-green-500 bg-opacity-10 px-2 py-1
rounded border border-green-500">

      {feature.points} PTS

    </div>

    <div className="text-yellow-400 text-xs font-mono bg-yellow-500 bg-opacity-10 px-2 py-
1 rounded border border-yellow-500">

      {feature.difficulty}

    </div>

  </div>

  <h3 className="text-xl font-bold text-white mb-3">{feature.title}</h3>

  <p className="text-green-300 leading-relaxed mb-4">{feature.description}</p>

  <div className="text-green-400 text-sm font-mono bg-green-500 bg-opacity-5 px-3 py-2
rounded border border-green-500">

    {feature.command}

  </div>

</div>

</div>

</div>

)

}}

</div>

</div>

</section>
```

```

    { /* How It Works */ }

    <section className="px-4">

      <div className="max-w-6xl mx-auto">

        <div className="bg-black border border-cyan-500 rounded-xl p-8 lg:p-12">

          <div className="text-center mb-12">

            <h2 className="text-2xl lg:text-4xl font-bold text-white mb-4 glitch" data-
text="MISSION_PROTOCOL">

              MISSION_PROTOCOL

            </h2>

            <p className="text-cyan-300 max-w-2xl mx-auto">

              Standard operating procedure for successful infiltration

            </p>

          </div>

          <div className="grid md:grid-cols-2 lg:grid-cols-4 gap-6 lg:gap-8">

            {howItWorks.map((item, index) => (

              <div key={index} className="text-center group">

                <div className="w-16 h-16 bg-gradient-to-r from-green-500 to-cyan-600 rounded-full flex
items-center justify-center text-white font-bold text-xl mx-auto mb-4 border border-cyan-400 group-
hover:border-green-400 transition-all">

                  {item.step}

                </div>

                <div className="text-green-400 text-sm font-mono bg-green-500 bg-opacity-10 px-3 py-1
rounded border border-green-500 mb-3">

                  {item.command}

                </div>

                <h3 className="text-lg font-bold text-white mb-2">{item.title}</h3>

                <p className="text-green-300 text-sm">{item.description}</p>

              </div>

            ))}

          </div>

        </div>

      </div>

```

```
    </div>
  </div>
</div>
</section>
```

```
{/* Recent Activity */}
```

```
<section className="px-4">
  <div className="max-w-6xl mx-auto">
    <div className="bg-black border border-green-500 rounded-xl p-8">
      <div className="flex items-center justify-between mb-6">
        <h2 className="text-2xl lg:text-3xl font-bold text-white glitch" data-text="LIVE_ACTIVITY">
          LIVE_ACTIVITY
        </h2>
        <div className="flex items-center space-x-2 text-green-400">
          <Eye className="w-5 h-5" />
          <span className="text-sm">REAL-TIME FEED</span>
        </div>
      </div>

      <div className="space-y-3">
        {recentActivity.map((activity, index) => (
          <div key={index} className="flex items-center justify-between p-4 bg-black border border-
green-500 rounded-lg hover:border-cyan-400 transition-all group">
            <div className="flex items-center space-x-4">
              <div className="w-10 h-10 bg-gradient-to-r from-green-500 to-cyan-500 rounded-full flex
items-center justify-center">
                <span className="text-white font-bold text-sm">{activity.user[0]}</span>
              </div>
              <div>
```

```

        <div className="text-white font-semibold">{activity.user}</div>
        <div className="text-green-300 text-sm">{activity.action}</div>
      </div>
    </div>
    <div className="text-right">
      <div className="text-cyan-400 font-bold">{activity.points}</div>
      <div className="text-green-400 text-sm">{activity.time}</div>
    </div>
  </div>
  )}}
</div>
</div>
</div>
</section>

```

```

{ /* Sponsors Section */ }
<section className="px-4">
  <div className="max-w-6xl mx-auto">
    <div className="text-center mb-8">
      <h2 className="text-2xl lg:text-3xl font-bold text-white mb-4">
        STRATEGIC PARTNERS
      </h2>
      <p className="text-green-300">Supported by industry leaders in cybersecurity</p>
    </div>
    <div className="grid grid-cols-2 md:grid-cols-4 gap-6">
      {sponsors.map((sponsor, index) => (
        <div key={index} className="bg-black border border-cyan-500 rounded-lg p-6 text-center
        hover:border-green-400 transition-all group">

```

```

<div className={`text-lg font-bold mb-2 ${
  sponsor.tier === 'PLATINUM' ? 'text-yellow-400' :
  sponsor.tier === 'GOLD' ? 'text-yellow-300' :
  sponsor.tier === 'SILVER' ? 'text-gray-300' : 'text-amber-600'
}`}>
  {sponsor.name}
</div>

<div className="text-green-400 text-sm font-mono bg-green-500 bg-opacity-10 px-2 py-1
rounded border border-green-500">
  {sponsor.tier}
</div>
</div>
)}}
</div>
</div>
</section>

{/* CTA Section */}
<section className="px-4 pb-16">
  <div className="max-w-4xl mx-auto">
    <div className="bg-gradient-to-r from-green-900 via-black to-cyan-900 border border-green-500
rounded-xl p-8 lg:p-12 text-center relative overflow-hidden">
      <div className="absolute inset-0 bg-grid-pattern opacity-10"></div>

      <Lock className="w-12 h-12 text-cyan-400 mx-auto mb-6" />
      <h2 className="text-2xl lg:text-4xl font-bold text-white mb-4">
        READY FOR DEPLOYMENT?
      </h2>
      <p className="text-green-300 text-lg mb-8 max-w-2xl mx-auto">

```


Join the elite cybersecurity operators and test your skills in real-world scenarios

</p>

{!user ? (

<Link

to="/register"

className="inline-flex items-center justify-center px-8 py-4 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all transform hover:scale-105 border border-cyan-400 shadow-lg shadow-cyan-500/25 relative z-10"

>

<Server className="w-5 h-5 mr-2" />

COMMENCE_INITIALIZATION

</Link>

): (

<Link

to="/challenges"

className="inline-flex items-center justify-center px-8 py-4 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all transform hover:scale-105 border border-cyan-400 shadow-lg shadow-cyan-500/25 relative z-10"

>

<Zap className="w-5 h-5 mr-2" />

RESUME_OPERATIONS

</Link>

}}

</div>

</div>

</section>

{/* Footer */}

<footer className="bg-black border-t border-green-500 pt-12 pb-8">

<div className="max-w-6xl mx-auto px-4">

```
<div className="grid md:grid-cols-4 gap-8 mb-8">

  {/* Brand */}

  <div className="space-y-4">

    <div className="flex items-center space-x-3">

      <Terminal className="w-8 h-8 text-cyan-400" />

      <span className="text-xl font-bold text-white">CYBER_ARENA</span>

    </div>

    <p className="text-green-300 text-sm leading-relaxed">

      Advanced cybersecurity training platform for elite operators and penetration testers.

    </p>

    <div className="flex space-x-4">

      <a href="#" className="text-green-400 hover:text-cyan-400 transition-colors">

        <Github className="w-5 h-5" />

      </a>

      <a href="#" className="text-green-400 hover:text-cyan-400 transition-colors">

        <Twitter className="w-5 h-5" />

      </a>

      <a href="#" className="text-green-400 hover:text-cyan-400 transition-colors">

        <Linkedin className="w-5 h-5" />

      </a>

      <a href="#" className="text-green-400 hover:text-cyan-400 transition-colors">

        <Mail className="w-5 h-5" />

      </a>

    </div>

  </div>

  {/* Quick Links */}

  <div>
```

```
<h3 className="text-white font-bold mb-4 text-sm uppercase tracking-wider">NAVIGATION</h3>
```

```
<div className="space-y-2">
```

```
{['Challenges', 'Leaderboard', 'Rules', 'FAQ'].map((item) => (
```

```
<Link
```

```
key={item}
```

```
to={`/${item.toLowerCase()}`}
```

```
className="block text-green-300 hover:text-cyan-400 transition-colors text-sm"
```

```
>
```

```
{item}
```

```
</Link>
```

```
))}
```

```
</div>
```

```
</div>
```

```
{/* Resources */}
```

```
<div>
```

```
<h3 className="text-white font-bold mb-4 text-sm uppercase tracking-wider">RESOURCES</h3>
```

```
<div className="space-y-2">
```

```
{['Documentation', 'Tutorials', 'Blog', 'Community'].map((item) => (
```

```
<a
```

```
key={item}
```

```
href="#"
```

```
className="block text-green-300 hover:text-cyan-400 transition-colors text-sm"
```

```
>
```

```
{item}
```

```
</a>
```

```
))}
```

</div>

</div>

{/* Security */}

<div>

<h3 className="text-white font-bold mb-4 text-sm uppercase tracking-wider">SECURITY</h3>

<div className="space-y-2">

['Bug Bounty', 'Responsible Disclosure', 'Security Policy', 'Contact'].map((item) => (

<a

key={item}

href="#"

className="block text-green-300 hover:text-cyan-400 transition-colors text-sm"

>

{item}

))}

</div>

</div>

</div>

{/* Bottom Bar */}

<div className="border-t border-green-500 pt-8">

<div className="flex flex-col md:flex-row justify-between items-center space-y-4 md:space-y-0">

<div className="text-green-300 text-sm">

© 2024 CYBER_ARENA_CTF. ALL SYSTEMS SECURE.

</div>

<div className="flex space-x-6 text-sm text-green-300">

Privacy


```
left: 0;
width: 100%;
height: 100%;
}
```

```
.glitch::before {
  animation: glitch-1 0.5s infinite linear alternate-reverse;
  color: #ff00ff;
  z-index: -1;
}
```

```
.glitch::after {
  animation: glitch-2 0.5s infinite linear alternate-reverse;
  color: #00ffff;
  z-index: -2;
}
```

```
@keyframes glitch-1 {
  0% { transform: translate(0); }
  20% { transform: translate(-2px, 2px); }
  40% { transform: translate(-2px, -2px); }
  60% { transform: translate(2px, 2px); }
  80% { transform: translate(2px, -2px); }
  100% { transform: translate(0); }
}
```

```
@keyframes glitch-2 {
  0% { transform: translate(0); }
  20% { transform: translate(2px, -2px); }
```

```
40% { transform: translate(2px, 2px); }  
60% { transform: translate(-2px, -2px); }  
80% { transform: translate(-2px, 2px); }  
100% { transform: translate(0); }  
}
```

```
.matrix-rain {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background: linear-gradient(180deg, rgba(0,255,0,0.1) 0%, transparent 50%);  
  opacity: 0.1;  
  pointer-events: none;  
}
```

```
.bg-grid-pattern {  
  background-image:  
    linear-gradient(rgba(0, 255, 0, 0.1) 1px, transparent 1px),  
    linear-gradient(90deg, rgba(0, 255, 0, 0.1) 1px, transparent 1px);  
  background-size: 50px 50px;  
}
```

```
.terminal-text {  
  font-family: 'Courier New', monospace;  
}
```

```
`</style>
```

```
</div>
```

```
)  
}
```

```
export default Home"
```

```
src/pages/leaderboard.jsx
```

```
"import React, { useState, useEffect } from 'react'
```

```
import {
```

```
  Trophy,
```

```
  Medal,
```

```
  Crown,
```

```
  TrendingUp,
```

```
  Users,
```

```
  Award,
```

```
  Star,
```

```
  Terminal,
```

```
  Cpu,
```

```
  Network,
```

```
  Shield,
```

```
  Zap,
```

```
  Clock,
```

```
  Eye,
```

```
  User,
```

```
  Server,
```

```
  Binary,
```

```
  Scan
```

```
} from 'lucide-react'
```

```
import axios from 'axios'
```



```
const Leaderboard = () => {  
  const [leaderboard, setLeaderboard] = useState([])  
  const [loading, setLoading] = useState(true)  
  const [timeRange, setTimeRange] = useState('all')  
  const [terminalOutput, setTerminalOutput] = useState([])  
  const [currentUserRank, setCurrentUserRank] = useState(null)  
  
  useEffect(() => {  
    fetchLeaderboard()  
  }, [timeRange])  
  
  const addTerminalLine = (text, type = 'info') => {  
    setTerminalOutput(prev => [...prev, {  
      text,  
      type,  
      timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })  
    }])  
  }  
  
  const fetchLeaderboard = async () => {  
    try {  
      addTerminalLine('Accessing global rankings database...', 'system')  
      setLoading(true)  
      const response = await axios.get('/leaderboard')  
      setLeaderboard(response.data)  
  
      // Simulate finding current user rank (in real app, this would come from API)  
      const simulatedUserRank = Math.floor(Math.random() * response.data.length) + 1  
      setCurrentUserRank(simulatedUserRank)
```

```

    addTerminalLine(`Rankings loaded: ${response.data.length} operators detected`, 'success')
  } catch (error) {
    addTerminalLine('ERROR: Failed to access ranking system', 'error')
    console.error('Error fetching leaderboard:', error)
  } finally {
    setLoading(false)
  }
}

```

```

const getRankIcon = (index) => {
  switch (index) {
    case 0:
      return <Crown className="w-6 h-6 text-yellow-400" />
    case 1:
      return <Medal className="w-6 h-6 text-gray-300" />
    case 2:
      return <Medal className="w-6 h-6 text-orange-400" />
    default:
      return (
        <div className="w-6 h-6 flex items-center justify-center bg-gray-700 rounded-full border border-green-500">
          <span className="text-xs font-bold text-green-400">{index + 1}</span>
        </div>
      )
  }
}

```

```

const getRankColor = (index) => {

```

```
switch (index) {  
  case 0: return 'from-yellow-500 to-yellow-600'  
  case 1: return 'from-gray-400 to-gray-600'  
  case 2: return 'from-orange-500 to-orange-600'  
  default: return 'from-green-500 to-cyan-600'  
}  
}
```

```
const getRankBadge = (index) => {  
  switch (index) {  
    case 0: return { text: 'ELITE', color: 'text-yellow-400 border-yellow-400 bg-yellow-400 bg-opacity-10' }  
    case 1: return { text: 'VETERAN', color: 'text-gray-300 border-gray-300 bg-gray-300 bg-opacity-10' }  
    case 2: return { text: 'OPERATOR', color: 'text-orange-400 border-orange-400 bg-orange-400 bg-opacity-10' }  
    default:  
      if (index < 10) return { text: 'PRO', color: 'text-cyan-400 border-cyan-400 bg-cyan-400 bg-opacity-10' }  
      return { text: 'AGENT', color: 'text-green-400 border-green-400 bg-green-400 bg-opacity-10' }  
  }  
}
```

```
const timeRanges = [  
  { value: 'all', label: 'GLOBAL_RANKINGS', command: 'cat /var/log/global_rank' },  
  { value: 'weekly', label: 'WEEKLY_STANDINGS', command: './stats --period weekly' },  
  { value: 'daily', label: 'DAILY_PERFORMANCE', command: './stats --period daily' }  
]
```

```
if (loading) {  
  return (  

```

```

<div className="min-h-screen bg-gray-950 flex items-center justify-center">

  <div className="text-center">

    <div className="w-16 h-16 border-4 border-green-500 border-t-transparent rounded-full
animate-spin mx-auto mb-4"></div>

    <div className="text-green-400 font-mono">ACCESSING_RANKINGS...</div>

    <div className="text-green-600 text-sm mt-2">Decrypting leaderboard data</div>

  </div>

</div>

)
}

return (

  <div className="min-h-screen bg-gray-950 text-green-400 font-mono">

    <div className="max-w-7xl mx-auto px-4 py-8">

      { /* Terminal Header */ }

      <div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20
mb-8">

        <div className="flex items-center space-x-2 p-4 border-b border-green-500/30">

          <div className="w-3 h-3 bg-red-500 rounded-full"></div>

          <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>

          <div className="w-3 h-3 bg-green-500 rounded-full"></div>

          <div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- ranking_system</div>

        </div>

        <div className="p-6">

          <div className="flex items-center space-x-2 text-green-300 mb-4">

            <Terminal className="w-5 h-5" />

            <span className="text-sm">GLOBAL_OPERATOR_RANKINGS</span>

          </div>

```

```

    { /* Terminal Output */ }

    <div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-24 overflow-y-auto">

      {terminalOutput.map((line, index) => (

        <div key={index} className={`text-sm ${

          line.type === 'error' ? 'text-red-400' :

          line.type === 'success' ? 'text-green-400' :

          line.type === 'system' ? 'text-cyan-400' : 'text-green-300'

        }`}>

          <span className="text-gray-500">[{line.timestamp}]</span> {line.text}

        </div>

      )]}

    </div>

    { /* Header Section */ }

    <div className="text-center mb-8">

      <div className="inline-flex items-center justify-center w-20 h-20 bg-gradient-to-r from-yellow-500 to-orange-500 rounded-2xl mb-4 border border-yellow-400 shadow-lg shadow-yellow-500/25">

        <Trophy className="w-10 h-10 text-white" />

      </div>

      <h1 className="text-3xl lg:text-4xl font-bold text-white mb-2 glitch" data-text="OPERATOR_RANKINGS">

        OPERATOR_RANKINGS

      </h1>

      <p className="text-green-300">Elite cybersecurity operators performance metrics</p>

    </div>

  </div>
</div>

```

```

{/* Time Range Filters */}

<div className="flex justify-center mb-8">

  <div className="bg-black border border-green-500 rounded-lg p-2">

    <div className="flex flex-wrap justify-center gap-2">

      {timeRanges.map((range) => (

        <button

          key={range.value}

          onClick={() => setTimeRange(range.value)}

          className={`px-6 py-3 rounded-lg font-bold text-sm transition-all border ${

            timeRange === range.value

              ? 'bg-cyan-500 text-white border-cyan-400 shadow-lg shadow-cyan-500/25'

              : 'text-green-400 border-green-500 hover:border-cyan-400 hover:text-cyan-400'

            }}

        >

          {range.label}

        </button>

      )})}

    </div>

  </div>

</div>

```

```

{/* Stats Overview */}

<div className="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">

  <div className="bg-black border border-green-500 rounded-lg p-6 text-center hover:border-cyan-400 transition-all group">

    <Users className="w-8 h-8 text-cyan-400 mx-auto mb-3 group-hover:scale-110 transition-transform" />

    <div className="text-2xl font-bold text-white">{leaderboard.length}</div>

    <div className="text-green-300">ACTIVE_OPERATORS</div>

```

```

    </div>

    <div className="bg-black border border-green-500 rounded-lg p-6 text-center hover:border-cyan-400 transition-all group">

        <Award className="w-8 h-8 text-yellow-400 mx-auto mb-3 group-hover:scale-110 transition-transform" />

        <div className="text-2xl font-bold text-white">

            {leaderboard.reduce((sum, user) => sum + (user.solvedChallenges || 0), 0)}

        </div>

        <div className="text-green-300">TOTAL_BREACHES</div>

    </div>

    <div className="bg-black border border-green-500 rounded-lg p-6 text-center hover:border-cyan-400 transition-all group">

        <TrendingUp className="w-8 h-8 text-green-400 mx-auto mb-3 group-hover:scale-110 transition-transform" />

        <div className="text-2xl font-bold text-white">

            {leaderboard.reduce((sum, user) => sum + (user.totalPoints || 0), 0)}

        </div>

        <div className="text-green-300">TOTAL_POINTS</div>

    </div>

</div>

{/* Leaderboard Table */}

<div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20 overflow-hidden mb-8">

    {/* Table Header */}

    <div className="grid grid-cols-12 bg-gradient-to-r from-green-900 to-cyan-900 px-6 py-4 text-sm font-bold text-white border-b border-green-500">

        <div className="col-span-1">RANK</div>

        <div className="col-span-4">OPERATOR</div>

        <div className="col-span-2 text-center">BREACHES</div>

        <div className="col-span-2 text-center">POINTS</div>

```

```

<div className="col-span-2 text-center">LAST_ACTIVITY</div>

<div className="col-span-1 text-center">STATUS</div>

</div>

```

```

{leaderboard.length === 0 ? (
  <div className="text-center py-12">
    <TrendingUp className="w-16 h-16 text-green-400 mx-auto mb-4" />
    <h3 className="text-lg font-bold text-white mb-2">NO_DATA_AVAILABLE</h3>
    <p className="text-green-300">Initiate system penetration to appear in rankings</p>
  </div>
) : (
  <div className="divide-y divide-green-500 divide-opacity-30">
    {leaderboard.map((user, index) => {
      const rankBadge = getRankBadge(index)
      return (
        <div
          key={user.id}
          className={`grid grid-cols-12 px-6 py-4 items-center transition-all hover:bg-green-500
hover:bg-opacity-5 ${
            index < 3 ? 'bg-gradient-to-r from-yellow-500 to-orange-500 bg-opacity-5' : ''
          }}
        >
          { /* Rank */ }
          <div className="col-span-1 flex items-center">
            {getRankIcon(index)}
          </div>

          { /* Player Info */ }
          <div className="col-span-4">

```



```

<div className="flex items-center space-x-4">
  <div className={`w-12 h-12 rounded-full bg-gradient-to-r ${getRankColor(index)} flex
items-center justify-center text-white font-bold border-2 border-white border-opacity-20`}>
    {user.username?.charAt(0).toUpperCase()}
  </div>
  <div>
    <div className="font-bold text-white flex items-center space-x-2">
      <span>{user.username}</span>
      {index < 3 && <Star className="w-4 h-4 text-yellow-400" />}
    </div>
    <div className={`text-xs px-2 py-1 rounded border ${rankBadge.color} inline-block mt-
1`}>
      {rankBadge.text}
    </div>
  </div>
</div>
</div>

```

```

{/* Solved Challenges */}

```

```

<div className="col-span-2 text-center">
  <div className="text-lg font-bold text-cyan-400">
    {user.solvedChallenges || 0}
  </div>
  <div className="text-xs text-green-300">BREACHES</div>
</div>

```

```

{/* Points */}

```

```

<div className="col-span-2 text-center">
  <div className="text-lg font-bold text-yellow-400">

```

```

        {user.totalPoints || 0}
      </div>

      <div className="text-xs text-green-300">POINTS</div>
    </div>

    { /* Last Solve */ }

    <div className="col-span-2 text-center">
      <div className="text-sm text-green-300">
        {user.lastSolve
          ? new Date(user.lastSolve).toLocaleDateString()
          : 'INACTIVE'
        }
      </div>
    </div>

    { /* Status */ }

    <div className="col-span-1 text-center">
      <div className={`w-3 h-3 rounded-full mx-auto ${
        user.lastSolve && (Date.now() - new Date(user.lastSolve).getTime()) < 24 * 60 * 60 * 1000
          ? 'bg-green-500 animate-pulse'
          : 'bg-gray-500'
        }`} ></div>
    </div>
  </div>
)
}}
</div>
)}
</div>

```

```

{ /* Current User Position & CTA */

<div className="grid md:grid-cols-2 gap-6">

  { /* Current Position */

    {currentUserRank && (

      <div className="bg-black border border-cyan-500 rounded-lg p-6">

        <div className="flex items-center space-x-3 mb-4">

          <User className="w-6 h-6 text-cyan-400" />

          <h3 className="text-lg font-bold text-white">YOUR_POSITION</h3>

        </div>

        <div className="text-center">

          <div className="text-3xl font-bold text-cyan-400 mb-2">#{currentUserRank}</div>

          <div className="text-green-300 text-sm">GLOBAL_RANKING</div>

          <div className="w-full bg-gray-700 rounded-full h-2 mt-3">

            <div

              className="bg-gradient-to-r from-cyan-500 to-green-500 h-2 rounded-full"

              style={{ width: `${((leaderboard.length - currentUserRank) / leaderboard.length) * 100}%` }}

            ></div>

          </div>

        </div>

      </div>

    )}

  { /* CTA */

    <div className="bg-gradient-to-r from-green-900 via-black to-cyan-900 border border-green-500 rounded-lg p-6 text-center">

      <Zap className="w-8 h-8 text-cyan-400 mx-auto mb-3" />

      <h3 className="text-lg font-bold text-white mb-2">READY_TO_CLIMB?</h3>

      <p className="text-green-300 text-sm mb-4">

```

Execute more successful breaches to advance your ranking

</p>

<button className="px-6 py-3 bg-gradient-to-r from-green-600 to-cyan-600 text-white font-bold rounded-lg hover:from-green-700 hover:to-cyan-700 transition-all border border-cyan-400">

DEPLOY_OPERATIONS

</button>

</div>

</div>

{/* System Footer */}

<div className="bg-black border border-green-500 rounded-lg p-4 mt-8">

<div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-center text-sm">

<div>

<div className="text-green-300">RANKING_ALGO</div>

<div className="text-white font-bold">ELO_V2.4</div>

</div>

<div>

<div className="text-green-300">UPDATE_FREQ</div>

<div className="text-white font-bold">REAL_TIME</div>

</div>

<div>

<div className="text-green-300">DATA_SOURCE</div>

<div className="text-white font-bold">SECURE_DB</div>

</div>

<div>

<div className="text-green-300">VALIDATION</div>

<div className="text-white font-bold">ACTIVE</div>

</div>

</div>

```
</div>
```

```
</div>
```

```
<style jsx>{`
```

```
.glitch {  
  position: relative;  
}
```

```
.glitch::before,  
.glitch::after {  
  content: attr(data-text);  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
}
```

```
.glitch::before {  
  animation: glitch-1 0.5s infinite linear alternate-reverse;  
  color: #ff00ff;  
  z-index: -1;  
}
```

```
.glitch::after {  
  animation: glitch-2 0.5s infinite linear alternate-reverse;  
  color: #00ffff;  
  z-index: -2;  
}
```

```
@keyframes glitch-1 {  
  0% { transform: translate(0); }  
  20% { transform: translate(-2px, 2px); }  
  40% { transform: translate(-2px, -2px); }  
  60% { transform: translate(2px, 2px); }  
  80% { transform: translate(2px, -2px); }  
  100% { transform: translate(0); }  
}
```

```
@keyframes glitch-2 {  
  0% { transform: translate(0); }  
  20% { transform: translate(2px, -2px); }  
  40% { transform: translate(2px, 2px); }  
  60% { transform: translate(-2px, -2px); }  
  80% { transform: translate(-2px, 2px); }  
  100% { transform: translate(0); }  
}
```

```
`</style>
```

```
</div>
```

```
)
```

```
}
```

```
export default Leaderboard"
```

```
src/pages/login.jsx
```

```
"import React, { useState, useEffect } from 'react'
```

```
import { Link, useNavigate, useLocation } from 'react-router-dom'
```

```
import { useAuth } from '../contexts/AuthContext'
```

```
import {  
  Terminal,  
  Eye,  
  EyeOff,  
  Mail,  
  Lock,  
  Shield,  
  User,  
  Server,  
  Scan,  
  Key  
} from 'lucide-react'
```

```
const Login = () => {  
  const [formData, setFormData] = useState({  
    email: "",  
    password: ""  
  })  
  
  const [showPassword, setShowPassword] = useState(false)  
  const [error, setError] = useState("")  
  const [loading, setLoading] = useState(false)  
  const [terminalOutput, setTerminalOutput] = useState([])  
  const [accessAttempts, setAccessAttempts] = useState(0)  
  
  const { login } = useAuth()  
  const navigate = useNavigate()  
  const location = useLocation()  
  
  const from = location.state?.from?.pathname || '/challenges'
```

```

useEffect(() => {
  // Initial terminal message
  addTerminalLine('Authentication system initialized...', 'system')
  addTerminalLine('Waiting for user credentials...', 'system')
}, [])

const addTerminalLine = (text, type = 'info') => {
  setTerminalOutput(prev => [...prev, {
    text,
    type,
    timestamp: new Date().toLocaleTimeString('en-US', { hour12: false })
  }])
}

const handleChange = (e) => {
  setFormData(prev => ({
    ...prev,
    [e.target.name]: e.target.value
  }))
}

const handleSubmit = async (e) => {
  e.preventDefault()
  setError('')
  setLoading(true)
  setAccessAttempts(prev => prev + 1)

  addTerminalLine(`Access attempt #${accessAttempts + 1} initiated...`, 'system')

```



```
addTerminalLine('Verifying user credentials...', 'system')
```

```
const result = await login(formData.email, formData.password)
```

```
if (result.success) {
```

```
  addTerminalLine('Credentials verified successfully!', 'success')
```

```
  addTerminalLine('Granting system access...', 'system')
```

```
  addTerminalLine('Establishing secure session...', 'system')
```

```
  setTimeout(() => navigate(from, { replace: true }), 1500)
```

```
} else {
```

```
  addTerminalLine(`AUTHENTICATION FAILED: ${result.error}`, 'error')
```

```
  addTerminalLine('Security protocol engaged...', 'system')
```

```
  setError(result.error)
```

```
}
```

```
setLoading(false)
```

```
}
```

```
const simulateIntrusionDetection = () => {
```

```
  addTerminalLine('Intrusion detection system activated...', 'warning')
```

```
  addTerminalLine('Scanning for security threats...', 'system')
```

```
  addTerminalLine('All systems secure.', 'success')
```

```
}
```

```
return (
```

```
<div className="min-h-screen bg-gray-950 text-green-400 font-mono relative">
```

```
  {/* Matrix Background */}
```

```
<div className="fixed inset-0 bg-black opacity-90 z-0">
```

```
  <div className="matrix-rain"></div>
```

</div>

<div className="relative z-10 flex items-center justify-center min-h-screen py-12 px-4">

<div className="max-w-2xl w-full space-y-8">

{/* Terminal Header */}

<div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20">

<div className="flex items-center space-x-2 p-4 border-b border-green-500/30">

<div className="w-3 h-3 bg-red-500 rounded-full"></div>

<div className="w-3 h-3 bg-yellow-500 rounded-full"></div>

<div className="w-3 h-3 bg-green-500 rounded-full"></div>

<div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- authentication_gateway</div>

</div>

<div className="p-6">

{/* System Header */}

<div className="flex items-center space-x-2 text-green-300 mb-6">

<Terminal className="w-5 h-5" />

SECURE_AUTHENTICATION_PROTOCOL

</div>

{/* Terminal Output */}

<div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-32 overflow-y-auto">

{terminalOutput.map((line, index) => (

<div key={index} className={`text-sm \${

line.type === 'error' ? 'text-red-400' :

line.type === 'success' ? 'text-green-400' :

line.type === 'warning' ? 'text-yellow-400' :

```

        line.type === 'system' ? 'text-cyan-400' : 'text-green-300'
      } }>
      <span className="text-gray-500">[{line.timestamp}]/> {line.text}
    </div>
  )}
</div>

{/* Login Form */}
<form onSubmit={handleSubmit} className="space-y-6">
  {error && (
    <div className="bg-red-500 bg-opacity-10 border border-red-500 text-red-400 px-4 py-3
rounded-lg font-mono text-sm">
      <span className="text-red-400">[SECURITY BREACH]/> {error}
    </div>
  )}

  {/* Email Field */}
  <div>
    <label htmlFor="email" className="block text-sm font-bold text-green-400 mb-2">
      [IDENTITY] ACCESS_ID
    </label>
    <div className="relative">
      <Mail className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"
/>
      <input
        type="email"
        id="email"
        name="email"
        value={formData.email}

```

```

        onChange={handleChange}

        required

        className="w-full pl-10 pr-4 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"

        placeholder="user@cyber-arena.io"

    />
</div>
</div>

{/* Password Field */}
<div>

    <label htmlFor="password" className="block text-sm font-bold text-green-400 mb-2">
        [SECURITY] ENCRYPTION_KEY
    </label>

    <div className="relative">

        <Lock className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"
/>

        <input

            type={showPassword ? 'text' : 'password'}

            id="password"

            name="password"

            value={formData.password}

            onChange={handleChange}

            required

            className="w-full pl-10 pr-12 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"

            placeholder="Enter encryption key"

        />

```

```

    <button
      type="button"
      onClick={() => setShowPassword(!showPassword)}
      className="absolute right-3 top-1/2 transform -translate-y-1/2 text-green-400 hover:text-
cyan-400 transition-colors"
    >
      {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
    </button>
  </div>
</div>

```

```

  { /* Security Options */ }

  <div className="flex items-center justify-between p-4 bg-black border border-green-500
rounded-lg">
    <div className="flex items-center space-x-3">
      <input
        type="checkbox"
        id="remember"
        className="w-4 h-4 text-green-500 bg-black border-green-500 rounded focus:ring-green-
500"
      />
      <label htmlFor="remember" className="text-sm text-green-300">
        MAINTAIN_SESSION
      </label>
    </div>
    <button
      type="button"
      onClick={simulateIntrusionDetection}
      className="text-sm text-cyan-400 hover:text-cyan-300 font-bold flex items-center space-x-

```

```

>

<Scan className="w-4 h-4" />

<span>SYSTEM_SCAN</span>

</button>

</div>

{/* Submit Button */}

<button

  type="submit"

  disabled={loading}

  className="w-full bg-gradient-to-r from-green-600 to-cyan-600 text-white py-4 px-6
rounded-lg hover:from-green-700 hover:to-cyan-700 focus:outline-none focus:ring-2 focus:ring-cyan-
500 focus:ring-offset-2 focus:ring-offset-black disabled:opacity-50 disabled:cursor-not-allowed
transition-all font-bold border border-cyan-400 shadow-lg shadow-cyan-500/25 relative group"

  >

    {loading ? (

      <div className="flex items-center justify-center">

        <div className="w-5 h-5 border-2 border-white border-t-transparent rounded-full
animate-spin mr-3"></div>

        <span className="text-sm">AUTHENTICATING...</span>

      </div>

    ) : (

      <div className="flex items-center justify-center">

        <Shield className="w-5 h-5 mr-2" />

        <span>INITIATE_AUTHENTICATION</span>

      </div>

    )}

  </button>

</form>

```

```
{/* Registration Link */}

<div className="mt-6 pt-6 border-t border-green-500 border-opacity-30">

  <p className="text-center text-green-300 text-sm">

    No system access?{' '}

    <Link

      to="/register"

      className="text-cyan-400 hover:text-cyan-300 font-bold"

    >

      REQUEST_CLEARANCE

    </Link>

  </p>

</div>

</div>

</div>
```

```
{/* Demo Credentials - Hacker Style */}

<div className="bg-black border border-yellow-500 rounded-lg p-6 text-center">

  <div className="flex items-center justify-center space-x-2 text-yellow-400 mb-3">

    <Key className="w-4 h-4" />

    <h3 className="text-sm font-bold">TEST_CREDENTIALS</h3>

  </div>

  <div className="space-y-2 text-xs">

    <div className="text-green-400 font-mono">

      <span className="text-gray-500">ACCESS_ID:</span> admin@cyber-arena.io

    </div>

    <div className="text-green-400 font-mono">

      <span className="text-gray-500">ENCRYPTION_KEY:</span> Admin123!

    </div>

    <div className="text-cyan-400 text-xs mt-2">
```

```
[PRIVILEGE_LEVEL: ROOT_ACCESS]

</div>

</div>

</div>

{/* System Status */}

<div className="grid grid-cols-3 gap-4 text-xs text-center">

  <div className="bg-black border border-green-500 rounded p-3">

    <div className="text-green-400 font-bold">SECURE</div>

    <div className="text-gray-400">CONNECTION</div>

  </div>

  <div className="bg-black border border-cyan-500 rounded p-3">

    <div className="text-cyan-400 font-bold">ACTIVE</div>

    <div className="text-gray-400">SESSION</div>

  </div>

  <div className="bg-black border border-green-500 rounded p-3">

    <div className="text-green-400 font-bold">ENCRYPTED</div>

    <div className="text-gray-400">CHANNEL</div>

  </div>

</div>

</div>

</div>

<style jsx>{`

.matrix-rain {

  position: absolute;

  top: 0;

  left: 0;

  width: 100%;
```



```
height: 100%;  
background: linear-gradient(180deg, rgba(0,255,0,0.1) 0%, transparent 50%);  
opacity: 0.1;  
pointer-events: none;  
}
```

```
.blinking-cursor {  
  animation: blink 1s infinite;  
  color: #00ff00;  
}
```

```
@keyframes blink {  
  0%, 50% { opacity: 1; }  
  51%, 100% { opacity: 0; }  
}
```

```
`</style>
```

```
</div>
```

```
)
```

```
}
```

```
export default Login",
```

```
register.jsx
```

```
"import React, { useState } from 'react'
```

```
import { Link, useNavigate } from 'react-router-dom'
```

```
import { useAuth } from '../contexts/AuthContext'
```

```
import {
```

```
  Terminal,
```

```
  UserPlus,
```

```
  User,
```

```
Mail,  
Lock,  
Eye,  
EyeOff,  
Shield,  
Server,  
Key,  
Scan  
} from 'lucide-react'
```

```
const Register = () => {  
  const [formData, setFormData] = useState({  
    fullName: "",  
    username: "",  
    email: "",  
    password: "",  
    confirmPassword: ""  
  })  
  
  const [showPassword, setShowPassword] = useState(false)  
  const [showConfirmPassword, setShowConfirmPassword] = useState(false)  
  const [error, setError] = useState("")  
  const [loading, setLoading] = useState(false)  
  const [terminalOutput, setTerminalOutput] = useState([])  
  
  const { register } = useAuth()  
  const navigate = useNavigate()  
  
  const addTerminalLine = (text, type = 'info') => {  
    setTerminalOutput(prev => [...prev, { text, type, timestamp: new Date().toLocaleTimeString() }])  
  }  
}
```

```
}
```

```
const handleChange = (e) => {  
  setFormData(prev => ({  
    ...prev,  
    [e.target.name]: e.target.value  
  })))  
}
```

```
const handleSubmit = async (e) => {  
  e.preventDefault()  
  setError("")  
  setTerminalOutput([])
```

```
  addTerminalLine('Initializing user registration protocol...', 'system')  
  addTerminalLine('Validating input parameters...', 'system')
```

```
  // Validation
```

```
  if (formData.password !== formData.confirmPassword) {  
    addTerminalLine('ERROR: Password verification failed - mismatch detected', 'error')  
    setError('Passwords do not match')  
    return  
  }
```

```
  if (formData.password.length < 6) {  
    addTerminalLine('ERROR: Password strength insufficient - minimum 6 characters required', 'error')  
    setError('Password must be at least 6 characters long')  
    return  
  }
```

```
if (formData.username.length < 3) {  
  addTerminalLine('ERROR: Username validation failed - minimum 3 characters required', 'error')  
  setError('Username must be at least 3 characters long')  
  return  
}
```

```
addTerminalLine('All validations passed. Establishing secure connection...', 'success')  
setLoading(true)
```

```
const { confirmPassword, ...registerData } = formData  
const result = await register(registerData)
```

```
if (result.success) {  
  addTerminalLine('User account created successfully!', 'success')  
  addTerminalLine('Granting system access...', 'system')  
  addTerminalLine('Redirecting to challenge interface...', 'system')  
  setTimeout(() => navigate('/challenges'), 1500)  
} else {  
  addTerminalLine(`ERROR: Registration failed - ${result.error}`, 'error')  
  setError(result.error)  
}
```

```
setLoading(false)  
}
```

```
const passwordStrength = (password) => {  
  if (password.length === 0) return { strength: 0, label: '', color: 'bg-gray-500' }  
  if (password.length < 6) return { strength: 1, label: 'CRITICAL', color: 'bg-red-500' }
```

```

if (password.length < 8) return { strength: 2, label: 'WEAK', color: 'bg-orange-500' }
if (password.length < 10) return { strength: 3, label: 'SECURE', color: 'bg-yellow-500' }
return { strength: 4, label: 'STRONG', color: 'bg-green-500' }
}

```

```

const strength = passwordStrength(formData.password)

```

```

return (
  <div className="min-h-screen bg-gray-950 text-green-400 font-mono relative">
    {/* Matrix Background */}
    <div className="fixed inset-0 bg-black opacity-90 z-0">
      <div className="matrix-rain"></div>
    </div>

    <div className="relative z-10 flex items-center justify-center min-h-screen py-12 px-4">
      <div className="max-w-2xl w-full space-y-8">
        {/* Terminal Header */}
        <div className="bg-black border border-green-500 rounded-lg shadow-2xl shadow-green-500/20">
          <div className="flex items-center space-x-2 p-4 border-b border-green-500/30">
            <div className="w-3 h-3 bg-red-500 rounded-full"></div>
            <div className="w-3 h-3 bg-yellow-500 rounded-full"></div>
            <div className="w-3 h-3 bg-green-500 rounded-full"></div>
            <div className="text-green-400 text-sm ml-2">cyber-arena.ctf -- user_registration</div>
          </div>

          <div className="p-6">
            {/* System Header */}
            <div className="flex items-center space-x-2 text-green-300 mb-6">

```

```
<Terminal className="w-5 h-5" />

<span className="text-sm">SYSTEM_REGISTRATION_PROTOCOL</span>

</div>
```

```
{/* Terminal Output */}

<div className="bg-black border border-green-500 rounded-lg p-4 mb-6 h-32 overflow-y-
auto">

  {terminalOutput.map((line, index) => (

    <div key={index} className={`text-sm ${
      line.type === 'error' ? 'text-red-400' :
      line.type === 'success' ? 'text-green-400' :
      line.type === 'system' ? 'text-cyan-400' : 'text-green-300'
    }`}>

      <span className="text-gray-500">[{line.timestamp}]</span> {line.text}

    </div>

  )]}

  {terminalOutput.length === 0 && (

    <div className="text-gray-500 text-sm">

      [SYSTEM] Waiting for registration sequence initiation...

    </div>

  )}

</div>
```

```
{/* Registration Form */}

<form onSubmit={handleSubmit} className="space-y-6">

  {error && (

    <div className="bg-red-500 bg-opacity-10 border border-red-500 text-red-400 px-4 py-3
rounded-lg font-mono text-sm">

      <span className="text-red-400">[ERROR]</span> {error}

    
```

```
</div>
```

```
}}
```

```
<div className="grid md:grid-cols-2 gap-6">
```

```
  { /* Full Name */ }
```

```
  <div>
```

```
    <label htmlFor="fullName" className="block text-sm font-bold text-green-400 mb-2">
```

```
      [IDENTITY] FULL_NAME
```

```
    </label>
```

```
    <div className="relative">
```

```
      <User className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4" />
```

```
      <input
```

```
        type="text"
```

```
        id="fullName"
```

```
        name="fullName"
```

```
        value={formData.fullName}
```

```
        onChange={handleChange}
```

```
        required
```

```
        className="w-full pl-10 pr-4 py-3 bg-black border border-green-500 rounded-lg  
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-  
green-600 transition-all font-mono"
```

```
        placeholder="Enter full name"
```

```
      />
```

```
    </div>
```

```
</div>
```

```
  { /* Username */ }
```

```
  <div>
```

```
    <label htmlFor="username" className="block text-sm font-bold text-green-400 mb-2">
```

```

[ACCESS] USERNAME
</label>

<div className="relative">
  <User className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-
4" />

  <input
    type="text"
    id="username"
    name="username"
    value={formData.username}
    onChange={handleChange}
    required
    minLength={3}

    className="w-full pl-10 pr-4 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"

    placeholder="Choose username"
  />
</div>
</div>
</div>

{/* Email */}
<div>
  <label htmlFor="email" className="block text-sm font-bold text-green-400 mb-2">

    [CONTACT] EMAIL_ADDRESS

  </label>

  <div className="relative">

    <Mail className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"
  />

```



```

<input
  type="email"
  id="email"
  name="email"
  value={formData.email}
  onChange={handleChange}
  required

  className="w-full pl-10 pr-4 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"

  placeholder="user@domain.com"
/>
</div>
</div>

{/* Password */}
<div>
  <label htmlFor="password" className="block text-sm font-bold text-green-400 mb-2">
    [SECURITY] ENCRYPTION_KEY
  </label>
  <div className="relative">
    <Lock className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"
  />
    <input
      type={showPassword ? 'text' : 'password'}
      id="password"
      name="password"
      value={formData.password}
      onChange={handleChange}
      required

```

```

        minLength={6}

        className="w-full pl-10 pr-12 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"

        placeholder="Create encryption key"
    />

    <button

        type="button"

        onClick={() => setShowPassword(!showPassword)}

        className="absolute right-3 top-1/2 transform -translate-y-1/2 text-green-400 hover:text-
cyan-400 transition-colors"

    >

        {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}

    </button>

</div>

```

```

{/* Password Strength Meter */}

{formData.password && (

    <div className="mt-3 space-y-2">

        <div className="flex items-center space-x-2 text-xs">

            <span className="text-green-400">SECURITY_LEVEL:</span>

            <span className={`font-bold ${

                strength.strength === 1 ? 'text-red-400' :

                strength.strength === 2 ? 'text-orange-400' :

                strength.strength === 3 ? 'text-yellow-400' : 'text-green-400'

            }`}>

                {strength.label}

            </span>

        </div>

    </div>

    <div className="flex space-x-1">

```

```

    {[1, 2, 3, 4].map((level) => (
      <div
        key={level}
        className={`h-2 flex-1 rounded-full ${
          level <= strength.strength ? strength.color : 'bg-gray-700'
        }}
      />
    )]}
  </div>
</div>
)}
</div>

```

```

    { /* Confirm Password */ }
    <div>
      <label htmlFor="confirmPassword" className="block text-sm font-bold text-green-400 mb-
2">
        [VERIFY] CONFIRM_KEY
      </label>
      <div className="relative">
        <Key className="absolute left-3 top-1/2 transform -translate-y-1/2 text-green-400 w-4 h-4"
/>
        <input
          type={showConfirmPassword ? 'text' : 'password'}
          id="confirmPassword"
          name="confirmPassword"
          value={formData.confirmPassword}
          onChange={handleChange}
          required

```

```
        className="w-full pl-10 pr-12 py-3 bg-black border border-green-500 rounded-lg
focus:outline-none focus:ring-2 focus:ring-cyan-500 focus:border-cyan-500 text-white placeholder-
green-600 transition-all font-mono"
```

```
        placeholder="Verify encryption key"
```

```
    />
```

```
<button
```

```
    type="button"
```

```
    onClick={() => setShowConfirmPassword(!showConfirmPassword)}
```

```
        className="absolute right-3 top-1/2 transform -translate-y-1/2 text-green-400 hover:text-
cyan-400 transition-colors"
```

```
>
```

```
    {showConfirmPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
{/* Terms Agreement */}
```

```
<div className="flex items-start space-x-3 p-4 bg-black border border-green-500 rounded-lg">
```

```
<input
```

```
    type="checkbox"
```

```
    id="terms"
```

```
    required
```

```
        className="w-4 h-4 text-green-500 bg-black border-green-500 rounded focus:ring-green-
500 mt-1"
```

```
    />
```

```
<label htmlFor="terms" className="text-sm text-green-300">
```

```
    I acknowledge and agree to the{' '}
```

```
<Link to="/terms" className="text-cyan-400 hover:text-cyan-300 font-bold">
```

```
    SYSTEM_TERMS
```

```
</Link>{' '}
```

```
and{' '}
<Link to="/privacy" className="text-cyan-400 hover:text-cyan-300 font-bold">
  SECURITY_PROTOCOLS
</Link>
</label>
</div>
```

```
{/* Submit Button */}
<button
  type="submit"
  disabled={loading}
  className="w-full bg-gradient-to-r from-green-600 to-cyan-600 text-white py-4 px-6
rounded-lg hover:from-green-700 hover:to-cyan-700 focus:outline-none focus:ring-2 focus:ring-cyan-
500 focus:ring-offset-2 focus:ring-offset-black disabled:opacity-50 disabled:cursor-not-allowed
transition-all font-bold border border-cyan-400 shadow-lg shadow-cyan-500/25 relative group"
  >
  {loading ? (
    <div className="flex items-center justify-center">
      <div className="w-5 h-5 border-2 border-white border-t-transparent rounded-full
animate-spin mr-3"></div>
      <span className="text-sm">INITIALIZING_ACCESS...</span>
    </div>
  ) : (
    <div className="flex items-center justify-center">
      <Shield className="w-5 h-5 mr-2" />
      <span>EXECUTE_REGISTRATION</span>
    </div>
  )}
</button>
</form>
```

```
{/* Login Link */}

<div className="mt-6 pt-6 border-t border-green-500 border-opacity-30">

  <p className="text-center text-green-300 text-sm">

    Already have system access?{' '}

    <Link

      to="/login"

      className="text-cyan-400 hover:text-cyan-300 font-bold"

    >

      INITIATE_AUTHENTICATION

    </Link>

  </p>

</div>

</div>

</div>

</div>

</div>
```

```
<style jsx>{`

.matrix-rain {

  position: absolute;

  top: 0;

  left: 0;

  width: 100%;

  height: 100%;

  background: linear-gradient(180deg, rgba(0,255,0,0.1) 0%, transparent 50%);

  opacity: 0.1;

  pointer-events: none;

}
```

```
`}</style>
</div>
)
}
```

```
export default Register"
```

```
app.jsx
```

```
"// src/App.jsx
```

```
import React from 'react'
```

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom'
```

```
import { AuthProvider, useAuth } from './contexts/AuthContext.jsx'
```

```
import LoadingSpinner from './components/ui/LoadingSpinner.jsx'
```

```
import ErrorBoundary from './components/ui/ErrorBoundary.jsx'
```

```
// Layout Components
```

```
import MainLayout from './layouts/MainLayout.jsx'
```

```
import AdminLayout from './layouts/AdminLayout.jsx'
```

```
import UserLayout from './layouts/UserLayout.jsx'
```

```
// Your Existing Page Components
```

```
import Home from './pages/Home.jsx'
```

```
import Challenges from './pages/Challenges.jsx'
```

```
import Leaderboard from './pages/Leaderboard.jsx'
```

```
import Login from './pages/Login.jsx'
```

```
import Register from './pages/Register.jsx'
```

```
import Admin from './pages/Admin.jsx'
```

```
// Create router with proper future flags configuration
```

```
const createRouter = () => {  
  return Router  
}
```

```
// Protected Route Component
```

```
const ProtectedRoute = ({ children, requireAdmin = false }) => {  
  const { user, isAdmin } = useAuth()  
  
  if (!user) {  
    return <Navigate to="/login" replace />  
  }
```

```
  if (requireAdmin && !isAdmin) {  
    return <Navigate to="/" replace />  
  }
```

```
  return children  
}
```

```
// Public Route Component (redirect to dashboard if already logged in)
```

```
const PublicRoute = ({ children }) => {  
  const { user } = useAuth()  
  
  if (user) {  
    return <Navigate to="/dashboard" replace />  
  }
```

```
  return children  
}
```



```
// Inner App component that uses the auth context
```

```
function AppContent() {
```

```
  const { loading } = useAuth()
```

```
  if (loading) {
```

```
    return <LoadingSpinner />
```

```
  }
```

```
  const RouterComponent = createRouter()
```

```
  return (
```

```
    <RouterComponent
```

```
      future={{
```

```
        v7_startTransition: true,
```

```
        v7_relativeSplatPath: true,
```

```
      }})
```

```
    <
```

```
    <div className="min-h-screen bg-gray-950">
```

```
      <Routes>
```

```
        { /* Public Routes with Main Layout */ }
```

```
        <Route path="/" element={ <MainLayout /> } />
```

```
        <Route index element={ <Home /> } />
```

```
        <Route
```

```
          path="/login"
```

```
          element={
```

```
            <PublicRoute>
```

```
              <Login />
```

```
            </PublicRoute>
```

```

    }
  />
  <Route
    path="/register"
    element={
      <PublicRoute>
        <Register />
      </PublicRoute>
    }
  />
  <Route path="/leaderboard" element={<Leaderboard />} />
  <Route path="/challenges" element={<Challenges />} />
</Route>

```

{/* User Dashboard Routes with User Layout */}

```

<Route
  path="/dashboard/*"
  element={
    <ProtectedRoute>
      <UserLayout />
    </ProtectedRoute>
  }
>
  <Route index element={<Challenges />} />
  <Route path="challenges" element={<Challenges />} />
</Route>

```

{/* Admin Routes with Admin Layout */}

```

<Route

```

```

    path="/admin/*"
    element={
      <ProtectedRoute requireAdmin={true}>
        <AdminLayout />
      </ProtectedRoute>
    }
  >
    <Route index element={<Admin />} />
    <Route path="challenges" element={<Admin />} />
    <Route path="users" element={<Admin />} />
    <Route path="settings" element={<Admin />} />
  </Route>

  { /* Fallback Route */}
  <Route path="*" element={<Navigate to="/" replace />} />
</Routes>
</div>
</RouterComponent>
)
}

// Main App component that provides the Auth context and Error Boundary
function App() {
  return (
    <ErrorBoundary>
      <AuthProvider>
        <AppContent />
      </AuthProvider>
    </ErrorBoundary>
  )
}

```

```
)  
}
```

```
export default App",
```

```
main.jsx
```

```
"import { createRoot } from 'react-dom/client'  
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'  
import { AuthProvider } from './contexts/AuthContext.jsx'  
import ErrorBoundary from './components/ui/ErrorBoundary.jsx'  
import App from './App.jsx'  
import './index.css'
```

```
const queryClient = new QueryClient()
```

```
createRoot(document.getElementById('root')).render(  
  <ErrorBoundary>  
    <QueryClientProvider client={queryClient}>  
      <AuthProvider>  
        <App />  
      </AuthProvider>  
    </QueryClientProvider>  
  </ErrorBoundary>  
)"
```